Privacy-Preserving Machine Learning for Practitioners

Manuel V. Loureiro

Version 2026-10-21

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **2**
What each chapter is for . . . 3
Notation . . . 3
How to use the chapters . . . 3

1. Probability distributions **4**
1.1. Why this matters . . . 4
1.2. Means, variances and the square-root law . . . 4
1.3. Counts and rates: binomial and Poisson . . . 5
1.4. Tails and heavy tails . . . 5
1.5. Laplace and normal noise . . . 6
1.6. Normal noise in many dimensions . . . 7
1.7. The central limit theorem . . . 7
1.8. A density against its shifted copy . . . 8
1.9. Code . . . 8
1.10. Cheat sheet . . . 8
1.11. Further reading . . . 9

2. Concentration inequalities **10**
2.1. Why this matters . . . 10
2.2. Mean or variance only: Markov and Chebyshev . . . 10
2.3. Noise you added: Laplace and Gaussian . . . 11
2.4. Many numbers at once: the union bound . . . 11
2.5. Counts: the Chernoff bound . . . 11
2.6. Averages of bounded values: Hoeffding, Bernstein, Clopper–Pearson . . . 12
2.7. Functions and adaptive processes: McDiarmid and Azuma . . . 12
2.8. Code . . . 13
2.9. Cheat sheet . . . 14
2.10. Further reading . . . 15

3. Divergences **16**
3.1. Why this matters . . . 16
3.2. KL divergence . . . 16
3.3. Rényi divergence and the max-divergence . . . 17
3.4. Shifted normal and Laplace pairs . . . 17
3.5. Total variation . . . 18

3.6. Hockey-stick divergence . . . 18
3.7. f-divergences . . . 19
3.8. The data processing inequality . . . 19
3.9. Pinsker's inequality . . . 19
3.10. Code . . . 20
3.11. Cheat sheet . . . 20
3.12. Further reading . . . 21

4. Hypothesis testing **22**
4.1. Why this matters . . . 22
4.2. Tests and their two errors . . . 22
4.3. The Neyman–Pearson lemma . . . 22
4.4. Trade-off curves . . . 23
4.5. The Gaussian trade-off curve . . . 23
4.6. The Laplace trade-off curve . . . 23
4.7. Best advantage and total variation . . . 24
4.8. ROC summaries: why AUC hides the low-FPR regime . . . 24
4.9. The Bayesian view . . . 25
4.10. Measuring rates from finite trials . . . 25
4.11. Evaluating a detector at a low false positive rate . . . 26
4.12. Cheat sheet . . . 26
4.13. Further reading . . . 26

5. Information theory **27**
5.1. Why this matters . . . 27
5.2. Entropy . . . 27
5.3. Mutual information and data processing . . . 28
5.4. Min-entropy and one-guess leakage . . . 28
5.5. Shannon against min-entropy leakage . . . 29
5.6. Fano's inequality . . . 30
5.7. Code . . . 30
5.8. Cheat sheet . . . 31
5.9. Further reading . . . 32

6. Optimization **33**
6.1. Why this matters . . . 33
6.2. The constants of a loss, and how to enforce them . . . 33
6.3. Gradient descent and SGD . . . 34
6.4. Convex losses: the step size and the averaged iterate . . . 34
6.5. Strongly convex losses: a decaying step . . . 35
6.6. Gradient noise and minibatches . . . 35
6.7. Non-convex losses . . . 36
6.8. How far one example moves the model . . . 36
6.9. Code . . . 37
6.10. Cheat sheet . . . 37
6.11. Further reading . . . 38

7. Algorithmic stability and generalization **39**
7.1. Why this matters . . . 39
7.2. Training loss, test loss and the gap . . . 39
7.3. The gap and uniform stability . . . 39
7.4. High probability: the Bousquet–Elisseeff bound . . . 40
7.5. Regularized ERM is stable . . . 40
7.6. SGD is stable if you stop early . . . 41
7.7. An unstable algorithm: 1-nearest neighbour . . . 41
7.8. Overfitting is memorization of individual examples . . . 41
7.9. Choosing what to measure after looking . . . 42
7.10. Code . . . 42
7.11. Cheat sheet . . . 43
7.12. Further reading . . . 43

8. Cryptography basics **44**
8.1. Why this matters . . . 44
8.2. Which tool for which job . . . 44
8.3. Advantage . . . 45
8.4. Efficient adversaries and concrete security . . . 45
8.5. Hybrid arguments . . . 45
8.6. Pseudorandom generators and functions . . . 46
8.7. The birthday bound and the data limits of a key . . . 46
8.8. Encryption . . . 46
8.9. Simulation and secret sharing . . . 47
8.10. Learning with errors . . . 47
8.11. Code . . . 48
8.12. Cheat sheet . . . 49
8.13. Further reading . . . 49

II. Threat models and attacks **50**

9. Threat models and privacy games **52**
9.1. Why this matters . . . 52
9.2. Writing down the threat model . . . 52
9.3. Scoring an attack against its baseline . . . 53
9.4. Reporting: TPR at a low FPR . . . 54
9.5. Average case and worst case . . . 54
9.6. Code . . . 55
9.7. Cheat sheet . . . 56
9.8. Further reading . . . 56

10. Membership inference **57**
10.1. Why this matters . . . 57
10.2. Membership as a test about one record . . . 57
10.3. The quick check: the generalization gap . . . 58
10.4. LiRA: a likelihood-ratio test per example . . . 58
10.5. Running LiRA on your model . . . 59

10.6. Reading the result . . . 60
10.7. Who is exposed, and what helps . . . 60
10.8. Cheat sheet . . . 61
10.9. Further reading . . . 61

11. Attribute inference and model inversion **62**
11.1. Why this matters . . . 62
11.2. The attribute game . . . 62
11.3. The imputation baseline . . . 63
11.4. Model inversion: the MAP attack . . . 64
11.5. Confidence masking . . . 64
11.6. Deciding whether there is a leak . . . 65
11.7. Code . . . 65
11.8. Cheat sheet . . . 66
11.9. Further reading . . . 66

12. Property inference **67**
12.1. Why this matters . . . 67
12.2. The property game and its ceiling . . . 67
12.3. Why a property leaks when no record does . . . 68
12.4. Running the attack: shadow models and a meta-classifier . . . 68
12.5. Neural networks: choose the right representation . . . 69
12.6. Defences and their limits . . . 70
12.7. Code . . . 70
12.8. Cheat sheet . . . 71
12.9. Further reading . . . 71

13. Memorization and training-data extraction **72**
13.1. Why this matters . . . 72
13.2. Which memorization to measure . . . 72
13.3. What gets memorized: copies, model size and rare data . . . 73
13.4. Auditing with canaries and exposure . . . 74
13.5. Extraction without the prefix: generate, rank, verify . . . 74
13.6. Code . . . 75
13.7. Cheat sheet . . . 76
13.8. Further reading . . . 76

14. Gradient inversion **77**
14.1. Why this matters . . . 77
14.2. The attacker and what it sees . . . 77
14.3. Exact recovery from a fully connected layer . . . 77
14.4. Label recovery from the last layer . . . 78
14.5. Deep Leakage from Gradients . . . 78
14.6. Batches and averaging . . . 79
14.7. What to do as a deployer . . . 80
14.8. Cheat sheet . . . 81
14.9. Further reading . . . 81

15. Linkage and re-identification **82**
15.1. Why this matters . . . 82
15.2. Quasi-identifiers and the linkage attack . . . 82
15.3. Pseudonyms and hashed identifiers . . . 83
15.4. Predicting uniqueness before a release . . . 83
15.5. Sparse data: the Netflix attack . . . 84
15.6. Does releasing a sample protect? . . . 84
15.7. A pre-release risk check . . . 85
15.8. Code . . . 85
15.9. Cheat sheet . . . 86
15.10. Further reading . . . 86

16. Syntactic anonymity and how it breaks **87**
16.1. Why this matters . . . 87
16.2. What the adversary believes about a class . . . 87
16.3. k-anonymity and what it costs . . . 87
16.4. Homogeneity, background knowledge and l-diversity . . . 88
16.5. Measuring what a release teaches . . . 89
16.6. Skewness, similarity and t-closeness . . . 89
16.7. The composition attack . . . 90
16.8. Choosing and running an anonymization . . . 90
16.9. Code . . . 90
16.10. Cheat sheet . . . 91
16.11. Further reading . . . 92

17. Reconstruction attacks **93**
17.1. Why this matters . . . 93
17.2. Counts are linear equations . . . 93
17.3. Exact and random queries: how much error is enough . . . 93
17.4. Least squares: the attack to run on your system . . . 94
17.5. The Fundamental Law of Information Recovery . . . 95
17.6. Attacking a deployed query system . . . 95
17.7. Reconstructing a census block . . . 95
17.8. Auditing an aggregate release . . . 96
17.9. Code . . . 96
17.10. Cheat sheet . . . 97
17.11. Further reading . . . 98

III. Differential privacy foundations **99**

18. The definition of differential privacy **101**
18.1. Why this matters . . . 101
18.2. Neighbouring datasets . . . 101
18.3. The definition . . . 101
18.4. The privacy loss and δ . . . 102
18.5. Randomized response . . . 102
18.6. What ε means for an attacker . . . 103

18.7. How small δ must be . . . 104
18.8. What differential privacy does not protect . . . 104
18.9. Code . . . 105
18.10. Cheat sheet . . . 105
18.11. Further reading . . . 106

19. Post-processing, group privacy and composition **107**
19.1. Why this matters . . . 107
19.2. Post-processing . . . 107
19.3. Group privacy . . . 108
19.4. Sequential composition . . . 108
19.5. Parallel composition . . . 109
19.6. Keeping a budget . . . 109
19.7. Code . . . 110
19.8. Cheat sheet . . . 111
19.9. Further reading . . . 112

20. The Laplace and Gaussian mechanisms **113**
20.1. Why this matters . . . 113
20.2. Sensitivity . . . 113
20.3. The Laplace mechanism . . . 114
20.4. The Gaussian mechanism . . . 115
20.5. Laplace or Gaussian? . . . 116
20.6. Calibrating a release . . . 116
20.7. Code . . . 117
20.8. Cheat sheet . . . 118
20.9. Further reading . . . 118

21. Advanced and optimal composition **119**
21.1. Why this matters . . . 119
21.2. Which rule to use . . . 119
21.3. Basic and advanced composition . . . 120
21.4. Optimal composition . . . 120
21.5. Adaptive composition . . . 121
21.6. Code . . . 122
21.7. Cheat sheet . . . 123
21.8. Further reading . . . 123

22. Selection: exponential mechanism, report-noisy-max, sparse vector **124**
22.1. Why this matters . . . 124
22.2. Which mechanism to use . . . 124
22.3. The exponential mechanism . . . 125
22.4. Accuracy, and the cost of many candidates . . . 125
22.5. A private quantile or clipping bound . . . 125
22.6. Report-noisy-max and Gumbel noise . . . 126
22.7. The sparse vector technique . . . 126
22.8. When the sparse vector technique goes wrong . . . 127
22.9. Code . . . 128

22.10. Cheat sheet . . . 128
22.11. Further reading . . . 129

23. Query release and private multiplicative weights **130**
23.1. Why this matters . . . 130
23.2. Which method to use . . . 130
23.3. Noise on every answer . . . 131
23.4. Private multiplicative weights . . . 131
23.5. Synthetic data in practice . . . 132
23.6. Online release and SmallDB . . . 133
23.7. Code . . . 133
23.8. Cheat sheet . . . 134
23.9. Further reading . . . 135

24. Beyond global sensitivity **136**
24.1. Why this matters . . . 136
24.2. Local sensitivity, and why it leaks . . . 136
24.3. Clipping . . . 137
24.4. Subsample-and-aggregate . . . 138
24.5. Medians and quantiles: inverse sensitivity . . . 138
24.6. Propose-test-release . . . 139
24.7. Stability-based release . . . 139
24.8. Smooth sensitivity . . . 139
24.9. Cheat sheet . . . 139
24.10. Further reading . . . 140

25. Discrete and floating-point-safe mechanisms **141**
25.1. Why this matters . . . 141
25.2. Floating-point Laplace breaks differential privacy . . . 141
25.3. The snapping mechanism . . . 142
25.4. The discrete Laplace mechanism . . . 142
25.5. The discrete Gaussian mechanism . . . 142
25.6. Making a real-valued query integer . . . 143
25.7. Exact samplers, and choosing a library . . . 143
25.8. Code . . . 144
25.9. Cheat sheet . . . 144
25.10. Further reading . . . 145

26. Local, shuffle and distributed trust models **146**
26.1. Why this matters . . . 146
26.2. Choosing a trust model . . . 146
26.3. Central and local differential privacy . . . 146
26.4. Frequency estimation: k-ary randomized response and unary encoding . . . 147
26.5. Large and unknown domains: hashing and heavy hitters . . . 148
26.6. Numbers in one bit: local mean estimation . . . 148
26.7. Collecting every day: memoization . . . 148
26.8. The shuffle model and amplification by shuffling . . . 149
26.9. Distributed DP: splitting the noise . . . 149

26.10. Cheat sheet . . . 150
26.11. Further reading . . . 150

27. Lower bounds: what privacy costs **151**
27.1. Why this matters . . . 151
27.2. Choosing among many candidates: packing . . . 151
27.3. How many records you need . . . 152
27.4. The floor for approximate DP: fingerprinting . . . 153
27.5. Tracing attacks on released means . . . 153
27.6. Reconstruction as a floor on noise . . . 154
27.7. Cheat sheet . . . 154
27.8. Further reading . . . 155

28. Private mean estimation **156**
28.1. Why this matters . . . 156
28.2. Bounded data: privacy adds a lower-order term . . . 156
28.3. The cost of an unknown range . . . 157
28.4. Many dimensions: clip the norm . . . 158
28.5. Heavy tails: clip into the data . . . 158
28.6. Code . . . 159
28.7. Cheat sheet . . . 160
28.8. Further reading . . . 160

IV. Privacy accounting **161**

29. Rényi differential privacy **163**
29.1. Why this matters . . . 163
29.2. The RDP curve . . . 163
29.3. RDP curves of the basic mechanisms . . . 164
29.4. Composition adds curves . . . 164
29.5. Post-processing and group privacy . . . 165
29.6. Convert once, at the end . . . 166
29.7. Accounting with a library . . . 166
29.8. Cheat sheet . . . 167
29.9. Further reading . . . 168

30. From Rényi DP to (ε, δ) and to tests **169**
30.1. Why this matters . . . 169
30.2. What δ means for your users . . . 169
30.3. Mironov's and the improved conversion . . . 169
30.4. Which conversion your library uses . . . 170
30.5. Choosing the order . . . 170
30.6. The slack no conversion removes . . . 171
30.7. RDP, attacks and other guarantees . . . 172
30.8. Code . . . 172
30.9. Cheat sheet . . . 173
30.10. Further reading . . . 173

31. Concentrated differential privacy **174**
31.1. Why this matters . . . 174
31.2. The zCDP ledger . . . 174
31.3. Charging other kinds of step . . . 175
31.4. From ρ to (ε, δ) . . . 175
31.5. Groups and approximate zCDP . . . 176
31.6. Subsampled steps: keep them out of zCDP . . . 177
31.7. Code . . . 177
31.8. Cheat sheet . . . 178
31.9. Further reading . . . 179

32. f-DP and Gaussian differential privacy **180**
32.1. Why this matters . . . 180
32.2. Trade-off curves and f-DP . . . 180
32.3. μ-GDP, read as an attack . . . 181
32.4. Converting μ to (ε, δ) . . . 181
32.5. Composition: μ^2 adds . . . 182
32.6. The CLT estimate for DP-SGD . . . 182
32.7. RDP forgets part of the curve . . . 183
32.8. Code . . . 183
32.9. Cheat sheet . . . 184
32.10. Further reading . . . 185

33. Amplification by subsampling **186**
33.1. Why this matters . . . 186
33.2. One step under Poisson sampling . . . 186
33.3. Match the sampler to the accountant . . . 187
33.4. The accountant for DP-SGD . . . 188
33.5. Small q and the design of a run . . . 189
33.6. Other mechanisms . . . 189
33.7. Code . . . 189
33.8. Cheat sheet . . . 190
33.9. Further reading . . . 191

34. Numerical accounting with privacy loss distributions **192**
34.1. Why this matters . . . 192
34.2. The privacy loss distribution . . . 192
34.3. Composition is convolution . . . 193
34.4. Pessimistic and optimistic discretization . . . 193
34.5. Composition by FFT . . . 194
34.6. The subsampled Gaussian and DP-SGD . . . 194
34.7. Running it with a library . . . 195
34.8. Code . . . 195
34.9. Cheat sheet . . . 196
34.10. Further reading . . . 197

35. Amplification by shuffling and by iteration **198**
35.1. Why this matters . . . 198

35.2. Shuffled binary randomized response, exactly . . . 198
35.3. The clone bound for any local randomizer . . . 199
35.4. Choosing ε_0 and deploying a shuffler . . . 200
35.5. Amplification by iteration . . . 200
35.6. Hidden state: privacy that stops growing . . . 200
35.7. Code . . . 201
35.8. Cheat sheet . . . 202
35.9. Further reading . . . 203

36. Adaptive composition: filters, odometers and individual accounting 204
36.1. Why this matters . . . 204
36.2. Why plugging in the realized parameters fails . . . 204
36.3. Early stopping within a planned maximum . . . 205
36.4. The Rényi filter . . . 205
36.5. Filters for pure differential privacy . . . 206
36.6. Odometers . . . 206
36.7. Individual privacy accounting in DP-SGD . . . 207
36.8. Code . . . 207
36.9. Cheat sheet . . . 208
36.10. Further reading . . . 208

37. Choosing an accountant 209
37.1. Why this matters . . . 209
37.2. Every accountant on one run . . . 209
37.3. The inverse problem: noise for a target ε . . . 210
37.4. Where each accountant is tight . . . 210
37.5. Matching the accountant to the code . . . 211
37.6. What to report . . . 212
37.7. Checking your accountant . . . 212
37.8. Code . . . 213
37.9. Cheat sheet . . . 213
37.10. Further reading . . . 214

V. Differential privacy in machine learning 215

38. Private empirical risk minimization 217
38.1. Why this matters . . . 217
38.2. The setting and the noise . . . 217
38.3. Output perturbation . . . 218
38.4. Objective perturbation . . . 218
38.5. Gradient perturbation: noisy gradient descent . . . 219
38.6. Noisy gradient descent without a ridge . . . 219
38.7. How the three compare . . . 219
38.8. When the model is not convex . . . 220
38.9. Libraries . . . 220
38.10. Code . . . 221
38.11. Cheat sheet . . . 221

38.12. Further reading . . . 222

39. DP-SGD **223**
39.1. Why this matters . . . 223
39.2. The algorithm and its guarantee . . . 223
39.3. The implementation checklist . . . 224
39.4. Clipping and the step size . . . 224
39.5. Batch size at a fixed budget . . . 225
39.6. Making it fast . . . 225
39.7. What privacy costs in accuracy . . . 225
39.8. The Opacus workflow . . . 226
39.9. Cheat sheet . . . 227
39.10. Further reading . . . 228

40. Hyperparameter tuning and private selection **229**
40.1. Why this matters . . . 229
40.2. The procedure and the report . . . 229
40.3. The best of k runs can cost k runs . . . 230
40.4. Paying by composition . . . 230
40.5. A random number of runs . . . 231
40.6. Selecting by a noisy validation score . . . 232
40.7. Tuning the running example . . . 232
40.8. Cheat sheet . . . 233
40.9. Further reading . . . 234

41. DP-FTRL and matrix factorization **235**
41.1. Why this matters . . . 235
41.2. The matrix mechanism . . . 235
41.3. Choosing a factorization . . . 236
41.4. Several passes: banding and the data order . . . 236
41.5. Which method when . . . 237
41.6. Code . . . 238
41.7. Cheat sheet . . . 239
41.8. Further reading . . . 239

42. Private prediction: PATE and in-context learning **240**
42.1. Why this matters . . . 240
42.2. The noisy vote: GNMax . . . 240
42.3. How accurate a noisy vote is . . . 241
42.4. Private answers or a private model . . . 242
42.5. PATE: from private answers to a private model . . . 242
42.6. Data-dependent accounting and Confident-GNMax . . . 243
42.7. Private in-context learning . . . 243
42.8. Code . . . 244
42.9. Cheat sheet . . . 244
42.10. Further reading . . . 244

43. Public data and private fine-tuning **245**
43.1. Why this matters . . . 245
43.2. How many weights to train . . . 245
43.3. Linear probing first . . . 246
43.4. Low-rank adapters and full fine-tuning . . . 246
43.5. Which method at which budget . . . 247
43.6. What public data does not buy . . . 248
43.7. Code . . . 248
43.8. Cheat sheet . . . 249
43.9. Further reading . . . 249

44. User-level differential privacy **250**
44.1. Why this matters . . . 250
44.2. The privacy unit . . . 250
44.3. What an example-level model gives a user . . . 251
44.4. User-level DP-SGD . . . 251
44.5. Contribution bounding . . . 252
44.6. Heterogeneous users . . . 252
44.7. Which strategy . . . 253
44.8. Code . . . 253
44.9. Cheat sheet . . . 254
44.10. Further reading . . . 254

45. Disparate impact **255**
45.1. Why this matters . . . 255
45.2. Evaluating a private model per group . . . 255
45.3. What to expect . . . 256
45.4. Why small groups pay: clipping and noise . . . 257
45.5. Mitigations and what they cost . . . 257
45.6. Code . . . 258
45.7. Cheat sheet . . . 259
45.8. Further reading . . . 259

46. Choosing ε **260**
46.1. Why this matters . . . 260
46.2. Justifying a choice . . . 260
46.3. The pair and the whole curve . . . 261
46.4. What an attacker can believe . . . 262
46.5. How small δ must be . . . 263
46.6. Comparing with deployed values . . . 263
46.7. Code . . . 264
46.8. Cheat sheet . . . 264
46.9. Further reading . . . 265

47. Auditing private training **266**
47.1. Why this matters . . . 266
47.2. Which audit for which question . . . 266
47.3. Known bugs, and which audit catches them . . . 267

47.4. From an attack to a lower bound . . . 267
47.5. Canaries and threat models . . . 268
47.6. Tight audits of the mechanism . . . 268
47.7. One-run auditing . . . 269
47.8. Black-box audit of the trained network . . . 269
47.9. Auditing in continuous integration . . . 269
47.10. Code . . . 270
47.11. Cheat sheet . . . 270
47.12. Further reading . . . 271

48. Adaptive data analysis **272**
48.1. Why this matters . . . 272
48.2. What to do in practice . . . 272
48.3. Reusing a holdout set overfits . . . 273
48.4. Sample splitting and its cost . . . 273
48.5. Differential privacy implies generalization . . . 274
48.6. Answering with Gaussian noise . . . 274
48.7. The reusable holdout: Thresholdout . . . 275
48.8. Code . . . 276
48.9. Cheat sheet . . . 276
48.10. Further reading . . . 277

Appendices **278**

A. Glossary **278**

B. Index **281**

Preface

This is the practitioner edition of *Privacy-Preserving Machine Learning*. It is for engineers, data scientists and auditors who build or review machine-learning systems that handle personal data, and it tells them what each result says, when it applies, how to use it and what goes wrong, without proofs.

It covers the same topics as the full edition, in the same parts and chapters, but it is organised around the work rather than the theory. Each topic states the result with its formula, says why it matters, and gives a recipe with a worked example. It ends with what goes wrong. Each chapter closes with a cheat sheet and a link to the matching chapter of the full edition, which derives every result step by step. A separate exercise book, keyed to the full edition, has worked solutions.

You need some Python and NumPy, basic probability, and enough machine learning to know what a loss and stochastic gradient descent are.

How this book is organized

1. **Prerequisites** is a mathematical reference to dip into when a later chapter cites it.
2. **Threat models and attacks** shows how models and data leak.
3. **Differential privacy foundations** defines the guarantee and builds its mechanisms.
4. **Privacy accounting** tracks the guarantee through private training.
5. **Differential privacy in machine learning** trains, tunes and audits private models.

You do not need to read Part 1 first. With a background in statistics or machine learning, start at Part 2 or Part 3 and follow the links back when you need a fact.

The numbers in the text come from the scripts in the repository's `code/` directory, which the full edition also uses, and tests check that the scripts still produce them. To rerun one, run `uv run python code/<part>/<topic>.py`.

Part I.

Prerequisites

This part is a mathematical toolkit, and it contains no privacy yet. You do not need to read it first. Later chapters link back to the facts they use, so come here when a link sends you.

What each chapter is for

Two ideas run through the part. The first is the **likelihood ratio**, how many times more likely an output is under one dataset than under a neighbouring one. The second is the **effect of one input**, how far a single example can move a result.

Probability distributions in the full edition and Concentration inequalities in the full edition give you the noise families, their scales and the error bars you publish with noisy numbers, which Parts 3 and 4 use constantly. Divergences in the full edition compares the output distributions on two datasets with a single number, the language of Parts 3 and 4. Hypothesis testing in the full edition shows how to read a detector's trade-off curve at low false positive rates, which Parts 2 and 5 rely on. Information theory in the full edition bounds what an output reveals about a hidden value, which Part 2 uses.

Optimization in the full edition gives the constants, step sizes and clipping that Part 5 needs. Algorithmic stability and generalization in the full edition explains why regularization and early stopping limit how much a model depends on one example, and why a reused holdout set misleads you. Cryptography basics in the full edition teaches you to read a security claim stated as a game or a simulator, which later parts need.

Notation

Table 1.: Notation used throughout the book.

Symbol	Meaning
Pr, $\mathbb{E}$, Var	Probability, expectation, variance
$\mathcal{N}(\mu, \sigma^2)$, Lap($b$)	Normal and Laplace distributions
ln, $\log_2$	Natural logarithm (nats) and base-2 logarithm (bits)
$\|w\|$	Euclidean length of a vector
D, $\ell(w, z)$	A dataset, and the loss of a model w on an example z

How to use the chapters

Each chapter states its results, shows how to use them with a worked number, and links to its derivations in the full edition. Every number comes from a script you can rerun, such as `uv run python code/part01-prerequisites/concentration.py`. The scripts draw their randomness from seeded NumPy generators, `numpy.random.default_rng`, so reruns give the same numbers.

1. Probability distributions

1.1. Why this matters

Most of the random numbers in a machine-learning system are either counts or draws of noise. The number of mistakes a model makes on a test set is a count, and so is the size of a randomly sampled minibatch. The noise a system adds on purpose is the other kind, usually drawn from a Laplace or a normal distribution. Before you trust any of these numbers, you need to know where each one usually lands and how rare its large values are.

A handful of families covers almost every case. For a count, you need its spread, so that you can size a test set. For noise, you need its tail, so that you can calibrate its scale to an error bar.

1.2. Means, variances and the square-root law

The **mean** of a random quantity X, written $\mathbb{E}[X]$, is the long-run average of many independent draws of it. The **variance** measures the spread around it, as the average squared distance from the mean:

$$\operatorname{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}X)^2\big] = \mathbb{E}[X^2] - (\mathbb{E}X)^2. \tag{1.1}$$

The **standard deviation** σ is its square root, in the same units as the quantity, so it is the number to compare with the mean.

Three rules cover almost every calculation. The mean of a sum is the sum of the means, always. Scaling a quantity by a constant c multiplies its variance by c^2. And when two quantities are independent, their variances add:

$$\operatorname{Var}(X + Y) = \operatorname{Var}(X) + \operatorname{Var}(Y). \tag{1.2}$$

Without independence the last rule fails. Add a quantity to an exact copy of itself, and the variance of the sum is four times the original, not twice.

Together, the second and third rules give the square-root law. An average of n independent draws has the variance of one draw divided by n, so its standard deviation is that of one draw divided by $\sqrt{n}$. The law breaks when the draws are not independent, for example when one user contributes many rows, so count independent units, not rows.

1.3. Counts and rates: binomial and Poisson

A yes-or-no outcome, such as whether a test example is misclassified, is a **Bernoulli** variable. It is 1 with probability p, so its mean is p and its variance is $p(1-p)$. Counting the yeses over n independent trials gives a **binomial** count, with mean np and variance $np(1-p)$. Every measured rate, such as accuracy or a false-positive rate, is such a count divided by n, so its standard deviation is

$$\sqrt{\frac{p(1-p)}{n}}. \tag{1.3}$$

This formula is how you size a test set. Suppose a classifier errs on each of 2,000 test examples independently with probability 0.05. Then the measured error rate is typically within about half a percentage point of the true 5%. But if the true error rate is 0.001, the same test set expects only 2 mistakes, so the relative error of the count is about 70%. So what sets the relative precision is the expected number of events np, not n.

That is also why the normal approximation to a binomial count needs its variance $np(1-p)$ to be about 10 or more, and fails for rare events. The snippet in Section 1.9 turns a target precision into a test-set size. For a relative standard deviation of 10%, a 5% error rate needs 1,900 examples, while a 0.1% rate needs 99,900.

When the trials are many and each is rare, the binomial depends almost only on the expected count, written μ. It then becomes the **Poisson distribution** Poisson(μ). Its variance equals its mean, so its relative spread is $1/\sqrt{\mu}$, and independent Poisson counts add up to a Poisson count. The main use in this book is the batch size under Poisson sampling, where each example joins a batch independently with a sampling probability q. Out of n examples, the batch size is then binomial with mean qn. It is close to Poisson(qn), so its standard deviation is about $\sqrt{qn}$. For example, when each of 60,000 examples joins with probability 0.01, the standard deviation is about 24.5. A batch of 550 or 650 is then unremarkable.

So code that assumes a fixed batch size under this kind of sampling, such as a preallocated buffer or a per-batch average over a constant, is wrong. You can simulate the sizes with NumPy's `rng.binomial(n, q)` or `rng.poisson(q * n)`.

The waiting time until the first rare event follows the **exponential distribution** Exp(b), whose mean is the scale b. In this book it serves mainly as half of a Laplace distribution. The size of a Laplace draw is exponential, and the difference of two independent exponential draws is a Laplace draw, which is one way to sample it.

1.4. Tails and heavy tails

The mean and the variance describe the bulk of a distribution, but a guarantee that must hold except in rare cases depends on the **tail**, the chance of landing far from the mean. The Laplace tail shrinks exponentially with the distance, and the normal tail shrinks much faster, like the exponential of the squared distance. The tail bounds of Concentration inequalities in the full edition work through the **moment generating function**, the average of $e^{\lambda X}$ for a number λ you choose. A **heavy-tailed** quantity, such as an unclipped loss or gradient, has none, so those bounds do not apply to it, which is one reason to clip values before adding noise.

1.5. Laplace and normal noise

The two standard noise distributions are the Laplace and the normal. The **Laplace distribution** $\mathrm{Lap}(b)$ with scale b is an exponential density mirrored around zero. Its variance is $2b^2$, so its standard deviation is $\sqrt{2}\,b$, not b. A draw X lands at least a distance t from zero with probability

$$\Pr[|X| \geq t] = e^{-t/b}. \tag{1.4}$$

Setting this chance to 5% gives the 95% half-width $b \ln 20$, about three scales, which `scipy.stats.laplace(scale=b).ppf(0.975)` computes. So a count reported with Laplace noise of scale 2 is within 5.99 of the true count with 95% probability. The **normal distribution** has a mean μ and a standard deviation σ, written $\mathcal{N}(\mu, \sigma^2)$. Its 95% half-width is 1.96σ, from `scipy.stats.norm.ppf(0.975)`.

Figure 1.1 compares the two at the same variance. On a log scale the normal's density is a parabola and the Laplace's is two straight lines, so the Laplace has a sharper peak and much heavier tails.

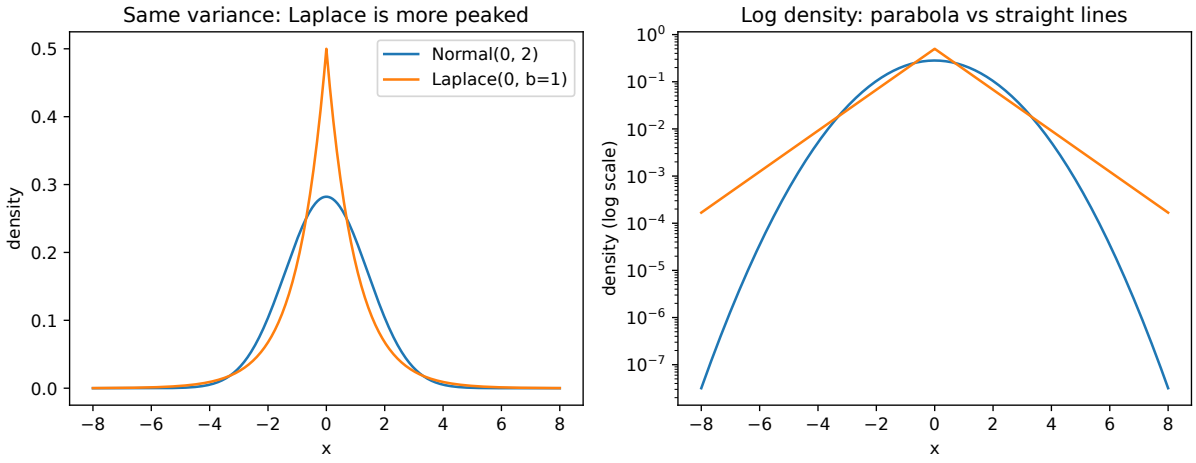


Figure 1.1.: Normal and Laplace densities with variance 2, on linear and log scales. The log of the normal density is a parabola; the log of the Laplace density is two straight lines.

Calibrating noise means solving the tail for the scale. Say the noise must stay within a width t of zero, except with a failure probability β. Then the Laplace scale and the normal standard deviation can be at most

$$b = \frac{t}{\ln(1/\beta)}, \qquad \sigma = \frac{t}{z_{1-\beta/2}}, \tag{1.5}$$

where $z_{1-\beta/2}$ is the normal quantile at $1 - \beta/2$. Table 1.1 shows that the two multipliers grow very differently as β shrinks.

Table 1.1.: How many standard deviations or scales a noise draw stays within, except with probability β.

Failure probability β	0.05	10^{-3}	10^{-6}
Normal, $z_{1-\beta/2}$ standard deviations	1.96	3.29	4.89
Laplace, $\ln(1/\beta)$ scales	3.00	6.91	13.82

The two families also combine differently. A weighted sum of independent normal quantities is again normal, with the weighted means added and the variances added with squared weights:

$$\sum_i a_i X_i \sim \mathcal{N}\Big(\sum_i a_i \mu_i,\ \sum_i a_i^2 \sigma_i^2\Big). \tag{1.6}$$

So the total noise of many noisy normal measurements is one normal draw that you can analyse exactly. A sum of Laplace draws is not Laplace, and its error bar needs the bound in Concentration inequalities in the full edition. Either way, a noisy mean computed as a noisy sum divided by n has its noise standard deviation divided by n, on top of the sampling spread of the data itself.

The most common bug with these distributions is a mismatched parameter. With `rng = numpy.random.default_rng(seed)`, NumPy's `rng.laplace(0, b, size)` and SciPy's `scipy.stats.laplace(scale=b)` take the scale b, whose standard deviation is $b\sqrt{2}$. By contrast, NumPy's `rng.normal(loc, scale)`, Python's `random.gauss(mu, sigma)` and `scipy.stats.norm(scale=...)` take the standard deviation, never the variance. So before you trust a noise level, check which one a call expects, and draw a million samples to confirm the variance.

1.6. Normal noise in many dimensions

Noise is often added to a whole vector, such as a model's parameters, as d independent normal coordinates with standard deviation σ each. Along any direction fixed in advance, this noise is a single draw from $\mathcal{N}(0, \sigma^2)$. Yet the vector's length is large and predictable, close to $\sigma\sqrt{d}$. For example, with a million parameters and a standard deviation of 0.01 on each, the noise vector has a length of about 10. So judge high-dimensional noise by its effect along the directions you care about, not by its total length.

The same picture explains noisy gradient descent. Suppose each example's gradient is clipped to a length of at most C, and normal noise with standard deviation σC per coordinate is added to the sum of a batch of B clipped gradients. The noise vector then has length about $\sigma C\sqrt{d}$, which can dwarf the clipped sum, whose length is at most BC. Along the gradient's own direction, though, the noise is only σC. Dividing the sum by the batch size divides both, leaving per-coordinate noise $\sigma C/B$ on the averaged gradient, so larger batches shrink the noise relative to the signal.

1.7. The central limit theorem

The **central limit theorem** says that a standardized sum of many independent draws with finite variance looks normal, whatever each draw's distribution, which is what justifies a normal error

bar on an average. The approach is slow for skewed data. For a sum of 30 exponential draws, the largest gap between its distribution function and the normal one is still 0.0243. And it describes the bulk, so for small tail probabilities use Concentration inequalities in the full edition instead.

1.8. A density against its shifted copy

Later parts keep asking how strongly one noisy observation can favour one underlying value over another that differs by a shift s. The answer is the ratio of the noise density to its shifted copy at the observation. Laplace noise with scale b caps this ratio at $e^{|s|/b}$, so no observation is very telling. Normal noise leaves it unbounded, so an observation far out in the tail can be overwhelming, although such observations are rare. With the shift and both scales set to 1, for example, Laplace noise never favours either value by more than a factor of about 2.72. Later parts rely on this difference, and Divergences in the full edition averages this ratio.

1.9. Code

The snippet prints the noise multipliers of Table 1.1 and sizes a test set.

```
import math
from statistics import NormalDist


def normal_multiplier(beta: float) -> float:
    """Number of standard deviations a normal draw stays within, except with probability bet
    return NormalDist().inv_cdf(1 - beta / 2)


def test_set_size(p: float, relative: float) -> int:
    """Test examples needed for a measured rate near p to have this relative standard deviat
    return math.ceil((1 - p) / (p * relative**2))


print(f"{normal_multiplier(0.05):.2f} {normal_multiplier(1e-3):.2f} {normal_multiplier(1e-6)
print(f"{math.log(1 / 0.05):.2f} {math.log(1 / 1e-3):.2f} {math.log(1 / 1e-6):.2f}")  # 3.00
print(test_set_size(0.05, 0.1), test_set_size(0.001, 0.1))  # 1900 99900
```

1.10. Cheat sheet

Table 1.2.: The families of this chapter, with what library calls usually expect.

Distribution	Mean	Variance	Tail	Library parameter
Binomial(n, p)	np	$np(1-p)$	Chernoff bound	n, p
Poisson(μ)	μ	μ	Chernoff bound	the mean μ
Lap(b)	0	$2b^2$	$\Pr[\lvert X\rvert \geq t] = e^{-t/b}$	scale b, not the standard deviation
$\mathcal{N}(\mu, \sigma^2)$	μ	σ^2	$\Pr[X - \mu \geq t] \leq \frac{1}{2}e^{-t^2/(2\sigma^2)}$	standard deviation σ, not the variance

Rules of thumb

- Variances add only for independent quantities, so count independent units, not rows.
- Size a test set by the expected number of events np, not by n, and don't use the normal approximation when np is small.
- A Laplace scale b is not its standard deviation, which is $\sqrt{2}\,b$, and NumPy's normal `scale` is the standard deviation, not the variance.
- Under Poisson sampling the batch size varies, with standard deviation about $\sqrt{qn}$.

1.11. Further reading

- Blitzstein & Hwang, *Introduction to Probability* (2nd ed., 2019), ch. 3–5 and 7: every distribution here, with worked examples.
- Vershynin, *High-Dimensional Probability* (2018), §3.1: why the length of a normal vector concentrates.

Derivations. Every result here is derived step by step in the full edition's chapter.

2. Concentration inequalities

2.1. Why this matters

Almost every number in a machine-learning system is random. Some are random by design, like noise deliberately added to a published count, and others because of how they were measured, like an error rate on a test set. Before you publish or act on such a number, you need an error bar for it. And before you collect data, you need to know how much data buys the error bar you want.

A concentration inequality gives you both. It bounds the chance that a random quantity lands more than some distance from its mean. This chapter calls that event a failure, its chance the **failure probability** β, and the distance the **width**. You use every inequality the same way: fix β and solve for the width, or fix the width and solve for the number of samples. Which inequality to use depends on what you know about the quantity, and the more you know, the narrower the error bar. At $\beta = 0.05$ the choice matters little, but at the one-in-a-million level that later chapters need, the wrong choice makes an error bar a hundred times too wide.

2.2. Mean or variance only: Markov and Chebyshev

Markov's inequality needs only the average of a quantity X that cannot be negative, such as a cost or a latency. It bounds the chance that X reaches a threshold a:

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}. \tag{2.1}$$

Setting the right side to β says that X stays below $\mathbb{E}[X]/\beta$ except with probability β. If requests take 2 ms on average, at most a quarter take 8 ms or more. The condition matters, because a quantity that can be negative can hide large values behind negative ones.

Chebyshev's inequality adds the variance. Write μ for the mean and σ^2 for the variance. The chance of landing a distance t from the mean is then

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.2}$$

So the width is $\sigma/\sqrt{\beta}$. For an average of n independent samples, divide it by $\sqrt{n}$. It holds for every distribution, which makes it safe when you distrust the shape of the data. The price is steep at small β, though. At a 95% level it asks for 4.5 standard deviations where a normal-shaped tail needs 2.4. At one in a million, it asks for 1,000 where the normal needs 5.3. So use it for a quick 95% error bar and for nothing stricter.

2.3. Noise you added: Laplace and Gaussian

When the randomness is noise you added yourself, you know its distribution exactly, so you read the width off its tail and need no inequality. Laplace noise with scale b stays within its width

$$b \ln(1/\beta) \tag{2.3}$$

of zero, except with probability β. For normal noise with standard deviation σ, multiply σ by the normal quantile at $1 - \beta/2$, which is 1.96 at 95%. The bound $\sigma\sqrt{2\ln(1/\beta)}$ is a convenient upper limit for mental arithmetic. In code, `scipy.stats.laplace.isf` and `scipy.stats.norm.isf` give the exact quantiles, as does the standard-library snippet in Section 2.8.

The typical use is the error bar you publish with a noisy count. If the count carries Laplace noise of scale 2, the published value is within 5.99 of the true one with 95% probability. Later parts choose the noise scale; this chapter only reads off its error bar. Publish that number next to the count, because an analyst who treats it as exact will over-read small differences.

A sum of k noisy counts needs care. Near its centre the total noise looks normal, with standard deviation $b\sqrt{2k}$, but its far tail decays like a single Laplace draw, so the normal approximation understates rare failures. For a sum of 10 draws, it understates the chance of a large total about ninefold. A valid one-sided width is the larger of

$$\sqrt{8kb^2 \ln(1/\beta)} \quad \text{and} \quad 4b \ln(1/\beta). \tag{2.4}$$

For two sides, use $\beta/2$, which gives 17.18 at 95% for 10 counts of scale 1. If that is too wide, simulate the noise a million times and read off the quantile, since you know its distribution exactly.

2.4. Many numbers at once: the union bound

A dashboard of k noisy counts should be right everywhere at once, not just count by count. The **union bound** says the chance that any of several failures happens is at most the sum of their chances. So give each count the failure probability β/k. Because the widths above depend on β only through a logarithm, this is cheap. For 1,000 counts with Laplace scale 2, the width that covers them all is 19.81, against 5.99 for one. The union bound needs no independence, so it is always valid, and for rare independent failures it is nearly exact.

2.5. Counts: the Chernoff bound

Batch sizes under random sampling, the records that match a query and the errors on a test set are all counts of independent yes-or-no outcomes. Write μ for the mean of a count S, and δ for a relative deviation between 0 and 1. The **Chernoff bound** then says

$$\Pr[S \geq (1+\delta)\mu] \leq e^{-\delta^2\mu/(2+\delta)}, \qquad \Pr[S \leq (1-\delta)\mu] \leq e^{-\delta^2\mu/2}. \tag{2.5}$$

For example, suppose each example joins a batch with probability 0.01, so that the batch size has mean 600. It leaves $600 \pm 20\%$ with probability at most 2.4×10^{-5} per step. Over 1,000 steps,

that is about 0.024. That is good enough for sizing memory, but code that silently assumes a fixed batch size will still break on the rare large batch, so handle it explicitly.

Two things go wrong in practice. The bound depends on the expected count, not on the number of trials, so a count of rare events has a wide relative error bar however many trials you run. And it needs independence: with several rows per user, count users.

2.6. Averages of bounded values: Hoeffding, Bernstein, Clopper–Pearson

Often you know only that each value lies in an interval of length R, as for a clipped loss or a 0/1 mistake. Then **Hoeffding's inequality** bounds the error of an average of n independent values. With probability $1 - \beta$, the average is within

$$R\sqrt{\frac{\ln(2/\beta)}{2n}} \tag{2.6}$$

of its mean. Its main use is planning, because it needs nothing you don't know in advance. Solving for n gives

$$n \geq \frac{R^2 \ln(2/\beta)}{2t^2} \tag{2.7}$$

for a width t, so measuring accuracy to within one percentage point at 95% takes 18,445 test examples. When you pick the best of m models on a validation set, replace β by β/m, because the winner is the one most likely to look too good.

Hoeffding's inequality is too loose for small rates, because it assumes the largest variance the range allows. For a classifier with 1 false positive in 1,000 negatives, it adds 0.039 to the measured rate. **Bernstein's inequality** uses the variance σ^2 instead, with a one-sided width of about

$$\sqrt{\frac{2\sigma^2 \ln(1/\beta)}{n}} + \frac{2\ln(1/\beta)}{3n} \tag{2.8}$$

for values in $[0, 1]$, and adds only 0.0045 here. Plugging in a variance estimated from the same data is a heuristic, though. For rates, use the exact binomial answer instead, the **Clopper–Pearson** interval. For the classifier, its 95% upper bound is 0.0047. With no errors at all it would be 0.0030, the "rule of three": zero failures in n trials bound the rate by about $3/n$. Plan with Hoeffding and report with Clopper–Pearson (Hypothesis testing in the full edition).

2.7. Functions and adaptive processes: McDiarmid and Azuma

McDiarmid's inequality extends Hoeffding's to any function of independent inputs, as long as changing one input, the i-th, moves the result by at most c_i. The tail is then $\exp(-2t^2/\sum_i c_i^2)$, and Algorithmic stability and generalization in the full edition uses it for the error of a retrained model. The c_i is the worst-case influence of one record, a quantity later parts return to.

Azuma's inequality gives the same tail for a running total whose steps may depend on the past but have expected value zero given it, a **martingale**, with c_i replaced by the length of each step's interval. Use it when an analyst or a training loop picks each step after seeing the last, which breaks the independence the other bounds need.

2.8. Code

The functions below compute the widths, sample sizes and rate bounds of this chapter, and print the numbers quoted above.

```python
import math
from statistics import NormalDist


def laplace_width(b: float, beta: float, k: int = 1) -> float:
    """All k draws of Lap(b) noise stay within this distance of zero, except with probabilit
    return b * math.log(k / beta)


def gaussian_width(sigma: float, beta: float, k: int = 1) -> float:
    """The same for N(0, sigma^2) noise, from the exact normal quantile."""
    return sigma * NormalDist().inv_cdf(1 - beta / (2 * k))


def laplace_sum_width(k: int, b: float, beta: float) -> float:
    """Two-sided width for the sum of k independent Lap(b) noise draws, from the bound."""
    log_term = math.log(2 / beta)
    return max(math.sqrt(8 * k * b**2 * log_term), 4 * b * log_term)


def chernoff_two_sided(mu: float, delta: float) -> float:
    """Chance that a count with mean mu lands more than a fraction delta from it (delta <= 1
    return math.exp(-(delta**2) * mu / (2 + delta)) + math.exp(-(delta**2) * mu / 2)


def hoeffding_n(width: float, beta: float, value_range: float = 1.0) -> int:
    """Samples needed for an average to be within width of its mean, except with probability
    return math.ceil(value_range**2 * math.log(2 / beta) / (2 * width**2))


def rate_upper_bound(errors: int, n: int, beta: float) -> float:
    """Exact (Clopper-Pearson) one-sided upper bound on a rate measured as errors out of n."

    def cdf(p: float) -> float:
        return sum(math.comb(n, i) * p**i * (1 - p) ** (n - i) for i in range(errors + 1))

    lo, hi = errors / n, 1.0
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if cdf(mid) > beta else (lo, mid)
    return hi
```

```
print(f"{laplace_width(2, 0.05):.2f}")  # 5.99
print(f"{laplace_width(2, 0.05, k=1000):.2f}")  # 19.81
print(f"{gaussian_width(2, 0.05):.2f}")  # 3.92
print(f"{laplace_sum_width(10, 1, 0.05):.2f}")  # 17.18
print(f"{chernoff_two_sided(600, 0.2):.1e}")  # 2.4e-05
print(hoeffding_n(0.01, 0.05))  # 18445
print(f"{rate_upper_bound(1, 1000, 0.05):.4f}")  # 0.0047
print(f"{rate_upper_bound(0, 1000, 0.05):.4f}")  # 0.0030
```

2.9. Cheat sheet

Table 2.1.: Which bound to use, and the width it gives. Two-sided unless marked; for two sides from a one-sided width, use $\beta/2$.

What you know	Use	Width at failure probability β
$X \geq 0$ and its mean	Markov	Stays below $\mathbb{E}[X]/\beta$
The variance σ^2	Chebyshev	$\sigma/\sqrt{\beta}$; only for a quick 95% bar
Laplace noise $\mathrm{Lap}(b)$	Its exact tail	$b\ln(1/\beta)$; k draws: $b\ln(k/\beta)$
Normal noise $\mathcal{N}(0,\sigma^2)$	Its exact quantile	$\sigma \cdot z_{1-\beta/2}$, at most $\sigma\sqrt{2\ln(1/\beta)}$
A sum of k Laplace draws	The sum bound, or simulation	$\max\left(\sqrt{8kb^2\ln(1/\beta)},\ 4b\ln(1/\beta)\right)$, one-sided
A count with mean μ	Chernoff	$\approx \sqrt{2\mu\ln(1/\beta)}$, one-sided
An average of n values in a range R	Hoeffding	$R\sqrt{\ln(2/\beta)/(2n)}$; plan n with it
The same, with a small variance	Bernstein	$\sqrt{2\sigma^2\ln(1/\beta)/n} + 2\ln(1/\beta)/(3n)$, one-sided
A rate: errors out of n	Clopper–Pearson	Exact; zero errors give about $3/n$ at 95%
One input moves f by at most c_i	McDiarmid	$\sqrt{\sum_i c_i^2\ln(1/\beta)/2}$, one-sided
Adaptive steps of length d_k	Azuma	$\sqrt{\sum_k d_k^2\ln(1/\beta)/2}$, one-sided

Rules of thumb

- Use the most you know: an exact distribution beats any bound. Knowing only the variance costs a width growing like $1/\sqrt{\beta}$, a normal-shaped tail like $\sqrt{\ln(1/\beta)}$, and a Laplace-shaped tail like $\ln(1/\beta)$.
- Publish the error bar with every noisy number.
- For k numbers at once, divide β by k.

- Match the sides: a one-sided width covers only one direction, so for a two-sided error bar use $\beta/2$.
- Hoeffding needs bounded values: clip unbounded data first, and use the clipping range as R.
- Plan sample sizes with Hoeffding; report rates with Clopper–Pearson.
- Never use the normal approximation for a guarantee in the far tail.
- Independence is a condition: with several records per person, count people.

2.10. Further reading

- Mitzenmacher & Upfal, *Probability and Computing*, ch. 4: Chernoff bounds with computer-science examples.
- Boucheron, Lugosi & Massart, *Concentration Inequalities* (2013): the reference, including Bernstein's inequality.

Derivations. Every result here except Bernstein's inequality is derived step by step in the full edition's chapter.

3. Divergences

3.1. Why this matters

Many questions in machine learning come down to putting a number on how different two distributions are. A feature drifts in production, and you want to know how far it moved. A test set changes slightly, and you want to know how far a classifier's accuracy can move. A **divergence** gives that number. It is zero when the two distributions are equal and grows as they become easier to tell apart. The cheat sheet says which divergence answers which question.

All of them summarize one quantity. Call the two distributions P and Q. For an outcome x, write $p(x)$ and $q(x)$ for the probabilities they give to it. Their quotient is the **likelihood ratio**

$$r(x) = \frac{p(x)}{q(x)}. \tag{3.1}$$

Its logarithm, written $L(x)$ and called the **log likelihood ratio**, is the evidence that the outcome gives for P over Q. Later parts use it to bound how much an output reveals.

Natural logarithms give divergences in nats, and base-2 logarithms give bits, with one nat about 1.443 bits. This chapter uses nats, so check the base of any library you use.

3.2. KL divergence

The **Kullback–Leibler (KL) divergence** is the average evidence per observation when the data come from P:

$$D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P\left[\ln \frac{p(X)}{q(X)}\right] = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.2}$$

It is never negative, and it is zero only when the two distributions are equal. It is not symmetric, because it averages under its first argument, so always say which direction you computed. Maximum likelihood minimizes the KL divergence of the model from the data, while variational inference minimizes the reverse direction.

Its most useful property is that it adds up over independent observations:

$$D_{\mathrm{KL}}(P_1 \times P_2 \,\|\, Q_1 \times Q_2) = D_{\mathrm{KL}}(P_1\|Q_1) + D_{\mathrm{KL}}(P_2\|Q_2). \tag{3.3}$$

So n independent observations carry n times the divergence of one. That makes it easy to work with many samples or many steps: add up the divergences, and convert to the quantity you need only at the end. Independence is a condition, though, because correlated observations partly repeat each other.

To compute a KL divergence from data, bin both samples on the same edges and call `scipy.stats.entropy(p, q)`, which normalizes the two histograms and returns nats, or sum `scipy.special.rel_entr(p, q)`. For example, take the feature histograms in Section 3.10, where the last of four bins was empty last month but holds a tenth of this month's data. The KL divergence of last month from this month is 0.1116 nats, or 0.1610 bits. In the other direction it is infinite, because this month has mass in a bin that last month never used. So add a small pseudocount to every bin or use coarser bins, and report the choice. Compare only estimates made with the same bins and sample sizes, because they change with both.

3.3. Rényi divergence and the max-divergence

The KL divergence reports the typical evidence. The **Rényi divergence** is a dial from the typical evidence to the worst case, set by a number α greater than 1 called its order:

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \mathbb{E}_Q[r^\alpha]. \tag{3.4}$$

As the order goes to 1 it becomes the KL divergence, and it never decreases as the order grows. As the order goes to infinity, it becomes the **max-divergence**, the largest log ratio that any outcome, or any event A, can give:

$$D_\infty(P\|Q) = \ln \max_x r(x) = \ln \max_A \frac{P(A)}{Q(A)}. \tag{3.5}$$

Like the KL divergence, every Rényi divergence adds up over independent observations.

The Rényi divergence also bounds how often the evidence is large. For every order, the chance under Q that one observation gives at least c nats of evidence for P is at most $\exp(-\alpha c + (\alpha - 1) D_\alpha)$, and the full edition derives this tail bound. Different orders give different bounds, and none is best for every question. So in practice you compute the divergence on a grid of orders, add each order's values over independent observations, and keep the order that gives the tightest final bound.

3.4. Shifted normal and Laplace pairs

The most common pair in practice is a noise distribution and a copy shifted by s, such as the same noise added to two values that differ by s. For normal noise with standard deviation σ, write $\mu = s/\sigma$ for the shift in standard deviations. Then

$$D_{\mathrm{KL}}(P\|Q) = \frac{\mu^2}{2}, \qquad D_\alpha(P\|Q) = \frac{\alpha \mu^2}{2}, \qquad D_\infty(P\|Q) = \infty. \tag{3.6}$$

At $\mu = 1$, for example, the KL divergence is 0.5. Order 2 gives 1, and order 10 already gives 5. Because the dependence on the shift is quadratic, a small shift costs very little. A drift of 0.1 standard deviations gives a KL divergence of only 0.005.

For Laplace noise with scale b, write $t = s/b$ for the shift in scales. The worst case is then just the shift:

$$D_\infty(P\|Q) = t, \tag{3.7}$$

and the KL divergence is $t + e^{-t} - 1$, which is 0.3679 at $t = 1$. The evidence never exceeds t, and it equals t on half the mass of P.

This difference decides how the two noises are analysed. Laplace noise has a finite worst-case ratio, which you read straight off the noise scale as s/b. Normal noise has an infinite max-divergence, because some rare outputs are overwhelming evidence. So any worst-case statement about normal noise must either leave out a small probability of rare outputs or use a finite Rényi order instead. Which pair is "more different" depends on the divergence, too. At equal variance and a shift of one Laplace scale, the Laplace pair has the larger divergence at small orders, but the normal pair overtakes it above order 2.98.

3.5. Total variation

The **total variation (TV)** says directly how often a test is right. It is the mass of P that does not overlap with Q:

$$\mathrm{TV}(P, Q) = \max_A |P(A) - Q(A)| = \frac{1}{2} \sum_x |p(x) - q(x)|. \tag{3.8}$$

It is symmetric and lies between 0 and 1. When P and Q are equally likely a priori, the best possible test from one observation is right with probability

$$\Pr[\text{correct}] = \frac{1}{2} + \frac{1}{2}\mathrm{TV}(P, Q). \tag{3.9}$$

No classifier can beat this. For two normal curves one standard deviation apart, the total variation is 0.3829, so the best test is right with probability 0.6915.

The total variation also bounds how far any fixed statistic between 0 and 1 can move. Accuracy is such a statistic, so when a test distribution moves by a total variation of at most 0.01, the accuracy of any fixed classifier moves by at most 0.01.

3.6. Hockey-stick divergence

Scale q up by a factor γ of at least 1, and measure the mass of p that still sticks out above γq. That mass is the **hockey-stick divergence**:

$$E_\gamma(P \| Q) = \max_A \left[P(A) - \gamma Q(A)\right] = \sum_x \left(p(x) - \gamma q(x)\right)_+. \tag{3.10}$$

Here $(y)_+$ is the positive part of y. With no scaling it is the total variation. It falls as γ grows, and reaches zero once γ reaches the largest likelihood ratio.

So the hockey-stick divergence measures how much of the difference between two distributions comes from likelihood ratios above γ, and later parts use it in that form. It also gives the best test when the two hypotheses are not equally likely, with γ set to the prior odds. For a normal pair shifted by μ standard deviations, with Φ the standard normal cumulative distribution function,

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{\ln \gamma}{\mu}\Big) - \gamma\, \Phi\Big(-\frac{\mu}{2} - \frac{\ln \gamma}{\mu}\Big). \tag{3.11}$$

It is positive for every γ, because the normal likelihood ratio is unbounded. At $\mu = 1$ it gives 0.1906 for $\gamma = 2$.

3.7. f-divergences

The KL divergence, the total variation and the hockey-stick divergence are all members of one family. Each is the average under Q of a convex function f of the likelihood ratio, with $f(1) = 0$:

$$D_f(P\|Q) = \mathbb{E}_Q[f(r(X))]. \tag{3.12}$$

The χ^2 divergence and the squared Hellinger distance are two more members. Every f-divergence obeys the data processing inequality below, and the Rényi divergence, which is an increasing function of one, inherits that.

3.8. The data processing inequality

You rarely use raw outputs, because you threshold a score, round a statistic, fine-tune a model, or pass it through any other algorithm. The **data processing inequality (DPI)** says that none of this can make two distributions easier to tell apart. Describe the processing as a **channel** K, which maps each input to a distribution over outputs, and write PK and QK for the two output distributions. Then every f-divergence and every Rényi divergence satisfies

$$D_f(PK\|QK) \le D_f(P\|Q). \tag{3.13}$$

That is because processing averages likelihood ratios together, and averaging can only weaken evidence.

The condition is that the processing behaves the same whatever the input came from, so it must not look at the original data again. Under that condition, thresholding, rounding or retraining on other data can never make the two distributions easier to tell apart. How much processing loses depends on the divergence. For two unit normal curves one standard deviation apart, a single bit that says whether the observation lies below the midpoint keeps 0.3090 of the KL divergence of 0.5. But it keeps all of the total variation, because it is the best test.

Every divergence here is an average or a maximum over outcomes, and only the maximum controls every output. A small KL divergence can hide a few outputs that are overwhelming evidence. So when every output matters, bound the worst case, through the max-divergence, a high Rényi order or the hockey-stick divergence, not only the average.

3.9. Pinsker's inequality

Pinsker's inequality converts a KL divergence into a bound on the total variation:

$$\mathrm{TV}(P, Q) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(P\|Q)}. \tag{3.14}$$

It is a quick conversion, but often loose, and it says nothing once the KL divergence exceeds 2. It also gives a floor on sample sizes. For a drift of 0.1 standard deviations, no test reaches 95% accuracy with fewer than 324 samples, while the exact answer for this normal pair is 1,083.

3.10. Code

The functions below compute a discrete KL divergence, the hockey-stick divergence of a normal pair, and the Pinsker floor on a sample size.

```
import math
from statistics import NormalDist

Z = NormalDist()


def kl(p: list[float], q: list[float]) -> float:
    """KL divergence of P from Q in nats; infinite if Q misses an outcome that P can produce
    total = 0.0
    for pi, qi in zip(p, q):
        if pi > 0:
            if qi == 0:
                return math.inf
            total += pi * math.log(pi / qi)
    return total


def gauss_hockey(mu: float, gamma: float) -> float:
    """Hockey-stick divergence E_gamma between N(0, 1) and N(mu, 1)."""
    c = math.log(gamma)
    return Z.cdf(mu / 2 - c / mu) - gamma * Z.cdf(-mu / 2 - c / mu)


def min_samples(kl_one: float, accuracy: float) -> int:
    """Pinsker: no test reaches this accuracy (equal priors) with fewer samples."""
    return math.ceil(2 * (2 * accuracy - 1) ** 2 / kl_one)


def gauss_samples(mu: float, accuracy: float) -> int:
    """Samples the best test needs for N(0, 1) against N(mu, 1), exactly."""
    return math.ceil((2 * Z.inv_cdf(accuracy) / mu) ** 2)


before, after = [0.5, 0.3, 0.2, 0.0], [0.4, 0.3, 0.2, 0.1]
print(f"{kl(before, after):.4f} {kl(after, before)}")  # 0.1116 inf
print(f"{kl(before, after) / math.log(2):.4f}")  # 0.1610
print(f"{gauss_hockey(1.0, 2.0):.4f}")  # 0.1906
print(min_samples(0.005, 0.95), gauss_samples(0.1, 0.95))  # 324 1083
```

3.11. Cheat sheet

Table 3.1.: Which divergence answers which question, for a normal pair shifted by μ standard deviations and a Laplace pair shifted by t scales.

Question	Divergence	Normal pair, shift μ	Laplace pair, shift t
Average evidence; drift; adds over samples	KL, $\mathbb{E}_P[\ln r]$	$\mu^2/2$	$t + e^{-t} - 1$
Typical to worst-case evidence, order by order	Rényi D_α	$\alpha\mu^2/2$	finite, at most t
Worst-case ratio over all events	D_∞, $\ln \max r$	∞	t
Best test accuracy on balanced data	TV	$2\Phi(\mu/2) - 1$	$1 - e^{-t/2}$
Mass of ratios above γ	E_γ	Equation 3.11	zero once $\ln \gamma \geq t$

Rules of thumb

- KL is not symmetric, so state its direction.
- Don't mix nats and bits.
- Normal noise has no finite worst-case ratio, so a worst-case statement about it must leave out rare outputs.
- A small average divergence does not control every output.
- Processing that does not look at the original data never increases any divergence.
- Add divergences over independent observations, and convert to TV or a tail bound at the end.

3.12. Further reading

- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014): the Rényi family and its properties.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2 and §11.6: KL and Pinsker's inequality.

Derivations. Every result here is derived step by step in the full edition's chapter.

4. Hypothesis testing

4.1. Why this matters

A spam filter must call an email legitimate or spam, and a fraud detector must call a transaction normal or fraudulent. A monitoring job must likewise decide from a batch of readings whether a sensor has drifted. In each case an observation came from one of two distributions, and you must decide which. So every detector and every binary classifier is a hypothesis test, and evaluating one means measuring a test's two error rates.

This chapter gives you the best possible test, the curve that says how good any test can be, and a way to measure a real detector's error rates on a finite set of trials. Throughout, a single summary number such as accuracy or the area under the curve hides how a detector behaves when false alarms must be rare. That regime is often the one that matters.

4.2. Tests and their two errors

The default assumption, called the **null hypothesis** H_0, is that the observation comes from a distribution P, such as the scores of normal transactions. The **alternative hypothesis** H_1 is that it comes from a distribution Q, such as the scores of fraudulent ones. A **test** decides whether to reject the null, which here means flagging the transaction.

A test can make two kinds of mistake. Flagging a normal case is a false positive, and its probability is the **false positive rate (FPR)**, written α. Missing a real case is a false negative, with probability β, the **false negative rate (FNR)**. The **power** $1 - \beta$ is the **true positive rate (TPR)**, the share of real cases the test catches. Catching more of them always means more false alarms. So the convention is to fix a budget for the FPR first, called the **level**, and then to find the most powerful test within it.

4.3. The Neyman–Pearson lemma

Write $p(x)$ and $q(x)$ for the probabilities, or densities, of an observation x under the two hypotheses. Their quotient is the **likelihood ratio**

$$\Lambda(x) = \frac{q(x)}{p(x)}, \tag{4.1}$$

which puts the alternative on top, unlike Divergences in the full edition. A **likelihood ratio (LR) test** picks a threshold η and rejects wherever the ratio exceeds it. The **Neyman–Pearson lemma** says that this is the most powerful test at its level. The reason is that rejecting x costs

$p(x)$ of false-alarm budget and buys $q(x)$ of power, so the best test spends its budget on the best exchange rates first. On discrete data the LR test may have to reject a boundary value only with some probability, a **randomized test**, to hit a budget exactly.

So the best test looks at an observation only through its likelihood ratio. A single global threshold on a raw score is not that test when the score behaves differently for different items. For example, some merchants' normal transactions score high anyway, while for others a high score is rare unless something is wrong. The better test models each merchant's own two score distributions and thresholds the ratio between them. With a number n of independent observations, the LR test thresholds a sum of log ratios. Its FNR at a fixed FPR then falls like

$$e^{-n\,\mathrm{KL}(P\|Q)}, \tag{4.2}$$

where $\mathrm{KL}(P\|Q)$ is the Kullback–Leibler (KL) divergence. This is the Chernoff–Stein lemma.

4.4. Trade-off curves

To say how hard it is to tell P from Q, record the best FNR at every level. As a function of the budget, this is the **trade-off function** $T(\alpha)$, and you compute it by sweeping the LR threshold. The **receiver operating characteristic (ROC)** curve holds the same information upside down, as TPR against FPR. Any particular detector traces its own ROC curve, which lies on or below the best one. Every trade-off curve lies on or below the coin-flip line $1-\alpha$ and is convex, because randomizing between two tests reaches every point between them.

4.5. The Gaussian trade-off curve

The most common pair in practice is a score that is normal under both hypotheses, with the same spread. Write μ for the distance between the two centres in standard deviations, and Φ for the standard normal cumulative distribution function. The best test thresholds at $\Phi^{-1}(1-\alpha)$, and its FNR is

$$T(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu). \tag{4.3}$$

In SciPy this is `norm.cdf(norm.ppf(1 - alpha) - mu)`, and at a tiny α use `norm.isf(alpha)` for the threshold, because $1-\alpha$ loses the digits that matter. So a single number μ fixes the best TPR at every FPR.

For example, take $\mu = 2$. At an FPR of 5%, the best test has a TPR of 0.6388. If false alarms must be rare, at a rate of 10^{-3}, the TPR drops to 0.1378.

4.6. The Laplace trade-off curve

For a Laplace distribution and a copy shifted by κ scales, the likelihood ratio is capped between $e^{-\kappa}$ and e^{κ} (Probability distributions in the full edition). No test can buy power faster than the cap per unit of FPR, so every test has

$$\mathrm{TPR} \le e^{\kappa}\,\mathrm{FPR}. \tag{4.4}$$

At $\kappa = 1$ and an FPR of 0.05, the best TPR is 0.1359. The full curve has two more pieces, which the full edition gives. The same cap holds for any pair whose likelihood ratio is bounded by e^{κ}, so at a low FPR every test on such a pair has a very small TPR.

A Laplace pair at $\kappa = 1$ and a normal pair at $\mu = 1.0301$ have the same best advantage. At a level of 10^{-6}, though, the best test on the normal pair has 36 times the power, because the unbounded normal ratio lets it find rare, overwhelming observations. So plan for the level you will actually use.

4.7. Best advantage and total variation

The **advantage** of a test is its TPR minus its FPR, called Youden's index in medicine. The best advantage over all tests is the **total variation (TV)** distance of Divergences in the full edition:

$$\max(\mathrm{TPR} - \mathrm{FPR}) = \mathrm{TV}(P, Q) = \frac{1}{2} \sum_{x} |q(x) - p(x)|. \tag{4.5}$$

It is reached by the test that rejects exactly where q exceeds p. For the normal pair it is $2\Phi(\mu/2) - 1$, or 0.3829 at a shift of one standard deviation. Accuracy on a balanced set of positives and negatives measures the same point. But that point says nothing about the ends of the curve, as the Laplace and normal pairs above show. So never report only accuracy or advantage for a detector.

4.8. ROC summaries: why AUC hides the low-FPR regime

Detectors are often compared by the **area under the curve (AUC)** of their ROC curves. It is the chance that a random positive scores higher than a random negative:

$$\mathrm{AUC} = \Pr[S_{+} > S_{-}]. \tag{4.6}$$

But the AUC weighs mostly false-alarm rates nobody would operate at, while the question is often whether a detector can flag some cases with confidence, as when a fraud team can review only a few transactions.

Two detectors with standard normal scores on negatives make the point. Detector A's positives score normal with mean 1 and standard deviation 1. Detector B's positives are more spread out, with mean 1.5811 and standard deviation 2. Both have an AUC of 0.7602. At an FPR of 10^{-3}, though, A catches 0.0183 of the positives, while B catches about 12 times as many. A few of B's spread-out positives score far above every negative, and those are exactly the cases a strict threshold catches.

So plot the full ROC curve on log–log axes, and report the TPR at fixed low FPRs such as 10^{-2} and 10^{-3}. In scikit-learn, `sklearn.metrics.roc_curve(y_true, y_score)` returns the FPRs and TPRs to read them from, and `roc_auc_score(y_true, y_score)` gives the AUC for comparison with older work. Don't read a TPR at an FPR below one over the number of negatives, though, because there the curve rests on a single false positive or none, and it is noise.

4.9. The Bayesian view

Often you know a base rate π, such as one transaction in a thousand being fraud. Bayes' rule says that the odds of H_1 after seeing x are the odds before, multiplied by the likelihood ratio:

$$\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]} = \frac{\pi}{1-\pi} \times \Lambda(x). \tag{4.7}$$

So the decision with the fewest errors says H_1 when the ratio exceeds $(1-\pi)/\pi$, which is again an LR test.

The same rule applies to an alarm, whose likelihood ratio is the TPR divided by the FPR. Take the test with $\mu = 2$ in a population where 1% are positive. At an FPR of 0.05, a flagged case is positive with probability only 0.114, so most alarms are false. At an FPR of 10^{-3}, the test flags fewer cases, but a flagged case is positive with probability 0.582. So when positives are rare, the FPR decides whether an alarm is worth believing.

4.10. Measuring rates from finite trials

You measure a detector's FPR by counting false positives among n test negatives. If each negative is flagged independently with the same probability, the count k is binomial, and its relative uncertainty is about $1/\sqrt{k}$. So the precision of a measured rate depends on how many events you see, not on n alone.

The **Clopper–Pearson interval** gives the range of true rates that make the observed count unsurprising. Write $1-a$ for the confidence. Its lower end l and upper end u solve

$$\Pr[\mathrm{Bin}(n, l) \geq k] = \frac{a}{2}, \qquad \Pr[\mathrm{Bin}(n, u) \leq k] = \frac{a}{2}. \tag{4.8}$$

It is exact, because it uses the binomial itself, and conservative, because it covers the true rate at least $1-a$ of the time. In SciPy, `scipy.stats.binomtest(k, n).proportion_ci(confidence_level=0.95, method="exact")` computes it, and statsmodels' `proportion_confint(k, n, alpha=0.05, method="beta")` does the same.

To claim that an FPR is below a target, use the upper end alone. The upper end of a two-sided 90% interval is a one-sided 95% bound, which is 0.0047 for 1 false positive in 1,000 negatives. With no events at all, the one-sided 95% bound is about $3/n$, the **rule of three**. So certifying an FPR below 10^{-3} takes 2,995 negatives with no false positive.

Even 10 false positives in 10,000 negatives give the interval $[0.00048, 0.00184]$, from about half to almost twice the target. As a rule of thumb, measuring an FPR of α to within about 20% takes on the order of $100/\alpha$ negatives. A common shortcut fails here, because the **Wald interval**, a normal approximation, is too narrow and collapses to $[0, 0]$ with no events.

4.11. Evaluating a detector at a low false positive rate

These results give a recipe for evaluating any detector or binary classifier.

1. Fix the target FPRs, such as 10^{-2} and 10^{-3}.
2. Plan the trials with about $100/\alpha$ negatives for a target FPR α.
3. Choose the threshold on one split and measure the rates on another, because a threshold tuned on the evaluation set looks better there than it is.
4. Report the TPR at each target FPR, with Clopper–Pearson intervals on both rates, and the full ROC curve on log–log axes.
5. Compare two detectors at the same FPR, never by AUC or accuracy alone. On a shared evaluation set, a paired test such as McNemar's (`statsmodels.stats.contingency_tables.mcnemar`) compares the positives each one catches.

4.12. Cheat sheet

Table 4.1.: The questions of this chapter and their answers.

Question	Answer
What is the best test?	Threshold the likelihood ratio, per item if items differ (Neyman–Pearson)
How good can any test be?	The trade-off curve; normal pair $\Phi(\Phi^{-1}(1-\alpha)-\mu)$
What does a capped ratio allow?	$\mathrm{TPR} \le e^{\kappa}\,\mathrm{FPR}$
What do accuracy and advantage measure?	Best advantage = TV, one point of the curve
Why is AUC not enough?	Equal AUC can hide a 12-fold gap in TPR at an FPR of 10^{-3}
What does a base rate do?	Posterior odds = prior odds × TPR/FPR
How sure is a measured rate?	Clopper–Pearson; zero events give about $3/n$ at 95%

Rules of thumb

- Use Clopper–Pearson intervals, never the Wald interval, for rare rates.

4.13. Further reading

- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves and AUC as a ranking probability.
- Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): why the Wald interval fails and what to use instead.

Derivations. Every result here is derived step by step in the full edition's chapter.

5. Information theory

5.1. Why this matters

You often observe one thing and want to know how much it tells you about another thing you cannot see. A word in an email hints at whether the email is spam, and a noisy sensor reading carries a trace of a machine's true state. Later parts ask how much a published output reveals about hidden data. Information theory turns "how much" into a number, in two different ways, because there are two different questions.

The first question is how many bits the output saves, on average, when you describe the secret. Shannon entropy and mutual information answer it, so they are the language of compression, of the cross-entropy loss and of feature selection. The second question is how much more likely a single best guess becomes once you have seen the output. Min-entropy and vulnerability answer that one, so they are the language of password strength and of anyone who gets one try. The two can rank the same channels in opposite orders, and both are averages, so neither says how exposed one particular value is.

Logarithms here are base 2, so every quantity is in **bits**. One nat, the unit of natural logarithms, is about 1.443 bits. SciPy's `scipy.stats.entropy(pk, base=2)` returns bits, but its default and scikit-learn's `sklearn.metrics.mutual_info_score(labels_true, labels_pred)` return nats, so convert before you compare numbers.

5.2. Entropy

The **entropy** of a random variable X is its average surprisal, the logarithm of one over the probability of the value that occurs. Writing $p(x)$ for the probability that X takes the value x, it is

$$H(X) = -\sum_{x} p(x) \log p(x), \tag{5.1}$$

where an outcome of probability zero adds nothing. It is also the average number of bits per value that the best possible code needs. For a coin that lands heads with probability p, it is the **binary entropy** $h(p)$, which is 1 bit for a fair coin and zero for a certain one. Even a coin with an 11% chance of heads still carries $h(0.11) = 0.4999$ bits.

Entropy lies between zero, when one value is certain, and the logarithm of the number of possible values, when every value is equally likely. A uniform 4-digit PIN, for example, has $\log_2 10{,}000 = 13.2877$ bits. The same bound sizes a combination of columns, such as birth date, sex and postcode. Its entropy is at most the logarithm of the number of combinations. Once it approaches the logarithm of the number of rows, many rows have a combination of their own.

For two variables, the **conditional entropy** $H(X \mid Y)$ is the uncertainty about X left after seeing Y, averaged over the values of Y. It is never larger than $H(X)$, but only on average. A particular observation can increase the uncertainty, for example when a rare word makes a probably-legitimate email look equally likely to be spam.

5.3. Mutual information and data processing

How much does an output Y tell you about a secret X? The **mutual information** $I(X;Y)$ is the drop in uncertainty it causes on average. It is also the Kullback–Leibler (KL) divergence between the joint distribution of the pair and the product of the two separate distributions:

$$I(X;Y) = H(X) - H(X \mid Y) = D(P_{XY} \,\|\, P_X P_Y). \tag{5.2}$$

So it is never negative, it is zero only when the two variables are independent, and it is symmetric. Unlike a correlation, it detects dependence of any kind. Decision trees call it the **information gain**, and many feature-selection methods rank features by it.

To estimate mutual information from data, `mutual_info_score` puts the observed frequencies of two discrete columns into Equation 5.2. For continuous features, `mutual_info_classif` and `mutual_info_regression` in `sklearn.feature_selection` use a nearest-neighbour estimator instead. Both are biased. Plug-in estimates from small samples run high, so independent variables rarely give exactly zero. Compare an estimate with the same estimate on data whose pairing you have shuffled, which shows what independence alone produces.

The **data processing inequality (DPI)** says that processing cannot add information. Suppose a third variable Z is computed from Y alone, possibly with fresh randomness of its own. This is written $X \to Y \to Z$, and then

$$I(X;Z) \le I(X;Y). \tag{5.3}$$

The condition is essential, because a Z that also looked at X directly could know more. In practice, denoising, feature extraction and training are all processing steps, so a bound on what the raw output reveals covers everything anyone computes from it.

A **channel** is the table of output probabilities for each input, and its **capacity** is the largest mutual information over all input distributions. It is the highest rate at which error-correcting codes can send data reliably. For the binary symmetric channel (BSC), which flips each bit with probability p, it is $1 - h(p)$.

5.4. Min-entropy and one-guess leakage

A guesser who gets one try wins with the probability of the most likely value. That probability is the **vulnerability** $V(X)$, and its negative logarithm is the **min-entropy**:

$$V(X) = \max_x p(x), \qquad H_\infty(X) = -\log V(X). \tag{5.4}$$

Min-entropy is never larger than Shannon entropy, and the two are equal only for a uniform distribution. A value with a min-entropy of k bits is as hard to guess in one try as a uniform string of k bits, whatever its Shannon entropy.

A PIN shows why Shannon entropy is the wrong measure of guessing difficulty. Suppose 10% of people choose 1234, and the rest choose uniformly among the other PINs. Then the Shannon entropy barely moves, to 12.4278 bits. But the min-entropy falls to 3.3219 bits, because the guess "1234" succeeds one time in ten.

Now let the guesser see an output Y through a channel C, whose entry $C[x, y]$ is the probability of output y when the secret is x. Write π for the **prior**, how likely each secret is to begin with, so that each entry of the joint table is $\pi(x)C[x, y]$. The best strategy is the maximum a posteriori (MAP) guess, the secret most likely given the output. Its success probability is called the **posterior vulnerability**, and it is the sum of the column maxima of the joint table:

$$V(X \mid Y) = \sum_y \max_x \pi(x)C[x, y]. \tag{5.5}$$

The **min-entropy leakage** is the logarithm of the factor by which the output multiplies the chance of a correct guess:

$$\mathcal{L} = \log \frac{V(X \mid Y)}{V(X)}. \tag{5.6}$$

So a leakage of 1 bit doubles the guesser's chance, which is a statement you can act on.

This is how to quantify one-guess risk on a small table. List the secret's prior, write down the channel from secret to output, and compute the two vulnerabilities. In the example of Section 5.7, a secret with four values has a best prior guess that is right with probability 0.4. With the output, the guesser wins with probability 0.66, so the leakage is 0.7225 bits. Two shortcuts give wrong answers. Comparing channel entries instead of joint entries ignores the prior. Averaging the min-entropy of each posterior takes the logarithm too early, and gives 0.6548 bits instead of the correct conditional min-entropy of 0.5995 bits.

When you do not know the prior, use the worst case over all priors. That is the **min-capacity**, which needs only the column maxima of the channel itself, and which a uniform prior attains:

$$\mathcal{ML}(C) = \log \sum_y \max_x C[x, y]. \tag{5.7}$$

For the example channel it is 1.2630 bits. For a channel that reports a hidden bit correctly with probability 0.75 and flips it otherwise, it is 0.5850 bits. So whatever the prior, seeing the output multiplies the chance of guessing the bit by at most 1.5.

5.5. Shannon against min-entropy leakage

The two leakages agree for a channel that reveals the same bits every time. They disagree, by a lot, for a channel that reveals the secret completely on a few outputs and nothing on the rest. Shannon leakage averages the logarithm, so a rare full reveal contributes little. Min-entropy leakage takes the logarithm of a success probability, which the same reveal dominates. Table 5.1 compares two deterministic channels on a uniform 64-bit secret.

Table 5.1.: Shannon and min-entropy leakage of two deterministic channels on a uniform 64-bit secret.

Channel	Shannon leakage	Min-entropy leakage	One-guess success
Reveal all if the low 3 bits are zero	8.1686 bits	61.0000 bits	0.1250
Reveal the low 9 bits	9.0000 bits	9.0000 bits	2.78×10^{-17}

Shannon leakage says the second channel leaks more, yet a guesser with one try does better against the first by a factor of more than 10^{15}. So low mutual information does not mean that nothing is revealed. A channel can have little mutual information with its secret and still give some values away completely.

Min-entropy leakage has the same blind spot in a milder form, because it too is an average over secrets. A channel that reveals a 32-bit secret only when it is all zeros has a min-entropy leakage of exactly 1 bit, and a far smaller Shannon leakage. Yet for the one secret made of zeros, it reveals everything. Any measure that averages over the secret can hide a value that is fully exposed. So when every value matters, you need a worst-case bound on the likelihood ratio of each output, which later parts build.

5.6. Fano's inequality

Mutual information can also show that a problem is hard. **Fano's inequality** concerns a secret that takes one of M values. It says that a guess computed from the output, wrong with probability P_e, must satisfy

$$H(X \mid Y) \le h(P_e) + P_e \log(M - 1). \tag{5.8}$$

So if much uncertainty remains after the output, every guess must often be wrong. For a label uniform over 8 classes, with features that carry at most 1 bit about it, every classifier errs at least 0.3730 of the time. Later parts use the same tool to show that some estimates need many samples.

5.7. Code

This snippet computes the vulnerabilities, the min-entropy leakage and the min-capacity for a secret with four values and three outputs, and the min-capacity of a channel that flips a bit with probability 0.25.

```
from math import log2

prior = [0.4, 0.3, 0.2, 0.1]
channel = [  # channel[x][y] = probability of observation y given secret x
    [0.8, 0.2, 0.0],
    [0.1, 0.8, 0.1],
    [0.0, 0.5, 0.5],
```

```
    [0.0, 0.2, 0.8],
]


def posterior_vulnerability(prior: list[float], channel: list[list[float]]) -> float:
    """Success probability of the best one-shot guess: the sum of the joint table's column m
    joint = [[px * c for c in row] for px, row in zip(prior, channel)]
    return sum(max(col) for col in zip(*joint))


def min_capacity(channel: list[list[float]]) -> float:
    """Leakage in bits under the worst prior: log of the sum of the channel's column maxima.
    return log2(sum(max(col) for col in zip(*channel)))


v_prior = max(prior)
v_post = posterior_vulnerability(prior, channel)
print(f"{v_prior:.4f} {v_post:.4f}")  # 0.4000 0.6600
print(f"{log2(v_post / v_prior):.4f}")  # 0.7225
print(f"{min_capacity(channel):.4f}")  # 1.2630
flip = [[0.75, 0.25], [0.25, 0.75]]  # report the hidden bit correctly with probability 0.75
print(f"{min_capacity(flip):.4f} {2 ** min_capacity(flip):.4f}")  # 0.5850 1.5000
```

5.8. Cheat sheet

Table 5.2.: Which quantity answers which question.

Question	Quantity	Formula
How uncertain is a secret, in bits?	Entropy	$H(X) = -\sum_x p(x) \log p(x)$, at most $\log\lvert\mathcal{X}\rvert$
How much does an output tell on average?	Mutual information	$I(X;Y) = H(X) - H(X \mid Y)$
Can post-processing reveal more?	Data processing	$I(X;Z) \le I(X;Y)$ when $X \to Y \to Z$
How likely is one guess to succeed?	Vulnerability	$V = \max_x p(x)$, $H_\infty = -\log V$
How much does the output help one guess?	Min-entropy leakage	$\log V(X \mid Y)/V(X)$
The worst case over priors?	Min-capacity	$\log \sum_y \max_x C[x, y]$
Is any single value fully exposed?	None of these	Needs a worst-case bound on each output

Rules of thumb

- Check whether a library returns bits or nats.
- Compare a mutual information estimate with the same estimate on shuffled data, because small-sample estimates run high.
- Measure guessing difficulty with min-entropy, never with the Shannon entropy of a password or PIN distribution.
- Low mutual information does not mean nothing is revealed, and an average leakage never shows that every value is safe.

5.9. Further reading

- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2 and 7: the standard reference for every Shannon result here.
- Smith, "On the Foundations of Quantitative Information Flow" (FoSSaCS 2009): vulnerability, and why Shannon leakage is the wrong measure for one-try guessing.

Derivations. Every result here is derived step by step in the full edition's chapter.

6. Optimization

6.1. Why this matters

Almost all training is done by gradient descent (GD), or by its cheaper and noisier variant, stochastic gradient descent (SGD), and two practical questions come up every time. The first is how to set the step size and the number of steps, and how much noise in the gradients slows training down. The second is how much the trained model depends on any one training example, which is the quantity that later parts of the book have to control.

Both are answered by a few constants of the loss and by the variance of the random gradients. So the first job is to know these constants for your loss, or to enforce them.

6.2. The constants of a loss, and how to enforce them

Throughout, the model is a parameter vector w, and the training set has n examples. Write $\ell(w, z)$ for the loss on an example z, and $z_1, \dots, z_n$ for the training examples. Training minimizes the **empirical risk**, the average loss

$$F(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i), \tag{6.1}$$

and this chapter writes w^* for its minimizer and F^* for its smallest value.

A loss is L**-Lipschitz** when its gradient never has length more than L, so it caps how far one example can push a model. It is **smooth** when its gradient changes by at most a constant β per unit of distance. That bounds how far you can trust a step, and it sets the safe step size $1/\beta$.

A loss is λ**-strongly convex** when it curves upward by at least λ in every direction, so that a point with a small gradient must lie close to the minimizer. The ratio $\kappa = \beta/\lambda$ is the **condition number**, the ratio of the steepest curvature to the flattest. Table 6.1 gives the constants for the losses this book uses, for features of length at most 1.

Table 6.1.: The constants of common losses, for feature vectors of length at most 1.

Loss	Lipschitz	Smooth	Strongly convex
Logistic, $\ln(1 + e^{-y\langle w,x\rangle})$	$L = 1$	$\beta = \frac{1}{4}$	No
Hinge, $\max(0, 1 - y\langle w, x\rangle)$	$L = 1$	No, it has a kink	No

Loss	Lipschitz	Smooth	Strongly convex
Squared, $\frac{1}{2}(\langle w, x\rangle - y)^2$	No, the gradient grows with w	$\beta = 1$	Only if the feature vectors span every direction
Any of these plus $\frac{\lambda}{2}\|w\|^2$	Only on a bounded set: $L + \lambda r$ within radius r	$\beta + \lambda$	λ

The table shows that the constants come from three choices you control. Normalizing every feature vector to length at most 1 is what makes the logistic and hinge losses 1-Lipschitz, so a single unnormalized outlier breaks the guarantee. A ridge term $\frac{\lambda}{2}\|w\|^2$ is the usual source of strong convexity, because the logistic loss has none of its own. But the ridge term makes the gradient grow with the parameters. So you also keep w in a ball by **projection**, which pulls it back to the nearest point of the ball after any step that leaves it.

For a neural network, whose constants nobody can compute, you instead **clip** each example's gradient to a chosen length C. Clipping enforces the Lipschitz bound by construction, with $L = C$, but only if it happens per example. In PyTorch, `torch.nn.utils.clip_grad_norm_` clips the gradient of a whole batch, which bounds nothing about any single example. Per-example clipping, which Part 5 uses, needs per-example gradients.

6.3. Gradient descent and SGD

Gradient descent repeatedly steps against the full gradient, with a step size written η. The safe step is $1/\beta$, because smoothness guarantees that such a step never overshoots. On a strongly convex risk the error then shrinks by a factor $1 - 1/\kappa$ per step. So a badly conditioned problem is slow, which is why rescaling features to similar ranges speeds training up.

SGD replaces the full gradient with that of one example, or the average over a small random **minibatch**. The random gradient equals the true gradient on average, but only if the examples are sampled uniformly, and a sampler that favours some examples minimizes a reweighted risk instead. Its scatter around the true gradient, the variance, is what every SGD rate pays for. With a constant step it keeps the iterates jittering around the minimizer, and averaging them or shrinking the step removes the jitter.

6.4. Convex losses: the step size and the averaged iterate

The basic guarantee covers convex losses with bounded gradients, including the hinge loss. It runs **projected SGD** inside a closed convex set that contains the minimizer. It returns the **averaged iterate** $\bar{w}_T$, the average of the parameters over a run of T steps. Write G^2 for a bound on the average squared length of the random gradients, and R for a bound on the distance from the start to the minimizer. With the step size

$$\eta = \frac{R}{G\sqrt{T}}, \qquad \mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{RG}{\sqrt{T}}. \tag{6.2}$$

For an L-Lipschitz loss, $G = L$.

To use it, take R as the radius of the projection ball and L from Table 6.1, choose T, and start from the step the formula gives. For logistic regression on the ball of radius 5, a run of 10,000 steps gives a step of 0.05. No method does better than this $1/\sqrt{T}$ rate on this class.

Treat the bound as a guarantee and the step as a starting point. The formula assumes the worst gradients and the farthest start, so on the full edition's logistic problem the best step on a grid is 3.6 times larger. So tune the step on a grid around the formula's value, and return the averaged iterate, which the guarantee covers, rather than the last one.

6.5. Strongly convex losses: a decaying step

With a ridge term the rate improves from $1/\sqrt{T}$ to $1/T$, so halving the error takes twice the steps rather than four times. Use the steps $2/(\lambda(t+1))$ at step t, and average the iterates with weights proportional to t. Then

$$\mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{2G^2}{\lambda(T+1)}. \tag{6.3}$$

The simpler schedule $1/(\lambda t)$ with a uniform average costs an extra factor that grows like $\ln T$. Here G bounds the gradients on the projection ball, since a ridge term makes them grow with the weights.

In practice, scikit-learn's `SGDClassifier` with `learning_rate="optimal"` uses a step of this $1/(\lambda t)$ shape, with its `alpha` equal to λ, and `average=True` returns an averaged iterate. Its `LogisticRegression` is parameterized differently. It multiplies the summed loss by `C` instead of penalizing the weights, so its ridge strength in this chapter's terms is $\lambda = 1/(Cn)$. A large `C` therefore means a weak ridge term.

The schedule needs a value of λ no larger than the true curvature. If you overestimate it, the steps shrink too fast and the iterates can stall far from the minimizer.

6.6. Gradient noise and minibatches

Gradients get noisier from small batches, low-precision arithmetic or compression, and later parts add noise to them on purpose. Write σ_1^2 for the variance of a single-example gradient. A batch of B examples divides it by B, and drawing the batch without replacement does slightly better:

$$\mathbb{E}\|g_B - \nabla F(w)\|^2 = \frac{\sigma_1^2}{B} \cdot \frac{n-B}{n-1}. \tag{6.4}$$

Noise of a fixed size added to the summed gradient of a batch is also divided by B when the sum is averaged. So larger batches dilute added noise, which is why noisy training prefers them.

Now add noise of mean zero whose average squared length is σ^2. It simply adds to G^2, so the convex recipe becomes

$$\eta^* = \frac{R}{\sqrt{(L^2+\sigma^2)T}}, \qquad \mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{R\sqrt{L^2+\sigma^2}}{\sqrt{T}}. \tag{6.5}$$

To reach the same bound, the number of steps must grow by the factor $1 + \sigma^2/L^2$, so noise of size $\sigma = 3L$ multiplies the steps you need by 10. On the full edition's logistic problem with 10,000 steps, the formula's step falls from 0.0500 without noise to 0.0158 at $\sigma = 3$, and the error at that step grows about 20-fold. The best step on a grid falls too, from 0.178 to 0.056. So a step tuned without noise is too large once noise is added, and the formula's step is conservative throughout. Retune on a grid whenever the noise level changes.

6.7. Non-convex losses

Neural networks have non-convex losses, and no gradient method can promise to find their global minimum. With a smooth loss and a step of at most $1/\beta$, SGD only guarantees a point where the gradient is small. Noise slows that down, just as it does for convex losses. A small gradient can sit at a saddle point or a poor local minimum, so nothing here says how far the risk is from its best value. A step above $2/\beta$ can even raise the risk, which is what a diverging loss curve usually means.

6.8. How far one example moves the model

Regularized **empirical risk minimization (ERM)** minimizes the average loss plus a ridge term,

$$J(w; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i) + \frac{\lambda}{2} \|w\|^2, \tag{6.6}$$

where D is the training set and $w^*(D)$ its minimizer. If each loss is convex and L-Lipschitz, replacing one example moves the minimizer by at most

$$\|w^*(D) - w^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.7}$$

One example tilts the gradient by at most $2L/n$, and strong convexity turns a small tilt into a small move. The loss on any example then changes by at most $2L^2/(\lambda n)$, a property called **uniform stability**, which the next chapter in the full edition turns into a generalization bound.

Later parts calibrate noise to this per-example influence, so read the bound as a design rule. Ten times more data gives a tenth of the movement, and halving λ can double it. For the logistic loss with $\lambda = 0.1$ and 1,000 examples, the bound is 0.02, while the largest movement observed on the full edition's data is 6.66×10^{-3}.

The bound needs strong convexity, and without it no bound of this kind exists. The absolute loss is convex and 1-Lipschitz, but its minimizer is the median, and replacing one example can move the median from 0 to 1 however large n is. For a non-convex loss, one example can decide which of two valleys is deeper. So for neural networks, any statement about the influence of one example must be about the training algorithm, not about the minimizer.

6.9. Code

This calculator gives the step size of convex SGD, with or without gradient noise, and the movement bound for regularized ERM.

```python
import math


def sgd_step(radius: float, lipschitz: float, steps: int, noise: float = 0.0) -> float:
    """Step size that balances the convex SGD bound."""
    return radius / math.sqrt((lipschitz**2 + noise**2) * steps)


def erm_movement(lipschitz: float, lam: float, n: int) -> float:
    """Largest move of the minimizer when one example is replaced."""
    return 2 * lipschitz / (lam * n)


print(f"{sgd_step(5, 1, 10_000):.4f}")  # 0.0500
print(f"{sgd_step(5, 1, 10_000, noise=1):.4f}")  # 0.0354
print(f"{sgd_step(5, 1, 10_000, noise=3):.4f}")  # 0.0158
print(f"{erm_movement(1, 0.1, 1000):.3f}")  # 0.020
```

6.10. Cheat sheet

Table 6.2.: The results of this chapter.

Result	Conditions	Bound or rule
SGD, convex	Gradients bounded by G, start within R	Step $R/(G\sqrt{T})$; error $\leq RG/\sqrt{T}$, averaged iterate
SGD, strongly convex	Steps $2/(\lambda(t+1))$, t-weighted average	$2G^2/(\lambda(T+1))$
Minibatch of B out of n	Uniform, without replacement	Variance $\frac{\sigma_1^2}{B} \cdot \frac{n-B}{n-1}$
Added gradient noise σ	Mean zero	$G^2 = L^2 + \sigma^2$; steps needed grow by $1 + \sigma^2/L^2$
Regularized ERM	Convex, L-Lipschitz loss	Minimizer moves $\leq 2L/(\lambda n)$

Rules of thumb

- Normalize features to length at most 1, add a ridge term, and project onto a ball. That gives you L, λ and R, and without the normalization L is not 1.
- For a model whose constants you cannot compute, clip each example's gradient. Clipping the batch's gradient bounds nothing per example.

- Never keep a step size tuned without noise once noise is added. Retune it on a grid, and return the averaged iterate.
- Read scikit-learn's `C` as $\lambda = 1/(Cn)$, not as λ.
- One example moves a regularized minimizer by at most $2L/(\lambda n)$. Without strong convexity, as in neural networks, there is no such bound.

6.11. Further reading

- Bottou, Curtis & Nocedal, "Optimization Methods for Large-Scale Machine Learning" (SIAM Review 2018): noise, step sizes and batch sizes in practice.
- Bubeck, "Convex Optimization: Algorithms and Complexity" (2015): the rates and their lower bounds.

Derivations. Every result here is derived step by step in the full edition's chapter.

7. Algorithmic stability and generalization

7.1. Why this matters

What matters is a model's loss on new data, not on its training data. The difference between the two is the **generalization gap**, and you measure it on data the model has not seen. This chapter tells you in advance which knobs shrink it, which examples carry it, and when a held-out measurement stops being honest.

The one idea is that **a model generalizes when it does not depend much on any single training example.** For regularized training and for stochastic gradient descent (SGD), that dependence shrinks with a stronger regularizer and a shorter run. It can also be measured example by example, and Part 2 shows why it matters beyond accuracy.

7.2. Training loss, test loss and the gap

Take a training set S of examples drawn independently from one distribution $\mathcal{D}$. Write n for the number of examples, and $A(S)$ for the model that a learning algorithm A returns. A loss $\ell(w, z)$ that is never negative measures how badly a model w does on an example z. The **test loss** $R(w)$ is the average loss on a new example from $\mathcal{D}$, and the **training loss** $\hat{R}_S(w)$ is the average over the training set. The gap of the trained model is

$$\text{gap} = R(A(S)) - \hat{R}_S(A(S)). \tag{7.1}$$

For a model fitted to the data, the training loss is biased downward, because the algorithm chose the model that does well on these particular examples. So estimate the test loss on a held-out set that played no part in training and comes from the same distribution as future data.

The chapter's running example is logistic regression on features in 10 dimensions with noisy labels, plus a ridge penalty whose strength is written λ.

7.3. The gap and uniform stability

The tool here is the **replace-one neighbour** $S^{(i)}$, the training set with example i replaced by a fresh draw from $\mathcal{D}$. The expected gap is then exactly the average damage that training without an example does to that example's loss:

$$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\Big[\ell\big(A(S^{(i)}), z_i\big) - \ell\big(A(S), z_i\big)\Big]. \tag{7.2}$$

It holds for any algorithm, because a training example and a fresh draw are interchangeable. In other words, overfitting is dependence on individual examples.

To bound that dependence, call an algorithm β**-uniformly stable** when replacing any one training example by any other moves the loss by at most β. That must hold at every test point z:

$$\big|\ell(A(S), z) - \ell(A(S^{(i)}), z)\big| \le \beta. \tag{7.3}$$

Every term of the identity then lies within β of zero, so the expected gap does too:

$$\big|\mathbb{E}[\text{gap}]\big| \le \beta. \tag{7.4}$$

It works for large models too, but β is a worst case over training sets, replacements and test points, so it can sit far above the true gap.

7.4. High probability: the Bousquet–Elisseeff bound

You train once, so you may want a high-probability bound. The bound of Bousquet & Elisseeff gives one for losses bounded by some maximum, but only when β falls faster than $1/\sqrt{n}$, and it is loose at realistic sizes. At 95% confidence, for regularized logistic regression with 10,000 examples at $\lambda = 0.1$, it adds 0.614 to the training loss, which says nothing useful. So measure the gap on held-out data, and use the bounds only to compare settings.

7.5. Regularized ERM is stable

Regularized **empirical risk minimization (ERM)** covers ridge regression and regularized logistic regression. It picks the weights that minimize the training loss plus a ridge penalty,

$$w_S = \arg\min_w \Big(\hat{R}_S(w) + \frac{\lambda}{2}\|w\|^2 \Big). \tag{7.5}$$

Suppose each loss is convex and L-Lipschitz, which means that its gradient never has length more than L (Optimization in the full edition). Then replacing one example moves the weights by at most $2L/(\lambda n)$, and the algorithm is stable with

$$\beta = \frac{2L^2}{\lambda n}. \tag{7.6}$$

To compute β, normalize the features to length at most 1, which makes $L = 1$ for the logistic and hinge losses, and plug in λ and n. Logistic regression with $\lambda = 0.1$ and 1,000 examples then has $\beta = 0.02$. In scikit-learn, `LogisticRegression(C=...)` sets $\lambda = 1/(Cn)$, so a large `C` means a weak penalty and a large β.

On the full edition's running example, the measured gap falls like $1/n$, as the bound predicts, but it stays between about 3% and 13% of the bound. So the bound tells you how the gap scales with λ and n, not how large it is. Doubling λ halves β, but a larger penalty also pulls the weights away from the best fit. The test loss is therefore U-shaped in λ, so choose λ on held-out data and use the bound to know which direction helps.

Both conditions matter. Squared loss is Lipschitz only on a bounded region, so ridge regression needs bounded features and labels. Without convexity, one flipped label can move the minimizer from one valley to another, however large n is.

7.6. SGD is stable if you stop early

Large models are trained by SGD rather than to an exact minimizer. Write η_t for the step size at step t, and T for the number of steps. Hardt, Recht & Singer (2016) showed that

$$\beta \le \frac{2L^2}{n} \sum_{t=1}^{T} \eta_t. \tag{7.7}$$

With a constant step η over a number of epochs E, the bound becomes $2L^2\eta E$. So for a fixed number of epochs, more data does not improve it.

The bound needs a convex, L-Lipschitz and smooth loss, with steps small enough not to overshoot. It also needs examples picked uniformly at random, so shuffling once per epoch, as most implementations do, is not covered as stated.

The practical reading is that training time is a stability knob, which makes early stopping a trade-off rather than a trick. Take the running example without regularization, with 100 examples and a step of 0.25. One epoch then gives $\beta \le 0.5$. The gap grows from 0.036 after one epoch to 0.121 after 40. The test loss is lowest after about 5 epochs and rises afterwards, while the training loss keeps falling. The bound gets the direction right but the size wrong, by a factor of 165 after 40 epochs. For non-convex losses the bound grows polynomially in the number of steps, so treat it as a direction, not a number.

7.7. An unstable algorithm: 1-nearest neighbour

The nearest neighbour (NN) rule with one neighbour, 1-NN, predicts the label of the closest training point. Like a lookup table that stores the training set, it has training error 0 and, under the 0-1 loss, stability $\beta = 1$. That is because replacing a point by the same point with the opposite label flips the prediction there. So with noisy labels its gap never closes, and in the running example its test error never falls below 0.338. In scikit-learn, `KNeighborsClassifier(n_neighbors=1)` is this rule, and its perfect training score is a warning, not a result.

7.8. Overfitting is memorization of individual examples

Each term of Equation 7.2 measures how much one example depends on its own presence, and it is called the example's **memorization score**, or **self-influence**,

$$\mathrm{mem}_i(S) = \ell(A(S^{(i)}), z_i) - \ell(A(S), z_i). \tag{7.8}$$

Their average is the expected gap, so the scores show which examples carry it.

To measure a score, train on S and record the example's loss. Then replace the example by a fresh draw, retrain with the same settings, and record its loss again. That costs one retraining per example, which is affordable only for small models. For a large model, score a sample of examples, or train many models on random subsets and compare each example's loss with and without it.

The scores are uneven in a predictable way. A typical example is fitted well whether or not it is present, because its neighbours carry the same information. An atypical one, such as a mislabelled

point or a member of a rare subgroup, is fitted only if it is there itself. In the running example, the examples whose label goes against the trend have three times the average score of the others. Regularization flattens that profile. A small average gap therefore leaves room for a few heavily memorized examples, so a small train-test gap does not mean that no example was memorized. Look at the largest scores, not only the mean, and see Part 3 for a stronger, worst-case guarantee. Feldman (2020) shows that on data with a long tail of rare subgroups some memorization is needed for accuracy, so a large score is not always a defect.

7.9. Choosing what to measure after looking

A held-out set is honest only for questions fixed before you look at it. Call a **question** a rule that gives each example a score between 0 and 1, such as whether a model classifies it correctly. Questions fixed before the holdout set is drawn are safe. For some number k of them on n examples, their largest error is on average at most $\sqrt{\ln(2k)/(2n)}$. But you usually look at a verdict, change something, and ask again. Each new question then depends on the holdout set, so you have become a learning algorithm trained on it, and the bound no longer holds.

Freedman's paradox shows the size of the effect. A holdout set has 1,000 examples, each with many random features and a label independent of them all, so no rule beats 50% on new data. An analyst keeps the features that agree with the label noticeably more or less than half the time, and measures their majority vote on the same holdout set. With 1,000 features, the questions fixed in advance stay within the bound of 0.0616. But the vote, chosen after looking, reports an accuracy of 0.8053.

The fix is **sample splitting**, which means never letting an example answer a question it helped to choose. Choosing the features on one half and measuring the vote on the other brings the accuracy back to 0.5018. You need a fresh part of the data for every round of choices, not for every question. So split the data with scikit-learn's `train_test_split` before you start, and tune with `cross_val_score` on the training part. Then use the test set once, for the final verdict.

7.10. Code

This calculator gives the stability of regularized ERM and of SGD, and prints the numbers quoted above.

```
import math


def erm_beta(lipschitz: float, lam: float, n: int) -> float:
    """Uniform stability of ridge-regularized ERM."""
    return 2 * lipschitz**2 / (lam * n)


def sgd_beta(lipschitz: float, step: float, epochs: float) -> float:
    """Stability bound for SGD with a constant step on a convex, smooth, Lipschitz loss."""
    return 2 * lipschitz**2 * step * epochs
```

```
print(f"{erm_beta(1, 0.1, 1000):.3f}")  # 0.020
print(sgd_beta(1, 0.25, 1))  # 0.5
print(sgd_beta(1, 0.25, 40))  # 20.0
```

7.11. Cheat sheet

Table 7.1.: The results of this chapter.

Result	Conditions	Statement
Gap identity	Examples drawn independently from one distribution	Expected gap = average memorization score
Expected gap	β-uniformly stable	$\lvert\mathbb{E}[\text{gap}]\rvert \le \beta$
Regularized ERM	Convex, L-Lipschitz loss	$\beta = 2L^2/(\lambda n)$
SGD	Convex, smooth, L-Lipschitz, small steps	$\beta \le 2L^2\eta E$ for E epochs
Fixed questions	k questions fixed before the holdout set is drawn	Largest error $\le \sqrt{\ln(2k)/(2n)}$ on average

Rules of thumb

- Measure the gap on held-out data. The two knobs that shrink it are the regularization strength and the training time. For ERM, β is proportional to $1/\lambda$, and for SGD to ηE.
- A training error of 0 is a warning, not a result.
- A small train-test gap does not rule out memorized outliers. The atypical examples are memorized most, so inspect the largest scores.
- A holdout set that helped choose a question is a training set for it. Never tune on the test set, give each round of choices its own data, and use the final test set once.
- Do not report a generalization bound without checking that it is below the loss you measure.

7.12. Further reading

- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): uniform stability and the high-probability bound.
- Hardt, Recht & Singer, "Train Faster, Generalize Better: Stability of Stochastic Gradient Descent" (ICML 2016): SGD stability and early stopping.

Derivations. Every result here is derived step by step in the full edition's chapter.

8. Cryptography basics

8.1. Why this matters

To build or review a system that hides data, you need to know which tool does the job, how much data one key can protect, and what a security claim promises. The last question has a precise answer, because security is a game. A challenger secretly flips a fair coin to pick one of two worlds, such as encrypting one message or another. An **adversary** sees the result and guesses which world it is in. A scheme is secure when no efficient adversary beats the coin by more than a tiny amount.

8.2. Which tool for which job

Never build your own scheme. Use a vetted library, and pick the construction by the job, as in Table 8.1.

Table 8.1.: The construction to use for each job, and the mistake to avoid.

Job	Use	In Python	What goes wrong
Encrypt data	Authenticated encryption with associated data (AEAD): AES in GCM (Galois/Counter Mode), or ChaCha20-Poly1305	`AESGCM` or `ChaCha20Poly1305` from the `cryptography` package	A nonce reused under one key reveals data
A keyed pseudonym or tag	A pseudorandom function, in practice the hash-based message authentication code (HMAC)	`hmac.new(key, msg, hashlib.sha256)`	A plain hash of an identifier is reversed by hashing every candidate
Keys, nonces, identifiers	A cryptographic random number generator	`secrets.token_bytes`, `secrets.token_hex`, `secrets.randbelow`	The `random` module is predictable
Split a value among holders	Additive secret sharing modulo a prime	`secrets.randbelow(p)`	Shares not reduced modulo p leak

Job	Use	In Python	What goes wrong
Compute on encrypted values	Lattice-based homomorphic encryption	Microsoft SEAL, its Python wrapper TenSEAL, or OpenFHE	Every operation spends a noise budget

8.3. Advantage

The simplest adversary sees one sample and answers with a bit. It is called a **distinguisher**, written A, and its answer 1 means "I think this sample came from the second world". Write X and Y for the distributions of the sample in the two worlds. The **advantage** of the distinguisher is the gap between how often it says 1 in the two worlds:

$$\mathrm{Adv}_A(X, Y) = \big|\Pr[A(X) = 1] - \Pr[A(Y) = 1]\big|. \tag{8.1}$$

A distinguisher whose answer 1 points to the second world wins the coin game with probability $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}_A$, so the advantage is twice its edge over a coin flip. For example, take a fair coin against a coin that lands heads with probability 0.6. Saying 1 on heads has advantage 0.1, and so it wins with probability 0.55.

No distinguisher, however long it runs, can do better than the **total variation (TV) distance** between the two distributions (Divergences in the full edition).

So if that distance is 0, security is **perfect**, and if it is tiny, it is **statistical**. Most of cryptography is **computational** instead, where no efficient adversary can find where the two worlds differ.

8.4. Efficient adversaries and concrete security

Textbooks define security asymptotically, requiring a **negligible** advantage, one that eventually falls below one over every polynomial in the key length. But "eventually" says nothing about the key length you use, so never read an asymptotic claim as a concrete one.

Practitioners use **concrete security** instead. A scheme has **κ-bit security** if no attack does better than trying keys by brute force over 2^κ keys. An attacker that runs for time t then gets an advantage of at most about

$$\frac{t}{2^\kappa}. \tag{8.2}$$

Take an attacker who tests 10^{18} keys per second for a year. Against a 128-bit key, it succeeds with probability only 9.3×10^{-14}. That is why AES uses keys of 128 bits or more.

8.5. Hybrid arguments

Proofs compare two worlds through a chain of intermediate worlds called **hybrids**, each differing from the next in one thing, and the advantages of the steps add up. So if one sample cannot be told apart, a number m of independent samples cost at most m times the one-sample advantage, a safe but pessimistic factor.

8.6. Pseudorandom generators and functions

A **pseudorandom generator (PRG)** stretches a short random seed into a long string that looks random to any efficient observer. Stream ciphers such as ChaCha20 are treated as PRGs. A **pseudorandom function (PRF)** is a keyed function, written F_k for the key k, whose outputs look like a random table to anyone without the key. HMAC is assumed to be one, which is what makes a keyed pseudonym safe. A plain hash has no key, so anyone can hash every possible identifier and match the results.

8.7. The birthday bound and the data limits of a key

Many arguments fail only if two random values coincide. Write q for the number of values drawn independently and uniformly from a set of size N. The **birthday bound** says that the chance of a repeat is

$$1 - e^{-q(q-1)/(2N)} \le \Pr[\text{collision}] \le \frac{q(q-1)}{2N}. \tag{8.3}$$

A collision becomes as likely as not after about $1.18\sqrt{N}$ draws, so random values of κ bits start to collide after about $2^{\kappa/2}$ draws. This needs uniform values, because others collide more often.

Collisions come early in practice. With the hashing trick, for example, 1,000 features hashed into about a million buckets collide with probability 0.379. Random 32-bit identifiers reach even odds of a clash at 77,164 records, as the snippet computes. So size identifiers until $q^2/(2N)$ is tiny for the largest q you expect.

The same bound limits how much data one key may protect, because treating a block cipher as a PRF costs the birthday bound on the blocks processed. With 128-bit blocks, this **switching lemma** reaches one half at 2^{64} blocks, so change keys long before that, and ciphers with 64-bit blocks need new keys far sooner.

8.8. Encryption

The **one-time pad** encrypts a message m as $m \oplus K$, the bitwise exclusive or (XOR) with a uniformly random key K as long as the message. That gives **perfect secrecy**, but only with a key as long as all the data. It must also never be reused, because the XOR of two ciphertexts under one pad is the XOR of the two messages.

Practical schemes use short keys and aim for security under **chosen-plaintext attack (CPA)**. In its game, the adversary may ask for encryptions of any messages it likes, then guesses which of two messages of equal length a challenge encrypts. Ciphertexts usually reveal length, so pad messages when their length is sensitive.

The game forces encryption to be randomized. If the same message always gave the same ciphertext, the adversary would ask for the encryption of its first message and compare, with advantage 1. Deterministic tokens of email addresses can even be reversed, because anyone who can compute the token reverses it by running a dictionary of addresses through it. The fix is a fresh random value for every encryption, called a **nonce** and written r, drawn from a set $\mathcal{R}$

of possible values. The PRF's output on r then serves as a one-time pad, and the ciphertext is $(r, m \oplus F_k(r))$. After q requested encryptions, its advantage is at most

$$2\,\mathrm{Adv}_{\mathrm{PRF}} + \frac{q}{|\mathcal{R}|}. \tag{8.4}$$

The second term is a real attack, because a repeated nonce reuses a pad.

Over the life of a key, any two nonces may repeat, so the birthday bound sets the limit. Take the 96-bit nonces of AES in GCM. If they are random, the chance of any repeat after 2^{32} messages is about 1.2×10^{-10}, and it grows with the square of the count. So rotate keys well before that, or use a counter as the nonce. And since a CPA-secure scheme does nothing against tampering, use authenticated encryption.

8.9. Simulation and secret sharing

Many procedures among several participants must reveal the result and nothing else, and **simulation** makes "nothing else" precise. Write $\mathrm{View}(x)$ for everything a participant sees on the inputs x, and $L(x)$ for the leakage, what it is allowed to learn. The procedure reveals nothing more if an efficient **simulator** S, given only $L(x)$, can fake a view with the same distribution:

$$S(L(x)) \equiv \mathrm{View}(x). \tag{8.5}$$

Whether revealing $L(x)$ is acceptable is a separate decision.

Additive secret sharing is the basic example. Take a number h of holders and a prime p. To split a secret x, draw $h-1$ shares uniformly at random modulo p, and choose the last share so that all of them add up to x:

$$s_h = x - (s_1 + \cdots + s_{h-1}) \bmod p. \tag{8.6}$$

Any $h-1$ shares are then independent uniform numbers whatever the secret is, so a simulator just outputs random numbers and the security is perfect. Holders can add their shares of two secrets to get shares of the sum. Part 3 builds on this, and the sum itself is never hidden.

The wrap-around modulo p is a condition, not a detail, because without it the range of a share moves with the secret. With a mask uniform on 0 to 15 and no reduction, the shares of the secrets 0 and 7 have a TV distance of 7/16. So choose p larger than any sum you will compute, and reduce every result modulo p.

8.10. Learning with errors

Learning with errors (LWE) is the hard problem behind post-quantum key exchange and homomorphic encryption. It hides a secret vector s behind slightly wrong linear equations modulo p, and each sample pairs a uniformly random vector a with a number

$$b = \langle a, s \rangle + e \bmod p. \tag{8.7}$$

Here $\langle a, s \rangle$ is the inner product, and e is a small random error drawn fresh for each sample.

Regev encryption shows how such schemes compute on ciphertexts. It hides a bit in a fresh sample, and decryption works while the error stays below a quarter of p. Adding ciphertexts adds the bits, but it also adds the errors. So if every error lies within an error bound B of zero, a sum of k ciphertexts is guaranteed to decrypt when

$$k < \frac{p}{4B}. \tag{8.8}$$

With $p = 4096$ and $B = 32$, the guarantee covers 32 ciphertexts, but independent errors partly cancel, so failures become measurable only around 256.

This **noise budget** is the central parameter of homomorphic encryption. SEAL, TenSEAL and OpenFHE implement schemes such as BFV for exact integers and CKKS for approximate real numbers. In each, a larger modulus buys more operations at the cost of bigger, slower ciphertexts.

8.11. Code

This snippet shares and adds secrets, and sizes identifiers and nonces.

```
import secrets


def union_bound(q: int, n: int) -> float:
    """Upper bound on the chance that q uniform draws from n values repeat."""
    return q * (q - 1) / (2 * n)


def share(secret: int, holders: int, p: int) -> list[int]:
    """Split a secret into additive shares modulo p."""
    shares = [secrets.randbelow(p) for _ in range(holders - 1)]
    return [*shares, (secret - sum(shares)) % p]


def reconstruct(shares: list[int], p: int) -> int:
    return sum(shares) % p


def even_odds_draws(n: int) -> int:
    """Smallest number of uniform draws from n values with a repeat at least half the time."
    no_repeat, q = 1.0, 0
    while no_repeat > 0.5:
        no_repeat *= 1 - q / n
        q += 1
    return q


p = 2**61 - 1
```

```
a, b = share(42, 3, p), share(100, 3, p)
print(reconstruct(a, p))  # 42
print(reconstruct([x + y for x, y in zip(a, b)], p))  # 142
print(even_odds_draws(2**32))  # 77164
print(f"{union_bound(2**32, 2**96):.1e}")  # 1.2e-10
```

8.12. Cheat sheet

Table 8.2.: The results of this chapter.

Result	Statement
Advantage	Wins with probability $\frac{1}{2} + \frac{1}{2}\text{Adv}$; at most the TV distance
κ-bit security	Time t buys an advantage of at most about $t/2^{\kappa}$
Birthday bound	Collisions likely after about $\sqrt{N}$ draws
CPA security	Deterministic schemes fail; so does a repeated nonce
Additive sharing mod p	Any $h-1$ shares are uniform; without reduction mod p they leak

Rules of thumb

- Never build your own scheme, and draw keys, nonces and shares from `secrets`, never from `random`.
- Random values of κ bits collide after about $2^{\kappa/2}$ draws. Size identifiers, nonces and key lifetimes accordingly.
- Pseudonymize identifiers with a keyed function such as HMAC. A plain hash of an email or a phone number is not anonymization.
- Never read an asymptotic security claim as a concrete one.
- Secret sharing hides the inputs, never the result.

8.13. Further reading

- Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed., 2020), ch. 2–3: perfect secrecy, pseudorandomness and CPA security.
- Lindell, "How to Simulate It — A Tutorial on the Simulation Proof Technique" (2017): the simulation paradigm step by step.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part II.

Threat models and attacks

This part teaches you to attack your own models and releases before someone else does. You need the attacks even if you plan to use differential privacy, because they are how you check that an implementation keeps its promise. Every attack here is a game, in which a model or a release depends on a hidden secret and an adversary sees what the threat model allows. Its guess counts only by how much it beats a baseline that never looks at the model.

Start with Threat models and privacy games in the full edition, and then read Membership inference in the full edition. After those two, pick the chapter whose secret matches your question. For a trained model, membership inference asks whether one record was used, and Attribute inference in the full edition seeks a person's sensitive field. Property inference in the full edition seeks a fact about the whole training population, and Memorization and extraction in the full edition seeks text the model memorized. In federated learning, Gradient inversion in the full edition rebuilds a client's input from its updates.

For released data, read the last three chapters in order, because together they tell one story about releasing less. Records without names still fall to linkage (Linkage and re-identification in the full edition). Records hidden in crowds still give away sensitive values (Syntactic anonymity in the full edition). And accurate aggregates alone can give away the whole dataset (Reconstruction attacks in the full edition). This is the Fundamental Law of Information Recovery: answering too many statistics too accurately reveals the data. So "we removed the names" and "we only release aggregates" are not anonymization, and the next part starts from differential privacy instead.

Two lessons drive every audit in this part. First, report the true positive rate at a low false positive rate, such as one in a thousand, not accuracy or the area under the curve. Second, the worst-off records leak first, such as outliers, duplicates and people with a unique combination of attributes, so an average over all records misleads. A failed attack proves little, because an attack only gives a lower bound on the leak.

At the end of this part you will be able to write a threat model, run a membership audit you can defend, test a table for unique records, and decide whether a dataset called anonymized really is. The full edition's scripts in `code/part02-attacks/` reproduce every number at toy scale.

9. Threat models and privacy games

9.1. Why this matters

A hospital trains a model on patient records and offers it to doctors through a web service. Whether the model is private has no answer until you say **against whom**. One attacker reads only predicted labels, while another holds the weights, knows the other patients or slips crafted records into the training data. A **threat model** pins these assumptions down. Without one, "the model leaks" is a claim nobody can test.

Once the threat model is fixed, an attack becomes a **game**. The adversary guesses a hidden **secret**, such as whether one patient's record was used, from what the threat model lets it see. This chapter gives you a worksheet for the threat model, the baseline every attack must beat, and the way to report a result. Behind all three is one idea: an attack counts only by how much it beats a guess made without the model.

9.2. Writing down the threat model

Before you run or read an attack, answer five questions.

1. **Goal: what is the secret?** A membership attack asks whether a record was in the training data, and an attribute attack seeks a sensitive field of a known record. A reconstruction attack seeks the records themselves, and a property attack a statistic of the whole dataset.
2. **Access: what does the adversary observe?** The weakest is label-only access, the model's decisions on inputs the adversary chooses. Next is black-box access to scores such as confidences or losses, then white-box access to the weights. The strongest is the **transcript**, everything training exposed along the way, such as per-step gradients.
3. **Behaviour: does it follow the rules?** An honest-but-curious adversary only analyses what it legitimately sees. A **malicious** one may choose its queries adaptively or insert crafted records into the training data, which is called **poisoning**.
4. **Trust: who sees the raw data?** In the **central** model a trusted curator holds the data and the adversary sees only the release. In the **local** model nobody is trusted, so each person's data are protected on their own device before collection.
5. **Side information: what does it know already?** It may know the population, a public dataset, some or all of the other records, or the target's non-sensitive fields.

For example, a service that returns only the top class gives label-only access, while published weights give white-box access to anyone. A federated-learning server holds the transcript of per-client updates, and it may be malicious. One model can sit in several such deployments at once, and its risk is set by the most exposed one.

Access is ordered, because each level can compute the one below it. A label is a function of the scores, and the scores are a function of the weights. So for the same model and side information, the best attack can only get stronger along the chain

$$\text{labels} \;\le\; \text{scores} \;\le\; \text{weights} \;\le\; \text{transcript}. \tag{9.1}$$

The same holds for side information and behaviour, since an adversary can always forget what it knows or choose not to deviate. It also means that rounding or noising an output can never make the best attack stronger, though it may not weaken it much either.

The chain compares the *best* attack at each level, and the attacks people run are often far from it. Against a simple anomaly detector, calibrating each target's score against how typical the target is raised a black-box attack's advantage from 0.0407 to 0.2789, the white-box optimum. So "the attack we tried failed" proves little. Membership inference in the full edition builds that calibration into a practical attack.

Side information is the axis you control least, so assume an adversary who knows every record except the target's unless you can argue otherwise. Trust decides who the adversary is. If the curator is inside the threat model, protection must happen before the data leave the user (Local, shuffle and distributed trust models).

9.3. Scoring an attack against its baseline

State every attack in the same four parts: the challenger, the secret, the adversary's view and its guess. The simplest games ask the adversary to guess one hidden bit. In the membership game, the bit says whether the target's record is in the training set, which makes the target a **member**.

Two rates describe an attack. Its **true positive rate (TPR)** is the fraction of members it flags, and its **false positive rate (FPR)** is the fraction of non-members it wrongly flags. Neither rate depends on how often members occur, but the win rate does, which is why it misleads. Write π_1 for the probability that the target is a member and $\pi_0 = 1 - \pi_1$ for the opposite. The attack then wins with probability

$$\Pr[\text{win}] = \pi_1 \,\text{TPR} + \pi_0 \,(1 - \text{FPR}). \tag{9.2}$$

If only one target in ten is a member, always saying "not a member" wins 90% of the games. So an attack that is right "90% of the time" may reveal nothing. So score every attack against a **baseline**, the best score of a guess that never looks at the model.

With a fair prior, half members and half non-members, the baseline is one half. Write the attack's **advantage** as $\text{Adv} = \text{TPR} - \text{FPR}$. The win rate is then

$$\Pr[\text{win}] = \frac{1}{2} + \frac{1}{2}\,\text{Adv}. \tag{9.3}$$

So an attack with 55% accuracy at a fair prior has advantage 0.1. Whether 55% is a problem depends on where those wins come from, which the next section shows how to read. Because both rates are measured inside one world, the advantage does not change with the prior. No attack's advantage exceeds the total variation distance between the adversary's views in the two worlds (Divergences), which differential privacy later bounds.

9.4. Reporting: TPR at a low FPR

The advantage is the best height of the attack's **receiver operating characteristic (ROC)** curve above the diagonal. It says nothing about the region where false positives are rare, which is where privacy breaks. An attack that flags 1% of members with no false alarms has a tiny advantage, but it identifies those members with certainty. But the 55%-accurate attack of the last section may flag members and non-members at similar rates, and then it identifies nobody with confidence. So Carlini et al. (2022) report membership attacks by their TPR at a low FPR such as 10^{-3}, or lower, on log axes. This part does the same. The full ROC ceiling of the two worlds is their trade-off curve, which Gaussian differential privacy turns into a guarantee.

A flagged person cares about a third number, the chance that the verdict is right. That is the **precision**,

$$\Pr[\text{member} \mid \text{flagged}] = \frac{\pi_1 \,\mathrm{TPR}}{\pi_1 \,\mathrm{TPR} + \pi_0 \,\mathrm{FPR}}. \tag{9.4}$$

When members are rare, even a small FPR applied to the many non-members swamps the true positives (Jayaraman et al., 2021). So a precision is meaningless without the prior it assumes.

Take a count of people with a condition, released with Gaussian noise of standard deviation 1, and an adversary who knows the other records. The count is centred on 50 without the target and on 51 with it. The best attack thresholds the count, and its advantage is 0.3829. Held to an FPR of 10^{-3}, it catches members with probability 0.0183. That is fewer than 2% of members, but when it fires at a fair prior its precision is 0.948. If only one target in a hundred is a member, the precision drops to 0.156.

A report you can act on states the following.

- The threat model, from the worksheet above, and the game.
- The TPR at a stated low FPR, 10^{-3} or lower, and the ROC curve on log axes.
- The number of non-members the FPR rests on, and a confidence interval for both rates, such as a Clopper–Pearson interval (Hypothesis testing).
- The baseline, and the prior behind any precision.
- Whether targets were random records or chosen worst cases.

To compute these, scikit-learn's `roc_curve` returns the FPR and TPR at every threshold from labels and scores, and `auc` gives the area under that curve. Then `numpy.interp` reads off the TPR at your target FPR. `scipy.stats.binomtest(k, n).proportion_ci(method="exact")` gives the Clopper–Pearson interval for a count. An FPR of 10^{-3} rests on few false positives unless you have far more than 1,000 non-members. To see a given number of false positives at an FPR α, you need about that number divided by α.

9.5. Average case and worst case

An evaluation can draw the data and the target from the population, which gives the **average case**. Or it can let the adversary know or pick the other records and the target, which gives the

worst case. Audits of real models measure the first, but a guarantee must control the second. The best advantages are ordered:

$$\text{does not know the others} \;\le\; \text{knows them} \;\le\; \text{chooses them.} \tag{9.5}$$

Not knowing the other records acts like extra noise that blurs the two worlds.

The gap can be large. Suppose the noisy count has 100 other records. The adversary who knows them reaches an advantage of 0.3829, and one who does not reaches only 0.0779. But a malicious adversary who planted the other records knows them, so a defence that relies on the crowd fails against exactly the adversary it most needs to stop.

Averaging over targets hides the worst-treated person in the same way. A mechanism that publishes one record chosen at random from 1,000 has an average-case advantage of only 0.001, yet every run exposes one person completely.

So treat an average-case number as a lower bound on the risk, never as an upper bound. Audits narrow the gap by planting **canaries**, records built to be as exposed as possible (Memorization and training-data extraction in the full edition). The definition of differential privacy bounds the worst case directly, for every pair of datasets that differ in one record.

9.6. Code

The snippet computes the best TPR at any FPR, and the precision, when the attack's score is standard normal for non-members and shifted by 1 for members, as in the noisy count.

```
from statistics import NormalDist

Z = NormalDist()


def tpr_at_fpr(fpr: float, shift: float) -> float:
    """Best TPR at a given FPR when the attack score is N(0, 1) for non-members, N(shift, 1)
    return 1 - Z.cdf(Z.inv_cdf(1 - fpr) - shift)


def precision(tpr: float, fpr: float, prior: float) -> float:
    """Fraction of 'member' verdicts that are right when a fraction `prior` of targets are m
    return prior * tpr / (prior * tpr + (1 - prior) * fpr)


advantage = 2 * Z.cdf(0.5) - 1  # best TPR - FPR for a shift of 1: the total variation dista
tpr = tpr_at_fpr(1e-3, 1.0)
print(f"{advantage:.4f}")  # 0.3829
print(f"{tpr:.4f}")  # 0.0183
print(f"{precision(tpr, 1e-3, 0.5):.3f}")  # 0.948
print(f"{precision(tpr, 1e-3, 0.01):.3f}")  # 0.156
```

9.7. Cheat sheet

Table 9.1.: The results of this chapter. Φ is the standard normal distribution function, `scipy.stats.norm.cdf`.

Quantity	Formula	Use it for
Advantage	$\mathrm{TPR} - \mathrm{FPR}$	Comparing attacks at any prior; 0 means no attack
Precision	$\pi_1\mathrm{TPR}/(\pi_1\mathrm{TPR} + \pi_0\mathrm{FPR})$	What a flagged person faces; state the prior
Access chain	labels $\leq$ scores $\leq$ weights $\leq$ transcript	More access never weakens the best attack
Knowledge	not knowing $\leq$ knowing $\leq$ choosing the other records	An average-case number is a lower bound
Noisy count, rest known	$2\Phi(\frac{1}{2\sigma}) - 1$	Advantage on one person's bit, noise σ
Noisy count, m others unknown	$\approx 2\Phi(1/(2\sqrt{m/4 + \sigma^2})) - 1$	The unknown others add variance $m/4$

Rules of thumb

- Fill in the five-question worksheet before running or reading any attack. Two audits with implicit threat models cannot be compared.
- Compare every attack with its baseline. Accuracy near the prior is no attack, and accuracy or the area under the curve alone says nothing.
- A balanced test set hides the precision at a realistic prior, so state the prior with any precision.
- Check what really leaves an "API only" system, because logits, embeddings and shared checkpoints move it up the access chain.
- A better attack on the same view can matter more than more access, so calibrate scores per target.
- Assume the adversary knows the other records, and audit with worst-case canaries.

9.8. Further reading

- Salem et al., "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023): every inference attack written as a game. Read §§3–5.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): why attacks should be reported by TPR at a low FPR on log-scaled ROC curves.

Derivations. Every result here is derived step by step in the full edition's chapter.

10. Membership inference

10.1. Why this matters

A hospital publishes a model that predicts a disease, and you hold a patient's record. **Was that record in the training set?** If it was, the patient was treated there for that disease. This is **membership inference**, the standard measure of leakage, since a model that cannot hide whether a record was used will hide little else.

The one idea of this chapter is that membership is a test about one record. Some records are predicted well whether or not the model trained on them, so their outputs reveal almost nothing. Others are predicted well only if it did. An attack that applies one rule to every record mixes the two kinds and identifies almost nobody. The attack to use in an audit is the likelihood ratio attack (LiRA). It compares the model's output on a record with how that same record fares under models trained with and without it.

10.2. Membership as a test about one record

The game is the one of Threat models and privacy games in the full edition. The attacker gets a training member or a fresh record with equal chance. It is scored by its **true positive rate (TPR)** and **false positive rate (FPR)**, and its advantage is the TPR minus the FPR.

Fix one target record, written z, and look at the model's output on it, such as its loss. Over many training runs with z, that output has one distribution, and over runs without z it has another. Call them the **IN** and **OUT** distributions of z. The best test thresholds their likelihood ratio (Hypothesis testing).

A simple case sets the scale. Suppose the attack's score is normal with standard deviation 1 and mean 0 for non-members, and shifted for members by an amount s, called the **separation**. Then, writing Φ for the standard normal cumulative distribution function and α for the FPR you allow, the best TPR is

$$\mathrm{TPR}(\alpha) = 1 - \Phi(\Phi^{-1}(1-\alpha) - s). \tag{10.1}$$

For example, at an FPR of 10^{-3} a separation of 0.5 gives a TPR of only 0.0048, but a separation of 4 gives 0.8185. Typical records have small separations and outliers have large ones, so the low-FPR end of any attack is made of outliers.

The numbers below come from the full edition's experiment, which trains logistic regression on random halves of a pool in which 5% of points are outliers. The attacker attacks 128 target models, with 64 shadow models of its own.

10.3. The quick check: the generalization gap

Before any attack, compare the training and test errors. A model fits its training data, so its loss is lower on members. Yeom et al. (2018) showed that the attack "member if correctly classified" has an advantage equal to the gap between the errors:

$$\text{Adv} = \text{test error} - \text{training error}. \tag{10.2}$$

In the running example, the gap between the two errors, and so the advantage, is 0.1168.

This gives a lower bound for free, since a model with a generalization gap leaks membership by at least the gap. The converse fails, because the gap is an average over records, and an average hides who is exposed. Yeom et al. even construct models that leak with no gap at all, so **a small gap does not mean a private model.** At a low FPR the loss attack is near chance, as Table 10.1 shows.

The classic attack of Shokri et al. (2017) trains **shadow models**, copies of the target trained on the same population, and learns to tell their members from non-members. It learns a rule that treats all records alike, so it is mostly superseded, but shadows remain the way to calibrate a threshold for the target.

10.4. LiRA: a likelihood-ratio test per example

LiRA, from Carlini et al. (2022), estimates each record's own IN and OUT distributions from shadow models trained with and without it. Its statistic is the logit of the model's confidence. Writing p_y for the confidence in the true label y, the logit is

$$\phi = \ln \frac{p_y}{1 - p_y}. \tag{10.3}$$

LiRA uses it because it is close to normal over retrainings, while the confidence itself piles up against 1. For each record, fit a normal distribution to its logit under the shadow models that trained on it, with the mean μ_{in} and the standard deviation σ_{in}. Fit another to the shadow models that did not, with mean μ_{out} and standard deviation σ_{out}.

Online LiRA scores the target model's output ϕ_z by how much more likely it is under the IN fit than the OUT fit. Writing $\mathcal{N}(\phi; \mu, \sigma^2)$ for the normal density, the score is

$$\ln \Lambda_z = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in}}, \sigma_{\text{in}}^2) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out}}, \sigma_{\text{out}}^2). \tag{10.4}$$

Offline LiRA uses only OUT models, which can be trained once and reused for every target record. It asks how many OUT standard deviations the output lies above its usual value,

$$\frac{\phi_z - \mu_{\text{out}}}{\sigma_{\text{out}}}. \tag{10.5}$$

Both put one threshold on the score for all records. Online LiRA is stronger, because it weighs each record's evidence by its own separation, while offline LiRA is cheaper. Table 10.1 compares them with the older attacks by their area under the curve (AUC) and by their TPR at a low FPR.

Table 10.1.: Every attack on the running example: 128 target models, 64 shadow models, 64,000 member and 64,000 non-member pairs.

Attack	AUC	TPR at FPR 10^{-3}
Loss threshold (Yeom)	0.577	0.0009
Shadow attack model (Shokri)	0.577	0.0008
LiRA offline	0.632	0.0030
LiRA online	0.629	0.0149
LiRA online, global variance	0.623	0.0177

The table shows why the AUC is the wrong summary, because by AUC the attacks are close. But at an FPR of 10^{-3} the loss and shadow attacks are at chance, since their TPR is no higher than the FPR. Online LiRA's TPR is 17.0 times the loss attack's there, although both use the same statistic, so comparing each record with itself is the whole gain.

The main practical failure is noisy fits. Each record's variance is estimated from half the shadow models. An underestimated variance makes an ordinary output look like a far-tail event, and that produces false positives exactly at the strict thresholds that matter. Carlini et al. therefore recommend one **global variance**, estimated from all records at once, when shadow models are few. For example, with 16 shadow models online LiRA reaches a TPR of only 0.0023 at an FPR of one in a thousand with per-record variances. With a global variance it reaches 0.0151.

10.5. Running LiRA on your model

The full-scale benchmark, CIFAR-10 with 16 shadow models, takes hours of GPU time, but the recipe is the same for any model.

1. **Split the data.** For each of $N + 1$ models, where N is the number of shadows, choose a random half of the training pool. One model plays the target, and rotating that role adds pairs.
2. **Train** every model with exactly the target's recipe, augmentation included.
3. **Compute the logit** for each model and record. Compute it from the class scores f before the softmax, as $f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, which never forms $1 - p_y$ and so never rounds to 0 for a confident model. Carlini et al. also query a few augmented copies of each input, such as flipped images, and fit over all of them.
4. **Fit** IN and OUT normals for each record over the shadows, with a global variance if you have fewer than about 64.
5. **Score** each pair of model and record with Equation 10.4, or with Equation 10.5 if you only have OUT models.
6. **Evaluate.** Pool all pairs, sweep one threshold, and plot the receiver operating characteristic (ROC) curve on log axes.

The cost is $N + 1$ training runs, so the shadow count is the budget decision, and a global variance makes 16 shadows enough.

The members and non-members must come from the same distribution, and violating this is the most common way a membership audit goes wrong. If non-members come from a later period,

another site or another preprocessing pipeline, the attack measures that shift rather than the model. So draw both from one pool, split at random, as step 1 does.

When shadows cost too much, there are cheaper options. Offline LiRA reuses one set of OUT models. Reference-model attacks, such as the relative membership inference attack of Zarifzadeh et al. (2024), are designed to stay strong with only a few reference models. Against a service that returns labels only, a label-only attack measures how much an input must be perturbed before its label flips (Choquette-Choo et al., 2021).

For the computation, `scipy.stats.norm.logpdf` gives the two terms of the online score, and `sklearn.metrics.roc_curve` gives the curve. TensorFlow Privacy's membership-inference tests take losses or logits of members and non-members in an `AttackInputData` object, and `run_attacks` runs simple threshold attacks on them. LiRA's reference implementation lives in the same repository. The Adversarial Robustness Toolbox offers `MembershipInferenceBlackBox`, a trained attack model, and `LabelOnlyDecisionBoundary`, and ML Privacy Meter implements reference-model attacks.

10.6. Reading the result

Report the TPR at an FPR of 10^{-3} or lower, with the ROC curve on log axes (Threat models and privacy games in the full edition), because averages hide breaches. An attack that exposes 5% of members with certainty can have the same AUC as one that nudges every score slightly, yet catch 38 times as many members.

Measuring a low FPR takes many non-members, because an FPR is estimated by counting false positives. A count of k events has a relative error of about $1/\sqrt{k}$. So to see k false positives at an FPR α, you need about k/α non-members. For example, the running example's 64,000 non-members give about 64 false positives at one in a thousand. Even then, the Clopper–Pearson 95% interval for online LiRA's FPR runs from 0.00077 to 0.00128.

Then check three things.

- **The TPR's interval is clear of the FPR.** If not, the attack is at chance there, like the loss attack in Table 10.1.
- **The top-scored records make sense.** Inspect them, because they are usually outliers, rare subgroups or mislabelled points.
- **The thresholds were set on separate data.** Setting them on the data that measure the rates is usual but optimistic.

10.7. Who is exposed, and what helps

The exposed records are those whose IN and OUT distributions lie far apart. A record's separation is large when the rest of the data contradict its label or its features are unusual. In the running example, at an FPR of one in a thousand, a share of 0.873 of the members online LiRA finds are outliers.

That points to what does and does not help.

- **Removing the exposed records does not.** Carlini et al. (2022b) removed the most exposed records and retrained, and the next layer became exposed. They call this the onion effect.
- **Closing the generalization gap is not enough.** More data or stronger regularization shrink the average, but the low-FPR end is set by the worst records.
- **Masking the output does not bound the leak,** because label-only attacks still work.
- **Differential privacy does.** It bounds every record's separation at once. Private training provides it, and an audit with planted canaries checks it.

Expect a trade-off, because memorizing rare cases can be what makes a model accurate on long-tailed data (Feldman, 2020).

10.8. Cheat sheet

Table 10.2.: Which attack or check to use.

Situation	Use	Notes
A first look, no attack budget	Test error − training error	A lower bound on the leak; a small gap proves nothing
An audit with few shadow models	Online LiRA with a global variance	16 shadows are usable this way
A label-only service	A label-only attack	Masking the scores does not stop membership inference

Rules of thumb

- Audit with LiRA, not with a global loss threshold, which is near chance at a low FPR.
- Draw members and non-members from one pool, split at random, and train shadows with the target's exact recipe.
- Overfitting implies membership leakage, but no overfitting does not imply its absence. A failed weak attack is not evidence of safety.

10.9. Further reading

- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022): LiRA online and offline, the global variance, and the protocol to follow in an audit.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018): the loss attack and its link to the generalization gap.

Derivations. Every result here is derived step by step in the full edition's chapter.

11. Attribute inference and model inversion

11.1. Why this matters

A hospital's public model predicts a lung disease from a patient's age band and whether they smoke. An insurer knows your age and, from a claim, that you have the disease. It can use the model to guess whether you smoke, but that alone is not a privacy failure, since a published study on smoking would allow the same guess. The question that matters is this. **Does the model reveal more about your attribute because your record was in its training data?**

Attribute inference tries to recover one sensitive field of a record, such as smoking status or a genotype, from its other fields and a model. The one idea of this chapter is to separate two kinds of success. **Population inference** works equally on everyone, and no defence that limits what one record contributes can stop it. The individual leak is the extra success on people who were in the training data.

11.2. The attribute game

"The attack recovered 78% of smoking statuses" means nothing on its own, so Yeom et al. (2018) defined a game (Threat models and privacy games in the full edition). In it, the adversary wants the sensitive attribute s and already knows the public features x. It also sees the label y, which the model predicts.

The challenger trains a model, then picks the target either from the training set, making it a **member**, or fresh from the population, with equal chance. The adversary sees the target's x and y and queries the model, and then guesses s. The score is the **attribute advantage**,

$$\mathrm{Adv}^{\mathrm{attr}} = \Pr[\text{guess right} \mid \text{member}] - \Pr[\text{guess right} \mid \text{non-member}]. \tag{11.1}$$

Population inference helps members and non-members equally, so it cancels in the difference, and what is left is what the model knows about its own training records.

A toy experiment shows how small the advantage can be. Against 200 small logistic regressions, model inversion is right on 0.7832 of members. Yet the attribute advantage is only 0.0569. Most of the attack's 78% would be there even if the person had never been in the training data.

Any attribute attack is also a membership attack that says "member" when its guess is right, with the same advantage. So a model that leaks nothing about membership leaks nothing about attributes, though the converse fails. As a membership test it is weak, because in the toy experiment being right is only 1.08 times as likely for a member as for a non-member.

11.3. The imputation baseline

The **imputation baseline** is the success of the best possible guess of s from x and y, made by someone who knows the population exactly. Jayaraman & Evans (2022) found that many published attacks do no better than imputation, so this is the number to beat.

The best guess for each pair of x and y is the most probable value of s, the maximum a posteriori (MAP) guess. Write $p(s, x, y)$ for the probability that a record from the population has these three values. The MAP guess then succeeds with probability

$$\text{baseline} = \sum_{x,y} \max_{s} p(s, x, y). \tag{11.2}$$

This is also a ceiling for non-members. A model trained without the target is independent of it, so no attacker does better on a non-member. Attacks that ignore the model have zero advantage, because without the model a member is just another draw from the population.

Table 11.1 works this out for the smoking example. Smokers make up 30% of the population and tend to be older, and the disease is more likely with age and with smoking.

Table 11.1.: The smoking table: the joint distribution of smoking status s, age band x and disease y, with the column maxima in bold and the probability of smoking in each column.

$p(s, x, y)$	young, no disease	young, disease	old, no disease	old, disease
non-smoker ($s = 0$)	**0.378**	0.042	**0.196**	0.084
smoker ($s = 1$)	0.072	**0.048**	0.054	**0.126**
Pr[smoker $\mid x, y$]	0.1600	0.5333	0.2160	0.6000

With no observation, the best guess is "non-smoker", right 70% of the time. But the chance of smoking crosses one half in the two disease columns, so there the MAP guess becomes "smoker". Its success is the sum of the bold entries, 0.748.

Every one of these numbers comes from the population, not from any model. So dropping the sensitive column from the model's inputs does not stop this inference, because the correlates in x and y remain. As Dwork & Roth (2014) put it, a study showing that smoking causes cancer discloses a fact about the world, not about its participants. Do not count such a finding as a breach. Population inference can still hurt, but it is a question for policy and fairness.

In practice you never know $p(s, x, y)$, so estimate the baseline from data. Take records from the same population that the model never saw, then fit the best imputer you can on half of them and measure its success on the rest. With a few discrete fields, the imputer is a table of the most common s in each cell, as in the snippet of Section 11.7. For continuous fields, train instead any good classifier to predict s from x and y, such as scikit-learn's `HistGradientBoostingClassifier`. The Adversarial Robustness Toolbox packages this as `AttributeInferenceBaseline`. Your imputer may fall short of the true ceiling, so the estimate errs low. An attack that seems to beat it on non-members points to a weak imputer, not a leak.

11.4. Model inversion: the MAP attack

The imputation guess needs the probability of the label given the other fields, $p(y \mid x, s)$, and a model that predicts the label from them estimates it. Fredrikson et al. (2014) called the attack built on this **model inversion**. Writing $f_y(x, s)$ for the model's confidence in the known label when it is fed the public features and a candidate value of s, the **MAP inversion attack** guesses

$$\hat{s}(x, y) = \arg\max_s \; p(s \mid x)\, f_y(x, s). \tag{11.3}$$

The first factor comes from the adversary's knowledge of the population, and using the overall frequency $p(s)$ instead throws away what the public features say. The second comes from one query per candidate, which against a scikit-learn model is a call to `predict_proba`, reading the column of the known label.

Take an old patient with the disease, and a calibrated model, one whose confidences equal the population's probabilities. An old person smokes with probability 0.3913, and the model gives the disease a probability of 0.7 for a smoker. So "smoker" scores 0.2739 against 0.1826 for "non-smoker", and the attack guesses "smoker". With a calibrated model, inversion simply *is* imputation, with zero advantage.

A trained model differs from the population in two ways. On members it is too sure of the truth, which is the generalization gap of Membership inference in the full edition seen through the attribute. On non-members its confidences are noisy, so its guesses fall below the ceiling. In the toy experiment, inversion beats the ceiling of 0.7517 on members but not on non-members. So measuring an attack only on fresh records understates what it achieves on members.

More data closes both gaps. With 25 training records the advantage is 0.1610, and with 1,600 it is only 0.0052.

The same idea can invert a whole input. For example, `MIFace` in the toolbox recovers a typical face of a class, which is a privacy problem when the class is one person.

11.5. Confidence masking

Many interfaces round their confidences or return only the top label, a defence that Fredrikson et al. (2015) suggested. Such **confidence masking** cannot help the best attacker, because whoever holds the full answers can apply the mask themselves. But it bounds nothing about the gap between members and non-members, which is the leak, and the gap can shrink or grow.

Table 11.2.: MAP inversion success through three interfaces, averaged over 200 models trained on 100 records each. Guessing from x alone gives 0.7045, and imputation with the true population 0.7517.

Interface	Members	Non-members	Advantage
Full confidences	0.7832	0.7262	0.0569
Confidences rounded to one decimal	0.7836	0.7253	0.0583
Top label only	0.7741	0.7092	0.0650

In Table 11.2, rounding changes almost nothing. With labels only, non-member success drops close to the guess from x alone, but member success drops less, so the advantage grows. Masking has hurt population inference more than the individual leak, and label-only membership attacks exploit the same asymmetry.

So don't count a drop in overall attack success as a privacy gain. Less overfitting shrinks the member gap, and differential privacy bounds it for every attacker and every output format.

11.6. Deciding whether there is a leak

An audit of attribute leakage follows four steps.

1. **Estimate the baseline** from held-out data, as in Section 11.3.
2. **Run MAP inversion**, or your strongest attack, on training members and on as many held-out non-members. The Adversarial Robustness Toolbox's `AttributeInferenceBlackBox` is a ready-made attack. For synthetic data, Anonymeter's `InferenceEvaluator` measures the same risk.
3. **Compare the two rates**, with a confidence interval for each. If member success is not clearly above non-member success, this attack finds no individual leak.
4. **Classify what remains.** Success up to the baseline is population inference, a policy question. Success on members beyond non-members is the individual leak, which more data or differential privacy addresses.

11.7. Code

The snippet estimates the imputation baseline from held-out records of the smoking population, and its estimate lands close to the exact 0.748.

```
import random
from collections import Counter
from collections.abc import Hashable

P_SMOKER = 0.3
P_OLD = {0: 0.4, 1: 0.6}  # Pr[old | s]
P_DISEASE = {(0, 0): 0.1, (0, 1): 0.3, (1, 0): 0.4, (1, 1): 0.7}  # Pr[disease | s, old]
Record = tuple[int, Hashable]  # (sensitive value s, the fields the attacker sees)


def draw(rng: random.Random) -> Record:
    s = int(rng.random() < P_SMOKER)
    old = int(rng.random() < P_OLD[s])
    return s, (old, int(rng.random() < P_DISEASE[(s, old)]))


def imputation_baseline(fit: list[Record], test: list[Record]) -> float:
    """Learn the most common s for each observed cell on `fit`, and return its success on `t
```

```
    counts: dict[Hashable, Counter[int]] = {}
    for s, seen in fit:
        counts.setdefault(seen, Counter())[s] += 1
    fallback = Counter(s for s, _ in fit).most_common(1)[0][0]
    guess = {seen: c.most_common(1)[0][0] for seen, c in counts.items()}
    return sum(guess.get(seen, fallback) == s for s, seen in test) / len(test)


rng = random.Random(0)
held_out = [draw(rng) for _ in range(20_000)]
print(f"{imputation_baseline(held_out[:10_000], held_out[10_000:]):.4f}")  # 0.7468
```

11.8. Cheat sheet

Table 11.3.: The results of this chapter.

Quantity	Formula or rule	What it tells you
Attribute advantage	Success on members − success on non-members	The individual leak; report both rates too
Imputation baseline	$\sum_{x,y} \max_s p(s, x, y)$	Population inference; estimate on held-out data
MAP inversion	$\hat{s} = \arg\max_s p(s \mid x)\, f_y(x, s)$	One query per candidate value; use $p(s \mid x)$, not $p(s)$
Masking	Cannot help the best attacker	But the member/non-member gap can grow

Rules of thumb

- Report success on members, on non-members, and the imputation baseline. One success rate, such as 80%, says nothing about privacy until you know the other two.
- An attribute attack is a weak membership test, so measure membership with a dedicated attack.
- Rounding confidences or returning labels only changes the attack, not what the model memorized.

11.9. Further reading

- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015): inversion from confidences, face reconstruction, and rounding as a countermeasure.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022): the imputation baseline, and what it takes to beat it.

Derivations. Every result here is derived step by step in the full edition's chapter.

12. Property inference

12.1. Why this matters

A hospital trains a readmission model on its 1,000 patients and shares it with a research partner. Suppose you have checked that no attack on the model can tell, much better than a coin flip, whether any one patient was in the training set. The partner can still learn whether 45% or 55% of the hospital's patients are over 65, and be right almost every time.

None of these secrets is about a single record. Each is a **property** of the training set as a whole, such as the fraction of records in some subgroup. **Property inference** is the attack that recovers it from a trained model. The one fact to keep from this chapter is that a property changes many records at once, so it moves the model far more than any single record does. That is why defences sized for one record do not cover it, and why the attack gets *easier*, not harder, as the dataset grows.

12.2. The property game and its ceiling

Threat models and privacy games in the full edition writes every attack as a game. In the property game, a challenger flips a fair bit and trains a model on records drawn with one of two subgroup shares, t_0 or t_1, depending on the bit. The attacker knows both shares, has data from both groups and can train models the way the victim does. It sees the weights or only the model's answers, and guesses the bit. Each dataset carries a single secret, so the natural score is the **accuracy**, the chance that the guess is right. It is tied to the true positive rate (TPR) and the false positive rate (FPR) by

$$\text{accuracy} = \tfrac{1}{2}(1 + \text{TPR} - \text{FPR}), \tag{12.1}$$

so a coin flip scores one half.

First ask whether the property is a secret at all. Learning that smokers get more lung cancer is the purpose of a medical study, not a breach. A property is a harm to reveal only when the property describes the *data owner* rather than the world, such as one hospital's patient mix or one lab's machine configuration. When the dataset is a small sample of a public population, its property is public anyway.

There is a ceiling on what any attacker can learn. When each record shows its group, no model can leak more than the count of subgroup records, so the best advantage is the total variation (TV) distance between the two distributions of that count (Hypothesis testing). Write n for the number of records. For the hospital's 1,000 patients the ceiling is 0.9985, so it is no comfort, only a scale against which to read a measured attack. An attacker who can add a few records of its own to the training set can make a property far easier to infer (Mahloujifar et al. 2022).

12.3. Why a property leaks when no record does

The simplest release, the count of subgroup records, shows why. One attacker asks whether one subgroup record is in the data, while another asks whether the share was t_0 or t_1. Write t for the population share in the first question. Then the best membership advantage is about

$$\mathrm{Adv}_{\mathrm{mem}} \approx \frac{1-t}{\sqrt{2\pi(n-1)t(1-t)}}, \tag{12.2}$$

which falls like $1/\sqrt{n}$. Write $\bar{t}$ for the midpoint of the two shares and Φ for the standard normal distribution function. The best property advantage is about

$$\mathrm{Adv}_{\mathrm{prop}} \approx 2\Phi\Big(\frac{\sqrt{n}\,|t_1-t_0|}{2\sqrt{\bar{t}(1-\bar{t})}}\Big) - 1, \tag{12.3}$$

which rises towards 1.

Table 12.1.: Exact best advantages from a released count of subgroup records, which the formulas match closely. Property: $t_0 = 0.45$ against $t_1 = 0.55$. Membership: one subgroup record, population share $t = 0.5$.

n	Property	Membership
100	0.6827	0.0398
1,000	0.9985	0.0126
10,000	1.0000	0.0040

In Table 12.1, at 10,000 records, the attacker who asks about one patient is right with probability 0.502, while the one who asks about the share is never wrong to four decimals.

Two practical rules follow. Size and gap enter only through $\sqrt{n}\,|t_1 - t_0|$, so doubling the gap has the same effect as four times as much data. And moving from share t_0 to t_1 changes about $n|t_1 - t_0|$ records, which for the hospital is 100 of its 1,000 patients. Differential privacy (DP) protects one record at a time. Its group-privacy guarantee for a group of records multiplies the privacy level ε by the size of the group, here 100 (Properties of differential privacy). So a model trained with DP at a useful ε still reveals the property, and a small membership risk says nothing about it either.

12.4. Running the attack: shadow models and a meta-classifier

Nobody can write down the distribution of a real model's weights, so the attacker samples it instead (Ateniese et al. 2015). The recipe is the shadow-model idea of Membership inference in the full edition, with whole models as the data points.

1. From auxiliary data, draw many datasets of the victim's size with share t_0 and as many with share t_1.
2. Train a **shadow model** on each one, with the victim's architecture, hyperparameters and training code.

3. Turn each shadow model into a feature vector, either its parameters or its outputs on fixed probe inputs.
4. Train a classifier, the **meta-classifier**, to tell the two shares apart from those vectors, and measure its accuracy on fresh shadow models.
5. Apply it to the victim's model.

A linear meta-classifier is a sensible default. Models fitted by minimizing an average loss on many records have roughly normal parameters, and for those the best possible test is linear. In scikit-learn, `LogisticRegression` from `sklearn.linear_model` serves well, and `LinearDiscriminantAnalysis` from `sklearn.discriminant_analysis` is the linear test itself. Fit either on the shadow features with the share as the label, and read its accuracy with `score` on held-out shadow models. No mainstream privacy library ships a standard property-inference attack, so this short pipeline is the usual tool.

```
meta = LogisticRegression(max_iter=1000).fit(shadow_features, shadow_shares)
print(meta.score(heldout_features, heldout_shares))   # attack accuracy
```

The full edition also predicts how well the best attack can do. Write θ' for the slope of the fitted parameters, how fast they move as the share changes, and Σ_1 for their noise, the covariance that one record would give. Their ratio is a constant κ, and for a small gap the best accuracy is about

$$\text{accuracy} \approx \Phi(\kappa\sqrt{n}\,|t_1 - t_0|/2), \qquad \kappa = \sqrt{\theta'^{\top}\Sigma_1^{-1}\theta'}. \tag{12.4}$$

The released count has $\kappa = 2$ at a midpoint of one half, so a model leaks as much as the count would on a fraction $(\kappa/2)^2$ of the records. If the property does not change what the model fits, nothing leaks to first order. If it does, the leak is a side effect of fitting the data well, and no care in the training code removes it.

For example, the full edition's logistic regression never sees the group but has $\kappa = 0.647$, so it leaks the share about as well as one record in ten would.

What goes wrong is mostly on the attacker's side. With few records and a small gap, noise in the shadow models hides the shift, so the measured attack falls short of the best one and a weak result is not evidence that the model is safe. Report the measured accuracy, and the predicted one next to it when you can estimate κ. The auxiliary data must also match the victim's population, or the meta-classifier learns the difference between the populations rather than the share.

12.5. Neural networks: choose the right representation

A neural network can be written down in many ways. Swap two hidden units, together with their incoming and outgoing weights, and the network computes the same function. With the tanh activation, flipping the signs of one unit's weights also changes nothing. Training treats all these equivalent weight vectors alike, so every raw weight has the same mean under both shares, and a meta-classifier fed the flattened weights sees nothing (Ganju et al. 2018).

The fix is a representation that ignores the order of the units. The full edition compares four on small tanh networks, for shares 0.2 against 0.8. A linear meta-classifier on raw parameters scores 0.497, which is chance, while one on the network's outputs at 16 fixed probe inputs scores 1.000.

Sorting the units by their outgoing weights recovers most of the signal, but it is fragile when two units have nearly equal weights. Sums over the units, the idea behind DeepSets (Zaheer et al. 2017), need no sorting and extend to deeper networks layer by layer. Probes do best, because the property changes the function the network computes and probes measure it directly.

Probe outputs need only query access, so a model behind an application programming interface is not safe either, and Zhang, Tople & Ohrimenko (2021) attack properties through queries alone.

12.6. Defences and their limits

Because a property is a large group of records, there is no cheap defence. A released statistic hides it only if its noise is on the scale of $n|t_1 - t_0|$ records, and that much noise also destroys what the statistic was for. Removing the group column does not help, because whatever correlates with the group still moves the model.

If the property is a secret (Section 12.2), the strongest option is not to release a model trained on that mix, and query access is no way out. Pooling data with other owners is no fix, because Zhang, Tople & Ohrimenko (2021) attack exactly that multi-party setting, and in federated learning the same attack can target one client's updates. If you must release, run the attack recipe with your own data as the auxiliary data. Then report its accuracy for the shares an adversary would plausibly compare, and state in the release notes that DP protects individuals only.

12.7. Code

The functions below compute the ceiling, the TV distance between the two binomial counts, and the best membership advantage, and print numbers of Table 12.1.

```
from math import exp, lgamma, log


def binom_pmf(n: int, t: float) -> list[float]:
    """Pr[Bin(n, t) = k] for k = 0..n, in log space."""
    return [exp(lgamma(n + 1) - lgamma(k + 1) - lgamma(n - k + 1) + k * log(t) + (n - k) * l


def property_ceiling(n: int, t0: float, t1: float) -> float:
    """Best advantage of any property attack when the group is visible: TV(Bin(n, t0), Bin(n
    return 0.5 * sum(abs(p - q) for p, q in zip(binom_pmf(n, t0), binom_pmf(n, t1)))


def membership_best(n: int, t: float) -> float:
    """Best advantage for one subgroup record from the same count: (1 - t) max_k Pr[Bin(n -
    return (1 - t) * max(binom_pmf(n - 1, t))
```

```
print(f"{property_ceiling(100, 0.45, 0.55):.4f} {property_ceiling(1000, 0.45, 0.55):.4f}")
print(f"{membership_best(100, 0.5):.4f} {membership_best(10000, 0.5):.4f}")  # 0.0398 0.0040
```

The full edition's script, `code/part02-attacks/property_inference.py`, runs the shadow-model attacks and reproduces every other number in this chapter.

12.8. Cheat sheet

Table 12.2.: The results of this chapter.

Question	Answer
How to score an attack	Accuracy $= \frac{1}{2}(1 + \text{TPR} - \text{FPR})$; a coin flip scores $\frac{1}{2}$
How the leak scales	Through $\sqrt{n}\,\lvert t_1 - t_0\rvert$; grows with n, while membership falls like $1/\sqrt{n}$
Predicted accuracy of a model	$\Phi(\kappa\sqrt{n}\,\lvert t_1 - t_0\rvert/2)$; the count has $\kappa = 2$ at a midpoint of $\frac{1}{2}$
Which representation for a network	Probe outputs first, then sums over units or canonical order; never raw weights
What DP gives	Protection for one record; little for a property, which changes $n\lvert t_1 - t_0\rvert$ records

Rules of thumb

- First ask whether the property describes the data owner. If it describes the world, it is not a secret.
- Record-level DP, or a low membership risk, does not imply property privacy.
- A meta-classifier at chance on raw weights means the representation is wrong, not that the model is safe.
- Shadow datasets must come from the target's distribution, or the attack learns the wrong difference.

12.9. Further reading

- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018): the practical attack on neural networks, neuron sorting and the DeepSets meta-classifier.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021): black-box property inference, including properties of attributes that were removed from the training data.

Derivations. Every result here is derived step by step in the full edition's chapter.

13. Memorization and training-data extraction

13.1. Why this matters

A language model trained on scraped text sees phone numbers, addresses, keys and private messages. Ask it to continue "Anna Smith's phone number is", and it may finish the sentence with the real number. Nobody queried a database, and no breach leaked anything, because the number is stored in the model's weights and comes out through its normal interface. This is **training-data extraction**. Membership inference in the full edition needs the record in hand to ask whether it was used, while extraction hands the attacker the record itself.

One idea runs through the chapter. Fix the context in which a secret appears, say "my pin is", and list every possible filler from the most likely to the least likely under the model. The **rank** of the true secret is its position in that list, which is also the number of guesses an attacker needs if it tries the fillers in that order. A model has memorized a string when it ranks it far above the strings that could have stood in its place.

13.2. Which memorization to measure

The notions of memorization below give rates that differ by orders of magnitude, so a report must say which one it measures. All use greedy decoding, which writes out the most likely next token at each step.

Table 13.1.: The notions of memorization and when to use each. Discoverable implies extractable, and verbatim implies approximate.

Definition	The string counts as memorized if	Attacker it fits	Use it for
Discoverable (Carlini et al. 2023)	Its training prefix makes greedy decoding produce it	One who holds the prefix	Measuring memorization on a model you own
Extractable (Carlini et al. 2021)	Some prompt, any at all, makes greedy decoding produce it	One who has only the model	The real risk; found by attacks, not measured
k-eidetic	It is extractable and appears in at most k documents	As extractable	Flagging the dangerous case, $k = 1$
Approximate (Ippolito et al. 2023)	Greedy decoding comes within a set edit distance of it	Any reader of the output	Near copies, such as a number with one digit changed

Discoverable memorization is the one you can measure. Carlini et al. (2023) prompt with 50 tokens of a training sequence, decode 50 tokens greedily and count exact matches. Treat any verbatim rate as a lower bound on the leak. Ippolito et al. (2023) show that a filter blocking verbatim training text is easy to bypass with prompts that change the style, while approximate copies still come out.

Deployed systems sample rather than decode greedily, which turns memorization into a matter of budget. A sample at temperature 1 writes out a string x after a prompt with the model's probability p of that string. Write β for the chance of failure the attacker accepts. Then a number m of independent samples suffices when

$$m \geq \frac{\ln(1/\beta)}{p}. \tag{13.1}$$

For example, a string of probability 10^{-3} needs about 4,600 samples for 99% confidence. So report a sampling probability or a budget alongside the greedy rate, because a string that greedy decoding misses can still come out after a few thousand queries. Measure with the temperature and decoding settings you deploy.

13.3. What gets memorized: copies, model size and rare data

Carlini et al. (2023) ran that test on open models, and the largest reproduced at least 1% of its training data. Memorization grew log-linearly with three things, so it rises by about the same amount each time one of them is multiplied by a fixed factor.

The first is the number of copies of a string. Kandpal, Wallace & Raffel (2022) found that a sequence repeated ten times is generated about a thousand times more often than one seen once.

The second is model size. A larger model separates contexts more finely, so rare strings end up with parameters of their own, and they are memorized from fewer copies. The third is prompt length, since a longer prefix pins down the context. Rare data are exposed first, because an atypical example can be fitted only by memorizing it (Algorithmic stability and generalization).

These findings give the order of defences. **Deduplication** is the cheapest, because it lowers the copies of every repeated string. Lee et al. (2022) found that models trained on deduplicated data emit memorized text about ten times less often without losing quality, and released their exact-substring tool as `deduplicate-text-datasets`. For near duplicates, MinHash with locality-sensitive hashing is the usual method, for example `MinHash` and `MinHashLSH` in the `datasketch` package. Deduplication does nothing for a string seen once in a large model, though, because there one copy is enough.

The second defence is to remove personal data before training. A detector of personally identifiable information (PII), such as Microsoft's Presidio, finds names, phone numbers and similar fields so you can mask them. It misses whatever its patterns do not cover, so measure what remains with canaries rather than trusting the filter. Filtering the model's outputs is weaker still, because approximate copies pass a verbatim filter. The only defence with a guarantee for singletons is training or fine-tuning with differential privacy.

13.4. Auditing with canaries and exposure

Discoverable memorization is all or nothing, while an auditor wants a graded answer. Carlini et al. (2019) proposed planting **canaries**, random secrets of a fixed format such as "my pin is" followed by five random digits. You know each canary's value and the number of values it could have taken, so after training you can see exactly where the model ranks it. Write $\mathcal{R}$ for the set of possible values, $|\mathcal{R}|$ for their number and $\hat{r}$ for the planted value. The **exposure** is the number of bits of search the model saves an attacker:

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{13.2}$$

Each bit halves the guesses the attacker needs, and an exposure near the maximum, $\log_2 |\mathcal{R}|$, means the secret can actually be extracted.

What makes exposure an audit tool is its known behaviour when nothing was memorized. A canary that was never inserted lands at a uniformly random rank, so for any threshold of t bits,

$$\Pr\left[\text{exposure}(\hat{r}) \geq t\right] \leq 2^{-t}. \tag{13.3}$$

So $2^{-\text{exposure}}$ bounds a p-value, the chance that an uninserted canary would look at least this memorized. The average exposure of an uninserted canary is about 1.44 bits.

The recipe for a real model follows.

1. Draw a few dozen canaries in a format your data could contain, each with a fresh random value. Insert each one a chosen number of times, from once to many.
2. Train as usual, and keep some canaries drawn but never inserted as a control.
3. For each canary, score its value and many random values of the same format by the model's log-probability after the prefix. With Hugging Face `transformers`, pass the tokens as `input_ids` and as `labels`, with the prefix positions of `labels` set to -100. The returned `loss` is then the mean cross-entropy over the value's tokens.
4. Estimate the rank as the share of random values that score at least as high as the canary, and convert it to exposure.
5. Check that the controls average about 1.44 bits, then report the exposure against copies. TensorFlow Privacy ships secret-sharer utilities that compute exposure, but the snippet in Section 13.6 does the same.

Three things go wrong. First, scoring m random values cannot resolve exposures above $\log_2 m$ bits, because a canary that beats them all only tells you that its rank fraction is below about $1/m$. Carlini et al. (2019) extrapolate by fitting a skew-normal to the scores, because a plain normal fit overestimates exposure. Second, ties hide memorization, because when every candidate gets the same score, every exposure is 0. Third, the canary format must look like your data, or the model may treat it as unlike anything it learned. Comparing two training runs, such as with and without deduplication, then tests the defence.

13.5. Extraction without the prefix: generate, rank, verify

A real attacker has only the model. Carlini et al. (2021) extracted data from GPT-2 in three steps. The attacker generates many samples, ranks them by how likely each is to be memorized,

and checks the top of the ranking against public sources. The ranking is the hard step, because a model finds a string likely either because it is ordinary text or because it was memorized. A **reference model**, trained on similar data but not on the target's training set, stands in for ordinary text. Write $p_\theta(x)$ for the target's probability of a string x, and $p_{\text{ref}}(x)$ for the reference's. Rank by the log-likelihood ratio

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x), \tag{13.4}$$

which is the best possible ranking for a fixed rate of false alarms (Hypothesis testing).

Raw likelihood puts first the strings that any model finds likely, such as software licences and boilerplate, which reveal nothing, and the ratio cancels them. A crude reference works. Carlini et al. (2021) used a smaller model of the same family, and also the target's own likelihood on lowercased text and the compressed length of the text, `len(zlib.compress(text.encode()))` from Python's standard library. With Hugging Face `transformers`, `model.generate` with `do_sample=True` draws the samples, and passing `output_scores=True` and `return_dict_in_generate=True` lets `model.compute_transition_scores` return each token's log-probability.

An attacker without the prefix extracts less than the discoverable rate suggests, but production models are still not safe. Nasr et al. (2023) asked ChatGPT to repeat one word forever, and about 200 US dollars of queries extracted over 10,000 unique memorized examples, so alignment hides memorization from casual use without removing it.

13.6. Code

These functions compute exposure, its p-value bound and the sample budget for an audit.

```
from math import ceil, log, log2


def exposure(rank: int, space: int) -> float:
    """Bits of guessing the model saves: log2 |R| - log2 rank."""
    return log2(space) - log2(rank)


def null_p_value(exposure_bits: float) -> float:
    """Upper bound on the chance that a canary never inserted scores this high."""
    return 2.0**-exposure_bits


def samples_bound(p: float, beta: float) -> int:
    """Samples that extract a string of probability p, except with probability beta."""
    return ceil(log(1 / beta) / p)


print(f"{exposure(1, 10**5):.2f}")  # 16.61
print(f"{null_p_value(10):.5f}")  # 0.00098
print(samples_bound(1e-3, 0.01))  # 4606
```

The full edition's script, `code/part02-attacks/memorization.py`, runs every experiment in this chapter.

13.7. Cheat sheet

Table 13.2.: The results of this chapter.

Question	Answer
Which rate to report	Discoverable for your own model, approximate as well as verbatim, with the decoding method stated
Samples to extract a string of probability p	$\ln(1/\beta)/p$
What drives memorization	Copies, model size and prompt length, each log-linearly; rare data first
Exposure of a canary	$\log_2\lvert\mathcal{R}\rvert - \log_2$ rank; at most 16.61 bits for five digits
Is the exposure significant	p-value at most $2^{-\text{exposure}}$; uninserted canaries average 1.44 bits
Ranking generations	$\ln p_\theta(x) - \ln p_{\text{ref}}(x)$, never raw likelihood
Defences, in order	Deduplicate, scrub PII, then train with differential privacy for a guarantee on singletons

Rules of thumb

- A verbatim rate is a lower bound on the leak, not a measure of it.
- Deduplicate first, but know that neither deduplication nor a PII filter removes a secret seen once.
- Plant canaries at several copy counts, not only once, with uninserted controls, and score enough candidates to resolve the exposure you care about.
- Test with the decoding settings you deploy, and count approximate as well as verbatim matches.

13.8. Further reading

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, exposure and how to estimate it.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): how memorization scales with duplication, model size and prompt length.

Derivations. Every result here is derived step by step in the full edition's chapter.

14. Gradient inversion

14.1. Why this matters

A phone keyboard learns from what its users type, and the raw text never leaves the phones. Instead, a server sends the current model to each phone, the phone computes the gradient of the loss on its own few examples, and it sends back only that gradient. The server averages the gradients of many phones and takes a training step. This is **federated learning (FL)**, often called private because "the data stay on the device".

This chapter asks what a single client's gradient reveals about its data, and the answer is often everything. A gradient holds one derivative for every parameter, and many of them are simple functions of the input, so the input can be read back. Averaging over a batch protects much less than people expect. So treat a per-client update as being as sensitive as the data it came from.

14.2. The attacker and what it sees

The attacker is the server. It is **honest-but-curious** in the sense of Threat models and privacy games in the full edition, so it follows the protocol, sees each client's own update and knows every parameter, since it chose them. A client holds a batch of B examples, each an input with its label. Write θ for all the parameters, and $\ell(\theta; x_i, y_i)$ for the loss on example i with input x_i and label y_i. The client sends the average gradient

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x_i, y_i). \tag{14.1}$$

The numbers in this chapter come from the full edition's experiments, on small grey-scale images and a network with one hidden layer of 32 units. Its gradient gives 2,212 numbers, one per parameter, against 64 unknown pixels per image.

14.3. Exact recovery from a fully connected layer

The most common first layer, a fully connected layer with a bias, gives its input back by one division. Each unit j multiplies the input x by its weights and adds a bias. Write δ_j for the rate at which the loss responds to that unit's output, its backward signal. The chain rule then makes the weight gradient and the bias gradient

$$\nabla_W \ell = \delta \, x^\top, \qquad \nabla_b \ell = \delta. \tag{14.2}$$

So row j of the weight gradient is the input scaled by δ_j, and entry j of the bias gradient is that same scale. For a single example, dividing one by the other gives the input exactly, for every unit whose bias gradient is not zero:

$$x = \frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j}. \tag{14.3}$$

Everything above the layer, including the depth of the network, the loss and the label, sits inside the scale δ_j and cancels. So the attack needs no optimization and no knowledge of the data, and it checks itself, because every usable unit gives the same answer. In the experiment, every switched-on unit gives back the image to rounding error. At a deeper fully connected layer, the same division gives that layer's input.

Switched-off ReLU units give nothing, but one active unit is enough. Without a bias, every row still points along x, so a one-dimensional search for the scale finds it. What stops the division is a convolutional first layer, because its filter is reused at every position of the image, so its gradient is a sum that no division separates. For such models the attacker searches instead (Section 14.5).

14.4. Label recovery from the last layer

The label is often the most sensitive part of an example, such as a diagnosis. It can be read off the last layer without any search (Zhao, Mopuri & Bilen 2020). The rule needs a last layer with a bias, followed by the softmax and the cross-entropy loss. Write p for the predicted probabilities and e_y for the vector with a 1 at the position of the label y. The last layer's bias gradient is then $p - e_y$, which is negative only at the true label. So for a single example

$$y = \arg\min_k (\nabla_{b_2} \ell)_k. \tag{14.4}$$

Without the bias, the same sign pattern shows in the last layer's weight gradient when the hidden values cannot be negative. The one failure is a very confident model, whose true-label probability rounds to exactly 1, so the sign is lost.

A batch hides which example has which label, but not how many examples there are of each class. Write n_k for the number of examples of class k, and p_{ik} for the probability the model gives that class on example i. Then

$$n_k = \sum_{i=1}^{B} p_{ik} - B\,(\nabla_{b_2} \ell)_k. \tag{14.5}$$

The server does not know the sum of predictions, but it knows the model. So it estimates the sum from its own data of the same kind, and rounds the result. With that estimate, every count came out exact in all batches of 4, but larger batches miss more often, and the share exact falls to 52.5% for batches of 64.

14.5. Deep Leakage from Gradients

When the division does not apply, the attacker searches. **Deep Leakage from Gradients (DLG)** (Zhu, Liu & Han 2019) guesses an input and computes the gradient it would produce. It then moves the guess until that gradient matches the observed one. Write R for the **matching**

loss, the squared distance between the two gradients. With the labels from Section 14.4, the attack looks for the guesses that make it smallest:

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \Big\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2 . \tag{14.6}$$

In PyTorch the attack is a short loop. Its key call is `torch.autograd.grad` with `create_graph=True`, which keeps the gradient differentiable so that the optimizer can follow the slope of R.

```
import torch
import torch.nn.functional as F

loss = F.cross_entropy(model(x_guess), y_known)
grads = torch.autograd.grad(loss, model.parameters(), create_graph=True)
R = sum(((g - g_obs) ** 2).sum() for g, g_obs in zip(grads, observed_grads))
R.backward()   # then one optimizer step on x_guess; repeat
```

Three things decide whether it works. The activation must have a slope that varies, so Zhu et al. replaced the ReLU by the sigmoid. The search can get stuck, because R is not convex, but the final value of R tells you which case you are in. A loss at rounding error means an exact match, while one that stops falling above it means a restart from a new random guess. Finally, a failed run is an optimization failure, not a lack of information. Geiping et al. (2020) compared gradients by direction and added a smoothness penalty, and recovered recognizable images from trained networks and from batches. In the full edition's experiment, one image comes back exact to about 13 digits after 600 steps of the Adam optimizer.

14.6. Batches and averaging

Clients usually send the average over a batch, or the parameter change after several local steps. Averaging breaks the division and makes the search harder, but much of the structure survives, because each example contributes one outer product to the first-layer gradient, so for a batch of B

$$\nabla_{W_1} \ell = \frac{1}{B} \sum_{i=1}^{B} \delta_{1,i}\, x_i^\top , \qquad \operatorname{rank} \nabla_{W_1} \ell \le B. \tag{14.7}$$

Every row of the gradient is a combination of the B inputs. So the server learns the few directions that every example is built from, which already rules out almost every input.

The division still recovers an example exactly when one ReLU unit is switched on for that example and off for the rest of the batch, because the unit then **isolates** the example. Suppose the m hidden units switched on like independent fair coins. Each example would then be isolated with probability $1 - (1 - 2^{-B})^m$, so an even chance at a batch of 10 needs only 710 units. Real data are worse than that formula, because images of the same kind make a unit fire on many examples or on few. With 1,024 units and a batch of 16, the formula predicts that 1.6% of the batch is exposed, while in fact 69.1% is. So wide layers with real data expose most examples of batches of 10 or more.

DLG suffers more from batching, and from a batch of 4 the median run gets stuck. That is a harder search, not less information, since the isolation attack recovered all four images of such a batch.

Several local steps, as in federated averaging, hamper the search but do not protect a single example. Every step's first-layer gradient is an outer product with the same input, so the division still works on the total change. And a malicious server that chooses the weights can isolate most examples of even a large batch by design. It plants units that fire only on rare inputs or sort inputs into narrow bins (Boenisch et al. 2023; Fowl et al. 2022).

14.7. What to do as a deployer

For an FL deployment, ask who can see one client's update, using Table 14.1.

Table 14.1.: Gradient leakage by FL setting, and the response.

Setting	What leaks to the server	What to do
One example per update, fully connected first layer with bias	The input exactly, and the label	Never deploy without protection
Small batches, any architecture	Labels or their counts, isolated examples, most of the rest by search	Treat each update as raw data
Large batches or several local steps	Label counts, the span of the batch, isolated examples	Not a defence on its own
Server not trusted to choose weights	Most examples of any batch	Verify the model clients receive
Updates protected by differential privacy (DP)	Bounded by the privacy level	The only protection with a guarantee

Two protections change the picture. **Secure aggregation** is a cryptographic protocol that lets the server see only the sum over many clients, never one client's update. That removes the per-client view that every attack here uses. It needs a minimum number of clients per sum, though, because a sum over very few clients is close to a single update. So larger cohorts help, and clients that drop out mid-round must not leave a small group exposed.

Secure aggregation alone is not enough, because the sum and the final model still leak and a server can still choose malicious weights. For a guarantee, add training with DP. In DP federated averaging, each client's update is clipped to a fixed norm and noise is added to the sum. That bounds what any update reveals, whoever sees it. Opacus provides per-example clipping and noise for PyTorch through its `PrivacyEngine`, and Flower and TensorFlow Federated both offer secure aggregation and differentially private aggregation of client updates. Heuristic defences, such as pruning small gradient entries or mixing inputs, raise the cost of the search but carry no guarantee.

To check your own setup, attack it with your own architecture, batch size and number of local steps. Geiping's open-source `breaching` framework implements the published attacks for PyTorch

models, and the full edition's script, `code/part02-attacks/gradient_inversion.py`, runs every attack in this chapter.

14.8. Cheat sheet

Table 14.2.: The results of this chapter.

Attack	Formula	Needs	Fails when
Exact recovery, one example	$x = (\nabla_{W_1}\ell)_{j,:}/(\nabla_{b_1}\ell)_j$	Fully connected first layer with bias	Convolutional first layer; batches, in part
Label, one example	$y = \arg\min_k(\nabla_{b_2}\ell)_k$	Softmax and cross-entropy	p_y rounds to 1
Label counts, batch	$n_k = \sum_i p_{ik} - B(\nabla_{b_2}\ell)_k$	Last-layer bias; an estimate of $\sum_i p_{ik}$	Large B with a poor estimate
Deep Leakage from Gradients	Minimize the matching loss R	A smooth network; the labels	Search stuck (final $R \gg 0$); larger batches
Span and isolation, batch	Rank $\leq B$; isolated examples recovered exactly	Fully connected first layer, ReLU units	Never safe; real data are isolated often
Secure aggregation alone	Hides single updates	Enough clients per sum	The sum and the model still leak; add DP

Rules of thumb

- "We share gradients, not data" is not privacy, because a per-client update is as sensitive as the data it came from.
- Do not rely on batch size or local steps, which leave the label counts, the span and every isolated example exposed.
- Do not trust a server that controls the model's weights.
- Secure aggregation needs large cohorts and safe handling of dropouts, and only DP bounds what any update reveals.

14.9. Further reading

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019): the gradient-matching attack, with reconstructions of images and text.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020): cosine matching with an image prior, and attacks on trained networks, local steps and batches.

Derivations. Every result here is derived step by step in the full edition's chapter.

15. Linkage and re-identification

15.1. Why this matters

A hospital wants to share its discharge records with researchers. It deletes the names but keeps the postcode, the date of birth, the sex and the diagnosis, because the research needs them. But anyone who holds a voter roll, with names next to those same three columns, can join the two tables and put a name back on most rows. This is a **linkage attack**, and putting a name back on a "de-identified" record is **re-identification**.

To judge whether a release is identifying, measure identifying information in bits, because bits add up. Write n for the number of people in a population. Singling out one of them takes $\log_2 n$ bits, which is 27.9 bits for the United States of 1990 and 32.9 bits for the whole world. An attribute with v equally likely values carries $\log_2 v$ bits, and the bits of independent attributes add. Once the bits an adversary knows exceed $\log_2 n$ by a few, almost everyone is unique.

15.2. Quasi-identifiers and the linkage attack

Direct identifiers, like a name or an email address, name a person on their own, while **sensitive attributes**, like a diagnosis, are what the release is about. **Quasi-identifiers** sit in between. None of them names anyone alone, but together they often single out one person, and they appear in other datasets next to names. The most common "de-identification" removes the direct identifiers and keeps the quasi-identifiers, which is exactly the gap the linkage attack uses.

To score a release you hold, group its n records into **equivalence classes**, the sets of records with identical quasi-identifiers, and write J for the number of classes. An adversary who knows the target is in the release, and knows the target's quasi-identifiers, finds a target alone in its class for sure and a target in a class of k records one time in k. Averaged over targets, its success probability is

$$\Pr[\text{win}] = \frac{J}{n}, \tag{15.1}$$

and no adversary that sees only the quasi-identifiers can do better.

For example, the hospital release in the Code section uses ZIP code, birth date and sex, and its records fall into 5 classes for a success of 0.625. Three of its records are unique, so anyone on a voter roll with one of those combinations has their diagnosis exposed with certainty. Latanya Sweeney estimated from the 1990 census that 87% of Americans were likely unique on those three attributes, because together they carry at most 31.1 bits. Coarser geography helps, and with the county in place of the ZIP code the figure fell to 18%.

If every class holds at least k records, the success is at most $1/k$, but a class whose records share one diagnosis still leaks it (Syntactic anonymity and how it breaks in the full edition).

15.3. Pseudonyms and hashed identifiers

Many releases replace each identifier by a **pseudonym**, a code that stays the same across a person's records, and such codes fail in two ways. The first is that an unkeyed hash, such as MD5, can be reversed whenever the possible identifiers can be listed. The adversary hashes every candidate once and looks each pseudonym up, which takes seconds to hours for phone, licence or identity numbers. The second failure survives any key. Because the pseudonym ties a person's records together, linking any one record to them reveals all the others.

New York City's 2013 taxi release showed both. Driver licences had been replaced by unkeyed MD5 hashes, and there were only about 2 million possible licences. So Vijay Pandurangan hashed every one in minutes, and public lists of licence holders put a name on every driver's trips and income. Passengers were exposed by the trips themselves, because a pickup minute and place single out one trip.

So treat a pseudonymized release as identifiable, as the GDPR does, since it counts pseudonymized data as personal data. If you must keep a join key, derive it with a keyed hash, a hash-based message authentication code (HMAC) such as Python's `hmac.new(key, msg, hashlib.sha256)`, and keep the key away from the data. That stops the dictionary attack, but the key still groups each person's records into a history that is itself identifying. Use a fresh key for each release, because one pseudonym reused across releases joins them.

15.4. Predicting uniqueness before a release

For a release you are still designing, you need a model rather than a count. Call a person's combination of attribute values their **cell**, write p_c for the probability of cell c, and assume that people fall in cells independently. The expected fraction of the n people who share their cell with nobody is then

$$\frac{\mathbb{E}U}{n} = \sum_c p_c (1 - p_c)^{n-1}. \tag{15.2}$$

When all cells are equally likely and there are 2^b of them, so that the attributes carry b bits, the formula becomes

$$\frac{\mathbb{E}U}{n} \approx \exp\big(-2^{\log_2 n - b}\big). \tag{15.3}$$

This fraction depends only on the bits beyond $\log_2 n$, and it is steep. Half the people are unique at 0.53 bits beyond it, and 99% at 6.64 bits.

Use Equation 15.3 for a first estimate. For the Sweeney attributes it predicts that 0.899 of Americans are unique, close to her 87%. Real attributes are skewed, and skew usually lowers uniqueness, so the uniform estimate is pessimistic. The full edition also gives a floor on uniqueness from the collision probability, the chance that two random people share a cell. Correlated attributes carry fewer bits than their sum. They need a model of the joint distribution, such as the Gaussian copula of Rocher, Hendrickx and de Montjoye (2019).

15.5. Sparse data: the Netflix attack

Ratings, purchases and locations have no quasi-identifier columns, but they are even more identifying. In 2006 Netflix released about 100 million movie ratings, with subscribers' names replaced by random numbers. Narayanan and Shmatikov (2008) matched its records to public reviews. Their attack scores how well each record r agrees with what the adversary knows, a few items i with approximate values aux_i:

$$\text{Score}(\text{aux}, r) = \sum_{i \in \text{aux}} w_i \, \text{Sim}(\text{aux}_i, r_i). \tag{15.4}$$

Here the agreement Sim is 1 when the record's rating r_i is within one star of the known value, and 0 otherwise. The weight w_i favours rare items, because agreeing on an obscure film says much more than agreeing on a hit. Write s_i for the fraction of records that rated item i. The weight $\ln(1/s_i)$ works well, and the item carries $\log_2(1/s_i)$ bits.

On Netflix, eight ratings identified 99% of records, even with two of them wrong and dates known only to within two weeks. Bits explain it again, since a film that 1% of subscribers rated carries 6.64 bits, and three such films single out one subscriber. Errors in the auxiliary data cost little because the score adds up evidence rather than demanding a perfect match.

The top score is the best guess only if the target is in the release. When it may not be, the **eccentricity test** declares a match only if the best score stands out. Write S_1 and S_2 for the two largest scores, and σ for the standard deviation of all scores. The test requires the gap to clear a threshold φ,

$$\frac{S_1 - S_2}{\sigma} \geq \varphi. \tag{15.5}$$

Calibrate the threshold on people known not to be in the data, because the published threshold of 1.5 falsely matched 26.3% of outsiders on the full edition's synthetic ratings. So always report the rate at a stated false positive rate (Hypothesis testing).

15.6. Does releasing a sample protect?

Statistical agencies often release only a sample, arguing that a sample-unique record may not be unique in the population. Write π for the **sampling fraction** and λ for the expected number of other people in the target's cell. Start from a known person who has exactly one match in the sample, and write κ for the probability that the match is right. Or start from a record alone in its cell in the sample, and write ξ for the probability that it is alone in the population too. For a rare cell,

$$\kappa \approx \frac{1}{1 + \lambda(1 - \pi)}, \qquad \xi \approx e^{-\lambda(1-\pi)}. \tag{15.6}$$

Since $1 - \pi$ is at most 1, sampling can never push them below $1/(1 + \lambda)$ and $e^{-\lambda}$, so it helps only when cells are crowded. For a rare cell, a match from a person is right with probability 0.910 even in a 1% sample.

In the full edition's synthetic population, 0.8301 of the sample-unique records in a 1% sample were still unique in the population, and the adversary can tell which ones from the sample itself. So estimate uniqueness for the population, never just for the sample. Rocher, Hendrickx and de

Montjoye predicted that 99.98% of Americans would be correctly re-identified in any dataset with 15 demographic attributes, and that sampling changes this very little. So a small sample protects only people who were not unique anyway.

15.7. A pre-release risk check

1. **List the quasi-identifiers.** Include every column an adversary could find elsewhere next to a name, such as demographics, dates, locations, employers and rare events like a hospital stay in the news.
2. **Measure the release.** Group by the quasi-identifiers and report J/n, the fraction of unique records and the fraction in classes smaller than your k. In pandas, `df.groupby(qi_cols).size()` gives the class sizes.
3. **Estimate the bits for planned releases.** Add $\log_2$ of each attribute's number of values, and if the total is several bits above $\log_2 n$, expect most people to be unique.
4. **Reduce the risk.** Coarsen geography and dates first, since they carry the most bits, then suppress the records that remain unique.
5. **Know when to stop.** If the analysis needs fine-grained or sparse data, release aggregates or a differentially private output instead (The definition of differential privacy).

Several tools automate these steps. ARX, an open-source Java application, measures risk and searches for generalizations, and the R package sdcMicro does the same from an object built with `createSdcObj`. To test a release by actually linking it, use the Python `recordlinkage` package, Splink, or Anonymeter's `SinglingOutEvaluator` and `LinkabilityEvaluator`. No tool knows what auxiliary data exist, so the choice of quasi-identifiers stays your judgement.

15.8. Code

The snippet computes the risk of a release you hold, and the uniform estimate of Equation 15.3 for Sweeney's attributes.

```
from collections import Counter
from math import exp, log2


def linkage_report(rows: list[tuple[str, ...]]) -> tuple[float, float, int]:
    """Linkage success J/n, fraction of unique records, and smallest class size of a release
    sizes = Counter(rows).values()  # one count per distinct quasi-identifier combination
    n = len(rows)
    return len(sizes) / n, sum(1 for k in sizes if k == 1) / n, min(sizes)



release = [
    ("02138", "1961-03-14", "F"), ("02138", "1972-11-02", "M"), ("02138", "1972-11-02", "M")
    ("02139", "1958-06-21", "F"), ("02139", "1990-01-01", "F"), ("02139", "1990-01-01", "F")
```

```
    ("02139", "1990-01-01", "F"), ("02141", "1983-09-30", "M"),
]
success, unique, k = linkage_report(release)
print(f"{success:.3f} {unique:.3f} {k}")  # 0.625 0.375 1
cells = 40_000 * 365 * 80 * 2  # ZIP code x birth date over 80 years x sex
print(f"{log2(cells):.1f} {exp(-248.7e6 / cells):.3f}")  # 31.1 0.899
```

The full edition's script, linkage.py, computes every number in this chapter.

15.9. Cheat sheet

Table 15.1.: The results of this chapter.

Question	Result	Use it for
Bits to name one person	$\log_2 n$	33 for the world, 28 for the US, 13 for 10,000 people
Risk of a release you hold	Success J/n; at most $1/k$ if every class has k records	Target known to be in the release
Fraction unique, uniform cells	$\exp(-2^{\log_2 n - b})$	50% at $\log_2 n + 0.53$ bits, 99% at $\log_2 n + 6.64$
Fraction unique, known frequencies	$\sum_c p_c(1-p_c)^{n-1}$	Planned releases with independent attributes
Sparse data	Score with rarity weights, then the eccentricity test	Calibrate φ on outsiders at a small false positive rate
Releasing a sample	$\xi \approx e^{-\lambda(1-\pi)}$	Sampling helps only in crowded cells

Rules of thumb

- Add up the bits of every attribute an adversary might know. Above $\log_2 n$, assume most people are unique, and about 7 bits more make 99% unique.
- Estimate uniqueness for the population, not just the sample, and assume the adversary holds auxiliary data you have not seen.
- Hashing a licence, phone or identity number is not anonymization, because anyone can hash every possible value. Key the hash, and never reuse a pseudonym across releases.

15.10. Further reading

- Sweeney, "Simple Demographics Often Identify People Uniquely" (2000): the 87% estimate.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (2008): the Netflix attack.

Derivations. Every result here is derived step by step in the full edition's chapter.

16. Syntactic anonymity and how it breaks

16.1. Why this matters

Linkage and re-identification in the full edition showed that deleting names does not anonymize a table, because the remaining quasi-identifiers, such as age, ZIP code and sex, single out most people. Between 1998 and 2008, researchers answered with definitions that constrain the released table itself. They are called **syntactic** because you can check each one by reading the published table.

Statistics offices, clinical data sharing under the HIPAA expert-determination route, and tools such as ARX and sdcMicro still use these definitions, so you will meet them in audits. The short answer to whether they are enough is that each is a property of one table against an adversary whose knowledge is fixed in advance. One unanticipated fact, or a second release, voids the guarantee.

16.2. What the adversary believes about a class

To anonymize a table, the custodian applies a **generalization**, which turns an age into a range, a ZIP code into a prefix, and so on. Records with the same generalized quasi-identifiers form an **equivalence class**, or class. An adversary who knows the target's quasi-identifiers can find the target's class E, but the records in it look alike. So if it knows nothing else, its belief about the target's sensitive value is the share of the class with that value.

Call that belief its **posterior**. Write $|E|$ for the size of the class, and $n_E(v)$ for the number of its records with a sensitive value v. Then

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|} = P_E(v). \tag{16.1}$$

Background knowledge works by elimination. Ruling out a value crosses out its records, and the remaining shares grow, which is the calculation behind every attack in this chapter.

16.3. k-anonymity and what it costs

Sweeney and Samarati proposed **k-anonymity** in 1998. A release is k-anonymous if every class has at least k records, for a crowd size k the custodian chooses. Generalizing gets there,

with **suppression** of the records that still stand out. The linkage success of Linkage and re-identification in the full edition is the number of classes J over the number of records n, and it is then capped:

$$\Pr[\text{win}] = \frac{J}{n} \le \frac{1}{k}. \tag{16.2}$$

The guarantee is about names only, and it assumes the adversary knows the quasi-identifiers and nothing more. In sparse data every column is a quasi-identifier, so k-anonymity destroys a ratings matrix (Aggarwal, 2005).

What k to use is a policy choice that the mathematics does not settle, so data-sharing agreements fix it. What the mathematics does say is how coarse the quasi-identifiers must be. Counted in bits as in the linkage chapter, hiding 99% of people in classes of at least 2 allows about 9 bits fewer than making 99% unique.

Anonymizers search for a generalization that fits the data. For example, **Mondrian** keeps splitting groups of records at the median of one quasi-identifier, as long as both halves keep at least k records. Table 16.1 gives what each criterion costs on the full edition's synthetic hospital table, where mean width is the average share of each quasi-identifier's range that a class covers.

Table 16.1.: Mondrian on a synthetic hospital table of 10,000 patients. Certain disclosure is the fraction of records in a single-diagnosis class, and guess accuracy the fraction whose diagnosis is the most common one in their class. Imputation from age alone guesses right for 0.536.

Criterion	Class sizes	Mean width	Certain disclosure	Guess accuracy
$k = 5$	5–11	0.016	0.0232	0.593
$k = 5$, distinct $\ell = 3$	5–33	0.022	0.0000	0.574
$k = 5$, recursive $(2, 3)$	5–641	0.113	0.0000	0.417
$k = 5$, $t = 0.15$	14–304	0.324	—	0.222

At $k = 5$ a class covers only 1.6% of each range, so k-anonymity itself is cheap. The last two columns are about the diagnosis, which it does not protect.

16.4. Homogeneity, background knowledge and l-diversity

Machanavajjhala and colleagues (2006) pointed out that k-anonymity hides who a record belongs to, but not what it says. If all three records of Alice's class say heart disease, her posterior is 1 without anyone finding her record, which is the **homogeneity attack**. If Bob's class says cancer, cancer and flu, a neighbour who knows he did not have flu learns he has cancer, which is the **background knowledge attack**. In the hospital table at $k = 5$, 2.3% of patients sit in single-diagnosis classes.

l-diversity asks each class to hold enough different values. Its distinct version asks for at least ℓ different values, and its entropy version asks for at least $\log_2 \ell$ bits of entropy. Neither caps the

share of the most common value, and a class over eight values can be entropy 3-diverse with one value in 73% of its records.

The recursive version targets background knowledge directly. Sort a class's value counts from largest to smallest, and write r_1 for the largest count and r_ℓ for the ℓ-th largest. A class is **recursive (c, ℓ)-diverse**, for a constant c the custodian picks, if

$$r_1 < c\,(r_\ell + r_{\ell+1} + \dots). \tag{16.3}$$

That condition caps the posterior even after the adversary rules out $\ell - 2$ values:

$$\Pr[S_{\text{target}} = v \mid \text{release}, \text{background}] < \frac{c}{1+c}. \tag{16.4}$$

So l-diversity protects against certainty, not likelihood, since the cap is already one half at $c = 1$. On the hospital table, distinct 3-diversity removes the certain disclosures at little cost, while recursive diversity forces some classes to hold hundreds of patients.

16.5. Measuring what a release teaches

Compare every posterior with what the adversary would believe with no release, not with zero. The comparison is the **imputation baseline** of Attribute inference and model inversion in the full edition, the most likely sensitive value given the target's own quasi-identifiers under the population distribution. Only success beyond it is about the individual.

On the hospital table, guessing the most likely diagnosis for each patient's age, with no release, is right for 53.6%, and a 5-anonymous release is right for 59.3%. So most of what the release teaches is the link between age and diagnosis, which any useful study would report, and only the excess of 0.057 is a leak.

16.6. Skewness, similarity and t-closeness

Li, Li and Venkatasubramanian (2007) found two attacks that l-diversity misses. In the **skewness attack**, a class that is half HIV is 2-diverse, yet in a table with 1% HIV it raises its members' posterior fifty-fold. In the **similarity attack**, a class of three stomach diseases is 3-diverse but reveals a stomach disease. **t-closeness** answers both. A release is t-close if every class's distribution is within a distance t of the whole table's.

With the **total variation (TV)** distance (Divergences), no belief moves by more than t. Write $P_E(A)$ and $P(A)$ for the chance that a value from the class, or from the table, lands in a set A. Then

$$|P_E(A) - P(A)| \le t. \tag{16.5}$$

The bound is additive, which protects rare values badly. A value held by 1% of the table may rise to 16% at $t = 0.15$. Total variation also treats all diagnoses as equally different, so against the similarity attack Li and colleagues used the **earth mover's distance (EMD)**, under which similar values are close.

t-closeness also fights the purpose of the release, because at $t = 0.15$ the hospital table's guess accuracy falls far below what imputation from age achieves.

16.7. The composition attack

Ganta, Kasiviswanathan and Smith (2008) asked what happens with two releases. Suppose that two hospitals in one city each publish a properly k-anonymous or ℓ-diverse table, and a patient was treated at both. Write E_A and E_B for the patient's classes in the two releases. The adversary keeps only the diagnoses that appear in both,

$$\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}. \tag{16.6}$$

When one candidate is left, the diagnosis is **pinned down**.

The full edition simulates two hospitals that each publish a 5-anonymous release. Together, the releases pin down the diagnosis of 9.9% of their shared patients, more than three times what either pins alone. With distinct 3-diversity neither release pins anyone, yet together they still pin 18 patients.

No single-table definition can stop this, because the intersection of two tables is a table that neither custodian checked. A definition that survives must compose, hold whatever the adversary knows, and bound the ratio of beliefs to protect rare values. The definition of differential privacy starts from these requirements, and that is why regulators and the Census Bureau have moved towards it (Reconstruction attacks in the full edition).

16.8. Choosing and running an anonymization

1. **Fix the quasi-identifiers** with the risk check of Linkage and re-identification in the full edition. Choosing them too narrowly is the commonest failure, since every column an adversary could know counts.
2. **Choose the criteria.** Use k-anonymity against linkage, add distinct l-diversity when a single-diagnosis class would be a disclosure, and use t-closeness only when the sensitive attribute is skewed or has similar values, since it costs the most.
3. **Run the anonymizer.** ARX, an open-source Java tool, takes a generalization hierarchy for each quasi-identifier, the privacy models above and a suppression limit, and searches for the generalization that loses the least information.
4. **Audit the output yourself.** In pandas, `df.groupby(qi_cols)[sensitive].agg(["size", "nunique"])` gives each class's size and number of distinct values, and the Code section computes entropy ℓ and distance from the table. The Python package pycanon also reports k, ℓ and t for a table. Report guess accuracy next to the imputation baseline.
5. **Move to differential privacy** when overlapping views or other releases of the same people exist, when the sensitive attribute is rare, or when the data are sparse.

16.9. Code

The snippet audits a 3-anonymous release and prints the smallest class size, the fewest distinct values, and each class's entropy-equivalent ℓ and distance from the table.

```
from collections import Counter, defaultdict
from math import log2


def classes(rows: list[tuple[str, str, str]]) -> list[Counter[str]]:
    """Count the diagnoses in each class of (ZIP, age, diagnosis) rows."""
    groups: defaultdict[tuple[str, str], Counter[str]] = defaultdict(Counter)
    for zip_code, age, diagnosis in rows:
        groups[(zip_code, age)][diagnosis] += 1
    return list(groups.values())


def entropy_l(counts: Counter[str]) -> float:
    """The number of equally common values with the same entropy."""
    total = sum(counts.values())
    return 2 ** -sum(c / total * log2(c / total) for c in counts.values())


def tv_to_table(counts: Counter[str], table: Counter[str]) -> float:
    """Total variation between a class's distribution and the whole table's."""
    nc, nt = sum(counts.values()), sum(table.values())
    return sum(abs(counts[v] / nc - table[v] / nt) for v in table) / 2


rows = [
    ("130**", "<30", "heart disease"), ("130**", "<30", "heart disease"), ("130**", "<30", "
    ("148**", "4*", "cancer"), ("148**", "4*", "flu"), ("148**", "4*", "heart disease"),
    ("130**", "3*", "cancer"), ("130**", "3*", "cancer"), ("130**", "3*", "flu"),
]
table = Counter(diagnosis for _, _, diagnosis in rows)
found = classes(rows)
print(min(sum(e.values()) for e in found))  # 3
print(min(len(e) for e in found))  # 1
print(" ".join(f"{entropy_l(e):.2f}" for e in found))  # 1.00 3.00 1.89
print(" ".join(f"{tv_to_table(e, table):.3f}" for e in found))  # 0.556 0.111 0.444
```

The full edition's script, syntactic_anonymity.py, implements Mondrian and the attacks.

16.10. Cheat sheet

Table 16.2.: The syntactic definitions and the attacks on them.

Definition or attack	Condition	What it gives, and where it ends
k-anonymity	Every class has at least k records	Linkage at most $1/k$; nothing about the sensitive attribute
Distinct, entropy l-diversity	At least ℓ values, or entropy at least $\log_2 \ell$	No cap on the top value's share
Recursive (c, ℓ)-diversity	$r_1 < c(r_\ell + r_{\ell+1} + \dots)$	Posterior below $c/(1+c)$ after ruling out $\ell - 2$ values
t-closeness (TV)	Each class within t of the table	Every belief moves by at most t, so a 1% value may reach $1\% + t$
Composition	Intersect the target's classes in two releases	No single-table definition prevents it

Rules of thumb

- k-anonymity protects names, not attributes, so check what each class's sensitive values reveal.
- Measure attribute disclosure against imputation from the population, or you will count the study's findings as leaks.
- A rare sensitive value needs a bound on the ratio of beliefs, not on their difference.
- Assume another release exists, because a guarantee about one table is not a guarantee about a person.

16.11. Further reading

- Machanavajjhala and colleagues, "l-Diversity" (2006), and Li, Li & Venkatasubramanian, "t-Closeness" (2007).
- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (2008): the intersection attack.

Derivations. Every result here is derived step by step in the full edition's chapter.

17. Reconstruction attacks

17.1. Why this matters

A hospital keeps one sensitive bit per patient, recording whether that patient tested positive. It only answers counting questions such as "how many of the patients on this list tested positive?", with a little noise added to each answer. Dinur and Nissim (2003) showed that this is not safe unless the noise is large. Each answer is a linear equation in the patients' bits, and enough equations with small enough errors determine the unknowns.

The protection can fail in two ways. The error can be too small, or there can be too many answers, because independent errors are averaged away as the attacker asks more. Dwork and Roth call this the **Fundamental Law of Information Recovery**: overly accurate answers to too many questions destroy privacy. Any aggregate release must be checked against both.

17.2. Counts are linear equations

Write the secret as a list of bits $d = (d_1, \dots, d_n)$, one for each of n people. A **subset-sum query** names a set of people and asks how many of them have bit 1. The attacker's m queries form a matrix A of zeros and ones, with one row per query and one column per person, so the true answers are Ad.

The party that answers, the **curator**, publishes the answers with an error vector e,

$$a = Ad + e. \tag{17.1}$$

Write E for a bound on the size of every error. The attacker is scored by the fraction of bits it recovers, against one half from guessing (Threat models and privacy games in the full edition).

17.3. Exact and random queries: how much error is enough

An attacker who could ask all 2^n subset sums would get at most $4E$ bits wrong, whatever errors the curator chose. So a curator whose error is a vanishing fraction of n is **blatantly non-private**. A distortion the attacker can simulate, such as rounding to a multiple of 4, is not noise at all, because the attacker can round each candidate's answers too.

Nobody can ask 2^n questions, but random ones suffice. Each random query includes every person with probability one half. The Dinur–Nissim decoder solves a **linear program**, searching for fractional bits whose answers lie within E of the published ones, and then rounds. The price is that the error must now be well below $\sqrt{n}$, not just below n, because a random query picks up

each wrong bit only half the time. A number of queries of order $n \log n$ then suffices. The proved constants are pessimistic, and in practice least squares already works at errors of about $0.2\sqrt{n}$ with $4n$ queries.

17.4. Least squares: the attack to run on your system

Least squares needs no solver and no value of E. It picks the vector whose answers come closest to the published ones in squared error,

$$\hat{x} = (A^\top A)^{-1} A^\top a, \tag{17.2}$$

and then clips and rounds each entry. Suppose the curator adds independent normal noise, and write σ for its standard deviation. A bit then comes out wrong when its error crosses one half, so write Φ for the standard normal cumulative distribution function. The expected fraction of bits recovered is then about

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right). \tag{17.3}$$

Plug your system's numbers into it. At a fixed number of queries per person only $\sigma/\sqrt{n}$ matters, so $\sqrt{n}$ is the scale of the threat, and recovery degrades smoothly rather than switching off at a threshold. On 500 people with 2000 queries, the prediction is within 0.013 of every simulated value in Table 17.1.

Table 17.1.: Least-squares reconstruction of $n = 500$ bits from $m = 2000$ random subset sums with Gaussian noise.

$\sigma/\sqrt{n}$	0.1	0.25	0.5	1	2
Recovered, simulated	1.0000	0.9655	0.8130	0.6550	0.5875
Recovered, Equation 17.3	1.0000	0.9585	0.8069	0.6676	0.5858

The last column shows how a curator can look safe and still fail. With noise of twice $\sqrt{n}$, four queries per person recover only 0.5875 of the bits. But if the curator answers any number of queries at that noise level, 106.1 queries per person are predicted to recover 90%. So the noise must grow like $\sqrt{m}$ to hold the attack back.

Errors chosen against the decoder hurt more than random noise of the same size. Least squares amplifies errors most along the direction its queries see least clearly. A curator that pushed every error that way caused 143 wrong bits, where normal noise of the same size caused about 17. So test a system with errors aimed at the decoder, too.

17.5. The Fundamental Law of Information Recovery

Noise of order $\sqrt{n}$ can stop reconstruction without giving privacy. A curator that answers from a secret random sample of half the people hides the others completely, yet enough queries reveal every sampled person's bit exactly. The definition of differential privacy protects every individual instead.

Exact answers are also cheap to exploit. An exact count of up to n people carries at most $\log_2(n+1)$ bits. So any attack needs at least about $n/\log_2(n+1)$ counts to pin down the n bits, and least squares with exact answers needs only n, close to that floor. Coarser answers carry fewer bits each, but they are no defence, because an attacker can simply ask more.

Weaker goals need far fewer aggregates. For example, a genome study's published allele frequencies reveal whether a given person took part, as Homer and colleagues showed in 2008 (Lower bounds).

17.6. Attacking a deployed query system

In 2017 Aircloak offered a bounty for breaking Diffix, its commercial system for counting queries in a restricted form of SQL. Diffix added noise for each condition in a query, so the noise grew with the length of a query but not with the number of queries. The noise was sticky, so repeating a query gained nothing. Its language forbade OR, so naming an arbitrary subset took many noisy conditions.

Cohen and Nissim (2020) picked random short descriptions instead, and took whatever subset each one selected. A query multiplied a record's public identifier by a prime, raised it to a fractional power, and kept the record when a chosen digit was divisible by 2 or 5. Diffix counted that as one condition, so each query carried little noise. On a real banking dataset, 3,500 such queries recovered every secret bit, with reported noise of standard deviation 4.

Least squares predicts this in advance. Write ρ for the fraction of records a query includes, and v for the average of $\rho(1-\rho)$ over the queries. From m queries, least squares then recovers about

$$\Phi\left(\frac{\sqrt{v\,(m-n-1)}}{2\sigma}\right) \tag{17.4}$$

of the n bits. For the smallest range in the attack, of 73 records, the prediction is near-total recovery. Holding the attack to 75% would have needed noise 4.9 times larger. The lesson applies to any system, because many distinct queries average away noise that is fixed per query. A restricted language does not help either, since any short formula that splits records independently of their secrets acts as a random query.

17.7. Reconstructing a census block

Each cell of a published table is a query. Garfinkel, Abowd and Martindale (2019) showed that a constraint solver rebuilds a census block from its statistics in seconds. The full edition repeats this with a fictional block of seven people. For six groups of them it publishes a count, a median age and a mean age, and means to two decimals give exact sums of ages.

A decoder lists candidate blocks and discards each one that contradicts a published number, and exactly one block survives. So eighteen numbers determine the age, sex, race and marital status of all seven people.

Real publications are more exposed, because the same people appear in thousands of overlapping cells. The usual protections also leave most constraints exact. For example, swapping changes a few households and leaves the rest, while coarsening and suppression remove some constraints but keep the others. A suppressed cell even adds an inequality, because it says the count is below the threshold.

The Census Bureau ran this attack on its own 2010 publications. It rebuilt a record for every person counted, and about 46% of the records matched a real person exactly on block, sex, age, race and ethnicity. Linking the records to commercial data then confirmed the identity of about 17% of the population. So the Bureau moved the 2020 Census to differential privacy, using its TopDown algorithm, and published the source code of its disclosure avoidance system.

17.8. Auditing an aggregate release

1. **Count the equations.** If the published cells or allowed queries outnumber the people, reconstruction is possible in principle.
2. **Classify the error.** Rounding, swapping and suppression are not noise, because the attacker can simulate them or work around them. Fresh noise per answer is averaged away, and sticky noise only stops repeats, while restricting or auditing the allowed queries fails because ordinary-looking formulas act as random queries. Only noise calibrated to a budget over all answers holds up.
3. **Predict least squares.** Plug the noise and the number of queries into Equation 17.3 or Equation 17.4. A prediction far above one half means the system leaks.
4. **Run the attack.** For noisy counts, solve with numpy's `np.linalg.lstsq`, as below, or pose the linear program with SciPy's `scipy.optimize.linprog`, bounding each unknown between 0 and 1. For tables of counts, medians and means, write each cell as a constraint on whole-number unknowns. Then solve the integer program with SciPy's `scipy.optimize.milp`, PuLP or Gurobi, and count the solutions.
5. **Fix it with a budget.** If the attack works, move to noise that grows with the number of answers, which is what differential privacy provides.

```
A = np.random.default_rng(0).integers(0, 2, size=(m, n))   # random subset-sum queries
x = np.linalg.lstsq(A, answers(A), rcond=None)[0]           # least-squares estimate
recovered = np.mean(np.clip(x, 0, 1).round() == d)          # against a test secret d
```

17.9. Code

The functions below predict least-squares recovery at your noise level and the queries per person that reach a target.

```
from math import sqrt
from statistics import NormalDist

PHI = NormalDist()


def ls_recovered(n: int, m: int, sigma: float) -> float:
    """Predicted fraction of n secret bits that least squares recovers from m noisy random s
    sd = 2 * sigma * sqrt(n / ((n + 1) * (m - n - 1)))  # spread of each decoded bit's error
    return PHI.cdf(0.5 / sd)  # a bit comes out wrong when its error crosses 1/2


def queries_per_person(target: float, sigma_over_sqrt_n: float) -> float:
    """m/n at which least squares recovers a fraction `target` of the bits, for noise sigma
    return 1 + (4 * sigma_over_sqrt_n * PHI.inv_cdf(target)) ** 2



n, m = 500, 2000
print(" ".join(f"{ls_recovered(n, m, c * sqrt(n)):.4f}" for c in (0.1, 0.25, 0.5, 1, 2)))  #
print(f"{queries_per_person(0.9, 2.0):.1f}")  # 106.1
```

The full edition's script, reconstruction.py, runs the attacks themselves.

17.10. Cheat sheet

Table 17.2.: The results of this chapter.

Setting	Result	Use it for
All 2^n subset sums	At most $4E$ wrong bits	Against the worst queries, error must be of order n
Random queries	About $n \log n$ suffice if the error is well below $\sqrt{n}$	Error must be of order $\sqrt{n}$ or more
Least squares, noise σ	Recovered $\approx \Phi(\sqrt{m-n}/(4\sigma))$	Predicting an attack at your noise level
Diffix-style fixed noise	Averaged away by many distinct queries	Syntax limits and sticky noise are no defence
Exact counts	At most $\log_2(n+1)$ bits each	Attacks need only about $n/\log n$ to n of them
Published tables	Each cell is a constraint	Run a solver and count the solutions

Rules of thumb

- The threat scale of a random query is $\sqrt{n}$, and errors well below it reveal the database.

- Budget the error over all answers, because more answers average away independent noise.
- Aggregates, small-cell suppression and swapping are not safe by themselves, so try to reconstruct your own release before publishing it.
- Stopping reconstruction is not privacy, so ask what the system guarantees for each individual.

17.11. Further reading

- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020): the attack on Diffix.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (2019): a census block rebuilt with a solver.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part III.

Differential privacy foundations

The previous part showed that anonymization and access control do not stop releases from leaking. This part introduces **differential privacy (DP)**, the defence the rest of the book builds on. It is a property of the release process, not of a dataset. Changing one person's data may change the probability of any output by at most a factor e^{ε}, set by a privacy level ε, plus a small slack δ. That one condition caps every attack of Part 2. So every DP claim must state three things, namely ε, δ and the unit of privacy, such as a record, a user or a user-day.

Table 17.3.: Notation used throughout Part 3.

Symbol	Meaning
ε, δ	Privacy level and slack
$\Delta_1 f$, $\Delta_2 f$	Sensitivity, the most one record moves the answer
$\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$	Laplace noise of scale b, normal noise of standard deviation σ

The part teaches one workflow. First fix the unit of privacy and the neighbouring relation, as The definition of differential privacy in the full edition explains. Then clip the data, bound its sensitivity and calibrate a mechanism to it (The Laplace and Gaussian mechanisms in the full edition). Next, account for every release (Post-processing, group privacy and composition in the full edition and Advanced and optimal composition in the full edition). Finally, post-process the outputs freely, since that costs nothing.

You need those four core chapters to ship a private statistic, while the next ones are per task. Selection in the full edition picks the best of many options. Query release in the full edition answers many queries through a synthetic dataset and Beyond global sensitivity in the full edition handles data whose worst case is far from typical. Read Discrete and floating-point-safe mechanisms in the full edition before sampling noise yourself, and Local, shuffle and distributed trust models in the full edition when nobody can be trusted with the raw data. The last two chapters help you size a study. Lower bounds in the full edition says when an accuracy is impossible at your ε, and Private mean estimation in the full edition says how many records a private mean needs.

The full edition proves every result, and this edition shows how to use them. Part 4 then replaces hand composition with accountants, and Part 5 applies all of it to training models. The tools used from here on are OpenDP, Tumult Analytics, diffprivlib, Google's `dp_accounting` and Opacus.

18. The definition of differential privacy

18.1. Why this matters

Every attack in Threat models and attacks runs an algorithm on one of two datasets that differ in one person, and guesses from the output which one was used. **Differential privacy (DP)** makes the two outputs look alike against every test, including tests nobody has invented yet. It is a property of the algorithm, not of the data, so you can check it before any data are touched. This chapter teaches you to read a privacy claim, which means knowing what it protects, what it promises and what it leaves out.

18.2. Neighbouring datasets

The first question about any claim is what "differ in one record" means, and the **neighbouring relation** answers it. Under **add/remove**, two datasets are neighbours when one is the other with one record added or taken away. The guarantee then hides whether a person is in the data at all. Under **replace**, the two datasets have the same size and differ in the value of one record, so only what the record says is hidden. A replacement is a removal followed by an addition, so a guarantee at a privacy level ε converts as

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{18.1}$$

The reverse fails, since only add/remove hides how many records there are.

You also choose the **unit of privacy**, what one record is. At the **example level**, it is one training example or one event, and at the **user level**, all the data of one person. Under example-level DP, a user who wrote 200 messages is a group of 200 records. The guarantee weakens quickly with the size of a group (Post-processing, group privacy and composition in the full edition), so check that the claim protects the person. The guarantee must also hide every record the data format allows, even a person aged 900, which is why private algorithms clip their inputs before adding noise.

18.3. The definition

Call a randomized algorithm that takes a dataset and returns an output a **mechanism**, written M. Write D for a dataset and D' for one of its neighbours. The definition asks that no set of outputs become much more likely when one record changes. It measures "much more likely" by a factor e^{ε}, for a non-negative privacy parameter ε, and allows a small additive slack δ on top. A mechanism is (ε, δ)**-differentially private** if every pair of neighbours and every set S of outputs satisfy

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{18.2}$$

The probabilities come only from the mechanism's own randomness, with the data held fixed.

When $\delta = 0$ the guarantee is called **pure DP**, and otherwise **approximate DP**. One consequence matters in practice. A private mechanism must be random, because a deterministic one gives some output that has probability 0 on a neighbour. So rounding, aggregating or suppressing small cells gives no guarantee by itself, however coarse the result looks.

18.4. The privacy loss and δ

For an output y, write $p_{M(D)}(y)$ for its probability, or its density, when the mechanism runs on D. The **privacy loss** of y is the logarithm of how much more likely the output is on D than on D',

$$L(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{18.3}$$

A loss of 0 means that the output says nothing about which dataset was used. Independent outputs multiply their likelihood ratios, so their losses add, which is why the accountants of Part 4 work with the loss. Pure ε-DP is exactly the condition that every output has a loss between $-\varepsilon$ and ε. Laplace noise meets it, while the loss of Gaussian noise grows without limit in the far tail (The Laplace and Gaussian mechanisms in the full edition).

Under approximate DP, δ covers the outputs whose loss exceeds ε. It counts each one only by how far its probability on D goes beyond the ratio bound. This weighted total is called the **hockey-stick divergence**. So δ is not the chance that the loss exceeds ε, which is a valid but often loose choice. Take a count with Gaussian noise of standard deviation 2, at $\varepsilon = 1$. Its loss exceeds ε with probability 4.006×10^{-2}. The exact δ is 6.830×10^{-3}, less than a sixth of that.

Each ε has its own smallest δ, and the function $\delta(\varepsilon)$ is the mechanism's **privacy curve**. A quoted pair is one point on it, so report the noise parameters with the pair, and others can recompute the whole curve. Numerical accounting computes these curves from the distribution of the loss.

18.5. Randomized response

The first private mechanism comes from survey design (Warner, 1965). Each person tells the truth with some probability t a little above one half, and lies otherwise, so a single answer proves nothing about them. A person who answers truthfully on heads, and with a second coin toss on tails, tells the truth with probability $\frac{3}{4}$, which gives pure DP at $\varepsilon = \ln 3$. In general, **randomized response** at level ε tells the truth with probability

$$t = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{18.4}$$

The guarantee is ε per person however many people answer, provided their coins are independent, but only under replace. Write $\bar{y}$ for the share of "yes" reports among the n people. The estimate of the true fraction and its standard deviation are

$$\hat{\pi} = \frac{\bar{y} - (1 - t)}{2t - 1}, \qquad \mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon/2}}{(e^{\varepsilon} - 1)\sqrt{n}}, \tag{18.5}$$

which is about $1/(\varepsilon\sqrt{n})$ for small ε. For 1,000 people, the two-coin scheme gives 0.0274, about twice the sampling error that the survey already has. That is the price of the **local** model, in which nobody sees a true answer (Local, shuffle and distributed trust models in the full edition). diffprivlib implements the mechanism as `mechanisms.Binary`.

18.6. What ε means for an attacker

A membership attacker looks at the output and decides whether to accuse the target of being in the data, so it has two error rates (Hypothesis testing). Its **false positive rate (FPR)** is how often it accuses a target who is out. Its **true positive rate (TPR)** is how often it accuses one who is in. The outputs on which it accuses form a set, which the definition bounds. So under add/remove, every attack at every FPR satisfies

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta. \tag{18.6}$$

This is the form to use, because Part 2 judged attacks by their TPR at a low FPR. For example, take $\varepsilon = 1$ and $\delta = 10^{-5}$. Then at an FPR of 10^{-3}, no attack reaches a TPR above 0.002728. As the FPR falls towards 0, the δ term takes over, so at a very low FPR the guarantee rests almost entirely on δ.

Read backwards, the cap turns an attack into an audit. Measure an attack's FPR and TPR, and solve the cap for the smallest ε consistent with them,

$$\varepsilon \ge \ln \frac{\mathrm{TPR} - \delta}{\mathrm{FPR}}. \tag{18.7}$$

Suppose a model is claimed to be $(2, 10^{-5})$-DP, and an attack reaches a TPR of 0.05 at an FPR of 0.001. If those rates are exact, ε is at least 3.91, so the claim is false. Audits plant a **canary**, a record built to be as easy to detect as possible, because the guarantee must hold for the worst record (Memorization and extraction).

Two summaries are easier to explain to a lawyer or an auditor. The **membership advantage**, the TPR minus the FPR, says how much better than a coin flip an attacker does. Under (ε, δ)-DP it is at most

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}, \tag{18.8}$$

which is the hyperbolic tangent of $\varepsilon/2$ when $\delta = 0$. The posterior bound speaks about beliefs. One output can multiply an adversary's odds between two neighbours by at most e^{ε}, whatever it believed before. So a prior belief π that the data are D rises at most to

$$\Pr[D \mid y] \le \frac{e^{\varepsilon}\pi}{e^{\varepsilon}\pi + 1 - \pi}. \tag{18.9}$$

Table 18.1 turns a claimed ε into both numbers.

Table 18.1.: What a pure ε-DP guarantee promises against the strongest attacker.

ε	Advantage bound	Posterior from a prior of 0.01	Posterior from a prior of 0.5
0.1	0.0500	0.0110	0.5250
1	0.4621	0.0267	0.7311
3	0.9051	0.1687	0.9526
8	0.9993	0.9679	0.9997

So an ε below 1 is strong, 1 to 5 is moderate, and around 10 the worst-case guarantee says little. Real deployments often sit higher, and Choosing ε discusses how to set and defend a value. The full trade-off curve between the two error rates is the subject of f-DP and Gaussian differential privacy.

18.7. How small δ must be

A small δ barely moves the advantage bound, but the advantage is an average that hides harm to a few people. For example, take a mechanism that publishes each record in full, independently with probability δ. Under add/remove it is $(0, \delta)$-DP, because a record changes the output only when its own coin publishes it. Yet over n records it publishes at least one with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{18.10}$$

With 10,000 records and $\delta = 1/n$, someone's full record is published with probability 0.6321.

Treat δ as a per-person budget for catastrophe. Choose it well below $1/n$, for example $1/n^{1.5}$, and let n count people rather than records.

18.8. What differential privacy does not protect

DP compares a world with your record and a world without it, so anything that can be learned in both worlds is outside the guarantee. A study that never used Alice's record may show that smokers get cancer at a high rate. That changes what anyone believes about Alice, a smoker, yet the study is 0-DP with respect to her. Correlated records are a second gap. A household of five that always answers alike is protected only at five times the per-record ε, unless the household is the unit of privacy. The third gap is the implementation, which needs its own review (Discrete and floating-point-safe mechanisms in the full edition).

When someone reports a privacy violation, ask three questions. Would the inference come out materially different without the person's record? Does it succeed exactly when the model is accurate? And when, and by whom, was the revealing information given up? Only the first decides. If the answer is no, as when public "likes" predict someone's politics, it is not a violation in the DP sense, even when it causes harm. A language model that completes a prompt with one user's identification number does fail it, because a model trained without her messages would not.

18.9. Code

This snippet prints rows of Table 18.1 and turns a measured attack into a lower bound on ε.

```
from math import exp, log


def advantage_bound(eps: float) -> float:
    """The largest TPR - FPR against any eps-DP mechanism."""
    return (exp(eps) - 1) / (exp(eps) + 1)


def posterior_bound(eps: float, prior: float) -> float:
    """The largest posterior after one output of an eps-DP mechanism."""
    return exp(eps) * prior / (exp(eps) * prior + 1 - prior)


def epsilon_lower_bound(tpr: float, fpr: float, delta: float) -> float:
    """The smallest eps consistent with an attack measured at (fpr, tpr)."""
    return log((tpr - delta) / fpr)


print(f"{advantage_bound(3.0):.4f}")  # 0.9051
print(f"{advantage_bound(8.0):.4f}")  # 0.9993
print(f"{posterior_bound(8.0, 0.01):.4f}")  # 0.9679
print(f"{posterior_bound(8.0, 0.5):.4f}")  # 0.9997
print(f"{epsilon_lower_bound(0.05, 1e-3, 1e-5):.2f}")  # 3.91
```

18.10. Cheat sheet

Table 18.2.: The results of this chapter.

Result	Formula	Notes
(ε, δ)-DP	$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$	All neighbours, all sets S
Add/remove to replace	$\varepsilon \to 2\varepsilon$	Only add/remove hides participation
Advantage and posterior	See Table 18.1	Odds move by at most a factor e^{ε}
Attack cap	$\text{TPR} \leq e^{\varepsilon}\,\text{FPR} + \delta$	Audit: $\varepsilon \geq \ln((\text{TPR} - \delta)/\text{FPR})$
Choosing δ	$\delta \ll 1/n$, with n people	$\delta = 1/n$ can expose someone

Rules of thumb

- Every claim must state ε, δ and the unit of privacy, with its neighbouring relation.
- When users contribute many rows, use user-level neighbours, not row-level DP.
- An ε is never a probability of re-identification. It bounds how far any output can shift an attacker's odds.
- A δ above one over the number of people allows exposing someone.
- A correct inference about a population is not a leak in the DP sense.

18.11. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition and what DP promises.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the hypothesis-testing reading behind the attack cap.

Derivations. Every result here is derived step by step in the full edition's chapter.

19. Post-processing, group privacy and composition

19.1. Why this matters

A private mechanism is never the whole system. Its output gets rounded, plotted or fed into a model, the people in your data may each own many records, and an analyst rarely asks only one question. So you need to know what guarantee holds for what you actually publish in each case, without writing a new proof for every pipeline.

Three properties of differential privacy (DP) give that answer. The first, post-processing, says that anything computed from a private output without reading the data again is free. The second, group privacy, says what a guarantee about one record promises for a person who owns several. And composition says that running several mechanisms on the same data costs at most the sum of their budgets. Together they turn privacy into bookkeeping, because you can prove each building block once and then keep a ledger of what you spend.

19.2. Post-processing

Take a private mechanism, written M. Write ε for its privacy level and δ for its slack, so that it is (ε, δ)-DP. Apply to its output any function f, which may use random coins of its own but never reads the data. Then the processed release is just as private,

$$f(M(D)) \text{ is } (\varepsilon, \delta)\text{-DP.} \tag{19.1}$$

Processing can merge outputs that point in different directions, and so it can lower the privacy loss, but it can never raise it.

This is what makes DP practical. Rounding a noisy count, clamping it at zero, or making noisy totals consistent with their parts costs no extra budget. A model trained on private synthetic data, a plot and a dashboard all inherit the guarantee with no further proof too. So a private system adds noise at one narrow point, such as noisy counts or noisy gradients, and treats everything after it as public. The same rule covers attackers. A membership-inference attack is a function of the output, so the attack cap of the definition holds even for attacks invented after the release.

The condition is that f never reads the data. “Take the noisy top category, then report its exact count” breaks it, because the second step is a new and non-private mechanism. So does “publish the noisy count, unless the true count is 0”, which peeks at the raw data to decide. A finer histogram computed after an odd bin catches your eye is a second mechanism too, whose cost adds by composition.

19.3. Group privacy

The definition protects one record, but a user of a messaging app contributes hundreds of messages. If the guarantee is per record and a person owns k records, you need a guarantee for datasets that differ in k records. For pure DP, the losses of the k steps from one dataset to the other add up, so ε-DP per record becomes $k\varepsilon$-DP for the group. With a slack, each step also adds its own δ, and every later step multiplies the earlier ones by e^{ε}. For a group of k records, an (ε, δ)-DP mechanism is

$$\Big(k\varepsilon,\ \delta_k\Big)\text{-DP}, \qquad \delta_k = \delta\,\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1} \le k\,e^{(k-1)\varepsilon}\delta. \tag{19.2}$$

The simpler form on the right is looser, so compute the sum when you can.

The price is steep. A user with 50 rows under a row-level guarantee of ε is protected only at 50ε. The slack grows even faster, because δ is multiplied by 12818.3 for a group of 10 at $\varepsilon = 1$ per record. That hurts, since δ was already chosen much smaller than one over the number of records. For example, a guarantee of $(0.5, 10^{-5})$ per record becomes $(5, 2.272 \times 10^{-3})$ for a user with 10 records.

So if you care about people, don't patch a per-record guarantee with group privacy. Define neighbours as datasets that differ in one person's records, and bound each person's contribution, for example by capping how many records a user may add. Then prove DP at that level directly. Group privacy is the fallback for when you cannot, and for correlated records such as households.

19.4. Sequential composition

An analyst asks several queries of the same data, each answered with fresh noise, and an adversary sees all the answers together. Write M_i for the i-th of k mechanisms and $(\varepsilon_i, \delta_i)$ for its budget. Releasing all their outputs is then

$$\Big(\sum_{i=1}^{k}\varepsilon_i,\ \sum_{i=1}^{k}\delta_i\Big)\text{-DP}. \tag{19.3}$$

This is **basic composition**. It holds because independent outputs have independent noise, so their privacy losses add. It needs every mechanism to use fresh coins and the same neighbouring relation.

The rule also covers **adaptive** composition, where each query is chosen after reading the earlier answers, as long as each budget is fixed in advance. For example, an eleventh query that runs only if the first ten look interesting is covered, provided its budget was fixed beforehand. Budgets chosen from the answers, as in "keep going until the estimate looks good", are another matter. They need a privacy filter, a rule that decides when to stop as the budget is spent (Adaptive composition). Pure DP with basic composition is the exception, since stopping before a fixed total is exceeded is always safe there.

Basic composition is loose for many releases. Ten Gaussian counts at $(0.5, 10^{-5})$ each cost an exact ε of 1.504, where basic composition charges 5. So use the accountants of Advanced and optimal composition in the full edition and Part 4 once releases number more than a handful.

19.5. Parallel composition

A histogram counts people in disjoint bins, and per-region statistics use disjoint subsets of the data. If you split the data by region, each region can spend the full budget, because one person's record falls in one region only. Split the possible records into disjoint parts by a public rule fixed in advance, so that each person lands in exactly one part. Then run a mechanism M_i with independent coins on the records in the i-th part. If each M_i is $(\varepsilon_i, \delta_i)$-DP under add/remove, the whole release is

$$\big(\max_i \varepsilon_i,\ \max_i \delta_i\big)\text{-DP under add/remove.} \tag{19.4}$$

This is why a histogram costs one ε, not one per bin. With Laplace noise of scale $1/\varepsilon$ on every bin, the histogram is ε-DP however many bins it has. Under replace, a record can move from one bin to another, so two bins change and the cost doubles to 2ε. The cost goes back to the maximum when each part is a fixed slot that belongs to one person, as in local randomized response.

The rule fixed in advance is the condition that is easiest to break. A split into "the top 10% of earners and the rest" moves its boundary when one high earner is added, so records cross it and both parts change. The same goes for partitioning on the list of keys that happen to appear in the data, because the list itself reveals who is present. Use fixed bins or a public key list, or treat a data-dependent split as a mechanism of its own.

19.6. Keeping a budget

Fix a total budget for a dataset before the first release, and charge every release against it. Suppose a dashboard over one patient table has a total of $(1, 10^{-5})$. A histogram of 20 diagnoses with $\varepsilon = 0.5$ on every bin costs only 0.5, by parallel composition. A mean age, released as a noisy sum and a noisy count at 0.125 each, costs 0.25 by sequential composition. A set of Gaussian counts at $(0.25, 10^{-5})$ uses the rest. Any further query must then be refused, or paid for from a new budget, and the snippet in Section 19.7 keeps exactly this ledger. Libraries keep the same ledger. OpenDP's `Context.compositor` takes a total budget and a number of queries. Its `context.query()` then charges each one. diffprivlib's `BudgetAccountant` tracks spending across its mechanisms, and Tumult Analytics sessions hold a total budget and evaluate each query against it.

Most real failures break a condition of the three properties, and the damage is usually total rather than a small loss. Table 19.1 lists the common cases.

Table 19.1.: What composes and what doesn't.

Situation	Composes?	What to do
A function of the output, with its own coins	Yes, for free	Process as much as you like
Another query on the same data, fresh noise	Yes, budgets add	Charge it to the ledger
A query chosen from earlier answers	Yes, budgets add	Fix its budget in advance

Situation	Composes?	What to do
Two releases that share one noise draw	No	Draw fresh noise for every query
"Post-processing" that reads the data again	No	Charge it as a new mechanism
A DP release beside an exact statistic of the same data	No	Release the statistic privately too
A split whose boundaries depend on the data	No, not in parallel	Fix the bins in advance
Guarantees under add/remove and replace in one ledger	Only after conversion	Convert, at 2ε

Shared noise is the clearest case. Release the number of smokers and the number of smokers aged 40 or over, adding the same Laplace draw to both. Each release alone is 1-DP, but their difference cancels the noise and gives the exact number of smokers under 40, so the pair is not DP at all. In code, the bug hides in two mechanisms seeded with the same random seed and in a cached noise value re-added to a related query. It also hides in noise reused across the shards of a distributed system. Returning the same noisy answer to a repeated identical query is fine, because that is post-processing of one release.

19.7. Code

The ledger below charges each release against a total fixed in advance, and refuses any release that would exceed it. It also computes group privacy, where hand calculations go wrong most often.

```
from math import exp


def group_privacy(eps: float, delta: float, k: int) -> tuple[float, float]:
    """The guarantee for a group of k records, from an (eps, delta) guarantee per record."""
    return k * eps, delta * sum(exp(j * eps) for j in range(k))


class Ledger:
    """Basic composition for releases from one dataset, with a total fixed in advance."""

    def __init__(self, eps_total: float, delta_total: float) -> None:
        self.eps_total, self.delta_total = eps_total, delta_total
        self.eps_spent, self.delta_spent = 0.0, 0.0

    def charge(self, name: str, eps: float, delta: float = 0.0) -> None:
        if self.eps_spent + eps > self.eps_total or self.delta_spent + delta > self.delta_to
            raise ValueError(f"{name}: over budget")
        self.eps_spent += eps
```

```
        self.delta_spent += delta

    def charge_parallel(self, name: str, budgets: list[tuple[float, float]]) -> None:
        """Mechanisms on disjoint parts of the data, under add/remove, cost the largest budg
        self.charge(name, max(e for e, _ in budgets), max(d for _, d in budgets))


ledger = Ledger(1.0, 1e-5)
ledger.charge_parallel("histogram, 20 bins", [(0.5, 0.0)] * 20)
ledger.charge("noisy sum", 0.125)
ledger.charge("noisy count", 0.125)
ledger.charge("Gaussian counts", 0.25, 1e-5)
print(ledger.eps_spent, ledger.delta_spent)  # 1.0 1e-05
try:
    ledger.charge("one more count", 0.1)
except ValueError as err:
    print(err)  # one more count: over budget
eps_k, delta_k = group_privacy(0.5, 1e-5, 10)
print(f"{eps_k:.1f} {delta_k:.3e}")  # 5.0 2.272e-03
print(f"{group_privacy(1.0, 1.0, 10)[1]:.1f}")  # 12818.3
```

19.8. Cheat sheet

Table 19.2.: The results of this chapter.

Property	Rule	Condition
Post-processing	$f(M(D))$ is (ε, δ)-DP	f never reads the data
Group privacy	$(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$ for k records	Prefer user-level neighbours
Basic composition	$(\sum \varepsilon_i, \sum \delta_i)$	Fresh noise, one neighbouring relation, budgets fixed in advance
Parallel composition	$(\max \varepsilon_i, \max \delta_i)$	Add/remove, parts fixed in advance; 2ε under replace

Rules of thumb

- Add noise once, as early as possible, and everything computed from the noisy output after it is free.
- Fix the total budget before the first release, and keep a ledger.
- Budgets add across queries on the same data, and take the maximum across disjoint parts of it.
- Use basic composition for a handful of releases, and an accountant when there are many.

- Never share a random seed between mechanisms, and never partition on the keys found in the data.

19.9. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): post-processing, group privacy and basic composition, the standard reference.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a query system built on post-processing and composition as its only rules.

Derivations. Every result here is derived step by step in the full edition's chapter.

20. The Laplace and Gaussian mechanisms

20.1. Why this matters

Suppose you run a hospital database and want to publish how many patients have diabetes with a differential privacy (DP) guarantee. The two mechanisms of this chapter are how almost every such release is built, up to the noisy gradients of private model training.

Both compute the exact answer of a query f on the dataset D. They then add fresh random noise, written Z, so that the mechanism publishes

$$M(D) = f(D) + Z. \tag{20.1}$$

Three decisions fix the whole mechanism. The **sensitivity**, how far one record can move the answer, sets the scale of the noise. The noise distribution sets the kind of guarantee, since Laplace noise gives pure DP while Gaussian noise needs a small slack δ. And a calibration formula sets the exact amount, through the privacy loss of The definition of differential privacy in the full edition.

20.2. Sensitivity

The noise must cover the largest change that one record can make to the answer. A query often returns a vector of d numbers, such as the bins of a histogram, so you need a size for a change to a vector. Laplace noise uses the ℓ_1 norm, the sum of the absolute changes. Gaussian noise uses the ℓ_2 norm, the square root of the sum of the squared changes. Writing $D \simeq D'$ for a pair of neighbours, the ℓ_p **sensitivity** of the query is the largest change between their answers,

$$\Delta_p f = \max_{D \simeq D'} \big\| f(D) - f(D') \big\|_p, \qquad p \in \{1, 2\}. \tag{20.2}$$

The two are never far apart, since

$$\Delta_2 f \le \Delta_1 f \le \sqrt{d}\, \Delta_2 f. \tag{20.3}$$

They are equal when one record moves a single coordinate.

The sensitivity also depends on the neighbouring relation. Under add/remove one dataset has one record more than the other, and under replace one record is swapped for another, so the size n is public. Write B for the largest value a record may take after clipping. Table 20.1 then gives the standard queries.

Table 20.1.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, values in $[0, B]$	B	B	B	B
Mean, values in $[0, B]$, n public	—	—	B/n	B/n
Histogram with k bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \geq j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

A replaced record leaves one histogram bin and joins another, which is why the relation must be named. The threshold counts are a **dense** query, in which one record can move every coordinate.

Compute the sensitivity from the domain, before you look at the data. A sensitivity read off the data, for example from the largest value present, makes the noise itself leak that value (Beyond global sensitivity in the full edition). For unbounded data the sensitivity is infinite, so clip every value to a fixed range first. For example, take the mean age of 1,000 patients, with ages clipped to at most 100. Under replace its sensitivity is 0.1, since no patient can move the mean by more than a tenth of a year.

20.3. The Laplace mechanism

The Laplace mechanism (Dwork et al., 2006) adds independent noise from the **Laplace distribution** to each coordinate, a two-sided exponential peaked at 0. Its scale, written b, sets its typical size. To be ε-DP at a privacy level ε, the mechanism sets the scale to

$$b = \frac{\Delta_1 f}{\varepsilon}. \tag{20.4}$$

The error of the mechanism is the noise itself. Each coordinate is off by b on average, and lands more than a distance t from the truth with probability $e^{-t/b}$. To bound all d coordinates at once, except with a small failure probability β, give each coordinate a share β/d of it. The width that results is

$$\Pr\Big[\max_i |Z_i| > b \ln \frac{d}{\beta}\Big] \leq \beta. \tag{20.5}$$

Publish this width with the release, and read it backwards as an error budget. Fix a target width t for one number and solve for the privacy level, which gives

$$\varepsilon = \frac{\Delta_1 f \, \ln(1/\beta)}{t}. \tag{20.6}$$

For example, a count that must be within 10 of the truth, with probability 0.95, needs an ε of 0.2996.

At the hospital, 1,234 of the 10,000 patients have diabetes. A count has sensitivity 1 under add/remove, so at $\varepsilon = 0.1$ the noise is Lap(10). Its 95% error bar is 29.96, a relative error of 0.0243.

The scale does not contain the size of the data, so the relative error of a count of size n is about $1/(n\varepsilon)$. But the sensitivity must include every number one record can move. If one record can change all of 10 released counts, noise calibrated for one count gives only 10ε.

20.4. The Gaussian mechanism

The Gaussian mechanism adds independent normal noise with mean 0 to each coordinate, with a standard deviation written σ. Its privacy loss is unbounded in the far tail, so it always carries a δ. In exchange, it is calibrated to the ℓ_2 sensitivity, which can be much smaller than the ℓ_1 one. Only the ratio of the sensitivity to the noise matters, so write $\mu = \Delta_2 f/\sigma$ for the number of noise standard deviations between the answers on two neighbours. Write Φ for the cumulative distribution function (CDF) of the standard normal distribution. The mechanism is then (ε, δ)-DP exactly when δ is at least

$$\delta_\mu(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{20.7}$$

The formula is how you calibrate. As μ grows, $\delta_\mu(\varepsilon)$ rises steadily from 0 to 1, so exactly one value μ^* hits your target δ. The smallest noise that works is then

$$\sigma^* = \frac{\Delta_2 f}{\mu^*}, \qquad \text{where } \delta_{\mu^*}(\varepsilon) = \delta, \tag{20.8}$$

and bisection finds μ^* in a few lines (Section 20.7). At the common setting of $\varepsilon = 1$ and $\delta = 10^{-5}$, this **analytic calibration** gives $\sigma^* = 3.7306\,\Delta_2 f$.

Many papers and libraries still use the classic calibration of Dwork and Roth (2014). For ε and δ both between 0 and 1, it sets

$$\sigma = \frac{\Delta_2 f \sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{20.9}$$

Table 20.2 compares the two at $\delta = 10^{-5}$.

Table 20.2.: Classic against analytic calibration of the Gaussian mechanism at $\delta = 10^{-5}$.

ε	Classic $\sigma/\Delta_2 f$	Analytic $\sigma^*/\Delta_2 f$	Ratio
0.1	48.4481	30.7496	1.576
1	4.8448	3.7306	1.299
8	0.6056	0.6002	1.009

For small ε the classic formula adds 30–70% more noise than needed. For large ε it is wrong, not just loose. At $\delta = 10^{-5}$ the two calibrations cross at $\varepsilon = 8.42$, and beyond it the classic σ is too small to give the claimed guarantee.

A Gaussian mechanism satisfies a whole curve of guarantees, one δ for each ε, and the accountants of Part 4 compose whole curves. So report $\sigma/\Delta_2 f$ with every Gaussian release, as well as the pair you chose. Accountants that work in zero-concentrated DP take the equivalent $\rho = \Delta_2 f^2/(2\sigma^2)$ (Concentrated differential privacy).

For the diabetes count at the same ε and $\delta = 10^{-5}$, the 95% error bar is 1.96 standard deviations, or 60.27. Laplace noise gave 29.96 with no δ at all, so for a single count it wins twice.

20.5. Laplace or Gaussian?

The choice depends on how one record spreads its change across the coordinates, as the root-mean-square error (RMSE) of each released number shows. Write σ_1 for the analytic noise level at unit ℓ_2 sensitivity. At the same ε, the errors per coordinate are then

$$\text{Laplace: } \frac{\sqrt{2}\,\Delta_1 f}{\varepsilon}, \qquad \text{Gaussian: } \Delta_2 f\,\sigma_1. \tag{20.10}$$

In a sparse query, such as a histogram under add/remove, one record moves one coordinate. The two sensitivities are then equal, so at the common setting Laplace noise wins at every d. In a dense query, by contrast, one record moves every coordinate. The Laplace error then grows like d while the Gaussian error grows only like $\sqrt{d}$, so Laplace noise is more accurate only while

$$d < \frac{(\varepsilon\sigma_1)^2}{2}. \tag{20.11}$$

At the common setting, the unit noise level is 3.7306, so for threshold counts Gaussian noise wins from 7 counts on.

Gaussian noise also wins when releases are composed many times. Several Gaussian releases compose exactly into one Gaussian mechanism, whose ℓ_2 sensitivity is the square root of the sum of the squared sensitivities. Private training needs both advantages. One example's gradient moves every coordinate of a model, so private training clips each gradient's ℓ_2 norm and calibrates Gaussian noise to that clip.

20.6. Calibrating a release

Every noisy release takes five steps.

1. Fix the neighbouring relation and the unit of privacy, usually one person under add/remove.
2. Clip every value to a range fixed in advance, and read the sensitivity off Table 20.1 or work it out from the worst record.
3. Choose Laplace noise for one number or a sparse vector, and Gaussian noise for a dense vector.
4. Set the scale with Equation 20.4 or Equation 20.7, and charge it to your budget (Post-processing, group privacy and composition in the full edition).
5. Draw the noise with a vetted library, and publish the error bar and the noise parameters with the result.

A mean under add/remove has no simple sensitivity, because its denominator moves too. Release instead a noisy sum and a noisy count, each with half of the budget, and divide one by the other. Dividing is post-processing, so it costs nothing more. When you do not know the range to clip to, Private mean estimation in the full edition shows how to find it privately.

In OpenDP, `dp.t.make_clamp` and `dp.t.make_sum` build the clipped query, and `dp.m.make_laplace` or `dp.m.make_gaussian` adds noise of a given scale. Its `dp.binary_search_param` then finds the scale for a target ε. IBM's diffprivlib has `mechanisms.Laplace` and `mechanisms.GaussianAnalytic`, whose `.randomise(x)` adds the noise, and `tools.mean(x, epsilon=..., bounds=...)` for a clipped mean. Plain `numpy.random.laplace` is for prototypes only, because textbook floating-point samplers can leak through the low-order bits of their output (Discrete and floating-point-safe mechanisms in the full edition).

20.7. Code

The analytic calibration needs only the normal CDF, which `math.erfc` provides. The snippet also computes Laplace error bars.

```
from math import erfc, exp, log, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_delta(eps: float, mu: float) -> float:
    """The exact delta of the Gaussian mechanism at eps, where mu = sensitivity / sigma."""
    return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)


def analytic_sigma(eps: float, delta: float, sensitivity: float = 1.0) -> float:
    """The smallest sigma for which the Gaussian mechanism is (eps, delta)-DP."""
    lo, hi = 1e-6, 100.0  # gaussian_delta grows with mu: bisect for gaussian_delta(eps, mu)
    for _ in range(60):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gaussian_delta(eps, mid) < delta else (lo, mid)
    return sensitivity / lo


def laplace_width(scale: float, beta: float, d: int = 1) -> float:
    """All d coordinates of Lap(scale) noise stay within this width, except with probability
    return scale * log(d / beta)


print(f"{analytic_sigma(1.0, 1e-5):.4f}")  # 3.7306
print(f"{analytic_sigma(0.1, 1e-5):.4f}")  # 30.7496
```

```
print(f"{laplace_width(1 / 0.1, 0.05):.2f}")  # 29.96
print(f"{log(1 / 0.05) / 10:.4f}")  # 0.2996
```

20.8. Cheat sheet

Table 20.3.: The results of this chapter.

Result	Formula	Notes
Laplace mechanism	$\mathrm{Lap}(\Delta_1 f/\varepsilon)$ per coordinate	Pure DP; error does not depend on n
Laplace error bar	$b\ln(d/\beta)$ for all d coordinates	Solve it for ε to budget error
Exact Gaussian δ	$\Phi(\frac{\mu}{2}-\frac{\varepsilon}{\mu})-e^{\varepsilon}\Phi(-\frac{\mu}{2}-\frac{\varepsilon}{\mu})$, $\mu=\Delta_2 f/\sigma$	Solve for σ by bisection
Classic Gaussian	$\sigma=\Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$	Too much noise below 1, unsafe above 8.42
Dense query	Laplace wins only for $d<(\varepsilon\sigma_1)^2/2$	Otherwise use Gaussian noise

Rules of thumb

- Compute the sensitivity from the domain, not the data.
- Calibrate Laplace noise to $\Delta_1 f$ and Gaussian noise to $\Delta_2 f$, never the other way round.
- Use the analytic Gaussian calibration, never the classic formula.
- Report $\sigma/\Delta_2 f$ for a Gaussian release, and an error bar for every release.
- When n is not public, never use B/n as the sensitivity of a mean.

20.9. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 and Appendix A: the Laplace mechanism and the classic Gaussian calibration.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy" (ICML 2018): the exact privacy curve and the bisection calibration.

Derivations. Every result here is derived step by step in the full edition's chapter.

21. Advanced and optimal composition

21.1. Why this matters

A differentially private system rarely answers just one query. A dashboard refreshes a hundred counts, and a training run takes thousands of noisy gradient steps. Each step on its own satisfies differential privacy (DP), with a privacy level ε and a small slack δ, as in The definition of differential privacy in the full edition. But what you report, and what you calibrate the noise to, is one guarantee for the whole run.

Post-processing, group privacy and composition in the full edition gave the first answer, **basic composition**, which adds up the ε of the steps. A hundred queries at $\varepsilon = 0.1$ then cost 10. Yet if you allow the whole run a slack of $\delta = 10^{-5}$, the same hundred queries are $(4.31, 10^{-5})$-DP. The gap exists because the privacy losses of the steps add up, and each loss is a random amount. Over k steps their total has a mean of only about $k\varepsilon^2/2$ and a spread of about $\sqrt{k}\,\varepsilon$, so it almost never reaches its maximum $k\varepsilon$.

21.2. Which rule to use

Table 21.1 sorts the common situations by what you know about the steps, with the total each rule gives for the hundred queries at $\varepsilon = 0.1$ and an overall $\delta = 10^{-5}$.

Table 21.1.: Which composition rule to use.

What you know	Use	Total ε for 100 steps at 0.1
Each step's ε, and you need $\delta = 0$	Basic composition	10
Each step's (ε, δ); you want a quick budget	Advanced composition	5.30
Each step's (ε, δ); you want the number to report	Optimal composition	4.31
The mechanism itself, such as Gaussian noise	An accountant from Part 4	Often far smaller
Privacy parameters chosen during the run	A privacy filter	—

For few steps, advanced composition can lose to basic composition, so always take the smaller of the two. To an auditor, report the accountant's number when you know the mechanism, and the optimal rule's number otherwise, with the full δ.

21.3. Basic and advanced composition

Basic composition says that k steps that are each (ε, δ)-DP are together $(k\varepsilon, k\delta)$-DP. It is the only rule that keeps $\delta = 0$.

Advanced composition (Dwork, Rothblum & Vadhan 2010) trades a little slack for a much smaller ε. Pick an extra slack δ' for the whole run. Then k steps that are each (ε, δ)-DP are together $(\varepsilon', k\delta + \delta')$-DP, with

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{21.1}$$

The first term is the spread of the total privacy loss, and the second is its average, the **drift**. The original theorem has $e^{\varepsilon} - 1$ in place of $\tanh(\varepsilon/2)$, which is about twice as large. Use the tanh form. The original form is useless for large steps. At $\varepsilon = 1$ per step its drift alone is $1.7183k$, more than basic composition's total k.

For the hundred queries, the spread term is 4.80 and the drift adds 0.50, which gives 5.30. Advanced composition only helps once $\sqrt{k}$ outgrows the factor $\sqrt{2\ln(1/\delta')}$. At $\varepsilon = 0.1$, the original form beats basic composition only from $k = 29$ steps on (Table 21.2).

Table 21.2.: Total ε' at $\delta' = 10^{-5}$ for k mechanisms that are each 0.1-DP.

k	Basic $k\varepsilon$	Advanced, original form	Advanced, Equation 21.1	Optimal
10	1.000	1.623	1.567	0.994
100	10.000	5.850	5.298	4.307
1,000	100.000	25.691	20.170	17.787
10,000	1,000.000	153.156	97.944	91.691

The most common use runs the theorem backwards. You have a total budget $(\varepsilon_{\text{total}}, \delta')$ and k steps, and you want each step's share. When $\varepsilon\sqrt{k}$ is small, the drift is negligible next to the spread, so each step can have about

$$\varepsilon = \frac{\varepsilon_{\text{total}}}{\sqrt{2k\ln(1/\delta')}}, \tag{21.2}$$

while basic composition would allow only $\varepsilon_{\text{total}}/k$. Once $\varepsilon\sqrt{k}$ is near 1 or above, the drift takes over and this shortcut gives too much.

21.4. Optimal composition

Kairouz, Oh & Viswanath (2015) found the exact best rule for steps that are known only by their (ε, δ). It is stated through randomized response at level ε, which tells the truth with probability $t = e^{\varepsilon}/(1 + e^{\varepsilon})$ and is the worst case. Write j for the number of lies among k reports, and $(x)_+$ for the larger of x and zero. Then the smallest δ at a total ε' is

$$\delta(\varepsilon') = 1 - (1-\delta)^k\bigl(1 - \delta_{\text{RR}}(\varepsilon')\bigr), \qquad \delta_{\text{RR}}(\varepsilon') = \sum_{j=0}^{k} \binom{k}{j} t^{k-j}(1-t)^j\bigl(1 - e^{\varepsilon' - (k-2j)\varepsilon}\bigr)_+. \tag{21.3}$$

No rule that sees only each step's (ε, δ) can do better.

To use it, evaluate the sum for a candidate ε' and search for the smallest one whose δ meets your target. For the hundred queries that gives 4.3068.

Two things go wrong when you compute it. The closed form in the paper divides two huge numbers, and for a thousand steps at $\varepsilon = 1$ it overflows double precision. The fix is to add the terms of Equation 21.3 through their logarithms, as the code below does. And when the steps have different privacy levels, the exact bound is computationally hard (Murtagh & Vadhan 2016), so use an accountant instead.

The bigger limit is that "optimal" means optimal for the class. A real mechanism has a whole curve of (ε, δ) pairs, and summarizing each step by one point forgets the rest of the curve. A thousand Gaussian steps with noise standard deviation 20, on a query of sensitivity 1, are exactly $(7.51, 10^{-5})$-DP. Converting each step to its best single pair and composing optimally gives 62.7, about 8 times worse. So whenever you know the mechanism, compose its whole curve with an accountant from Privacy accounting. In Google's `dp_accounting` library that looks like this:

```
import dp_accounting

step = dp_accounting.GaussianDpEvent(noise_multiplier=20.0)
accountant = dp_accounting.pld.PLDAccountant()
accountant.compose(dp_accounting.SelfComposedDpEvent(step, 1000))
print(accountant.get_epsilon(target_delta=1e-5))
```

It should print a value close to the exact 7.51. The same library's `dp_accounting.rdp.RdpAccountant` takes the same events, and Microsoft's `prv_accountant` package is a second numerical accountant to check against.

21.5. Adaptive composition

In practice the next query often depends on the last answer, as when a training loop computes each gradient at parameters set by earlier noisy ones. All the rules above still hold, as long as each step is (ε, δ)-DP whatever happened before it.

Two kinds of adaptivity break the rules. The first is choosing the privacy parameters during the run, so that each step's ε or the number of steps depends on earlier answers. A rule like "keep querying until the budget is spent" is of this kind. It needs a **privacy filter**, which is a stopping rule with its own theorem. Reporting the budget spent so far at every moment needs a **privacy odometer** instead. Adaptive composition covers both.

The second is letting the raw data steer the run. A stopping rule that reads only the noisy answers is post-processing and costs nothing. But a system that answers private counts until the true count of some group crosses a threshold reveals that count through how many answers it gives. Every answer can then be ε-DP, while the whole run is not DP for any finite ε.

21.6. Code

The snippet computes advanced and optimal composition, and inverts the optimal rule to give each step its share of a total budget. It prints the two totals quoted above, then plans a dashboard of 100 counts under a total budget of $(1, 10^{-5})$.

```
from math import exp, expm1, lgamma, log, log1p, sqrt, tanh


def advanced_epsilon(eps: float, k: int, delta_prime: float) -> float:
    """Advanced composition with the sharp drift term k * eps * tanh(eps / 2)."""
    return eps * sqrt(2 * k * log(1 / delta_prime)) + k * eps * tanh(eps / 2)


def optimal_delta(eps_total: float, eps: float, k: int, delta: float = 0.0) -> float:
    """Kairouz-Oh-Viswanath: the exact delta(eps_total) of k (eps, delta)-DP steps, in log s
    log_t, log_1mt = -log1p(exp(-eps)), -log1p(exp(eps))  # log of truthful / flipped probab
    logs: list[float] = []
    for j in range(k + 1):  # j of the k reports are flipped
        loss = (k - 2 * j) * eps
        if loss <= eps_total:
            break
        log_binom = lgamma(k + 1) - lgamma(j + 1) - lgamma(k - j + 1)
        logs.append(log_binom + (k - j) * log_t + j * log_1mt + log(-expm1(eps_total - loss)
    top = max(logs, default=0.0)
    delta_rr = exp(top) * sum(exp(x - top) for x in logs) if logs else 0.0
    return 1 - (1 - delta) ** k * (1 - delta_rr)


def optimal_epsilon(eps: float, k: int, target_delta: float, delta: float = 0.0) -> float:
    """The smallest total epsilon whose optimal delta is at most target_delta, by bisection.
    lo, hi = 0.0, k * eps
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if optimal_delta(mid, eps, k, delta) > target_delta else (lo, mid
    return hi


def per_step_epsilon(eps_total: float, k: int, target_delta: float) -> float:
    """The largest per-step epsilon whose k-fold optimal composition fits in (eps_total, tar
    lo, hi = eps_total / k, eps_total
    for _ in range(50):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if optimal_epsilon(mid, k, target_delta) <= eps_total else (lo, m
    return lo
```

```
print(f"{advanced_epsilon(0.1, 100, 1e-5):.4f}")  # 5.2981
print(f"{optimal_epsilon(0.1, 100, 1e-5):.4f}")  # 4.3068
eps0 = per_step_epsilon(1.0, 100, 1e-5)
print(f"{eps0:.4f} {1 / eps0:.1f}")  # 0.0271 37.0
print(f"{1.0 / sqrt(2 * 100 * log(1e5)):.4f}")  # 0.0208
```

In the dashboard line, basic composition gives each count $\varepsilon = 0.01$, so a count of sensitivity 1 gets Laplace noise of scale 100. Optimal composition allows each count 0.0271, which cuts the scale to 37.0, while the rule of thumb from advanced composition, 0.0208, sits in between.

21.7. Cheat sheet

Table 21.3.: The composition rules of this chapter. All of them hold for adaptively chosen queries.

Rule	Total for k steps that are each (ε, δ)-DP	When to use it
Basic	$(k\varepsilon,\ k\delta)$	Few steps, or $\delta = 0$ required
Advanced	$(\varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2),\ k\delta + \delta')$	Quick budgets; per-step share $\approx \varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$
Optimal	Equation 21.3, in log space, inverted by bisection	The number you report, when you know only (ε, δ)
Accountant	The whole privacy curve of each step	Whenever you know the mechanism

Rules of thumb

- Take the smaller of basic composition and whichever other rule you use, and never report basic composition for thousands of steps.
- Count every step's δ, because the slacks add up to $k\delta$.
- If you know the mechanism, compose its curve, because a single (ε, δ) per step can be 8 times too pessimistic.
- Fix the number of steps and their budgets before the run, and never change a step's ε after seeing results unless a privacy filter covers it.
- Let only released outputs, never the raw data, decide what runs next.

21.8. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.5: basic and advanced composition, in the form most libraries cite.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): the worst-case mechanism and the optimal rule.

Derivations. Every result here is derived step by step in the full edition's chapter.

22. Selection: exponential mechanism, report-noisy-max, sparse vector

22.1. Why this matters

Many questions about a dataset ask you to pick something rather than to compute a number. You might want the model that scores best on a validation set, a clipping bound for private training, or the first alarm among a thousand statistics.

Adding Laplace noise to every score and publishing them all would cost one release per candidate. The mechanisms of this chapter follow one rule instead, which is to **release the position, not the values**. Each one publishes only which candidate won, and one record can change that only a little, however many candidates there are. So under differential privacy (DP), a selection costs as much as a single query, and the number of candidates enters only the accuracy, through its logarithm.

22.2. Which mechanism to use

Table 22.1 matches common tasks to the mechanisms of this chapter. Throughout, the **sensitivity** Δ is the most that one person's record can change any candidate's score.

Table 22.1.: Which selection mechanism to use.

Task	Mechanism	Privacy cost
Pick the best of a fixed list	Exponential mechanism, sampled with Gumbel noise	ε
Pick the top c, such as the most frequent items	Gumbel noise, keep the c largest, or noisy max c times	$c\varepsilon$
Pick a number from a range, such as a quantile or a clipping bound	Exponential mechanism, piece by piece	ε
Find the first query in a stream above a threshold	AboveThreshold	ε, however long the stream
Find the first c such queries	AboveThreshold restarted c times	ε, with ε/c per run

22.3. The exponential mechanism

Each candidate r has a **score** $u(D, r)$ on the dataset D, such as the number of validation examples a model gets right. For a split in a private decision tree, it can be how well the split separates the labels. The **exponential mechanism** of McSherry and Talwar (2007) picks each candidate with probability proportional to an exponential of its score. With a privacy budget ε, it outputs candidate r with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\varepsilon\, u(D, r)/(2\Delta)\right)}{\sum_s \exp\left(\varepsilon\, u(D, s)/(2\Delta)\right)}. \tag{22.1}$$

It is ε-DP because one record shifts each weight, and their sum, by at most a factor $e^{\varepsilon/2}$.

Two conditions decide whether a use is correct. First, fix the list of candidates before you look at the data, because a list built from the data, such as a corpus's vocabulary, reveals records by itself. Second, drop the 2 in the exponent only when the scores are **monotone**, meaning that one record pushes all of them the same way. Counts under add/remove neighbours are monotone, but a histogram under replace neighbours is not, because moving one record lowers one count and raises another, and dropping the 2 there doubles the real cost.

22.4. Accuracy, and the cost of many candidates

The number of candidates k is free for privacy but not for accuracy, because more candidates means more bad ones to pick by accident. Write $\mathrm{OPT}(D)$ for the best score, and call the best score minus the chosen candidate's score the **gap**. For any $t > 0$, the gap stays below a threshold except with probability e^{-t}:

$$\Pr\left[\mathrm{OPT}(D) - u(D, M_E(D)) \geq \frac{2\Delta}{\varepsilon}(\ln k + t)\right] \leq e^{-t}. \tag{22.2}$$

On average the gap is at most $\frac{2\Delta}{\varepsilon}(\ln k + 1)$, so doubling the candidates adds only $\ln 2$ to the bracket.

For example, suppose you trained 50 models and want to release the best one, judged on a private validation set of 1,000 examples. Model r gets $900 - r$ examples right, and the score is that count, so $\Delta = 1$. At $\varepsilon = 1$ the mechanism picks the best model only 39% of the time. Yet it loses just 1.54 correct predictions on average, or 0.15% accuracy, because its mistakes are mostly near-ties.

Scoring each classifier in a finite class by minus its training mistakes makes the mechanism a private learner, which Hyperparameter tuning uses.

22.5. A private quantile or clipping bound

A common clipping bound for private training is the median of the gradient norms, and reading it off the data leaks it, so pick it with the exponential mechanism. First fix a range from 0 to an

upper end B before looking at the data, and clip every record into it. For the quantile q, such as the median at $q = 0.5$, score each point r of the range by

$$u(D, r) = -\big|(1-q)\,\#\{x_i < r\} - q\,\#\{x_i > r\}\big|, \tag{22.3}$$

where $\#\{x_i < r\}$ counts the records below r. One added or removed record moves this score by at most the larger of q and $1-q$, which is its sensitivity. The score is constant between consecutive sorted records, so the range splits into **pieces**. Pick a piece with probability proportional to its length times the weight of Equation 22.1, and return a uniform point inside it.

For the median, the guarantee bounds the **imbalance** of the output, the gap between the numbers of records on its two sides. Except with probability e^{-t}, it is at most $\frac{2}{\varepsilon}(\ln(B/L^\star) + t)$, where $L^\star$ is the total length of the best pieces. For example, take 1,000 lognormal gradient norms capped at $B = 10$, whose median is 0.9804. At $\varepsilon = 1$ the private median lands on average 0.0013 from it, and widening the range to $B = 1000$ leaves that unchanged, because B enters only through a logarithm. IBM's diffprivlib provides this as `diffprivlib.tools.quantile` and `diffprivlib.tools.median`, which take an `epsilon` and the range as `bounds`. Tumult Analytics has the same aggregations on its `QueryBuilder`. For a general list of candidates, diffprivlib has the `mechanisms.Exponential` class.

Two things go wrong in practice. The guarantee bounds the output's rank, not its value, so where the data near the quantile are sparse the output can land anywhere in a long piece. And the range must not come from the data, because setting B to the observed maximum reintroduces the leak.

22.6. Report-noisy-max and Gumbel noise

Report-noisy-max adds independent Laplace noise to each candidate's score, and outputs the candidate with the largest noisy score. With noise of scale $2\Delta/\varepsilon$ it is ε-DP for any number of candidates, and with monotone scores a scale of Δ/ε is enough. Release only the index of the winner, and spend extra budget on a fresh Laplace answer if you need its score.

On the 50 models at $\varepsilon = 1$, it picks the best with probability 0.4241, against 0.3935 for the exponential mechanism, but neither mechanism is always better.

Gumbel noise of scale $2\Delta/\varepsilon$ in place of the Laplace noise gives exactly the exponential mechanism. This **Gumbel-max trick** is the stable way to sample the mechanism, because computing the weights overflows for scores in the thousands. Keeping the c largest Gumbel-noised scores picks the top c in one shot, at the cost of c exponential mechanisms (Durfee & Rogers 2019).

22.7. The sparse vector technique

Now the queries arrive one at a time, and you want the first one that is high enough. The **sparse vector technique** (Dwork et al. 2009) compares each query with one noisy threshold and pays only for the query that fires. Its basic form, **AboveThreshold**, takes a threshold T and a stream of queries that share a sensitivity Δ. It splits the budget into ε_1 for the threshold and ε_2 for the queries.

1. Draw one threshold noise, written ρ, from $\mathrm{Lap}(\Delta/\varepsilon_1)$ for the whole run.
2. For each query, draw a fresh noise from $\mathrm{Lap}(2\Delta/\varepsilon_2)$ and add it to the query's answer. If the result is at least $T + \rho$, output "above" and halt. Otherwise output "below" and go on.

AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP for any number of "below" answers, even when each query is chosen after the earlier answers.

The accuracy does depend on the stream length k, which must be fixed before the run. With an equal split and failure probability β, every query is classified correctly unless its true value is within a margin

$$\alpha = \frac{8\Delta(\ln k + \ln(2/\beta))}{\varepsilon} \tag{22.4}$$

of the threshold. For example, monitor 1,000 counts against $T = 500$, at $\varepsilon = 1$. With $\beta = 0.05$ the margin is 84.77, so with probability 0.95 only counts within 84.77 of 500 can be misclassified.

To find the first c queries above the threshold, restart AboveThreshold with fresh threshold noise after each "above", at ε/c per run, and halt after the c-th. The margin then grows with c, so ask for as few alarms as you can. The split between ε_1 and ε_2 affects only accuracy, and Lyu, Su and Li recommend $\varepsilon_1 : \varepsilon_2 = 1 : (2c)^{2/3}$.

22.8. When the sparse vector technique goes wrong

Several published variants are not private (Table 22.2).

Table 22.2.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Actual privacy
Algorithm 3	Outputs the noisy value of each "above"	∞
Algorithm 4	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5	No noise on the queries	∞
Algorithm 6	Never halts	∞

The no-halt variant is the one to remember. It adds exactly as much noise per query as AboveThreshold, so a reviewer who checks noise scales will miss it. Yet at $\varepsilon = 1$, on neighbours with 40 low and then 40 high queries, its privacy loss is 18.1246.

So review the accounting, not just the noise. An implementation must draw fresh noise for every query and one threshold noise per run. A threshold redrawn for each query makes every "below" answer cost budget that nobody counted. It must halt after the allowed number of "above" answers, and it must never publish the noisy values it compared. Reusing one noise draw for all queries quietly turns it into Algorithm 5.

22.9. Code

The snippet picks a private quantile piece by piece, as a clipping bound would.

```
import random
from math import exp, log
from typing import Sequence


def private_quantile(data: Sequence[float], q: float, upper: float, eps: float, rng: random.
    """Exponential mechanism for the q-quantile over [0, upper], add/remove neighbours."""
    sensitivity = max(q, 1 - q)
    xs = sorted(min(max(x, 0.0), upper) for x in data)  # clip into the range fixed in advan
    edges = [0.0, *xs, upper]
    n = len(xs)
    logw = [  # piece j has j records below it and n - j above; empty pieces get no weight
        log(edges[j + 1] - edges[j]) - eps * abs((1 - q) * j - q * (n - j)) / (2 * sensitivi
        if edges[j + 1] > edges[j]
        else -1e300
        for j in range(n + 1)
    ]
    top = max(logw)
    j = rng.choices(range(n + 1), weights=[exp(w - top) for w in logw])[0]
    return edges[j] + rng.random() * (edges[j + 1] - edges[j])


gen = random.Random(0)
norms = [gen.lognormvariate(0.0, 0.75) for _ in range(1000)]  # gradient norms, median about
print(f"{sorted(norms)[499]:.4f} {sorted(norms)[500]:.4f}")  # 1.0273 1.0296
print(f"{private_quantile(norms, 0.5, 10.0, 1.0, random.Random(1)):.4f}")  # 1.0292
```

The private median lands between the two middle norms. A floating-point sampler like this one is fine for checking a calculation but not for production, as Discrete and floating-point-safe mechanisms in the full edition explains.

22.10. Cheat sheet

Table 22.3.: The selection mechanisms of this chapter, with k candidates or queries.

Mechanism	Privacy	Accuracy
Exponential mechanism, Equation 22.1	ε; drop the 2 only for monotone scores	Mean gap $\leq \frac{2\Delta}{\varepsilon}(\ln k + 1)$
Report-noisy-max, noise $\mathrm{Lap}(2\Delta/\varepsilon)$	ε, index only	Similar to the exponential mechanism

Mechanism	Privacy	Accuracy
AboveThreshold	$\varepsilon_1 + \varepsilon_2$	Margin $8\Delta(\ln k + \ln \frac{2}{\beta})/\varepsilon$, failure probability β

Rules of thumb

- Release positions, not values, because every released noisy value needs its own budget.
- Fix the candidate list, and the range of any number you pick, before you look at the data.
- Keep the factor 2Δ unless the scores are monotone under your neighbouring relation.
- A sparse vector implementation draws fresh noise per query and one threshold noise per run, halts, and publishes nothing it compared.

22.11. Further reading

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3, §3.4 and §3.6: the three mechanisms in their standard form.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (VLDB 2017): the budget split and the survey of broken variants.

Derivations. Every result here is derived step by step in the full edition's chapter.

23. Query release and private multiplicative weights

23.1. Why this matters

Many private releases are a large batch of simple statistics rather than a single number. A statistics office, for example, publishes every cross-tabulation of a census file. Each of these numbers is a **linear query**, the average over the records of a function that gives each record a value between 0 and 1. The most common batch is a set of **marginal tables**. Each table counts the records for every combination of values of a few attributes, and each count is a **cell**.

Adding noise to every answer is the direct approach, but the noise has to grow with the number of queries, and with millions of queries the answers drown. **Private multiplicative weights** fills the gap. It answers any set of linear queries with an error that grows only with the logarithm of their number. It also produces a **synthetic dataset**, a weighted list of possible records that answers every query in the set.

23.2. Which method to use

The choice depends on how many queries you have, compared with the number of records, and on how many attributes a record has (Table 23.1).

Table 23.1.: Which query-release method to use.

Situation	Use
Hundreds or thousands of queries, especially marginals	Gaussian noise on every answer, scaled to the structure of the set
The same, but pure differential privacy is required	Laplace noise on every answer
Far more queries than records, or queries not known in advance	Synthetic data
The same, with few attributes	Offline private multiplicative weights
Queries arrive one at a time, perhaps adaptively	Online private multiplicative weights
The same, with dozens of attributes	A graphical-model method such as MST or AIM

23.3. Noise on every answer

Write n for the number of records, which is public, and $|Q|$ for the number of queries in the set Q. Replacing one record moves each answer by at most $1/n$. So the ℓ_1 sensitivity of all the answers together is at most $|Q|/n$, and their ℓ_2 sensitivity is at most $\sqrt{|Q|}/n$.

Fix a target error α and a failure probability β. At privacy level ε, the Laplace mechanism of The Laplace and Gaussian mechanisms in the full edition keeps every error below α, with probability at least $1-\beta$, once

$$n \geq \frac{|Q| \ln(|Q|/\beta)}{\alpha\varepsilon}. \tag{23.1}$$

The Gaussian mechanism also allows a small slack δ, and it needs two further numbers. Write μ for the largest ratio of sensitivity to noise standard deviation that its exact calibration allows, which is 0.2367 at $\varepsilon = 1$ and $\delta = 10^{-6}$. And write z for the number that a standard normal variable exceeds with probability $\beta/(2|Q|)$. It then needs

$$n \geq \frac{\sqrt{|Q|}\, z}{\mu\alpha}. \tag{23.2}$$

So the records needed grow like $|Q|$ under pure differential privacy (DP), and like $\sqrt{|Q|}$ under approximate DP.

Structure in the queries helps a lot. Replacing one record lowers one cell of a marginal table and raises another, so each table adds only $2/n$ to the ℓ_1 sensitivity, whatever its number of cells. Take a running example of $n = 10{,}000$ records with eight yes/no attributes, so that there are 256 possible records. Every cell of every marginal table makes 6,560 cells in 255 tables. A uniform histogram, which ignores the data, has a largest error of 0.232 over these cells. At $\varepsilon = 1$, Laplace noise on every cell has a largest error of 0.497, which is worse than ignoring the data. Gaussian noise at $\delta = 10^{-6}$ does much better, with 0.037, and on this example nothing in this chapter beats it.

Two mistakes are common. Scaling each cell's noise to its own sensitivity of $1/n$ silently spends ε once per cell. And splitting ε evenly over thousands of separately noised queries with basic composition wastes most of the budget. Calibrate to the joint sensitivity instead, or use the composition in the full edition rules.

23.4. Private multiplicative weights

The engine is **multiplicative weights**, a method from online learning. It keeps a weight on each of N possible records, which together form a guessed histogram, starting from uniform. Each round it takes a query on which the guess is off by more than α. It then moves the weights of the records that query counts up or down to correct it. Online learning caps the number of such corrections at

$$\frac{4 \ln N}{\alpha^2}, \tag{23.3}$$

whatever the number of queries. That is why the privacy budget pays for corrections rather than for queries.

The private version, the **multiplicative weights exponential mechanism (MWEM)** of Hardt, Ligett and McSherry (2012), runs a fixed number T of rounds. Each round has three steps, and the first two each spend a budget ε_0.

1. **Select.** Pick a query with the exponential mechanism in the full edition, using the guess's error on each query as the score. The score has sensitivity $1/n$.
2. **Measure.** Release the chosen query's answer plus fresh Laplace noise of scale $1/(n\varepsilon_0)$.
3. **Update.** Apply one multiplicative-weights step with that query, toward the measurement.

The release is the average of the T guesses, and giving each step $\varepsilon_0 = \varepsilon/(2T)$ makes it ε-DP by basic composition. Write $\bar{p}$ for the released average. With probability at least $1-\beta$, every query q in the set has an error of at most

$$\max_{q} \left| q(\bar{p}) - q(D) \right| \le 2\sqrt{\frac{\ln N}{T}} + \frac{2}{n\varepsilon_0}\Big(\ln |Q| + 2 \ln \frac{2T}{\beta} \Big). \tag{23.4}$$

The first term is the learning error, which falls with more rounds. The second is the privacy noise, which grows with more rounds because each gets less budget. So fix T in advance by minimizing the bound, which uses only public quantities. On the running example at $\varepsilon = 1$ the bound picks 38 rounds, and the largest error is 0.078, against 0.098 with 10 rounds and 0.123 with 100.

So on 6,560 queries, private multiplicative weights beats Laplace noise but loses to Gaussian noise by a factor of about two. Its advantage is that its error grows only with $\ln |Q|$. For queries with no structure over 2^{20} possible records, it overtakes Gaussian noise only near 10^9 queries.

Two implementation rules matter. The measurement must use fresh noise, because reusing the exponential mechanism's internal scores would use the data without paying for it. And release the average of the guesses, not the last one, because the guarantee covers only the average.

23.5. Synthetic data in practice

The running time of private multiplicative weights grows with the number of possible records, because every round updates a weight for each one. Some such cost is unavoidable, because under standard cryptographic assumptions no efficient method produces private synthetic data that is accurate for every two-way marginal (Ullman & Vadhan 2011). So practical tools keep the loop but change its parts, in a pipeline of four steps.

1. **Select** a set of marginal tables to measure, from domain knowledge or privately with the exponential mechanism.
2. **Measure** each whole table with Gaussian noise, scaled to the joint sensitivity of the set.
3. **Generate** a compact graphical model that fits the noisy tables, instead of a full histogram.
4. **Sample** a synthetic dataset from the model.

Everything after the measurements is post-processing, so the synthetic data can be queried and shared at no further privacy cost. The same holds for repairs that make the noisy answers consistent, such as rounding negative counts up to zero or forcing the cells of a table to add up to its total. The 2020 US Census TopDown algorithm works this way at national scale. It measures many tables with discrete Gaussian noise, and then post-processes them into consistent, non-negative counts.

MST and AIM, both from McKenna and coauthors, are the standard methods, and their private-pgm library implements both. SmartNoise Synth, the `snsynth` package, wraps MWEM, MST and AIM behind one interface that fits a table at a given ε and samples synthetic rows. For the noisy-marginal baseline alone, OpenDP and Tumult Analytics provide Gaussian counts with the accounting done for you.

23.6. Online release and SmallDB

A query server that receives queries one at a time, perhaps chosen adaptively, can run the online form of private multiplicative weights (Hardt & Rothblum 2010). It answers from the guess when the sparse vector technique in the full edition finds the guess good enough, and pays only for updates. It is mainly a guarantee, used in Adaptive data analysis. SmallDB (Blum, Ligett & Roth 2008) picks a whole small dataset with one exponential mechanism. On the running example it would weigh about 10^{4568} candidates, so it shows what is possible rather than what you can run.

23.7. Code

This snippet runs private multiplicative weights with pure DP on four yes/no attributes, so 16 possible records, and all 80 marginal cells.

```
import itertools
import random
from math import exp, log, sqrt

Query = list[float]  # one value in [0, 1] for each possible record


def answer(q: Query, hist: list[float]) -> float:
    return sum(qx * px for qx, px in zip(q, hist))


def laplace(scale: float, rng: random.Random) -> float:
    return rng.expovariate(1 / scale) - rng.expovariate(1 / scale)


def private_mw(
    hist: list[float], n: int, queries: list[Query], eps: float, rounds: int, rng: random.Ra
) -> list[float]:
    """Select (exponential mechanism), measure (Laplace), update (multiplicative weights); p
    eps0 = eps / (2 * rounds)  # 2T steps under basic composition
    gamma = sqrt(log(len(hist)) / rounds)
    guess = [1 / len(hist)] * len(hist)
    total = [0.0] * len(hist)
    for _ in range(rounds):
```

```
        total = [t + g for t, g in zip(total, guess)]
        errors = [abs(answer(q, guess) - answer(q, hist)) for q in queries]
        top = max(errors)
        weights = [exp(eps0 * n * (e - top) / 2) for e in errors]  # score sensitivity 1/n
        q = rng.choices(queries, weights=weights)[0]
        measured = answer(q, hist) + laplace(1 / (n * eps0), rng)  # fresh noise
        sign = 1.0 if answer(q, guess) > measured else -1.0
        guess = [g * (1 - gamma * sign * qx) for g, qx in zip(guess, q)]
        guess = [g / sum(guess) for g in guess]
    return [t / rounds for t in total]  # release the average guess


d = 4
universe = list(itertools.product((0, 1), repeat=d))
queries = [
    [1.0 if all(x[a] == v for a, v in zip(attrs, vals)) else 0.0 for x in universe]
    for k in range(1, d + 1)
    for attrs in itertools.combinations(range(d), k)
    for vals in itertools.product((0, 1), repeat=k)
]
gen = random.Random(0)
n = 5000
records = [tuple(int(gen.random() < p) for p in (0.8, 0.6, 0.3, 0.2)) for _ in range(n)]
hist = [records.count(x) / n for x in universe]
synth = private_mw(hist, n, queries, 1.0, 20, random.Random(1))
print(len(queries))  # 80
print(f"{max(abs(answer(q, synth) - answer(q, hist)) for q in queries):.4f}")  # 0.0877
print(f"{max(abs(answer(q, [1 / 16] * 16) - answer(q, hist)) for q in queries):.4f}")  # 0.3
```

The synthetic histogram's largest error over the 80 cells is 0.0877, against 0.3866 for the uniform histogram.

23.8. Cheat sheet

Table 23.2.: The query-release methods of this chapter. N is the number of possible records, $|Q|$ the number of queries, β the failure probability, and μ and z the Gaussian calibration numbers of Section 23.3.

Method	Privacy	Records for error α on every query
Laplace on every answer	ε	$\lvert Q\rvert \ln(\lvert Q\rvert/\beta)/(\alpha\varepsilon)$
Gaussian on every answer	(ε, δ)	$\sqrt{\lvert Q\rvert}\, z/(\mu\alpha)$
Offline private multiplicative weights	ε or (ε, δ)	Grows with $\ln\lvert Q\rvert$ and $\ln N$; choose T from Equation 23.4

Method	Privacy	Records for error α on every query
Select, measure, generate (MST, AIM)	ε or (ε, δ)	Not a closed form; the practical choice for many attributes

Rules of thumb

- For hundreds or thousands of queries, add Gaussian noise to every answer, scaled to the structure of the query set.
- Use private multiplicative weights when the queries far outnumber the records, or when you want a synthetic dataset.
- Fix the number of rounds from the bound, never from the data, then measure with fresh noise and release the average guess.
- Beyond a few dozen attributes, use a graphical-model method such as AIM.
- Synthetic data from a generator trained without DP is not anonymised, however synthetic it looks.
- A generator is accurate on the marginals it was fit to by construction, so evaluate it on other queries.

23.9. Further reading

- Hardt, Ligett & McSherry, "A Simple and Practical Algorithm for Differentially Private Data Release" (NeurIPS 2012): MWEM, with experiments on real tables.
- McKenna, Mullins, Sheldon & Miklau, "AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data" (VLDB 2022): the select–measure–update loop as used in practice.

Derivations. Every result here is derived step by step in the full edition's chapter, except the hardness of efficient synthetic data (Ullman & Vadhan), which it cites.

24. Beyond global sensitivity

24.1. Why this matters

The Laplace and Gaussian mechanisms in the full edition set their noise by the **global sensitivity**. That is the most one record can move the answer, over every pair of neighbouring datasets that could exist. It is 1 for a count, but for many statistics it is useless. The median of ages between 0 and 100 has global sensitivity 100. So at a privacy level of $\varepsilon = 1$, the Laplace mechanism misses the median by 69.31 years in a typical release. The mean of incomes has no bound, and for a trained model nobody knows one. Yet on a sample of 1,001 ages, one person moves the median by only 0.1386 years.

This chapter shows how to add noise on the scale of the data you hold and still satisfy differential privacy (DP). Every technique respects one rule: **a noise scale computed from the data is itself a statistic of the data, so it must be private too.** Throughout, datasets have a fixed size n, and two datasets are neighbours when one replaces a single record of the other.

24.2. Local sensitivity, and why it leaks

The **local sensitivity** of a query f at your dataset D is the most that one record can move the answer, starting from D itself:

$$\mathrm{LS}_f(D) = \max_{D' \text{ a neighbour of } D} |f(D) - f(D')|. \tag{24.1}$$

For the median of an odd number of values, it is the larger of the two gaps between the middle value and its sorted neighbours.

The tempting release adds Laplace noise of scale $\mathrm{LS}_f(D)/\varepsilon$, and it is not private. For example, the five ages $(0, 0, 0, 0, 100)$ have median 0 and no gap around it, so they get no noise and the output is exactly 0. Their neighbour $(0, 0, 0, 100, 100)$ has the same median but a gap of 100 above it. Its output is spread out and never exactly 0, so an output of 0 names the first dataset with certainty. An observer reads the data from the width of the noise, not from its centre.

The same leak hides in any parameter set from the private data, such as a clip bound set to the observed maximum. The diffprivlib functions `diffprivlib.tools.mean` and `diffprivlib.tools.median` take a `bounds` argument, and if you leave it out they compute the bounds from the data and raise a `PrivacyLeakWarning`. Treat that warning as an error.

24.3. Clipping

Clipping forces a bound on unbounded data. Replace each non-negative value x by $\min(x, C)$ for a clip bound C that you choose. One record then moves the sum by at most C and the mean by at most C/n, so the clipped mean is ε-DP:

$$M(D) = \frac{1}{n}\sum_{i=1}^{n} \min(x_i, C) + \mathrm{Lap}\Big(\frac{C}{n\varepsilon}\Big). \tag{24.2}$$

Per-example gradient clipping in private training is the same move applied to Euclidean length, set in Opacus by the `max_grad_norm` argument of `PrivacyEngine.make_private` (Private stochastic gradient descent).

The price is a bias, because clipping removes the part of each value above C. Write X for a value drawn from the data's distribution, and $(x - C)_+$ for $\max(x - C, 0)$. For n independent values, the mean squared error is

$$\mathrm{err}(C)^2 = \underbrace{\mathbb{E}[(X - C)_+]^2}_{\text{bias}^2} + \underbrace{\frac{\mathrm{Var}(\min(X, C))}{n}}_{\text{sampling}} + \underbrace{\frac{2C^2}{n^2\varepsilon^2}}_{\text{noise}}. \tag{24.3}$$

Raising C shrinks the bias and grows the noise.

Take log-normal incomes in thousands with median 40, and a thousand people at $\varepsilon = 1$. Clipping at the median gives a root-mean-square error (RMSE) of 23.417, because half the incomes are cut back. The optimum is 331.34, which cuts off only 0.4% of the population. Its RMSE is 1.666, while the non-private mean has 1.649.

Choosing C is the step that goes wrong, so never set it to the observed maximum or an empirical percentile. Use this order instead:

1. Take C from domain knowledge, such as a legal maximum or a sensor range.
2. Otherwise, fix it from an error model like Equation 24.3, with a distribution assumed before you see any records.
3. Otherwise, spend a small slice of the budget on a private high quantile by inverse sensitivity (Section 24.5), for example with `diffprivlib.tools.quantile`, and clip at that value.
4. Charge that slice to the budget by composition in the full edition. Forgetting it is a common bug.

The libraries make the bounds explicit. OpenDP clamps with `dp.t.make_clamp` before `dp.t.make_sum`, diffprivlib takes `bounds` in `diffprivlib.tools.mean`, and Tumult Analytics takes them in `QueryBuilder(...).average(column, low, high)`.

Clipping only from above always underestimates the mean, and a two-sided clip cancels its bias only when the range is centred on the data, so estimate the centre privately first (Private mean estimation in the full edition). Under add/remove neighbours the size n is not public, so release a noisy clipped sum and a noisy count, and divide them.

24.4. Subsample-and-aggregate

When f is a model fit, a ratio or a pipeline you cannot analyse, **subsample-and-aggregate** treats it as a black box. Split the n records into B disjoint blocks, with a partition fixed in advance, for example by a hash of a record identifier. Run f on each block, and clip each answer to the range from 0 to a bound R fixed in advance. Then release the average of the clipped answers z_j plus Laplace noise:

$$M(D) = \frac{1}{B}\sum_{j=1}^{B} \mathrm{clip}_{[0,R]}(z_j) + \mathrm{Lap}\Big(\frac{R}{B\varepsilon}\Big). \tag{24.4}$$

One record lives in one block, so it moves the average by at most R/B, and the release is ε-DP for any f.

The number of blocks trades bias against noise. Write z for the output of f on one block and θ for the target. When clipping is inactive and the blocks are independent, the mean squared error is

$$\underbrace{\big(\mathbb{E}[z]-\theta\big)^2}_{\text{bias of } f \text{ on } n/B \text{ records}} + \underbrace{\frac{\mathrm{Var}(z)}{B}}_{\text{sampling}} + \underbrace{2\Big(\frac{R}{B\varepsilon}\Big)^2}_{\text{noise}}. \tag{24.5}$$

More blocks dilute the noise, but they make each block smaller, which raises the bias.

For example, estimate the rate of 10,000 exponential waiting times by one over the block mean, with $R = 5$. With 10 blocks the noise dominates and the RMSE is 0.7072, while with 1,000 blocks the bias dominates and it is 0.1120. The best grid point lies between them, at 250 blocks, where the RMSE is 0.0396. A non-private estimate has 0.0100, so privacy costs a factor of four, for an estimator whose sensitivity you never had to bound. Choose B from this error model before looking at the data.

24.5. Medians and quantiles: inverse sensitivity

For a median or a quantile, start with inverse sensitivity (Asi & Duchi 2020). For each candidate output t, it counts how many records you would have to change before the median equals t, and calls that count $\mathrm{len}(D, t)$. It then picks t with a probability that falls by a factor $e^{-\varepsilon/2}$ for each change needed:

$$p_D(t) \propto e^{-\varepsilon\,\mathrm{len}(D,t)/2}. \tag{24.6}$$

This is the exponential mechanism in the full edition with the count as a penalty. One record changes every count by at most 1, so the release is pure ε-DP.

For the median, the count is easy to compute. Sort the values, and pad them with 0 below and with the top of the range Λ above. The two gaps next to the median need one change, the next two out need two, and so on. So you pick a gap with probability proportional to its length times $e^{-\varepsilon\,\mathrm{len}/2}$, and then a point uniformly inside it. The function `diffprivlib.tools.quantile` works this way, given `quant`, `epsilon` and `bounds`.

On the 1,001 ages at $\varepsilon = 1$, the typical error is 0.0783 years. That is below even the local sensitivity, and it beats every other method of this chapter. It struggles on small or degenerate data, though. Smaller datasets have wider gaps, so on 101 ages the error grows to 0.696 years.

24.6. Propose-test-release

Propose-test-release (Dwork and Lei 2009) needs only a way to measure how stable a statistic is. You propose a bound b on the local sensitivity before looking at the data. The **distance to instability** is the number of records you must change before the local sensitivity can exceed b, and one record moves it by at most 1. So the mechanism adds $\mathrm{Lap}(1/\varepsilon)$ noise to the distance and refuses, which is written $\perp$, if the result is at or below $\ln(1/\delta)/\varepsilon$, where δ is a small slack. Otherwise it releases $f(D) + \mathrm{Lap}(b/\varepsilon)$, and the whole is $(2\varepsilon, \delta/2)$-DP.

The threshold is demanding, because at $\varepsilon = 1$ and $\delta = 10^{-6}$ it is 13.82 changes. For example, propose $b = 1$ year for the 1,001 ages. Their distance is 21, so the test passes with probability 0.999621, and the release misses by $\ln 2 = 0.693$ years in a typical release, at a cost of 2ε.

24.7. Stability-based release

One record usually cannot change a discrete answer, such as the most common diagnosis, so the **stability-based release** publishes it exactly after a private check. For the most common value, write g for its lead over the runner-up. Its stability score is

$$s = \max\left(0,\ \lceil g/2 \rceil - 1\right), \tag{24.7}$$

where $\lceil g/2 \rceil$ rounds $g/2$ up, and the score is 0 whenever one change could flip the answer. The release adds $\mathrm{Lap}(1/\varepsilon)$ to the score and refuses if the result is at or below $\ln(1/\delta)/\varepsilon$, but otherwise it outputs the exact mode, and the whole is $(\varepsilon, \delta/2)$-DP.

At the same settings as above, the smallest lead that answers 95% of the time is 35 records, whatever the number of categories. In a close race the release refuses, so plan for refusals downstream.

24.8. Smooth sensitivity

Smooth sensitivity (Nissim, Raskhodnikova and Smith 2007) keeps "answer plus noise", but it scales the noise to a bound that changes only slowly between neighbours. For pure DP that noise must have heavy tails, as in the Cauchy release $f(D) + \frac{6}{\varepsilon}\, S^*_{\varepsilon/6}(D)\, Z$, where S^* is the smooth sensitivity and Z a Cauchy draw. It pays off only on dense data, with n in the hundreds or more, and on the 1,001 ages it is about twelve times less accurate than inverse sensitivity. Never pair it with Laplace noise and claim $\delta = 0$, because the Laplace version needs a slack and a much smaller smoothing parameter.

24.9. Cheat sheet

Table 24.1.: Which technique to use for which release, with neighbours that replace one record.

Releasing	Technique	Release	Privacy
A median or quantile	Inverse sensitivity	Pick t with probability $\propto e^{-\varepsilon\,\mathrm{len}(D,t)/2}$	ε
Dense data with a smooth-sensitivity formula	Smooth sensitivity, Cauchy	$f(D) + \frac{6}{\varepsilon} S^*_{\varepsilon/6}(D)\, Z$	ε
A stable statistic with no formula	Propose-test-release	$\perp$ if $\mathrm{dist}_b + \mathrm{Lap}(\frac{1}{\varepsilon}) \le \frac{\ln(1/\delta)}{\varepsilon}$, else $f(D) + \mathrm{Lap}(\frac{b}{\varepsilon})$	$(2\varepsilon, \delta/2)$
A discrete answer with a clear winner	Stability-based release	The same test on a stability score, then $f(D)$ exactly	$(\varepsilon, \delta/2)$
A black-box estimator or model	Subsample-and-aggregate	Mean of B clipped block outputs $+\mathrm{Lap}(\frac{R}{B\varepsilon})$	ε
A mean or sum of unbounded values	Clipping	$\frac{1}{n}\sum \min(x_i, C) + \mathrm{Lap}(\frac{C}{n\varepsilon})$, bias $\mathbb{E}[(X-C)_+]$	ε

Rules of thumb

- Noise scaled to the local sensitivity is not private, and neither is a clip bound or range read off the data.
- For medians and quantiles, start with inverse sensitivity.
- A discrete answer can be released exactly once it leads by about $2\ln(1/\delta)/\varepsilon$ records.
- Fix clip bounds, block counts and proposals before looking at the data, never from its minimum or maximum, and charge any budget spent choosing them.
- Smooth sensitivity with Laplace noise needs a δ; never ship it as pure DP.

24.10. Further reading

- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007): smooth sensitivity, the median formula and sample-and-aggregate.
- Asi & Duchi, "Instance-optimality in Differential Privacy via Approximate Inverse Sensitivity Mechanisms" (NeurIPS 2020): inverse sensitivity, and why it is close to the best possible on every dataset.

Derivations. Every result here is derived step by step in the full edition's chapter.

25. Discrete and floating-point-safe mechanisms

25.1. Why this matters

The proof of the Laplace mechanism in the full edition is about densities on the real line. A computer has only floating-point numbers, a finite set with uneven gaps, so the code that claims to sample Laplace noise produces some other distribution, and the proof says nothing about it. Mironov (2012) showed that the difference matters, because the textbook Laplace sampler, run in double precision, leaks the true answer through the last few bits of its output.

The rule of this chapter is that **differential privacy (DP) is a property of the distribution the code actually produces.** There are two ways to make the code match the proof. The first keeps floating point and repairs the output, by rounding it to a coarse grid and clamping it to a fixed range. The second makes the query integer-valued and samples integer noise exactly, which gives the discrete Laplace and discrete Gaussian mechanisms. In practice you use a vetted library that does one of the two. So the answer to "is `x + np.random.laplace(0, b)` safe to ship?" is no, and the same goes for any numpy or scipy sampler.

25.2. Floating-point Laplace breaks differential privacy

DP lets one record raise the probability of any set of outputs by at most a factor e^{ε}, plus a small slack δ (The definition of differential privacy in the full edition). Now suppose that a set S of outputs can come from a dataset D but never from its neighbour D', so that seeing one of them names D with certainty. The factor e^{ε} then multiplies zero, and the whole probability of S has to be paid for by the slack. This is the **support argument**, which says that for every ε,

$$\delta \;\geq\; \Pr[M(D) \in S]. \tag{25.1}$$

So one output that only one input can reach rules out pure DP. Finding such outputs needs only a list of what each input can produce, so this is the quickest audit of an implementation.

The textbook sampler fails this audit. It adds to the true answer $f(D)$ the noise $\pm b \ln(1/U)$, where U is uniform between 0 and 1 and b is the noise scale. In exact arithmetic that is Laplace noise, but in floating point the logarithm and the sum are both rounded. Floats are dense near 0 and sparse far from it, because their spacing doubles at each power of 2. Take true answers 0 and 1. The outputs near 0 then lie on a fine grid under the first input and on a coarser one under the second, so most of them can come from only one input.

The full edition enumerates every output of a toy sampler with 12-bit floats and $b = 1$. There the outputs that only D can produce carry probability 0.4791, so the guarantee promised at $\varepsilon = 1$ is gone. More precision does not help, since about 48% of the probability sits on such outputs at every precision the full edition tries. Mironov found the same effect at the 53 bits of a double, in several DP libraries of the time.

25.3. The snapping mechanism

The leak lives in which float the output is, not in where it lands on the real line. So if the output must be a float, the low-order bits have to go. Mironov's **snapping mechanism** takes a query with sensitivity 1, a noise scale λ and a clamping bound B. Write Λ for the smallest power of 2 at or above λ. The mechanism clamps the true answer to the range from $-B$ to B, adds floating-point Laplace noise of scale λ, rounds the result to the nearest multiple of Λ, and clamps it again. Every input then reaches the same grid points.

Mironov proves that the snapped output is ε-DP at essentially the exact Laplace mechanism's level, $1/\lambda$, plus a floating-point term that is negligible for doubles and grows with B. Both clamps are needed, because rounding without clamping leaves the extreme outputs reachable from one input only.

Clamping biases answers near $\pm B$, so take B from the range the answer can take, knowing that ε grows with it.

25.4. The discrete Laplace mechanism

Most DP releases are counts and histograms, which take integer values, and integer noise for them has no low-order bits to leak. The **discrete Laplace** distribution keeps the Laplace density at the integers and normalizes it. Each step away from 0 multiplies the probability by the ratio $r = e^{-1/b}$, for a scale $b > 0$. Its probability mass function (PMF), the probability of each integer z, is

$$\Pr[Z = z] = \frac{1 - r}{1 + r}\, r^{|z|}, \qquad r = e^{-1/b}. \tag{25.2}$$

Adding this noise to an integer-valued query is the **geometric mechanism** of Ghosh, Roughgarden and Sundararajan (2009). Write $\Delta_1 f$ for the query's ℓ_1 sensitivity, the largest total change across its coordinates when one record changes. With independent noise on each coordinate, the mechanism is pure ε-DP with

$$\varepsilon = \frac{\Delta_1 f}{b}, \tag{25.3}$$

the same calibration as the continuous Laplace mechanism. The query must be integer-valued, or the two inputs would reach different outputs, which is the support problem again.

For example, a count released at $\varepsilon = 1$ needs the scale $b = 1$. The exact count then comes out with probability 0.4621, and the noise has variance 1.8413, just below the 2 of continuous noise.

25.5. The discrete Gaussian mechanism

The US Census Bureau used the integer analogue of the Gaussian mechanism (Canonne, Kamath and Steinke 2020) for the 2020 Census redistricting data. Pick a positive width σ. The **discrete Gaussian** $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ gives each integer x a probability proportional to $e^{-x^2/(2\sigma^2)}$. The mechanism adds a draw of it to an integer-valued query whose answer moves by an integer Δ between neighbours.

Its exact privacy curve has the continuous Gaussian's form. Write X for a draw of the noise. The smallest δ at a given ε is

$$\delta(\varepsilon) = \Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}\right] - e^{\varepsilon}\Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} + \frac{\Delta}{2}\right]. \tag{25.4}$$

The values differ, though. At $\varepsilon = 1$ and $\sigma = 1$, the discrete δ is 0.1414 where the continuous formula gives 0.1269, and in general the discrete curve wiggles around the continuous one. So compute Equation 25.4 directly, as the snippet in Section 25.8 does, and never report the continuous number for discrete noise.

Accounting by Rényi divergence is simpler. The discrete Gaussian's Rényi divergences never exceed the continuous ones, so it satisfies the same zero-concentrated DP guarantee. An accountant that composes Rényi divergences can therefore use the continuous Gaussian's bound unchanged. For $\sigma \geq 1$, treat its accuracy as the continuous Gaussian's.

25.6. Making a real-valued query integer

Discrete noise needs an integer query, but gradients and sums of real values are not integers. The standard recipe follows.

1. Clip each record's contribution, so that the real-valued query has a known sensitivity (Beyond global sensitivity in the full edition).
2. Divide by a step size s, small next to the noise, and round each value to an integer. Randomized rounding, which rounds up with probability equal to the fractional part, keeps the result unbiased.
3. Recompute the sensitivity after rounding, in units of s, because rounding can push two neighbours' values further apart. Calibrate to that larger value.
4. Add discrete Laplace or discrete Gaussian noise for that sensitivity, then multiply the result by s.

25.7. Exact samplers, and choosing a library

The discrete mechanisms are only as exact as their samplers. A coin flipped as `random() < exp(-gamma)` is inexact, because `exp` rounds and `random()` has only 53 bits. Canonne, Kamath and Steinke publish exact samplers built only from uniform random integers, and you call such samplers rather than write them.

An exact distribution is still not enough, because the running time is a second output. The number of rejection rounds depends on the value drawn, and Jin et al. (2022) show that such timing leaks are exploitable. When you review a library or a pipeline, ask these questions.

- Is the noise drawn by an exact sampler using only integer or rational arithmetic, or by a snapping-style repair, with no floating-point `exp`, `log` or `random()` inside the sampler?
- Do the random bits come from a cryptographic source, such as `secrets.SystemRandom()` in Python, rather than a seeded generator?
- Are the inputs and outputs clamped to a fixed range?

- Is the running time padded or fixed, so that rejection loops are hidden from the adversary?
- Does each release draw fresh noise?
- Does the reported guarantee match the mechanism actually used, including the discrete Gaussian's own $\delta(\varepsilon)$ and the sensitivity of the rounded query?

OpenDP's noise samplers, such as those behind `dp.m.make_laplace` and `dp.m.make_gaussian`, are designed to be exact, so they resist floating-point attacks, and on integer data they give the discrete Laplace and discrete Gaussian. The diffprivlib package offers discrete mechanisms as `diffprivlib.mechanisms.Geometric` and `diffprivlib.mechanisms.GaussianDiscrete`. Google's differential privacy library, which PyDP wraps for Python, also generates its noise with floating-point attacks in mind.

Private stochastic gradient descent adds continuous Gaussian noise to clipped gradients in floating point, so its proof (Private stochastic gradient descent) is about idealized code. Opacus offers `secure_mode=True` when you create its `PrivacyEngine`, which draws the noise from a cryptographically secure generator and is meant to blunt floating-point attacks. Turn it on for any model you release, and record in the privacy statement that the guarantee is for the idealized mechanism.

25.8. Code

Even with a vetted library, check the calibration. The snippet computes the discrete Gaussian's δ from Equation 25.4 by summing over the integers. Floating point is fine here, since it releases no data.

```
from math import ceil, exp


def discrete_gaussian_delta(eps: float, sigma: float, sens: int = 1) -> float:
    """Exact delta(eps) of the discrete Gaussian mechanism, summed over the integers."""
    reach = ceil(40 * sigma) + sens
    weight = {y: exp(-y * y / (2 * sigma**2)) for y in range(-reach - sens, reach + 1)}
    total = sum(weight[y] for y in range(-reach, reach + 1))
    delta = 0.0
    for y in range(-reach, reach + 1):  # outputs of f(D) = 0 and f(D') = sens
        p, q = weight[y] / total, weight[y - sens] / total
        delta += max(p - exp(eps) * q, 0.0)
    return delta


print(f"{discrete_gaussian_delta(1.0, 1.0):.4f}")  # 0.1414
print(f"{discrete_gaussian_delta(1.0, 2.0):.4f}")  # 0.0072
```

25.9. Cheat sheet

Table 25.1.: Which mechanism to use, and what it guarantees.

Situation	Use	Guarantee
Any production release	A vetted library, such as OpenDP	The library's documented guarantee
Any floating-point sampler	The support argument as an audit	$\delta \geq \Pr[M(D) \in S]$, with S reachable from D only
A real-valued output that must stay a float	Snapping: clamp, add noise, round to $\Lambda \geq \lambda$, clamp	About $1/\lambda$, the exact Laplace level
Counts, histograms, pure DP	Discrete Laplace, $\Pr[z] \propto e^{-\lvert z \rvert / b}$	$\varepsilon = \Delta_1 f / b$
Counts and sums, ℓ_2 sensitivity, secure aggregation	Discrete Gaussian, $\Pr[x] \propto e^{-x^2/(2\sigma^2)}$	$\delta(\varepsilon)$ by Equation 25.4; Rényi bounds as continuous
Gradients or other real values	Scale, round, recompute the sensitivity, add discrete noise	As the discrete mechanism used
Private training in Opacus	`secure_mode=True`	The idealized Gaussian guarantee

Rules of thumb

- Never ship `f(D) + laplace(b)` in floating point, including numpy or scipy samplers. Snap and clamp the output, or make the query integer-valued and use an exact sampler.
- Compute the discrete Gaussian's δ with Equation 25.4, because the continuous formula can understate it.
- Pad or fix the running time of rejection samplers.
- Turn on Opacus's `secure_mode` for any model you release.

25.10. Further reading

- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (CCS 2012): the floating-point attack and the snapping mechanism.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the discrete Gaussian and the exact samplers, the reference implementation to compare a library against.

Derivations. Every result here is derived step by step in the full edition's chapter.

26. Local, shuffle and distributed trust models

26.1. Why this matters

Every mechanism so far in this part assumed a **trusted curator**, which holds the raw data and releases only the mechanism's output. In this **central model**, the differential privacy (DP) guarantee covers the output, not the curator's database, its engineers or a court order for the data. Often the curator cannot be trusted that far, as when a browser vendor collects usage statistics from its own users.

The other three models trust less. In the **local model** nobody is trusted, and each person randomizes their own record before it leaves their device. In the **shuffle model** a shuffler is trusted only to cut the link between each report and its sender. In the **distributed model** cryptography and enough honest participants are trusted. The further upstream the noise enters, the fewer parties you trust and the more noise reaches the answer.

26.2. Choosing a trust model

Start from whom you cannot trust, then pay what that model costs. At a fixed privacy level, the error of a count stays constant in the central model, but in the local model it grows with the square root of the number of people. For a small histogram at the same guarantee, the full edition finds that at a hundred thousand people the local model's squared error is about 42,000 times the central one. Whatever you choose, state the trust model next to the privacy parameters, because the same numbers protect against different adversaries.

26.3. Central and local differential privacy

In the local model, each person sends only a **report**, the output of a **local randomizer** R run on their own record. Write ε_0 for the privacy level of one report. The randomizer is ε_0-DP when every set of reports S is at most e^{ε_0} times as likely under one record as under any other:

$$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{26.1}$$

With independent randomness per person, the whole collection is then ε_0-DP in the central sense. The guarantee covers only the randomized value, so a network address stored next to a report links it back to the person.

The basic local randomizer is **randomized response** on one bit. Each person reports their true bit with probability $e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, and the flipped bit otherwise. The analyst undoes the known

flipping to get an unbiased estimate $\hat{\pi}$ of the fraction of ones, whose standard deviation for n people is

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0}-1)\sqrt{n}}. \tag{26.2}$$

For example, survey ten thousand people at the level $\ln 3$. The estimated count then has a standard deviation of 86.6 people, while a trusted curator adding Laplace noise would be off by 1.287. So the local survey is 67.3 times noisier.

That factor is the model's, not the protocol's. No local protocol can estimate a fraction with error below order $1/((e^{\varepsilon_0}-1)\sqrt{n})$, and randomized response is optimal up to a constant. That is why local DP is used for the most frequent items in large populations, as in Apple's, Google's and Microsoft's telemetry, and almost never for finer ones. On a small user base its estimates are mostly noise. OpenDP provides randomized response as `dp.m.make_randomized_response_bool` for one bit and `dp.m.make_randomized_response` for a list of values.

26.4. Frequency estimation: k-ary randomized response and unary encoding

The workhorse of deployed local DP is **frequency estimation**. Each person holds one of a known list of values, such as emoji, and the analyst wants the fraction holding each one. Write k for the number of values.

In a **frequency oracle**, each report supports a value with a probability that is p if the person holds it and a smaller q otherwise. Count the reports C_v that support a value v. The debiased estimate of its fraction is then

$$\hat{f}_v = \frac{C_v/n - q}{p - q}. \tag{26.3}$$

There are two standard designs, and the pure-ldp Python package implements both. The first, **k-ary randomized response**, reports the true value with probability $e^{\varepsilon_0}/(e^{\varepsilon_0}+k-1)$, and any other value otherwise. The second, **optimized unary encoding** (Wang et al. 2017), writes the value as k bits with a single 1. It keeps that 1 with probability one half, and it turns each 0 into a 1 with probability $1/(e^{\varepsilon_0}+1)$.

Rare values decide between them, because their variance is the noise floor below which nothing can be seen. Times n, the two floors are

$$k\text{-ary: } \frac{e^{\varepsilon_0}+k-2}{(e^{\varepsilon_0}-1)^2}, \qquad \text{optimized unary: } \frac{4e^{\varepsilon_0}}{(e^{\varepsilon_0}-1)^2}. \tag{26.4}$$

So k-ary randomized response wins exactly when $k < 3e^{\varepsilon_0}+2$. Unary encoding's floor does not depend on k, and its price is a report of k bits. To size n, note that a value is visible only if its frequency is about three times the square root of the floor divided by n.

For example, a keyboard app wants the frequency of each of 128 emoji from a million users at $\varepsilon_0 = 1$. A rare emoji's estimate has a standard deviation of 0.0019 under optimized unary encoding, against 0.0066 under k-ary randomized response. An estimate must stand about three standard deviations above zero to be seen, so even the better design loses every emoji used by fewer than about 0.6% of users.

26.5. Large and unknown domains: hashing and heavy hitters

Unary reports need one bit per possible value, which **local hashing** avoids. Each person draws a hash function from a large public family, maps their value into one of w buckets, and sends the hash index with a w-ary randomized response of their bucket. Choose w from the privacy level, because near $w = e^{\varepsilon_0} + 1$ the variance equals that of optimized unary encoding, with reports of a few bits.

An oracle also needs the analyst to name each value in advance. Finding frequent values with no dictionary is the **heavy hitters** problem, and the standard answer grows prefixes (Bassily et al. 2017). Split the people into one group per symbol position, and let group ℓ report the first ℓ symbols of its value by local hashing. The analyst keeps the short prefixes whose estimates clear a threshold, extends each survivor by one symbol, and tests those on the next group. Each threshold comes from the normal tail of an absent value's estimate, so that a level raises few false alarms.

Deployed systems use these pieces. Google's RAPPOR, in Chrome, sends a unary encoding of a hashed value and tests candidates that the analyst supplies (Erlingsson, Pihur & Korolova 2014). Apple's count-mean sketch hashes each value and sends a unary encoding of its bucket.

26.6. Numbers in one bit: local mean estimation

Telemetry also collects numbers, such as minutes in an app, each between 0 and a known bound m. Ding, Kulkarni & Yekhanin (2017) showed that one bit per person is enough. A person with value x sends a bit that is 1 with a probability rising in a straight line, from the lie probability of randomized response at 0 to its truth probability at m:

$$\Pr[Y = 1] = \frac{1}{e^{\varepsilon_0} + 1} + \frac{x}{m} \cdot \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}. \tag{26.5}$$

The analyst undoes the line to get an unbiased mean. For a million users of an app at $\varepsilon_0 = 1$, the one-bit estimate has a standard deviation of 1.420 minutes. Fix the bound before collection and clip to it on the device, because a bound taken from the data would let one record change everyone's randomizer.

26.7. Collecting every day: memoization

Telemetry asks the same question every day, and fresh noise averages away. By basic composition in the full edition, each report of the same value adds its ε_0 to the cost. After a hundred daily reports of one bit at the level $\ln 3$, the bit is no longer in doubt.

The fix is **memoization**. The device draws one permanent randomized response per value and reuses it, so all reports of an unchanged value cost only that one response's level ε_∞. RAPPOR's homepage collection has $\varepsilon_\infty = 2.043$. Memoization fails when the value changes, since each new value costs another ε_∞. A user who changes their homepage every month reaches $\varepsilon = 24.5$ in a year.

For numbers that drift a little, Microsoft's Windows 10 telemetry uses **α-point rounding**, named after a secret random threshold α. A report then changes only when the value crosses that threshold. State such a guarantee as one about changes in behaviour, not as a per-round ε times the number of rounds.

26.8. The shuffle model and amplification by shuffling

In the **shuffle model** (Cheu et al. 2019), each person sends a local report to a **shuffler**. The shuffler strips identifiers and timing and forwards the reports in random order, so each report hides among the others (Amplification by shuffling and by iteration). Feldman, McMillan & Talwar (2021) give the bound to compute with. It needs enough people to hide among, which means

$$\varepsilon_0 \le \ln \frac{n}{16 \ln(2/\delta)}. \tag{26.6}$$

When this holds, the shuffled reports are (ε, δ)-DP for a small slack δ, with

$$\varepsilon \le \ln \left(1 + \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1} \left(\frac{8\sqrt{e^{\varepsilon_0} \ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n} \right) \right). \tag{26.7}$$

In practice you solve the bound for the local level. Roughly, a central ε allows ε_0 up to about $\ln(\varepsilon^2 n / \ln(1/\delta))$. For a central $\varepsilon = 1$ at $\delta = 10^{-6}$, ten thousand people may each randomize at $\varepsilon_0 = 3.504$, but at a thousand people the condition caps it at 1.460. The closed form is several times looser than the numerical bound of the same paper, so plan with the closed form, and deploy with that numerical bound, for which the authors publish code. When you quote the shuffled ε, say that it assumes a trusted shuffler.

Three things undo the amplification. A shuffler that colludes with the analyst leaves each report at its local ε_0, so keep that level reasonable. Fake senders provide no cover, so count only honest ones in Equation 26.7. And timing and message sizes must not leak, which is why real shufflers batch and pad their traffic.

26.9. Distributed DP: splitting the noise

The **distributed model** adds the central amount of noise once, in pieces. Each client adds a small share of noise to its value, and **secure aggregation** reveals only the sum to the server (Cryptography basics).

Gaussian noise splits cleanly. Write σ for the standard deviation of the central noise, and suppose each of the n clients adds normal noise of variance σ^2/n. The shares then sum to the central noise of variance σ^2, so the server's view is as private as the central Gaussian mechanism in the full edition. The small shares are also the weakness. When some number t of the clients collude with the server or drop out, only the honest clients' noise is left, of standard deviation

$$\sigma \sqrt{\frac{n - t}{n}}. \tag{26.8}$$

So decide in advance the largest fraction ρ of clients that may collude or drop out, and have each client add variance $\sigma^2/(n(1-\rho))$. For example, take a histogram summed through 100 clients and calibrated to $\varepsilon = 1$. If a fifth of the clients collude, the guarantee weakens to 1.128, and with only a tenth honest it is 3.525.

Secure aggregation works on integers modulo a large number, so clients round their scaled vectors and add integer noise (Discrete mechanisms in the full edition). The usual choices are Skellam noise (Agarwal, Kairouz & Liu 2021) and the distributed discrete Gaussian (Kairouz, Liu & Steinke 2021), which TensorFlow Federated implements in `tff.aggregators.DistributedDpSumFactory`.

26.10. Cheat sheet

Table 26.1.: Which trust model and protocol to use, by situation.

Situation	Use	Rule
Nobody may see raw records	Local randomized response	Only for frequent items in very large populations
Frequencies over k values	k-ary randomized response or optimized unary encoding	k-ary if $k < 3e^{\varepsilon_0} + 2$
A huge or unknown domain	Local hashing and growing prefixes	About $e^{\varepsilon_0} + 1$ buckets
A bounded number	One-bit mean	Fix the bound before collection
The same question every day	Memoization, α-point rounding for drifting numbers	Each new value costs another ε_∞
An anonymizing relay	Shuffle model	Plan with Equation 26.7, deploy with a numerical bound
Federated sums	Distributed Gaussian, Skellam or discrete Gaussian	Over-provision for colluding and dropped clients

Rules of thumb

- Local DP costs about $\sqrt{n}$ in error over central DP.
- Always state the trust model next to (ε, δ).

26.11. Further reading

- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): frequency oracles and local hashing, compared.
- Feldman, McMillan & Talwar, "Hiding Among the Clones" (FOCS 2021): the shuffling bounds, closed form and numerical.

Derivations. Every result here is derived step by step in the full edition's chapter.

27. Lower bounds: what privacy costs

27.1. Why this matters

Sooner or later someone shows you a design that looks too good. It releases a histogram over a million locations with differential privacy (DP) at a privacy parameter ε of 1 and no slack. It also promises an error of at most five records in every bin, with 95% probability. Before anyone tunes it, you want to know whether any mechanism could keep that promise.

A **lower bound** answers that question for every mechanism at once. It says that no mechanism satisfying the definition can be more accurate than some floor. So use it as a feasibility check. If a requirement is below the floor, the design is impossible, and the only ways out are more data, a coarser answer or a weaker guarantee. If your mechanism is already within a small factor of the floor, stop tuning it, because no algorithm can gain more than that factor.

Accuracy here has two numbers. The **error** α is how far the output may land from the true answer, and the **failure probability** β is the chance that it lands further away. Pure DP has no slack, while approximate DP allows a small slack δ (The definition of differential privacy in the full edition). The examples use $\varepsilon = 1$, $\beta = 0.05$ and, for approximate DP, $\delta = 10^{-6}$.

27.2. Choosing among many candidates: packing

Many releases must single out one of many candidates. A mode names the most common of N possible values, and a histogram over N bins must show which bin holds a spike. Under pure DP, the **packing lemma** prices this. Suppose that each of m candidates has a dataset on which a good answer must point to it. Suppose also that all these datasets lie within k records of one reference dataset. Then an ε-DP mechanism that is right with probability $1 - \beta$ on all of them needs

$$k \geq \frac{\ln\left(m(1-\beta)\right)}{\varepsilon}. \tag{27.1}$$

The cost grows only with the logarithm of the number of candidates, but under pure DP it never goes away.

For the mode, the candidates are the N values, and k is the **lead**, how many more times the top value appears than any other. Below a lead of $\ln(N(1-\beta))/\varepsilon$, no pure-DP mechanism finds the mode reliably. A histogram that is accurate to within α in every bin could find a spike of height $2\alpha + 1$, so its largest bin error obeys

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{27.2}$$

For a million locations, every ε-DP histogram has an error of at least 6.38 records in some bin, so the design from the opening is impossible. The Laplace histogram of The Laplace and Gaussian mechanisms in the full edition reaches 16.79, and since both numbers grow like $\ln N/\varepsilon$, Laplace is optimal up to a factor of about 2. Report-noisy-max from Selection in the full edition is optimal for the mode in the same sense, needing a lead of 18.44 where the floor is 13.76. Once a pure-DP histogram or selection uses one of these mechanisms, further tuning can win at most that factor.

The lemma needs pure DP. With a slack δ, the argument stops at a distance of about $\ln(1/\delta)/\varepsilon$, which is 13.82 records here, however many candidates there are. A real mechanism reaches that cap. The stability-based histogram never releases an empty bin, so its error does not grow with N at all (Beyond global sensitivity in the full edition). So when a huge and mostly empty universe puts the pure-DP floor too high, switch to approximate DP rather than tune.

Packing also prices not knowing where the data lie. Suppose that the records are normal with variance 1, and all you know is that their mean lies within a distance R of zero. Then estimating the mean to within α under pure DP needs at least $\ln(\lfloor R/\alpha \rfloor (1-\beta))/\varepsilon$ records. At an error of 0.1, the floor is only 16.07 records even when R is a million, and under approximate DP it is about $\ln(1/\delta)/\varepsilon$ whatever R is. A mean whose error grows in step with R is therefore paying for a clumsy mechanism, and Private mean estimation in the full edition gives one that pays only the logarithm.

27.3. How many records you need

An error in records becomes an error in fractions once you divide by the dataset size n, so every floor above is also a floor on n. Take a pure-DP histogram whose frequencies must all be within α. It needs

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{27.3}$$

Sampling costs records too. Estimating every frequency to within α from a random sample takes about $\ln(2N/\beta)/(2\alpha^2)$ records, by Hoeffding's inequality and a union bound (Concentration inequalities), and no estimator does much better. You need the larger of the two numbers. For a million cells at an error of 0.1, privacy needs at least 64 records while sampling needs up to 875. At an error of 0.01, privacy needs 638 and sampling 87,522. So privacy dominates only for coarse answers at strong privacy. When a plan is short of records for fine answers, sampling is the cause, and no better mechanism will fix it.

To size n for a target, solve the floor, or the error of the mechanism you plan to use, for n with a root finder such as `scipy.optimize.brentq`. For a model, an accountant such as Opacus's `get_noise_multiplier` or Google's `dp_accounting` gives the noise that a target ε allows, and you check the resulting error the same way.

Many attributes change the picture. Release the means of d yes/no attributes with an error of at most α in each. Packing over the 2^d answer patterns says that a pure-DP mechanism then needs about

$$n \geq \frac{d \ln 2}{\alpha\varepsilon} \tag{27.4}$$

records, linear in d. The Gaussian mechanism is approximate DP and calibrates its noise to the ℓ_2 sensitivity, so its requirement grows only like $\sqrt{d \ln d}$. Table 27.1 shows the two rates. From

$d = 2731$ attributes on, the Gaussian mechanism needs fewer records than any pure-DP mechanism can. So for outputs with many coordinates, such as a model's gradient, use approximate DP, because under pure DP the cost is linear in the dimension.

Table 27.1.: Records needed to release d attribute means with ℓ_∞ error 0.1 (on the scale $[-1, 1]$) with probability 0.95, at $\varepsilon = 1$ and, for the Gaussian, $\delta = 10^{-6}$.

	Pure DP, packing (lower)	Pure DP, Laplace (upper)	Approximate DP, Gaussian (upper)
$d = 1000$	6,921	198,070	10,836
$d = 100{,}000$	693,137	29,017,315	134,299

27.4. The floor for approximate DP: fingerprinting

Is $\sqrt{d}$ the price of approximate DP, or could a cleverer mechanism do better? **Fingerprinting** (Bun, Ullman & Vadhan 2014) says that it is the price. In its tight form (Steinke & Ullman 2016), any (ε, δ)-DP release of d attribute means with error α needs a number of records of order

$$n \gtrsim \frac{\sqrt{d \ln(1/\delta)}}{\alpha\varepsilon}. \tag{27.5}$$

The Gaussian mechanism matches this up to a factor of $\sqrt{\ln d}$. So once a release of many means uses well-calibrated Gaussian noise, stop looking for a better mechanism, and look for more records, fewer dimensions or a looser error instead.

Use this bound for its rate, not for its constants, which are poor. It also needs δ far below $1/n$, and so does every (ε, δ) claim. A mechanism that publishes each record outright with probability δ satisfies the definition, and at $\delta = 1/n$ it publishes about one record. So treat a claim with δ near $1/n$ as no guarantee at all.

27.5. Tracing attacks on released means

The fingerprinting argument is an attack that has been run on real data. In 2008, Homer et al. showed that the published variant frequencies of a genome-wide study reveal whether a given person's DNA was in it. So the US National Institutes of Health withdrew such summaries that year. The attacker holds a target's record y and the population means p. It scores how closely the released means $\hat{q}$ follow the target's deviations from the population:

$$Z(y) = \sum_{j=1}^{d} \hat{q}_j \, (y_j - p_j). \tag{27.6}$$

A member pulls the release toward its own record, so its score is shifted up, while a non-member's score is centred at zero. Without noise, the attack becomes strong once the number of attributes approaches the square of the number of people. Noise much larger than $\sqrt{d}/n$ per attribute defeats it, and the Gaussian mechanism adds noise of that order.

On the means of 50 people over 2,000 attributes, the full edition's simulation reaches a true positive rate (TPR) of 0.242 at a false positive rate (FPR) of 1% without noise. With the noise of a $(1, 10^{-5})$-DP Gaussian mechanism, that TPR falls to 0.014.

Use the score to audit any release of many averages, with the population means taken from a separate sample. Report the TPR at a low FPR, as in Membership inference, and compare it with the cap $\text{TPR} \le e^{\varepsilon}\,\text{FPR} + \delta$ that the definition in the full edition puts on every membership test. An audit that beats the cap has found a bug in the mechanism or in its accounting.

27.6. Reconstruction as a floor on noise

Reconstruction attacks recover almost all of n secret bits from about n random counting queries, whenever the noise per answer grows more slowly than $\sqrt{n}$ (Dinur & Nissim 2003). So a private mechanism that answers that many counting queries must add error of order $\sqrt{n}$ to each answer, and the Gaussian mechanism adds just that. With many more queries than records, answer from a private synthetic dataset instead (Query release in the full edition).

The same argument turns a measured attack into a statement about ε. Suppose the secret bits are independent, and each is equally likely to be 0 or 1. Then the definition's cap on guessing one bit says that no attack on an (ε, δ)-DP output recovers, on average, more than a fraction r of them, where

$$r \le \frac{e^{\varepsilon} + \delta}{e^{\varepsilon} + 1}, \qquad \text{so} \qquad \varepsilon \ge \ln \frac{r - \delta}{1 - r}. \tag{27.7}$$

At $\varepsilon = 1$, that limit is 73.11%. The bits must be uniform, because with biased bits a guesser beats one half from the prior alone.

For example, recovering 96.07% of the bits proves that ε is at least 3.20. But an attack bounds ε only from below, and narrowing the gap to the guarantee is the job of Auditing private training.

27.7. Cheat sheet

Table 27.2.: The floors of this chapter as feasibility checks, with the mechanism that comes closest to each.

Release	Floor: no mechanism does better	Closest mechanism
Mode of N values, pure DP	Lead $k \ge \ln(N(1-\beta))/\varepsilon$	Report-noisy-max
Histogram over N bins, pure DP	$\alpha \ge \frac{1}{2}(\ln(N(1-\beta))/\varepsilon - 1)$	Laplace, about twice the floor
Histogram or mode, approximate DP	About $\ln(1/\delta)/\varepsilon$, whatever N	Stability-based histogram
Mean within R of zero, pure DP	$n \ge \ln(\lfloor R/\alpha \rfloor(1-\beta))/\varepsilon$	Coarse then fine

Release	Floor: no mechanism does better	Closest mechanism
d means, pure DP	$n \gtrsim d \ln 2/(\alpha\varepsilon)$	Laplace, within a factor of order $\ln d$
d means, approximate DP	$n \gtrsim \sqrt{d \ln(1/\delta)}/(\alpha\varepsilon)$	Gaussian, within $\sqrt{\ln d}$
About n counting queries	Error of order $\sqrt{n}$ per answer	Gaussian
A fraction r of uniform bits recovered	Proves $\varepsilon \geq \ln \frac{r-\delta}{1-r}$	—

Rules of thumb

- Check a requirement against its floor before tuning. Below the floor, ask for more data, a coarser answer or a weaker guarantee.
- A floor holds for every mechanism, not just for bad ones, so no clever algorithm gets under it.
- Never promise per-cell accuracy for a large contingency table at small ε without checking the histogram floor, and use approximate DP for outputs with many coordinates.
- Stop tuning once a mechanism is within its known factor of the floor, about 2 for Laplace histograms and $\sqrt{\ln d}$ for Gaussian means.
- When sampling needs more records than privacy does, collect more data instead of changing the mechanism.
- Aggregates are not a defence. Never release many unnoised statistics from a small cohort, and audit releases of many averages with the tracing score.
- Keep δ far below $1/n$.

27.8. Further reading

- Vadhan, "The Complexity of Differential Privacy" (2017), §5: the three lower bounds side by side, and the stability-based histogram.
- Steinke & Ullman, "Between Pure and Approximate Differential Privacy" (Journal of Privacy and Confidentiality, 2016): the simplified fingerprinting lemma and the tight $\sqrt{d \ln(1/\delta)}$ bound for attribute means.

Derivations. Every result here is derived step by step in the full edition's chapter.

28. Private mean estimation

28.1. Why this matters

The mean is the statistic that private machine learning computes most often. Every step of private gradient descent clips per-example gradients, averages them and adds noise, and even a survey share is a mean of 0/1 answers. But what an analyst wants is usually the mean of the **population** the records came from. So the planning questions are how many records a private estimate needs, and how to choose a clipping range without looking at the data.

Differential privacy (DP) must hold for every dataset, including datasets your assumptions say cannot happen (The definition of differential privacy in the full edition). So an assumption about the data, such as "the records are normal", may make a mechanism more accurate, but it must never enter the privacy argument. Every mechanism below clips the data to a range, which keeps it private whatever the data are. Its noise is proportional to the width of that range, and the recipe of the chapter follows from it. Spend a small part of the budget finding where the data are, then clip there and spend the rest on the mean.

28.2. Bounded data: privacy adds a lower-order term

Throughout the chapter, datasets have a fixed size n, and neighbours differ in one replaced record. An estimate is judged by its **root-mean-square error** (RMSE), the typical distance between the estimate and the population mean.

Start with records in a known interval, such as yes/no answers coded 0 and 1. Write w for the width of the interval and σ^2 for the variance of a record. Release the average plus Laplace noise with scale $w/(n\varepsilon)$, which is ε-DP because one record moves the average by at most w/n (The Laplace and Gaussian mechanisms in the full edition). The RMSE then combines the sampling error and the noise:

$$\mathrm{RMSE} = \sqrt{\frac{\sigma^2}{n} + \frac{2w^2}{n^2\varepsilon^2}}. \tag{28.1}$$

To plan a study, set the RMSE equal to a target error α and solve for n. The snippet in the Code section does this exactly, and a simple upper bound is

$$n \le \frac{\sigma^2}{\alpha^2} + \frac{\sqrt{2}\,w}{\alpha\varepsilon}. \tag{28.2}$$

So sampling needs about σ^2/α^2 records whatever you do, and privacy adds at most about $w/(\alpha\varepsilon)$ more. For a yes/no survey in the worst case for sampling, a population that is half yes, an RMSE of 0.05 at $\varepsilon = 1$ needs 108 people instead of 100.

But the privacy term grows like $1/\varepsilon$, so it dominates at strong privacy and a coarse target. It also grows with the width w, the most one record can move, which is why the rest of the chapter is about making the range narrow.

Report the error in its three parts, which are sampling, the bias that clipping causes and the noise. With a public range that holds every record the bias is zero. But the later sections clip into the data, and a confidence interval that ignores that bias is too narrow. The formulas also treat n as public, which holds when neighbours replace a record. If the dataset size is itself private, release a noisy sum and a noisy count and divide, with the budget split between them.

Libraries follow this pattern. In diffprivlib, `tools.mean(x, epsilon=..., bounds=(lower, upper))` clips to the bounds and adds noise, and `tools.var` and `tools.quantile` do the same for variances and quantiles. If you leave out `bounds`, it computes them from the data and raises a `PrivacyLeakWarning`, because a range read off the observed minimum and maximum leaks them. Tumult Analytics' `QueryBuilder` takes the bounds as arguments of `average(column, low, high)`, and OpenDP likewise makes you declare the clamping bounds and the dataset size before it builds a mean.

28.3. The cost of an unknown range

Now let the records be normal with variance 1, with a mean known only to lie between $-R$ and R. Normal records rarely stray far from their mean. For 1,000 records, all of them lie within a distance τ of 4.604, except with a small failure probability β of 0.05.

The **naive estimator** clips every record to the widest place the data could be, $R + \tau$ on either side of zero, and adds Laplace noise for that width. It is private, but its error grows in step with R. With 1,000 records at $\varepsilon = 1$ and a range of a million, its RMSE of 2828.44 is useless. That is the price of a clumsy mechanism, because Lower bounds in the full edition shows that privacy forces a cost of only about $\ln R/\varepsilon$ records for the range.

Coarse then fine (Karwa & Vadhan 2018) comes close to that floor. In the coarse step, cut the range into bins of width 2τ, so that the records fill at most two bins. Pick one bin with the exponential mechanism in the full edition, scoring each bin by the number of records in it, and spend part of the budget, ε_1, on it. The fine step then clips every record to the chosen bin and its two neighbours, and it releases their average with Laplace noise for that window.

The fine step spends the rest of the budget, so the two steps together are ε-DP by composition (Post-processing, group privacy and composition in the full edition). Whether the result is accurate depends on the coarse step. Write m for the number of bins. Then the coarse step picks a full bin, except with probability β, once

$$n \geq \frac{4 \ln(m/\beta)}{\varepsilon_1}. \tag{28.3}$$

After that, the fine step's error does not depend on R at all. With a quarter of the budget on the coarse step, the full edition's 1,000 records give an RMSE of 0.0609 whether R is 10 or a million.

So choose by what you know. When the range is already narrow, the naive estimator wins, and with 1,000 records at $\varepsilon = 1$ the crossover is at $R = 13.81$. Beyond it, use coarse then fine. A

private quantile, such as diffprivlib's `tools.quantile`, can also do the locating. If you can accept approximate DP, with its small slack δ, you can drop R altogether. A stability-based histogram does the coarse step with about $\ln(1/\delta)/\varepsilon$ records whatever the range (Beyond global sensitivity in the full edition).

Two things go wrong. A budget split, bin width or β chosen after looking at the data leaks it. And a coarse step with too few records can pick an empty bin, so the fine step clips everything to the wrong place.

28.4. Many dimensions: clip the norm

Vectors in d dimensions, such as embeddings or gradients, follow the same recipe with a different measure of size. Clip each record's ℓ_2 norm, its length, to a radius C, so that one replaced record moves the average by at most $2C/n$. Then add Gaussian noise calibrated to that sensitivity (The Laplace and Gaussian mechanisms in the full edition). This is exactly the step of private gradient descent, which clips per-example gradients to a fixed norm (private training). With the radius close to the spread of the data, an error α needs about

$$n \approx \frac{d}{\alpha^2} + \frac{d\sqrt{\ln(1/\delta)}}{\alpha\varepsilon} \tag{28.4}$$

records, and the fingerprinting lower bound in the full edition shows that the privacy term cannot be avoided.

When you do not know where the mean is, the range problem returns, and a grid of bins would cost records in proportion to d. The **shrinking balls** of CoinPress (Biswas, Dong, Kamath & Ullman 2020) avoid the grid. Over a few rounds, they clip to a ball around the current guess, release a noisy average and shrink the ball, for about $\sqrt{d}\ln R/\varepsilon$ extra records. On 100,000 records in a thousand dimensions, the full edition's script finds an RMSE of 0.1736 against 0.1000 without privacy, because the privacy noise grows like d while sampling grows like $\sqrt{d}$. Fix the number of rounds and the budget split before seeing the data, because the rounds compose and too many of them eat the budget.

28.5. Heavy tails: clip into the data

Incomes, session lengths and per-example gradient norms have heavy tails. Suppose all you know is that the variance is at most σ^2. Then a clipping range wide enough to touch no record is enormous. The way out is to clip into the data on purpose and accept a bias.

Clip every record to within a half-width τ of a centre near the mean, found by a coarse step. With the centre at the mean, the bias is at most σ^2/τ, and the noise grows in proportion to τ. The bound on the total error is smallest at

$$\tau_b = \sigma\sqrt{\frac{n\varepsilon}{2\sqrt{2}}}, \tag{28.5}$$

which uses only n, ε and the variance bound. At that width, an error α needs about

$$n \approx \frac{\sigma^2}{\alpha^2} + \frac{8\sqrt{2}\,\sigma^2}{\alpha^2\varepsilon} \tag{28.6}$$

records. So with heavy tails privacy is no longer a lower-order cost, because its term has the same power of α as sampling. No ε-DP mechanism does better for every such distribution. A bound on a higher moment lowers the cost.

Table 28.1 shows the trade-off between bias and noise on Pareto records, a heavy-tailed distribution scaled to variance 1. The worst-case width $\tau_b = 18.80$ cuts the error from 0.4012 to 0.0607, and the best width for this distribution does better still.

Table 28.1.: Clipped private means of Pareto records with variance 1, 1,000 records, $\varepsilon = 1$, for three clipping half-widths.

Clipping half-width τ	Bias	Sampling sd	Noise sd	RMSE
141.42 (no record clipped)	0.0000	0.0313	0.4000	0.4012
18.80 (worst-case best, τ_b)	−0.0018	0.0293	0.0532	0.0607
5.36 (best for this distribution)	−0.0153	0.0249	0.0152	0.0329

So treat τ_b as a safe default and not as the best width. A narrower range usually does better, but it must still be chosen without looking at the records, from domain knowledge or with a private quantile (Beyond global sensitivity in the full edition). Setting the width to the observed maximum brings back the leak of a data-dependent noise scale.

28.6. Code

The snippet computes the records a bounded mean needs and the default clipping width for heavy tails.

```
from math import ceil, log, sqrt


def records_for_bounded_mean(alpha: float, sd: float, width: float, eps: float) -> int:
    """Smallest n with sd^2/n + 2 (width/(n eps))^2 <= alpha^2."""
    a, c = sd**2, 2 * (width / eps) ** 2
    return ceil((a + sqrt(a * a + 4 * c * alpha**2)) / (2 * alpha**2))


def heavy_tail_width(n: int, eps: float, sd: float) -> float:
    """The clipping half-width that balances the bias bound sd^2/tau against the noise."""
    return sd * sqrt(n * eps / (2 * sqrt(2)))


print(records_for_bounded_mean(0.05, 0.5, 1.0, 1.0))  # 108
print(f"{heavy_tail_width(1000, 1.0, 1.0):.2f}")  # 18.80
```

28.7. Cheat sheet

Table 28.2.: Private mean estimation at a glance, with $\tau_b = \sigma\sqrt{n\varepsilon/(2\sqrt{2})}$.

What you know	Mechanism	Records for error α (up to log factors)
Bounded, width w, variance σ^2	Mean plus $\mathrm{Lap}(w/(n\varepsilon))$	$\sigma^2/\alpha^2 + \sqrt{2}\,w/(\alpha\varepsilon)$
Normal, mean within R of zero	Coarse histogram, then clip to three bins	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln R/\varepsilon$
Same, approximate DP	Stability-based coarse step	$1/\alpha^2 + 1/(\alpha\varepsilon) + \ln(1/\delta)/\varepsilon$, no R
d dimensions, mean length at most R	Clip the norm, Gaussian noise; shrinking balls to find the mean	$d/\alpha^2 + d\sqrt{\ln(1/\delta)}/(\alpha\varepsilon) + \sqrt{d}\ln R/\varepsilon$
Variance at most σ^2 only	Clip to within τ_b of the centre	$\sigma^2/\alpha^2 + 8\sqrt{2}\,\sigma^2/(\alpha^2\varepsilon)$

Rules of thumb

- Let assumptions about the data affect only accuracy. The clipping range keeps the release private, for every dataset.
- Never clip to a range set by your ignorance, or to the observed minimum and maximum. Spend a quarter of the budget or less finding the data first.
- Fix every parameter, from the budget split to the bin width, before you see the data.
- With heavy tails, clip into the data, with a width chosen from n, ε and a variance bound, and put the clipping bias into every reported interval.
- If n is not public, divide a noisy sum by a noisy count.

28.8. Further reading

- Karwa & Vadhan, "Finite Sample Differentially Private Confidence Intervals" (ITCS 2018): coarse then fine for normal means and variances, under pure and approximate DP.
- Biswas, Dong, Kamath & Ullman, "CoinPress: Practical Private Mean and Covariance Estimation" (NeurIPS 2020): shrinking balls in practice, with code.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part IV.

Privacy accounting

A model trained with stochastic gradient descent (SGD) under differential privacy (DP), the method called DP-SGD, takes thousands of noisy steps, each on a random batch. The guarantee you report is a privacy level ε with a small slack δ, the chance that the promise fails for a person (Numerical accounting in the full edition defines it exactly). The composition theorems of Part 3 are far too loose for thousands of steps. An **accountant** does better, because it tracks a richer summary of each step's privacy cost that composes tightly, and converts the total to (ε, δ) once, at the end.

One DP-SGD run is the running example. Each step samples each example with probability 0.01 and adds noise with multiplier 1. After 10,000 steps, advanced composition certifies $\varepsilon = 24{,}379.55$ at $\delta = 10^{-5}$. But a privacy loss distribution accountant certifies only 6.24 for the same run (Choosing an accountant in the full edition). So an ε means nothing until you name the accountant that produced it, which is also why libraries and papers can disagree about the same run.

If you train with DP-SGD, read four chapters. Rényi differential privacy in the full edition is the curve that most accountants track. Amplification by subsampling in the full edition shows how much a random batch helps, provided the data loader samples the way the accountant assumes. Numerical accounting with privacy loss distributions in the full edition computes the tightest ε available, and Choosing an accountant in the full edition turns the part into a decision guide.

The other chapters serve narrower needs. From Rényi DP to (ε, δ) and to tests in the full edition explains the conversion behind every Rényi number. Concentrated differential privacy in the full edition budgets analytics queries with one number. f-DP and Gaussian differential privacy in the full edition reads a guarantee as a limit on attacks. Amplification by shuffling and by iteration in the full edition and Adaptive composition in the full edition cover shuffled pipelines and budgets decided during training.

The part uses the accountants of three libraries, Opacus, Google's `dp_accounting` and Microsoft's `prv_accountant`. Each chapter ends with a link to the full edition, which derives every result.

By the end, you will be able to account for a DP-SGD run with a library and check it against a second one. Above all, you will report ε with the accountant that produced it, so that someone else can reproduce it.

29. Rényi differential privacy

29.1. Why this matters

Suppose you train a model with 1000 steps of full-batch noisy gradient descent. Each step adds Gaussian noise of standard deviation 20 to a quantity that one person's record can move by at most 1. You want the differential privacy (DP) guarantee of the whole run, a privacy level ε at a small slack $\delta = 10^{-5}$. The tools of Part 3 turn each step into one such pair and then apply advanced composition (Composition). They give $\varepsilon = 68.01$, while the true answer is 7.51, so you would pay for a bound nine times too large in noise or in accuracy.

Rényi differential privacy (RDP) fixes this by keeping a whole curve for each step instead of one point. The curves of the steps simply add, with no δ spent along the way, and you convert the total into (ε, δ) once, at the end. For the same 1000 steps, this gives $\varepsilon = 8.84$, only 18% above the truth. It is what the RDP accountants of Opacus and Google's `dp_accounting` library track.

29.2. The RDP curve

Call P and Q the output distributions of a mechanism on two neighbouring datasets, which differ in one person's record. The **Rényi divergence** of Divergences measures how far apart they are. It has a parameter called the **order**, written α, which is any number above 1. Writing p and q for the densities of the two distributions, it is

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \int p^\alpha q^{1-\alpha}\, dy. \tag{29.1}$$

A larger order weighs more heavily the outputs that are much likelier under P than under Q. So a small order looks at the typical case, and a large order at the worst case.

A mechanism is (α, ε)**-RDP** if this divergence is at most ε for every pair of neighbours. The smallest such ε, as a function of the order, is the mechanism's **RDP curve** $\varepsilon(\alpha)$, and a higher curve means less privacy. Each point of it bounds one exponential moment of the privacy loss, the logarithm of p/q, and Numerical accounting in the full edition works with that loss directly.

The curve never decreases with the order. Its two ends tie it to familiar quantities. As the order falls towards 1, it tends to at least the Kullback–Leibler divergence between neighbours, the average privacy loss. At order infinity it is the largest privacy loss of any output, so an ε-DP mechanism is (α, ε)-RDP at every order. Accountants store the whole curve on a grid of orders. This part uses a grid of 154 orders, the default list of Opacus with 128, 256 and 512 added.

29.3. RDP curves of the basic mechanisms

To account for a mechanism you need its curve, and the three basic mechanisms have one in closed form (Mironov 2017). Each formula below assumes sensitivity 1, the most one record can move the answer. For the **Gaussian mechanism**, write σ for the noise multiplier, the noise standard deviation in units of the ℓ_2 sensitivity. Its curve is the straight line

$$\varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}. \tag{29.2}$$

The **Laplace mechanism** adds noise whose typical size is its scale b, and its curve is

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln\left(\frac{\alpha}{2\alpha - 1} e^{(\alpha-1)/b} + \frac{\alpha - 1}{2\alpha - 1} e^{-\alpha/b} \right). \tag{29.3}$$

Randomized response, which reports a bit truthfully with probability p and flips it otherwise, has a closed-form curve too, given in the full edition. Table 29.1 gives the three curves at a few orders.

Table 29.1.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, $p = 0.75$	0.7340	0.8473	1.0666	1.0940	1.0986

The Laplace and randomized-response curves level off at their pure DP levels. The Gaussian curve grows without limit, so the Gaussian mechanism is not ε-DP for any ε, but it is RDP at every finite order, and that is all an accountant needs.

These curves are for steps that use the whole dataset. A private step of stochastic gradient descent (SGD), called DP-SGD, works on a random batch and has a much smaller curve, which Amplification by subsampling in the full edition computes. You rarely type these formulas in, because libraries hold them. The `rdp_bank` module of autodp, for example, has `RDP_gaussian`, `RDP_laplace` and `RDP_randresponse`, which return the curve at one order.

29.4. Composition adds curves

Training runs thousands of mechanisms in sequence, and each may depend on what the earlier ones released, as a gradient step depends on the current model. RDP composes by adding curves order by order. Write ε_i for the curve of step i, and ε_M for the curve of a whole run of k steps. Then

$$\varepsilon_M(\alpha) \le \sum_{i=1}^{k} \varepsilon_i(\alpha) \qquad \text{for every order } \alpha. \tag{29.4}$$

The sum is exact when the steps are independent. Different mechanisms mix freely, because you only add numbers at the same order.

A pipeline that mixes mechanisms, such as a Gaussian count, a Laplace histogram and a randomized-response survey, is accounted the same way. In `dp_accounting` you wrap the events in a `ComposedDpEvent`, wrap a repeated one in a `SelfComposedDpEvent`, and compose the result. All the curves you add must describe the same notion of neighbours and the same unit of privacy, though. In `dp_accounting` 0.6.0, for example, the RDP accountant refuses a `RandomizedResponseDpEvent` under its default add-or-remove `neighboring_relation`, which a `PoissonSampledDpEvent` needs (Amplification by subsampling in the full edition explains the relations).

The condition that matters is that each step's curve is fixed in advance. Steps may adapt to earlier outputs, but the noise level and the number of steps must not be chosen from the data during the run. If they are, for example by stopping when a validation metric looks good, you need Adaptive composition in the full edition.

For Gaussian steps the sum has a useful shortcut, because adding k copies of the straight line gives the curve of a single Gaussian step with noise multiplier $\sigma/\sqrt{k}$. The shortcut also shows why DP-SGD needs subsampling. If this part's running example, 10,000 steps with noise multiplier 1, used the full dataset at every step, it would act like one step with noise multiplier 0.01. Its exact ε would be 5425.5, which is no guarantee at all.

29.5. Post-processing and group privacy

Post-processing is free in RDP. If you apply any randomized map g to the output without looking at the data again, the curve of the result is no larger at any order:

$$\varepsilon_{g \circ M}(\alpha) \le \varepsilon_M(\alpha). \tag{29.5}$$

So releasing the trained model or its predictions costs nothing beyond the training run, as long as g never reads the data.

Groups are harder. When one user contributes m records, you need a guarantee for all of them at once. RDP has a generic group rule, Mironov's, under which each doubling of the group halves the order and triples ε. It is loose, and for the Gaussian mechanism it overstates the exact value 1.5 times for two records and 3.375 times for eight.

So for Gaussian noise, skip the generic rule and use the group's sensitivity. A group of m records moves the answer by at most m times as much as one record, so the curve becomes

$$\varepsilon_{\text{group}}(\alpha) = \frac{m^2 \alpha}{2\sigma^2}, \tag{29.6}$$

which is m^2 times the single-record curve. Better still, clip per user and account at the user level (User-level privacy).

29.6. Convert once, at the end

After composing, you hold a curve, but you report an (ε, δ) pair. A conversion tries every order of the grid and keeps the one that gives the smallest ε, which libraries report as the best order. From Rényi DP to (ε, δ) and to tests in the full edition gives the formulas. The workflow is what matters here, because you compose the curves of all steps first and convert once, at the end.

The **moments accountant** of the original DP-SGD paper (Abadi et al. 2016) is this same workflow with different bookkeeping. It tracks $\lambda\varepsilon(\lambda+1)$, the logarithm of the moment generating function of the privacy loss, and its conversion is Mironov's, the original RDP conversion. So read an older paper's moments-accountant ε as an RDP number. Similarly, `compute_dp_sgd_privacy_statement` in TensorFlow Privacy 0.9.0 uses the RDP accountant of `dp_accounting` by default.

Table 29.2 shows what composing curves gains for runs of k Gaussian steps with $\sigma = 20$. It compares advanced composition with RDP, converted by Mironov's conversion, and with the exact value from the Gaussian's privacy curve (f-DP and Gaussian differential privacy in the full edition).

Table 29.2.: Total ε at $\delta = 10^{-5}$ for k Gaussian steps with $\sigma = 20$: advanced composition, RDP with Mironov's conversion (best order on the grid in brackets), and exact.

k	Advanced	RDP (best order)	Exact
1	0.847	0.251 (128)	0.160
10	3.270	0.771 (31)	0.561
100	13.423	2.524 (10.6)	1.993
1000	68.008	8.838 (4)	7.511

Advanced composition is 5 to 9 times the exact value, because each step was collapsed to one point before composing, while RDP gets tighter as the run grows. So converting each step to an (ε, δ) pair first and composing those pairs throws the gain away.

The best order moves with the run, from 128 for one step to 4 for 1000 steps. So a grid must cover orders from just above 1 to a few hundred, as the default grid of either library does. A best order on the edge of the grid means that the true optimum lies outside it, and the reported ε is too large.

29.7. Accounting with a library

In production, you describe each step to a library and let it compose and convert. Google's `dp_accounting` describes each mechanism as an event, so a DP-SGD step is a `GaussianDpEvent` inside a `PoissonSampledDpEvent`, and an `RdpAccountant` composes the events and converts the total. For example, these lines account for the running example in version 0.6.0.

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(
    sampling_probability=0.01,
    event=dp_accounting.GaussianDpEvent(noise_multiplier=1.0),
)
accountant = dp_accounting.rdp.RdpAccountant()       # or RdpAccountant(orders=[...])
accountant.compose(step, count=10_000)
eps, order = accountant.get_epsilon_and_optimal_order(target_delta=1e-5)
```

Opacus works differently, because its accountant is attached to the optimizer. There, `PrivacyEngine(accountant="rdp")` selects the class `opacus.accountants.RDPAccountant`, which records each update through `step(noise_multiplier=..., sample_rate=...)`. Its method `get_privacy_spent(delta=...)` then returns ε with the best order. Outside training, the functions `compute_rdp` and `get_privacy_spent` of `opacus.accountants.analysis.rdp` do the same in two calls. Ask for RDP explicitly when you want these numbers, since the default accountant of Opacus 1.6.0 is a numerical one.

Three checks catch most mistakes. First, if the best order the library returns sits at either end of the grid, pass a longer list of orders. Second, make sure the event matches the sampler your data loader runs, because a Poisson event for a shuffled fixed-size batch is a different mechanism (Amplification by subsampling in the full edition). Third, count each step once. In Opacus the optimizer already steps the accountant after every update, so a manual call to `step` on top of it charges each update twice.

29.8. Cheat sheet

Table 29.3.: The results of this chapter.

Result	Formula	Notes
(α, ε)-RDP	$D_\alpha(M(D)\|M(D')) \le \varepsilon$ for all neighbours	A curve over orders; $\varepsilon(\infty)$ is pure DP
Gaussian, noise multiplier σ	$\varepsilon(\alpha) = \alpha/(2\sigma^2)$	Exact; σ in units of the ℓ_2 sensitivity
Composition	$\varepsilon_M(\alpha) \le \sum_i \varepsilon_i(\alpha)$	Curves fixed in advance
k full-batch Gaussian steps	One Gaussian with $\sigma/\sqrt{k}$	Not for subsampled steps
Group of m, Gaussian	$m^2\alpha/(2\sigma^2)$	The generic group rule is loose

Rules of thumb

- Compose in RDP and convert once at the end. Never convert each step to $(\varepsilon_0, \delta_0)$ first.
- Keep a grid of orders from just above 1 to a few hundred, and check that the best order is not at its edge.

- Plan the noise level and the number of steps before training, or use an adaptive accountant.
- For a group of records under Gaussian noise, use the group's sensitivity, not the generic group rule.
- Add only curves computed under the same neighbouring relation and the same unit of privacy.

29.9. Further reading

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the curves of the basic mechanisms, group privacy and the conversion.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.

Derivations. Every result here is derived step by step in the full edition's chapter.

30. From Rényi DP to (ε, δ) and to tests

30.1. Why this matters

An accountant that tracks Rényi differential privacy (RDP) ends a training run holding a curve, with one value for each **order** α above 1 (Rényi differential privacy in the full edition). Nobody reports the curve, though. A paper, a model card or a regulator asks for a differential privacy (DP) guarantee, a privacy level ε with a small slack δ. So every RDP analysis ends with a conversion, and the conversion decides the number you publish.

The short answer is to use the **improved conversion**, which every maintained library already uses, and to check that the best order is inside the grid. For the Gaussian mechanism it still sits 5 to 10% above the exact ε, and only a different kind of accountant removes that slack.

30.2. What δ means for your users

An (ε, δ) guarantee says that one person's record changes the chance of any outcome by at most a factor e^{ε}, except for an extra probability δ. Strictly, δ is an average over the privacy loss of each output, defined in Numerical accounting in the full edition. In practice, treat δ as the chance that the guarantee fails for a person, possibly badly. Write n for the number of people in the data. Then δ must be much smaller than $1/n$, which is why The definition of differential privacy recommends a value such as $1/n^{1.5}$.

30.3. Mironov's and the improved conversion

The original conversion is **Mironov's conversion** (Mironov 2017). For a mechanism with RDP curve $\varepsilon(\alpha)$ and any order α, it says that the mechanism is (ε', δ)-DP with

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{30.1}$$

You minimize it over the grid of orders. The result is valid but loose, because the conversion charges every output whose privacy loss exceeds ε' with its whole probability, and then bounds that tail with Markov's inequality.

The **improved conversion** recovers most of that loss. Balle and coauthors, Canonne, Kamath and Steinke, and Asoodeh and coauthors found it independently in 2020. A mechanism with RDP curve $\varepsilon(\alpha)$ is (ε', δ)-DP, at every order, with

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}. \tag{30.2}$$

You again minimize over the grid, and if the result comes out negative, you report 0, since a guarantee at a negative ε also holds at 0. At any fixed order the improved value is lower by an amount that depends only on the order, 0.3612 at order 10, and more at smaller orders.

For a worked example, release the mean of a bounded statistic once with the Gaussian mechanism at noise multiplier 1, so that its curve is $\alpha/2$. At $\delta = 10^{-5}$, Mironov's conversion gives 5.299. The improved conversion gives 4.729, against an exact value of 4.377 (f-DP and Gaussian differential privacy in the full edition).

This explains why your library's ε can be lower than an older paper's for the same settings, since older papers and code used Mironov's conversion. Their numbers are loose, not wrong. So if two RDP accountants report values about 0.4 apart for the same run, with a best order near 10, one of them uses Mironov's conversion. On this part's running example, private stochastic gradient descent (SGD), called DP-SGD, Mironov's conversion gives 7.43 and the improved one 6.71.

30.4. Which conversion your library uses

Both widely used libraries use the improved conversion, as of Opacus 1.6.0 and `dp_accounting` 0.6.0. In Opacus, `RDPAccountant` calls the function `get_privacy_spent(orders=..., rdp=..., delta=...)` of `opacus.accountants.analysis.rdp`, and in `dp_accounting`, `RdpAccountant.get_epsilon(target_delta)` calls `compute_epsilon`. But their default grids of orders differ, and the grid matters as much as the formula. The Opacus list `RDPAccountant.DEFAULT_ALPHAS` stops at order 63, while the `DEFAULT_RDP_ORDERS` of `dp_accounting` go on to 1024.

So set the orders yourself when the default does not fit. In Opacus, pass `alphas=[...]` to the accountant's `get_privacy_spent` or `get_epsilon`. In `dp_accounting`, pass `orders=[...]` to the `RdpAccountant` and read the best order from `get_epsilon_and_optimal_order`. Opacus warns when the best order is at an end of its list, but `dp_accounting` does not, so there you compare the returned order with the ends of the list yourself.

One composed curve gives a guarantee at every δ, so you can report several pairs at no extra privacy cost. In `dp_accounting`, call `get_epsilon(target_delta)` once per δ, or `get_delta(target_epsilon)` for the reverse question. For the single release above, the improved conversion gives 4.729 at $\delta = 10^{-5}$ and 6.086 at 10^{-8}, as the snippet at the end prints.

30.5. Choosing the order

For a run of k Gaussian steps with noise multiplier σ, the composed curve is a straight line in the order. Its slope, written ρ, is $k/(2\sigma^2)$. Then the improved conversion's best order is close to

$$\alpha^* - 1 \approx \sqrt{\frac{\ln(1/\delta)}{\rho}} = \sigma\sqrt{\frac{2\ln(1/\delta)}{k}}. \tag{30.3}$$

More steps or less noise make the curve steeper, so they push the best order towards 1. But a smaller δ pushes it up only through the square root of a logarithm, so δ matters little. Table 30.1 shows the range a grid must cover.

Table 30.1.: Best orders of the improved conversion for the composed Gaussian at $\delta = 10^{-5}$.

σ	k	ρ	Best order (continuous / grid)	ε (continuous / grid)
10	1	0.005	40.525 / 41	0.375 / 0.375
1	1	0.5	5.432 / 5.4	4.728 / 4.729
1	100	50	1.472 / 1.5	96.035 / 96.116
1	10,000	5000	1.048 / 1.1	5475.791 / 5611.778

Long runs with little noise need orders close to 1, as the last row shows, and short runs with much noise need large orders. Subsampled DP-SGD steps have tiny curves at small orders, so a run with few steps or much noise can want orders above 63, the top of the Opacus default. So whenever the reported best order sits at either end of the grid, extend the grid in that direction and convert again.

30.6. The slack no conversion removes

No cleverer formula closes the gap between 4.729 and the exact 4.377 for the single release, because the curve itself lacks the information. The full edition computes the best conversion any method can get from the curve, and it matches the improved one to three decimals at every run length in Table 30.2.

Table 30.2.: ε at $\delta = 10^{-5}$ for k Gaussian steps with $\sigma = 10$, where $\mu = \sqrt{k}/\sigma$.

k	μ	Exact	Improved	Mironov
1	0.100	0.341	0.375	0.485
100	1.000	4.377	4.729	5.299
10,000	10.000	91.817	96.116	98.026

Every number in the table depends on the run only through μ, so the row with 100 steps is the single release again. The improved conversion exceeds the exact value by 10% for one step and by 5% for 10,000 steps, while Mironov's excess shrinks from 42% to 7%.

That 5 to 10% holds for the plain Gaussian. Subsampled DP-SGD steps have curves that are not straight lines, and the gap can be much larger (Choosing an accountant in the full edition measures it for the running example). When the slack matters, change the accountant rather than the formula. A privacy loss distribution (PLD) accountant (Numerical accounting in the full edition) tracks the whole distribution of the privacy loss, and it reaches the exact 4.377 for the single release.

30.7. RDP, attacks and other guarantees

An RDP curve also limits attacks. Each order bounds the power of any test that tries to tell whether your record was used. Write a for the test's false-positive rate. An (ε, δ) pair only caps the power at $e^{\varepsilon}a + \delta$, while at low false-positive rates the whole curve caps it far more tightly. So read attack risk from the curve, or better from a trade-off function (f-DP and Gaussian differential privacy in the full edition). Then compare it with what an audit measures (Auditing private training).

Mixing RDP with other guarantees works in one direction only. A pure ε-DP component has a curve, which Concentrated differential privacy in the full edition gives. But an (ε, δ) component with δ above 0 has none, because a mechanism that publishes your record with probability δ is $(0, \delta)$-DP yet has infinite Rényi divergence. So never feed an (ε, δ) pair into an RDP accountant, and take the curve from the mechanism behind it instead.

30.8. Code

This snippet converts the single release's curve on the part's grid of orders at three values of δ. To convert another mechanism, replace the curve by its RDP values on the same grid.

```
from math import log

ORDERS = [1 + x / 10 for x in range(1, 100)] + list(range(12, 64)) + [128, 256, 512]


def mironov(rdp: float, alpha: float, delta: float) -> float:
    return rdp + log(1 / delta) / (alpha - 1)


def improved(rdp: float, alpha: float, delta: float) -> float:
    """Balle et al. (2020); Canonne, Kamath & Steinke (2020). Clipped at 0."""
    return max(0.0, rdp + log((alpha - 1) / alpha) - (log(delta) + log(alpha)) / (alpha - 1))


def report(mu: float, delta: float = 1e-5) -> str:
    """Both conversions for a composed Gaussian with mu = sqrt(steps) / sigma."""
    curve = [a * mu**2 / 2 for a in ORDERS]
    mir = min((mironov(e, a, delta), a) for a, e in zip(ORDERS, curve))
    imp = min((improved(e, a, delta), a) for a, e in zip(ORDERS, curve))
    return "Mironov %.3f at order %g, improved %.3f at order %g" % (*mir, *imp)


print(report(1.0))  # Mironov 5.299 at order 5.8, improved 4.729 at order 5.4
print(report(1.0, 1e-6))  # Mironov 5.757 at order 6.3, improved 5.222 at order 5.9
print(report(1.0, 1e-8))  # Mironov 6.570 at order 7.1, improved 6.086 at order 6.7
```

The full edition's script, rdp_conversions.py, also computes the exact Gaussian values, the best possible conversion and the limits on tests.

30.9. Cheat sheet

Table 30.3.: The results of this chapter.

Result	Formula	Notes
What δ means	The chance the guarantee fails for a person	Keep it well below $1/n$
Mironov's conversion	$\varepsilon' = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$, best α	Valid but loose; older papers and code
Improved conversion	$\varepsilon' = \varepsilon(\alpha) + \ln \frac{\alpha-1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1}$, best α	Use this; clip at 0; Opacus and `dp_accounting` use it
Best order, composed Gaussian	$\alpha^* - 1 \approx \sigma\sqrt{2\ln(1/\delta)/k}$	Falls towards 1 as k grows
Unavoidable slack	5 to 10% of ε for the Gaussian	Larger for DP-SGD; use a PLD accountant
(ε, δ) to RDP	Impossible for $\delta > 0$	Take the curve from the mechanism

Rules of thumb

- Always use the improved conversion, which costs nothing.
- Check the best order. If it sits at either end of the grid, extend the grid and convert again.
- Never compare values of ε from different conversions or grids as if they were the same quantity.
- Choose δ well below $1/n$, and report ε at several values of δ from the same curve.
- Expect an RDP accountant to overstate the plain Gaussian's ε by 5 to 10%, and DP-SGD's by more. If that matters, use a PLD accountant.

30.10. Further reading

- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020): the improved conversion, and the region an RDP curve puts on tests.
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (2022 survey): privacy loss, RDP and conversions in one place, with their tightness.

Derivations. Every result here is derived step by step in the full edition's chapter.

31. Concentrated differential privacy

31.1. Why this matters

Suppose you run a pipeline of Gaussian releases, such as noisy counts or the noisy marginals behind a synthetic dataset. You would like to keep its differential privacy (DP) budget as a single number that you add to after each release. Rényi DP in the full edition describes each mechanism by a whole curve, its **Rényi differential privacy (RDP) curve**, with one value for each order of the Rényi divergence. The Gaussian mechanism's curve is a straight line through the origin, so one number, its slope, carries all of it.

Zero-concentrated differential privacy (zCDP) is the bookkeeping built on that number, and it is the most widely used form of concentrated differential privacy (CDP). It is the right ledger for releases built from Gaussian noise on the full data. It is the wrong ledger for training on subsampled minibatches, because it throws away almost all of the privacy that subsampling buys.

31.2. The zCDP ledger

A mechanism is zCDP when its RDP curve lies under a line through the origin. To state it, write D_α for the Rényi divergence of order α, and call two datasets neighbours when they differ in one person's record. Then a mechanism M is **ρ-zCDP** if, for all neighbouring datasets and all orders above 1,

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha. \tag{31.1}$$

Two properties make this a ledger. Composition adds the slopes, even when each step is chosen after seeing the earlier outputs, and processing an output never raises its ρ.

The Gaussian mechanism is the case the definition was built for. Write σ for the standard deviation of its noise and Δ for the query's ℓ_2 sensitivity, the most one record can move the query's value. Its RDP curve is exactly a line, so one step costs

$$\rho = \frac{\Delta^2}{2\sigma^2}. \tag{31.2}$$

With the noise multiplier convention, which measures σ in units of the sensitivity, a step with $\sigma = 1$ costs 0.5. Doubling the noise multiplier cuts the cost to a quarter.

The best-known user is the 2020 United States Census TopDown algorithm (Abowd et al. 2022). It answers many queries with the discrete Gaussian mechanism (Discrete and floating-point-safe mechanisms), which keeps counts as integers. Among libraries, Tumult Analytics offers a `RhoZCDPBudget`. In OpenDP, `dp.c.make_zCDP_to_approxDP` turns a zCDP measurement into

one with a privacy profile, and `dp.c.make_fix_delta` then fixes the slack so that you can read off the privacy level.

The Census example carries two warnings. First, ρ is not the privacy parameter ε of an (ε, δ) guarantee, where δ is the slack. A 0.5-zCDP step converts to about $\varepsilon = 4.7$ at $\delta = 10^{-5}$ (Section 31.4), so a reader who mistakes one for the other badly overrates the protection. Second, ρ covers only what passed through the noise. The Census Bureau released some statistics exactly, as a matter of policy, and these **invariants** sit outside the guarantee. A report should list such exact releases next to the budget.

31.3. Charging other kinds of step

A real pipeline mixes mechanisms, and Table 31.1 gives the charge in ρ for the common ones.

Table 31.1.: What each kind of step costs in a zCDP ledger.

Step	Charge in ρ	Notes
Gaussian, sensitivity Δ, noise σ	$\Delta^2/(2\sigma^2)$	Exact
Any pure ε-DP step	$\varepsilon^2/2$, or sharply $\varepsilon \tanh(\varepsilon/2)$	Poor fit once $\varepsilon \geq 2$
Exponential mechanism at ε	$\varepsilon^2/8$	Four times cheaper
An (ε, δ)-DP step	$\varepsilon^2/2$, plus δ in a second column	Approximate zCDP, Section 31.5
Any step, for a group of m records	m^2 times the step's ρ	Exact for the Gaussian
Poisson-subsampled Gaussian	$1/(2\sigma^2)$, whatever the sampling rate	No amplification, Section 31.6

A pure ε-DP step, such as a Laplace count or a randomized-response bit, changes the probability of any output by at most a factor e^{ε} between neighbours. The sharp charge $\varepsilon \tanh(\varepsilon/2)$ is the smallest that holds for all such steps, because randomized response is the worst case, and at $\varepsilon = 1$ it is 0.4621. The exponential mechanism's charge of $\varepsilon^2/8$ is due to Cesar and Rogers (2021).

The general rule breaks down for large ε. A pure DP step's RDP curve never rises above ε, while the line keeps climbing and passes that cap at order $2/\varepsilon$. So a 5-DP step charged $\varepsilon^2/2 = 12.5$ sits above its own cap at every order. Converting that charge back gives a worse ε than the 5 you started with, so keep such steps in RDP, where an accountant can charge them the exact curve of randomized response.

31.4. From ρ to (ε, δ)

You always report (ε, δ) in the end. Write ℓ for $\ln(1/\delta)$. Applying Mironov's conversion in the full edition to a line gives the **simple conversion** of Bun and Steinke (2016). It says that a ρ-zCDP

mechanism is (ε, δ)-DP with

$$\varepsilon = \rho + 2\sqrt{\rho\,\ell}. \tag{31.3}$$

The **improved conversion** of Canonne, Kamath and Steinke (2020) adds two negative correction terms at each order. It has no closed form, so you minimize it numerically, as the code below does. For the Gaussian the true ε is also known, from its privacy profile in f-DP and Gaussian differential privacy in the full edition. Table 31.2 compares the three.

Table 31.2.: ε at $\delta = 10^{-5}$ for a ρ-zCDP mechanism: the simple and improved conversions, and the exact ε of the Gaussian mechanism with the same ρ.

ρ	Simple Equation 31.3	Improved	Exact, Gaussian with the same ρ
0.1	2.246	1.914	1.760
0.5	5.299	4.728	4.377
1	7.786	7.077	6.573
5	20.174	19.047	17.857

At $\rho = 0.5$, the simple formula overshoots the truth by 21%, and the improved one by 8%. So use the improved conversion, or the exact Gaussian profile when every step is Gaussian.

For calibration you run the conversion backwards, from a target (ε, δ) to the largest total ρ that meets it. Solving the simple conversion for ρ shows that the target is met by any

$$\rho \le \left(\sqrt{\varepsilon + \ell} - \sqrt{\ell}\right)^2. \tag{31.4}$$

Then split ρ over the queries. If query number i has ℓ_2 sensitivity Δ_i and spends ρ_i, its noise needs the standard deviation

$$\sigma_i = \frac{\Delta_i}{\sqrt{2\rho_i}}. \tag{31.5}$$

For example, a release that must be $(1, 10^{-5})$-DP may spend at most $\rho = 0.02082$. A single query of sensitivity 1 then needs $\sigma = 4.901$. If you split the same budget equally over four queries, each gets a quarter of ρ and so needs twice the noise, 9.801.

That recipe works because the split happens in ρ, which adds, and never in ε, which does not. Suppose you have a budget of $\rho = 1$ and have spent half of it. Each half alone converts to 4.728, and two of those are far more than the whole run's 7.077, because the conversion grows like the square root of ρ. So keep budgets in ρ, subtract in ρ, and convert once, at the end. In OpenDP, `dp.binary_search_param` does the last step of calibration for you, by searching for the noise scale that meets a target. Use the ℓ_2 sensitivity here, because the ℓ_1 sensitivity of a vector query is never smaller and so overpays in noise.

31.5. Groups and approximate zCDP

A household, a user with many records or a hospital contributes several records, so you may need the guarantee for a group. Take a ρ-zCDP mechanism M and two datasets that differ in m

records. Bun and Steinke (2016) show that

$$D_\alpha(M(D) \,\|\, M(D')) \le m^2 \rho\, \alpha. \tag{31.6}$$

The square is exact for the Gaussian, since a group shifts the query m times as far. For example, a user with 4 records in a 0.1-zCDP release is protected at $\rho = 1.6$. Plain RDP's group bound is much weaker (Rényi DP in the full edition).

Some useful mechanisms, such as propose-test-release and stability-based histograms, are (ε, δ)-DP but cannot be zCDP at all. **Approximate zCDP** keeps them in the ledger by adding a second column. Each such step adds $\varepsilon^2/2$ to ρ and its slack to the second column, and at the end the second column's total is added to the δ of the conversion.

31.6. Subsampled steps: keep them out of zCDP

Differentially private stochastic gradient descent (SGD), DP-SGD for short, runs each step on a Poisson sample of the data. Each record enters the batch independently with probability q, the sampling rate, and this makes a step far more private than the Gaussian mechanism inside it (Amplification by subsampling in the full edition). But zCDP cannot see the gain. The subsampled step's RDP curve is tiny at small orders and bends up towards the unsubsampled line at large ones. A line that covers the whole curve must therefore have slope $1/(2\sigma^2)$, whatever the sampling rate.

The cost of ignoring this is enormous. On this part's running example of 10,000 steps with $q = 0.01$ and $\sigma = 1$, zCDP reports $\varepsilon = 5475.8$. RDP accounting of the same run gives 6.719, because the conversion only uses the small orders where the curve is still tiny.

Truncated concentrated differential privacy (tCDP) asks for the line only on low orders, so it sees some of the amplification. It is rarely used, because it does no better than RDP.

So if a pipeline mixes Gaussian releases with a DP-SGD run, keep all of it in RDP. A ρ-zCDP step is just the RDP line $\rho\alpha$, so it joins an RDP ledger without loss. In Google's `dp_accounting` library, for example, you add it to an `RdpAccountant` as a `GaussianDpEvent` with noise multiplier $1/\sqrt{2\rho}$. The training steps go in next to it as a `PoissonSampledDpEvent`.

31.7. Code

The whole zCDP ledger is a few lines of code. You charge each Gaussian step its ρ, add the charges, and convert once, and the last line splits a budget over four queries.

```
from math import log, sqrt


def zcdp_gaussian(sigma: float, sensitivity: float = 1.0) -> float:
    return sensitivity**2 / (2 * sigma**2)
```

```
def zcdp_to_eps_improved(rho: float, delta: float) -> float:
    """Minimized over 20,001 log-spaced orders, the full edition's fine grid."""
    best = float("inf")
    for i in range(20001):
        alpha = 1 + 10 ** (-4 + 8 * i / 20000)
        eps = rho * alpha + log(1 / delta) / (alpha - 1) + log(1 - 1 / alpha) - log(alpha) /
        best = min(best, eps)
    return best


def rho_for_target(eps: float, delta: float) -> float:
    """Largest rho whose simple conversion stays within eps at delta."""
    return (sqrt(eps + log(1 / delta)) - sqrt(log(1 / delta))) ** 2


def sigma_for_rho(rho: float) -> float:
    return 1 / sqrt(2 * rho)


rho = 10 * zcdp_gaussian(sigma=1.0)  # ten Gaussian steps with sigma = 1
print(f"{rho:.1f}")  # 5.0
print(f"{zcdp_to_eps_improved(rho, 1e-5):.3f}")  # 19.047
budget = rho_for_target(1.0, 1e-5)  # the whole release must be (1, 1e-5)-DP
print(f"{budget:.5f} {sigma_for_rho(budget):.3f}")  # 0.02082 4.901
print(f"{sigma_for_rho(budget / 4):.3f}")  # 9.801
```

31.8. Cheat sheet

Here ℓ stands for $\ln(1/\delta)$.

Table 31.3.: The tasks of this chapter; Table 31.1 gives the charge of each kind of step.

Task	Formula	Notes
Charge a Gaussian step	$\rho = \Delta^2/(2\sigma^2)$	Exact
Convert	Improved conversion, or $\rho + 2\sqrt{\rho\ell}$	8% and 21% above exact at $\rho = 0.5$
Calibrate	$\rho = (\sqrt{\varepsilon + \ell} - \sqrt{\ell})^2$, then $\sigma_i = \Delta_i/\sqrt{2\rho_i}$	Split the budget in ρ
Protect a group of m	$m^2\rho$	Exact for the Gaussian
Account a subsampled step	Not in zCDP	Use RDP or a privacy loss distribution

Rules of thumb

- Keep budgets in ρ, which adds, and convert to (ε, δ) once, at the end, with the improved conversion.
- Never report ρ as if it were ε. A 0.5-zCDP step is about 4.7-DP at $\delta = 10^{-5}$.
- Report the conversion you used, and any statistics released exactly.
- Calibrate Gaussian noise with the ℓ_2 sensitivity, not the ℓ_1.
- Charge a pure DP step $\varepsilon^2/2$ only for small ε. From about 2 up, keep it in RDP at its own ε.
- A pipeline with any DP-SGD in it belongs in RDP or a numerical accountant, not in zCDP.

31.9. Further reading

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): the definition and every rule of this chapter, in Sections 1–3.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020): the discrete Gaussian and the improved conversion.

Derivations. Every result here is derived step by step in the full edition's chapter.

32. f-DP and Gaussian differential privacy

32.1. Why this matters

Every privacy guarantee bounds how well an attacker can tell from a mechanism's output whether it ran on a dataset D or on a neighbour D' that differs in one record. Differential privacy (DP) answers with two numbers, and Rényi differential privacy (RDP) in the full edition with a curve of divergences. Zero-concentrated differential privacy (zCDP) in the full edition answers with one number. All three summarize a more complete answer, the attacker's best error rates, and this chapter works with that answer directly.

It pays off for two reasons. First, Gaussian noise gives a family of answers called **Gaussian differential privacy (GDP)**. Its one parameter, written μ, reads directly as the best success rate of a membership attack. Gaussian steps compose within the family, so a Gaussian pipeline has an exact guarantee in one number. Second, a central limit theorem (CLT) gives a quick estimate of μ for training with differentially private stochastic gradient descent (SGD), called DP-SGD. That estimate can understate the true privacy loss, so it must never be the number you report.

32.2. Trade-off curves and f-DP

The attacker's decision is a hypothesis test (Hypothesis testing), and a test can make two kinds of mistake. Write a for its **type I error**, how often it claims D' when the data was D. Write b for its **type II error**, how often it makes the reverse mistake. In a membership attack, where D' contains the target's record, a is the false positive rate and $1 - b$ the true positive rate. The **trade-off function** of a mechanism gives, at each a, the smallest b that any test can reach.

A guarantee is a floor under that curve. Write $T(M(D), M(D'))$ for the trade-off function of a mechanism M on two datasets. Dong, Roth and Su (2022) call the mechanism **f-differentially private (f-DP)** if, for all neighbours,

$$T(M(D), M(D')) \geq f. \tag{32.1}$$

The familiar guarantee with a privacy level ε and a slack δ is the special case whose floor is two straight lines,

$$f_{\varepsilon,\delta}(a) = \max\left(0,\ 1 - \delta - e^{\varepsilon}a,\ e^{-\varepsilon}(1 - \delta - a)\right). \tag{32.2}$$

Two straight lines make a crude floor. Randomized response and the Laplace mechanism both carry the label "$\varepsilon = 1$", but only randomized response sits on $f_{1,0}$, while the Laplace curve lies above it in the middle. So when you compare mechanisms, or audit one, compare curves.

32.3. μ-GDP, read as an attack

The Gaussian mechanism's curve has a simple form. Write σ for the noise multiplier, the standard deviation of the noise in units of the query's ℓ_2 sensitivity. Every Gaussian mechanism then reduces to telling a standard normal draw from one shifted by a distance $\mu = 1/\sigma$. Write Φ for the standard normal distribution function, so that $\Phi(t)$ is the chance that a standard normal draw is at most t. The best test's type II error at type I error a is

$$G_\mu(a) = \Phi(\Phi^{-1}(1-a) - \mu). \tag{32.3}$$

A mechanism is **μ-Gaussian differentially private (μ-GDP)** if G_μ is a floor for it. For the Gaussian mechanism this floor is the exact curve, so nothing is lost.

The parameter μ reads directly as an attack. Take a count released with $\sigma = 1$, so 1-GDP. At a 1% false positive rate, and whatever the attacker knows about the rest of the data, it detects the target's record at most 9.24% of the time. Table 32.1 gives this best true positive rate for three values of μ, next to the ε that each converts to.

Table 32.1.: Reading μ-GDP as an attack and as (ε, δ). The attack column is $1 - G_\mu(a)$; the code below computes all three.

μ	Best true positive rate at 0.1% false positives	At 1%	At 5%	ε at $\delta = 10^{-5}$
0.5	0.0048	0.0339	0.1261	1.9931
1	0.0183	0.0924	0.2595	4.3772
2	0.1378	0.3721	0.6388	9.9973

This table is the most useful way to explain a guarantee to someone who must decide whether it is enough. It is also the ceiling that a membership inference attack can reach, and that a privacy audit tests against, usually at low false positive rates. As a rough guide, Dong et al. suggest that $\mu \le 0.5$ is a reasonable guarantee and that $\mu \ge 6$ promises almost nothing.

32.4. Converting μ to (ε, δ)

To report (ε, δ), you turn the curve into pairs. Its **privacy profile** $\delta(\varepsilon)$ is the smallest slack that goes with each privacy level. For a curve that is unchanged when you swap D and D', the whole profile loses nothing, because together its pairs rebuild the curve. For μ-GDP the profile has a closed form (Dong et al., Corollary 2.13):

$$\delta(\varepsilon) = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big). \tag{32.4}$$

This is also the exact (ε, δ) curve of the Gaussian mechanism, the analytic calibration of The Laplace and Gaussian mechanisms (Balle & Wang 2018).

You use it in both directions, with any root finder, such as `scipy.optimize.brentq` on `scipy.stats.norm.cdf`, or the bisection in the code below. To report, fix δ and solve Equation 32.4 for ε by bisection, which works because the profile falls as ε grows. For example, 1-GDP is $(4.3772, 10^{-5})$-DP. To calibrate, fix the target pair, bisect for the largest μ that meets it, and set the noise multiplier to $1/\mu$. For example, a single $(1, 10^{-5})$-DP release needs $\sigma = 3.7306$.

One pair, on its own, loses a lot. At a type I error of 0.1, for example, the pair for 1-GDP allows a type II error as low as 0.0113. The curve G_1 itself requires 0.6109.

Do not calibrate with the classical formula instead, which sets the noise for a query of sensitivity Δ to

$$\sigma = \frac{\Delta\sqrt{2\ln(1.25/\delta)}}{\varepsilon}. \tag{32.5}$$

It is valid only for $\varepsilon < 1$, and even there it adds more noise than the exact curve requires. For a numerical accountant, `dp_accounting.pld.privacy_loss_distribution.from_gaussian_mechanism` builds the same exact curve as a privacy loss distribution.

32.5. Composition: μ^2 adds

Dong et al. show that composing f_i-DP steps gives a curve that depends only on the f_i, even when each step is chosen after seeing the earlier outputs. For Gaussian curves it stays Gaussian (Dong et al., Corollary 3.3):

$$G_{\mu_1} \otimes \cdots \otimes G_{\mu_k} = G_{\mu}, \qquad \mu = \sqrt{\mu_1^2 + \cdots + \mu_k^2}. \tag{32.6}$$

Here $\otimes$ stands for composition, and the quantity that adds up is μ^2. So full-batch noisy gradient descent is $\sqrt{k}/\sigma$-GDP after a number k of steps. Since that is the exact curve, no accountant can improve on it. The same holds for every pipeline of Gaussian noise without subsampling, such as analytics queries, the noisy vote counts of private aggregation of teacher ensembles (PATE) and matrix mechanisms.

The zCDP ledger does the same arithmetic, where ρ, the zCDP parameter, equals $\mu^2/2$. But its conversion loses some of the curve, while Equation 32.4 loses none.

Composing curves rather than pairs matters in practice. Four 1-GDP releases compose to 2-GDP, which is $(9.9973, 10^{-5})$-DP. Suppose you instead turn each release into a pair with a quarter of the δ budget, $(4.6902, 2.5 \times 10^{-6})$, and add the pairs by basic composition. You get $\varepsilon = 18.7607$, almost twice as much. So for a pipeline of Gaussian releases without subsampling, keep a ledger of μ^2 and convert once. Curves that are not Gaussian leave their family when composed, so for them you need a numerical accountant in the full edition.

32.6. The CLT estimate for DP-SGD

Most mechanisms are not Gaussian. But a central limit theorem for f-DP says that many steps that each leak little compose to nearly a Gaussian curve (Dong et al. 2022). DP-SGD is such a run.

Each step of DP-SGD samples a batch by Poisson sampling, which includes each record independently with a **sampling rate** q. It then clips each gradient and adds Gaussian noise (Amplification by subsampling in the full edition). With add/remove neighbours, Bu et al. (2020) show that k such steps are approximately μ-GDP with

$$\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}. \tag{32.7}$$

It shows that the parameter grows in proportion to the sampling rate and to the square root of the number of steps, and steeply as the noise falls.

The formula is an approximation, not a bound, and it errs on the unsafe side. A CLT controls the bulk of a distribution much better than its tails, where small values of δ are decided. After the 10,000 steps of this part's running example, the CLT gives $\varepsilon = 6.0071$ at $\delta = 10^{-5}$. A numerical accountant proves that the true ε is at least 6.1377, so the CLT number would be false (Gopi, Lee & Wutschitz 2021). Opacus offers this accountant as `accountant="gdp"`, implemented by `opacus.accountants.GaussianAccountant`, and it warns that the GDP accountant is experimental and can underestimate the privacy spent. For anything you report, use its default `"prv"` accountant or `"rdp"` (Choosing an accountant in the full edition). The formula also assumes Poisson sampling with add/remove neighbours (Amplification by subsampling in the full edition).

32.7. RDP forgets part of the curve

An RDP curve is a summary of the trade-off curve, and it forgets part of it even for the Gaussian. For 1-GDP the exact $\delta(1)$ is 0.1269. By any conversion in the full edition, its RDP curve certifies no better than 0.2149. So for Gaussian noise without subsampling, account in GDP.

32.8. Code

A GDP accountant needs only the normal distribution, which Python's `statistics.NormalDist` provides. The snippet computes the attack reading of μ, the conversions in both directions and the CLT estimate.

```
from math import exp, sqrt
from statistics import NormalDist

N = NormalDist()


def gdp_power(mu: float, fpr: float) -> float:
    """Best true positive rate of any attack on mu-GDP at a given false positive rate."""
    return 1 - N.cdf(N.inv_cdf(1 - fpr) - mu)


def gdp_delta(eps: float, mu: float) -> float:
    """The exact delta(eps) of mu-GDP."""
```

```
    return N.cdf(-eps / mu + mu / 2) - exp(eps) * N.cdf(-eps / mu - mu / 2)


def gdp_eps(mu: float, delta: float) -> float:
    """The eps at which mu-GDP reaches delta, by bisection."""
    lo, hi = 0.0, 500.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gdp_delta(mid, mu) > delta else (lo, mid)
    return hi


def gdp_mu(eps: float, delta: float) -> float:
    """The largest mu that meets (eps, delta), by bisection."""
    lo, hi = 1e-6, 100.0
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gdp_delta(eps, mid) < delta else (lo, mid)
    return lo


def clt_mu_dpsgd(q: float, sigma: float, steps: int) -> float:
    """CLT estimate of mu for Poisson-subsampled DP-SGD; not a bound."""
    return q * sqrt(steps * (exp(1 / sigma**2) - 1))


print(" ".join(f"{gdp_power(m, 0.001):.4f}" for m in (0.5, 1.0, 2.0)))  # 0.0048 0.0183 0.13
print(f"{gdp_eps(1.0, 1e-5):.4f}")  # 4.3772
mu = gdp_mu(1.0, 1e-5)  # calibrate one release to (1, 1e-5)
print(f"{mu:.4f} {1 / mu:.4f}")  # 0.2681 3.7306
mu = clt_mu_dpsgd(q=0.01, sigma=1.0, steps=10_000)
print(f"{mu:.4f} {gdp_eps(mu, 1e-5):.4f}")  # 1.3108 6.0071
```

32.9. Cheat sheet

Table 32.2.: The tasks of this chapter; Table 32.1 converts μ to attack rates and to ε.

Task	Formula	Notes
Read a guarantee as an attack	True positive rate $\leq 1 - f(a)$ at false positive rate a	The curve is the guarantee
Gaussian mechanism	$\mu = 1/\sigma$; $G_\mu(a) = \Phi(\Phi^{-1}(1-a) - \mu)$	Exact
Convert to (ε, δ)	$\delta(\varepsilon) = \Phi(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}) - e^\varepsilon \Phi(-\frac{\varepsilon}{\mu} - \frac{\mu}{2})$	Bisect for ε

Task	Formula	Notes
Calibrate	Bisect for the largest μ, then $\sigma = 1/\mu$	$(1, 10^{-5})$ needs $\sigma = 3.7306$
Compose Gaussian steps	$\mu = \sqrt{\sum_i \mu_i^2}$	Exact
Estimate DP-SGD	$\mu \approx q\sqrt{k(e^{1/\sigma^2} - 1)}$	Not a bound; under-reports ε

Rules of thumb

- The trade-off curve is the guarantee, and (ε, δ), RDP and ρ are summaries of it. Summarize as late as possible.
- For Gaussian noise without subsampling, keep a ledger of μ^2 and convert once, exactly.
- Explain a guarantee with its attack reading, and report μ next to any (ε, δ) pair.
- Calibrate from the exact curve, never from the classical $\sqrt{2\ln(1.25/\delta)}$ formula.
- Use the CLT, and Opacus's `gdp` accountant, only for quick estimates, never for the reported guarantee.

32.10. Further reading

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022): the source of f-DP and GDP.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): an accountant with error bounds, and examples where the CLT under-reports ε.

Derivations. Every result here is derived step by step in the full edition's chapter.

33. Amplification by subsampling

33.1. Why this matters

Most private models are trained with stochastic gradient descent (SGD) made to satisfy differential privacy (DP), a method called DP-SGD. Each step clips every example's gradient, sums the clipped gradients and adds Gaussian noise, whose standard deviation is the clipping norm times the **noise multiplier** σ. Suppose every record paid the full cost of a Gaussian mechanism at every step. With $\sigma = 1$, ten thousand steps would then cost a privacy level of $\varepsilon = 5425.5$ (Rényi DP in the full edition).

DP-SGD escapes this cost because each step looks only at a small random batch. Write q for the **sampling rate**, the probability that a given record lands in the batch. An attacker must then first guess whether the record was used at all, and that uncertainty is worth a lot. This effect is called **amplification by subsampling**. It depends on exactly how the batch was drawn, so the accountant, the code that adds up a run's privacy cost, must assume the sampler the training code actually runs. Getting that wrong is the largest single error in DP-SGD accounting.

33.2. One step under Poisson sampling

The simplest sampler is **Poisson sampling**, in which each record enters the batch by its own coin flip, with probability q, independently of every other record. Pair it with add/remove neighbours, datasets that differ by one record being present or absent. Balle, Barthe and Gaboardi (2018) give the tight result. If a step is (ε, δ)-DP on its own, where δ is the slack, then the same step run on a Poisson sample is (ε', δ')-DP with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{33.1}$$

The formula behaves very differently for small and large ε. For small ε the amplified cost is about $q\varepsilon$, so a 1% sample makes a step about a hundred times cheaper. With that sample, for example, a 1-DP step costs 0.0170. But for large ε the amplified cost is about

$$\varepsilon' \approx \varepsilon - \ln(1/q), \tag{33.2}$$

so sampling only subtracts a fixed amount. With the same sample, a 10-DP step still costs 5.3993. The switch happens where ε is near $\ln(1/q)$.

The result has two conditions. Each record's coin must be independent of the others, so that the record is drawn with probability exactly q whatever else is in the batch. And the batch must stay secret, because an attacker who sees the list of records drawn knows whether the target was used. The rule applies to any (ε, δ)-DP step, not only to the Gaussian. So a pure DP or

randomized-response release run on a secret Poisson sample is amplified too. For example, 1-DP randomized response on a 10% sample costs 0.1586. Per-step pairs compose badly over thousands of steps, though (Section 33.4).

33.3. Match the sampler to the accountant

Most training code does not draw Poisson batches. Some draws a fresh random batch of a fixed size, written m, from the n records at each step. Much more code shuffles the data once per epoch and cuts the list into consecutive batches, which is called **shuffle and partition**. Each sampler pairs naturally with a neighbouring relation, and Table 33.1 gives what to account for under each pairing.

Table 33.1.: What to account for under each combination of sampling scheme and neighbouring relation.

Sampling	Neighbours	What to account for
Poisson, rate q	add/remove	A Poisson accountant with q and σ
Fixed size m of n	add/remove	A Poisson accountant with rate m/n and noise multiplier $\sigma/2$ (Lebeda et al. 2024)
Fixed size m of n	replace-one	Per step, Equation 33.1 with $q = m/n$ and $\sigma/2$; no Poisson shortcut
Poisson, rate q	replace-one	About twice the add/remove cost for pure DP at small ε
Shuffle and partition	zero-out	No amplification formula; the Poisson number can be far too small

The factor of two in the noise comes from the gradient sum. Adding or removing a record moves the clipped sum by at most the clipping norm. But a record that enters a fixed-size batch pushes another one out, so the sum can move by twice as much. Twice the shift with the same noise is the same as the original shift with half the noise multiplier, and that is not a small change. This part's running example takes ten thousand steps with $q = 0.01$ and $\sigma = 1$. At $\delta = 10^{-5}$, the Poisson accountant gives it $\varepsilon = 6.71$. With fixed-size batches under add/remove it must be run at $\sigma = 0.5$, and it then gives 47.42, about seven times more.

Shuffle and partition is worse, because the source of amplification is missing. Within an epoch the record is in exactly one batch and is never absent, so the attacker only has to find which batch held it. Chua et al. (2024) build a dataset where the target's batch stands out, under zero-out neighbours, which set the target's gradient to 0. A test on it proves that one shuffled epoch of the

running example has ε of at least 4.06, while a Poisson accountant reports at most 1.21. So a Poisson number reported for shuffled training can be provably false.

So read the data loader before you trust a reported ε. A PyTorch `DataLoader` with `shuffle=True` and a `batch_size` is shuffle and partition, and a `tf.data` pipeline that shuffles and then batches is not Poisson sampling either. Opacus replaces the loader with a Poisson one by default: `PrivacyEngine.make_private` takes `poisson_sampling=True` unless you turn it off, and `opacus.data_loader.DPDataLoader.from_data_loader` converts a loader by hand. A Poisson loader gives batches of varying size, which is a quick check that it is in use. When the largest batches do not fit in memory, Opacus's `BatchMemoryManager` splits them into physical batches without changing the accounting (Private stochastic gradient descent). If you cannot use Poisson sampling, report under the matching row of Table 33.1, or say plainly that ε assumes a sampler you did not run.

33.4. The accountant for DP-SGD

Accountants for DP-SGD work in Rényi differential privacy (RDP), which describes a mechanism by a curve $\varepsilon(\alpha)$ over orders $\alpha > 1$ (Rényi DP in the full edition). Curves add over steps, even adaptive ones. So a run of k steps has the curve $k\,\varepsilon(\alpha)$, which is converted in the full edition to (ε, δ) once.

Mironov, Talwar and Zhang (2019) show that one Poisson-subsampled Gaussian step has, at integer orders, the curve

$$\varepsilon(\alpha) = \frac{\ln A_\alpha}{\alpha - 1}, \qquad A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\left(\frac{j^2 - j}{2\sigma^2}\right). \tag{33.3}$$

Here the index j counts the imagined copies of the run that used the record, and accountants compute fractional orders by numerical integration. On the running example the curve is tiny at low orders. At order 2, for example, it is 1.7181×10^{-4}, against 1 without sampling. Table 33.2 shows what the accountant gives over a run, against the per-step route that amplifies each step with Equation 33.1 and then composes the pairs.

Table 33.2.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ under four accounting routes.

Steps k	RDP, Mironov's conversion	RDP, improved conversion	Per step, basic composition	Per step, advanced composition
1,000	2.54	2.10	839.39	1258.64
10,000	7.43	6.71	11296.93	24379.55

The per-step route fails because each step keeps a tiny share of δ, which forces a large per-step ε that sampling barely reduces. So never compose per-step pairs for DP-SGD. The best order also falls as the run grows, to about 4 at ten thousand steps. So make sure the accountant's grid includes small and fractional orders, because a missing order can only raise ε.

In practice you use a library. Opacus's `RDPAccountant` and Google's `dp_accounting.rdp.RdpAccountant` both compute Equation 33.3, and in `dp_accounting` you describe a step as `PoissonSampledDpEvent(sampling_p`

`GaussianDpEvent(noise_multiplier)`. For fixed-size batches it has a `SampledWithoutReplacementDpEvent` for the replace-one relation, while Microsoft's `prv_accountant` describes the same step as a `PoissonSubsampledGaussianMechanism`. The value 6.71 is still an upper bound, and numerical accounting in the full edition computes the tight value.

33.5. Small q and the design of a run

For small q the curve has a simple leading term, which is what makes the trade-offs between batch size, noise and epochs visible:

$$\varepsilon(\alpha) \approx \frac{\alpha q^2 (e^{1/\sigma^2} - 1)}{2} \approx \frac{\alpha q^2}{2\sigma^2} \text{ for large } \sigma. \tag{33.4}$$

The per-step cost is quadratic in q, while a run of E epochs takes E/q steps. So the total RDP is about

$$\frac{E\, q\, \alpha}{2\sigma^2}. \tag{33.5}$$

At a fixed number of epochs, smaller batches are therefore cheaper in privacy.

That gives the recipe for planning a run. Fix the batch size B and the number of epochs E that your accuracy needs. Then the sampling rate is $q = B/n$ and the number of steps is $k = En/B$, where n is the size of the dataset you actually train on, after any filtering. A wrong n gives a wrong q, and so a wrong ε. Let the accountant solve for the noise multiplier that meets the target. Opacus does this with `get_noise_multiplier(target_epsilon=..., target_delta=..., sample_rate=..., epochs=...)`, or in one call with `make_private_with_epsilon`. Then check the result with a second accountant (Choosing an accountant in the full edition).

Use Equation 33.4 for planning only, never inside an accountant, because at high orders it is far too small. The closed-form bound of the original **moments accountant** (Abadi et al. 2016) has the same shape and is outdated.

33.6. Other mechanisms

For RDP, unlike for Equation 33.1, no rule returns every mechanism's subsampled curve exactly. General bounds for other mechanisms exist (Zhu & Wang 2019), but they are looser, so use the exact sum for the Gaussian.

33.7. Code

The accountant below handles Poisson sampling and add/remove neighbours. It computes Equation 33.3 in log space at integer orders only, so its value for the running example is a little above the 6.71 of fractional orders.

```python
from math import exp, lgamma, log


def subsampled_gaussian_rdp(alpha: int, q: float, sigma: float) -> float:
    """RDP at integer order alpha of one Poisson-subsampled Gaussian step (sensitivity 1)."""
    logs = [
        lgamma(alpha + 1) - lgamma(j + 1) - lgamma(alpha - j + 1)
        + (alpha - j) * log(1 - q) + j * log(q) + (j * j - j) / (2 * sigma**2)
        for j in range(alpha + 1)
    ]
    top = max(logs)
    log_a = top + log(sum(exp(t - top) for t in logs))  # log-sum-exp avoids overflow
    return log_a / (alpha - 1)


def dpsgd_eps(q: float, sigma: float, steps: int, delta: float) -> tuple[float, int]:
    """Compose over the steps, convert with the improved conversion at the best integer orde
    best = (float("inf"), 0)
    for alpha in range(2, 64):
        rdp = steps * subsampled_gaussian_rdp(alpha, q, sigma)
        eps = rdp + log((alpha - 1) / alpha) - (log(delta) + log(alpha)) / (alpha - 1)
        best = min(best, (eps, alpha))
    return best


print(f"{subsampled_gaussian_rdp(2, 0.01, 1.0):.4e}")  # 1.7181e-04
print("%.3f %d" % dpsgd_eps(0.01, 1.0, 10_000, 1e-5))  # 6.719 4
```

33.8. Cheat sheet

Table 33.3.: Which accounting to use for each sampler, and how to plan a run.

Situation	What to use	Notes
One step, Poisson, add/remove	$\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$	About $q\varepsilon$ for small ε, $\varepsilon - \ln(1/q)$ for large
DP-SGD, Poisson	RDP or numerical accountant with q, σ	6.71 for the running example
DP-SGD, fixed-size batches, add/remove	Poisson accountant with m/n and $\sigma/2$	About seven times the Poisson ε here
DP-SGD, shuffle and partition	No valid Poisson number	Switch to Poisson sampling

Rules of thumb

- Account for the sampler you run, and state the neighbouring relation next to ε.
- The sampler can move ε by a factor of seven, while the choice of conversion moves it by about 10%.
- Never compose per-step (ε, δ) pairs for DP-SGD. Compose RDP curves or privacy loss distributions, and convert once.
- Keep the small-q formula for planning, never for the reported number.
- Compute q from the dataset you actually train on, after filtering.
- A Poisson add/remove number does not hold for a replace-one claim.

33.9. Further reading

- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018): amplification for each sampler and neighbouring relation.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (IEEE SaTML 2025): fixed-size batches and other accounting pitfalls.

Derivations. Every result here is derived step by step in the full edition's chapter.

34. Numerical accounting with privacy loss distributions

34.1. Why this matters

You trained a model with differential privacy (DP), using the private form of stochastic gradient descent (SGD) called DP-SGD. The run took 10,000 steps, with sampling rate 0.01 and noise multiplier 1, and now you need the smallest ε for which the whole run is $(\varepsilon, 10^{-5})$-DP. Rényi differential privacy (RDP) gives an upper bound that may be loose by an amount you cannot see (Rényi DP in the full edition). The central limit theorem (CLT) of f-DP and Gaussian DP in the full edition gives an approximation that may land on either side of the truth.

A privacy loss distribution (PLD) accountant computes the answer itself. It returns an upper bound and a lower bound, and you know in advance how far apart they can be. On this run it brackets the true ε between 6.1377 and 6.2377, while RDP says 6.7127.

34.2. The privacy loss distribution

Call your dataset D and a neighbouring dataset D'. The mechanism's output then has a distribution P on the first and Q on the second. The **privacy loss** of an output y is the logarithm of how much likelier it is on D,

$$L(y) = \ln \frac{P(y)}{Q(y)}. \tag{34.1}$$

The loss is positive when the output favours D and negative when it favours D'. The output is random, so its loss is too, and the distribution of the loss of an output drawn from P is the **PLD**.

The PLD is all you need to compute the slack δ. The definition of DP lets each output be up to e^{ε} times likelier under D for free, and δ pays for the excess (The definition of differential privacy). Write $(x)_+$ for x when it is positive and 0 otherwise. Then for every ε,

$$\delta(\varepsilon) = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big] = P[L > \varepsilon] - e^{\varepsilon}\, Q[L > \varepsilon]. \tag{34.2}$$

In words, δ averages a weight over the losses. The weight is zero for losses up to ε and rises towards 1 above it, so δ reads only the upper tail of the PLD.

For the Gaussian mechanism the PLD has a closed form, which makes it the check for any accountant. Write μ for the query's sensitivity divided by the noise standard deviation, which is the shift in units of the noise. The PLD is then the normal distribution with mean $\mu^2/2$ and

variance μ^2. Write Φ for the standard normal distribution function. Putting the two normal tails into Equation 34.2 gives

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{34.3}$$

This is the exact δ of the Gaussian mechanism, and the privacy curve of μ-Gaussian differential privacy (GDP). Since δ falls as ε grows, you solve it for ε by bisection, as the snippet in the Code section does.

34.3. Composition is convolution

When steps run independently, the privacy loss of the whole run is the sum of the per-step losses, as Composition showed. The distribution of a sum of independent variables is the **convolution** of their distributions. So the PLD of k steps is the k-fold convolution of the per-step PLDs, and this step loses nothing. So all the error of a PLD accountant comes from computing the convolution on a machine.

Gaussian steps give reference values that every accountant must reproduce. The means and the variances of normal losses add, so k Gaussian steps with shift μ act like one step with shift $\sqrt{k}\,\mu$. Ten steps with $\mu = 1$ are therefore $(17.8566, 10^{-5})$-DP, and an accountant that misses this value should not be trusted on DP-SGD.

DP-SGD steps are not independent, because each gradient depends on the model that earlier outputs produced. The fix is a **dominating pair**, a fixed pair of distributions that is at least as easy to tell apart as the step, whatever happened before. Zhu, Dong and Wang (2022) prove that convolving the PLDs of dominating pairs gives a valid $\delta(\varepsilon)$ for the adaptive run. For the Gaussian and the Poisson-subsampled Gaussian, the pairs that libraries use lose nothing. You need a filter instead if the noise level or the number of steps is chosen from the outputs (Adaptive composition in the full edition).

34.4. Pessimistic and optimistic discretization

A computer puts the losses on a grid, the multiples of a spacing h. Rounding changes δ, but the weight in Equation 34.2 never decreases as the loss grows, so the direction of the rounding fixes the direction of the error. The **pessimistic** discretization rounds every loss up, and the **optimistic** one rounds every loss down. The order survives any number of convolutions, so for k steps and every ε,

$$\delta_{\text{optimistic}}(\varepsilon) \;\le\; \delta(\varepsilon) \;\le\; \delta_{\text{pessimistic}}(\varepsilon). \tag{34.4}$$

The width of this sandwich is known before you run anything. Each step's two roundings are at most one grid step apart, so after k steps the two values of ε at your target δ are at most kh apart. For a smooth PLD the width is almost exactly kh, so the error grows linearly in the number of steps.

The two sides are not equally strong. The rounded-up PLD still belongs to a real pair of distributions, which dominates the original, so the pessimistic value is a guarantee even for adaptive runs. The optimistic value is only a lower bound on what the mechanism costs, so it is

an error bar and a referee, never a number to report. The midpoint of the two sides, or rounding each loss to the nearest grid point, is a good estimate but is not a bound in either direction.

34.5. Composition by FFT

Production accountants convolve with the fast Fourier transform (FFT), padding each convolution so that no mass wraps around the window. If you write your own, pad too, because a wrapped result can err in either direction and is then no bound at all.

Two refinements of the grid are in use. The privacy loss random variable (PRV) accountant of Gopi, Lee and Wutschitz (2021) rounds each loss to the nearest grid point, after shifting the grid to keep the true mean. Its rounding errors then partly cancel, growing like $\sqrt{k}\,h$ instead of kh, and it still proves error bounds on ε. **Connect the dots** (Doroshenko et al. 2022), which Google's `dp_accounting` uses by default, builds the best pessimistic PLD on a given grid, so it allows coarser grids.

34.6. The subsampled Gaussian and DP-SGD

A DP-SGD step with Poisson sampling reduces to a one-dimensional pair (Amplification by subsampling in the full edition). In units of the clipping norm, the output without the record is $\mathcal{N}(0, \sigma^2)$, normal noise centred at 0 whose standard deviation is the noise multiplier σ. When the dataset holds the record, it is drawn with the sampling rate q, and when drawn it shifts the centre by 1. So with the record the output follows the mixture $(1-q)\,\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2)$. A guarantee for adding or removing one record must cover both orders of this pair. The **remove** direction puts the mixture in the role of P, and the **add** direction swaps the two.

The two directions behave very differently. A rare large output betrays that the record was used, so the remove direction's loss has a long right tail, while the add direction's loss stays below $-\ln(1-q)$. So one step at $\delta = 10^{-5}$ costs 0.1995 in the remove direction and only 0.0091 in the add direction. Neither direction is the larger in general, so compose both and report the larger. This pair also assumes Poisson sampling, and a loader that forms batches another way needs a different accountant (Amplification by subsampling in the full edition).

Table 34.1.: DP-SGD with $q = 0.01$ and $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps. The PLD columns use the grid $h = 10^{-5}$, each the larger of the two directions. RDP uses the improved conversion.

k	PLD upper	PLD lower	RDP	GDP CLT
1	0.1995	0.1994	0.9555	0.0368
1000	1.8332	1.8232	2.1014	1.6177
10000	6.2377	6.1377	6.7127	6.0071

Table 34.1 shows what each accountant costs. RDP is a valid bound but a loose one, 6.7127 against the PLD's 6.2377 after the full run. The CLT is worse, because it falls below the PLD lower bound at every k. So quoting the CLT's 1.6177 after 1000 steps would claim a guarantee

that the run does not have. The reason is that the subsampled PLD has a heavier right tail than a Gaussian, which is exactly the part that δ reads.

A PLD accountant is only as good as its grid. On the coarse grid $h = 10^{-3}$, the 1000-step bracket is a whole unit wide, and its upper end of 2.3279 is worse than RDP. A tenfold finer grid narrows the bracket tenfold but costs about ten times the memory.

34.7. Running it with a library

A maintained library should land inside Table 34.1's bracket. In Google's `dp_accounting`, the function `from_gaussian_mechanism` of the module `dp_accounting.pld.privacy_loss_distribution` builds the subsampled Gaussian. Its grid spacing is `value_discretization_interval`, whose default is 10^{-4}. Its argument `pessimistic_estimate` chooses the side of the rounding. Its default neighbouring relation is adding or removing one record, so it covers both directions and reports the larger. Then `self_compose` composes the steps and `get_epsilon_for_delta` reads off ε:

```
from dp_accounting.pld import privacy_loss_distribution as pld_lib

step = pld_lib.from_gaussian_mechanism(standard_deviation=1.0, sampling_prob=0.01,
    value_discretization_interval=1e-5, pessimistic_estimate=True)
upper = step.self_compose(10_000).get_epsilon_for_delta(1e-5)
```

Run it again with `pessimistic_estimate=False` for the lower bound.

Its event interface, `dp_accounting.pld.PLDAccountant`, composes a `PoissonSampledDpEvent` that wraps a `GaussianDpEvent`, and `get_epsilon(target_delta)` returns the pessimistic value. Microsoft's `prv_accountant` package takes the error you accept instead of a grid spacing. Its `PRVAccountant` takes a list of mechanisms, such as `PoissonSubsampledGaussianMechanism`, with `max_self_compositions`, `eps_error` and `delta_error`. Then `compute_epsilon(delta, num_self_compositions)` returns a lower bound, an estimate and an upper bound, and you report the upper one. The accountant is set up for at most `max_self_compositions` steps of each mechanism, so make that at least the number of steps you will run. Opacus uses the same accountant by default, through `PrivacyEngine(accountant="prv")`. If two libraries' brackets fail to overlap, one of them has a bug (Choosing an accountant in the full edition).

A numerical accountant is slower than RDP, but one run of it is cheap next to training. The cost adds up when you calibrate the noise, because a search evaluates the accountant at many noise levels. So search with RDP, then confirm the final noise level with the PLD.

34.8. Code

The snippet solves Equation 34.3 for ε, where k Gaussian steps with noise multiplier σ and sensitivity 1 have $\mu = \sqrt{k}/\sigma$.

```
from math import erfc, exp, sqrt


def normal_cdf(x: float) -> float:
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_delta(eps: float, mu: float) -> float:
    """Exact delta(eps) of the Gaussian PLD N(mu^2/2, mu^2)."""
    return normal_cdf(mu / 2 - eps / mu) - exp(eps) * normal_cdf(-mu / 2 - eps / mu)


def gaussian_epsilon(delta: float, mu: float) -> float:
    lo, hi = 0.0, 1000.0  # delta(eps) falls as eps grows: bisect for delta(eps) = delta
    for _ in range(200):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if gaussian_delta(mid, mu) > delta else (lo, mid)
    return hi


print(f"{gaussian_epsilon(1e-5, 1.0):.4f}")  # 4.3772
print(f"{gaussian_epsilon(1e-5, sqrt(10)):.4f}")  # 17.8566
print(f"{gaussian_epsilon(1e-5, 10.0):.4f}")  # 91.8173
```

34.9. Cheat sheet

Table 34.2.: Numerical accounting at a glance.

Question	Answer	Notes
Gaussian reference	PLD $\mathcal{N}(\mu^2/2, \mu^2)$; k steps: $\mu \to \sqrt{k}\,\mu$	4.3772, 17.8566, 91.8173 at $\mu = 1, \sqrt{10}, 10$
Which side to report	Pessimistic (rounded up)	Optimistic is the error bar; the midpoint is no bound
Grid spacing h	Width in ε at most kh	Choose h for your largest k, then check the width
Subsampled Gaussian	Compose add and remove, report the larger	Needs Poisson sampling

Rules of thumb

- Report the pessimistic ε, or a PRV accountant's upper bound, never the estimate or the optimistic value. Give the optimistic value as the error bar.
- Check the width of the bracket rather than trusting the default grid, because the width grows with the number of steps.

- Use RDP for a quick bound and never quote the CLT as a guarantee, because it can fall below the PLD lower bound.

34.10. Further reading

- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, with explicit error bounds.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022): accounting with the characteristic function of the PLD, dominating pairs and the two directions of Poisson subsampling.

Derivations. Every result here is derived step by step in the full edition's chapter.

35. Amplification by shuffling and by iteration

35.1. Why this matters

Uncertainty is worth privacy, as subsampling in the full edition showed, and two other kinds of it matter in practice. The first is about which report is yours. In the shuffle model (Local, shuffle and distributed trust models), each of n users randomizes their own data, and a **shuffler** strips identities and mixes the order. Each report satisfies local differential privacy (DP) at a level ε_0 that is usually large. But your report hides among the others, so the bag of reports satisfies central DP with an ε that falls like $1/\sqrt{n}$.

The second is uncertainty about what happened inside training. If only the final model is released, later noise also covers a record used early, and on a convex loss the privacy loss can even stop growing. Of the two, shuffling is a deployable tool with a computable guarantee, used for example in Prochlo (Bittau et al. 2017). Hidden-state amplification, by contrast, rarely applies to deep learning. All results use replace-one neighbours, two datasets of the same size that differ in the value of one record.

35.2. Shuffled binary randomized response, exactly

For the simplest local randomizer, the shuffled guarantee can be computed exactly. Each user holds a bit and sends it through binary randomized response (The definition of differential privacy). The user reports the true bit with probability $\theta = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$ and the flipped bit otherwise. After shuffling, the analyst learns only how many reports are 1, and your report moves that count by at most one.

So the guarantee is the privacy of a pair of count distributions. Let your bit be 0 in D and 1 in D', and suppose the other users all hold 0. Each of them reports 1 with probability $1-\theta$, so the number of their 1-reports is binomial, and we write $b(y)$ for its chance of being y. The count is then distributed as

$$P(y) = \theta\, b(y) + (1-\theta)\, b(y-1), \qquad Q(y) = (1-\theta)\, b(y) + \theta\, b(y-1), \tag{35.1}$$

when you hold 0 and when you hold 1. Its exact privacy curve is a sum over the possible counts, the same sum as in Numerical accounting in the full edition. You compute it in both directions and keep the larger, which the snippet in Section 35.7 does in a few seconds. Other values for the other users change it by less than 10^{-4} in the full edition's checks.

For intuition, the count is close to normal, and your report shifts its mean against a spread that grows like $\sqrt{n}$. So the shuffled count approximately satisfies Gaussian differential privacy (GDP)

(f-DP and Gaussian DP in the full edition), with a shift in standard deviations of

$$\mu \approx \frac{2\sinh(\varepsilon_0/2)}{\sqrt{n}}. \tag{35.2}$$

Don't deploy with this approximation, because it fails when the others send few 1-reports. For example, with 1000 users at $\varepsilon_0 = 4$, it gives 0.9667 where the exact value is 1.6910.

Shuffling never improves pure DP, the guarantee whose slack δ is zero. An all-zero count is still e^{ε_0} times likelier when you hold 0, however many users there are, and shuffling only makes it rare. So any shuffled guarantee below ε_0 needs $\delta > 0$.

35.3. The clone bound for any local randomizer

Real deployments send histograms, categories or noisy vectors, so you need a bound for every ε_0-DP **local randomizer**, the randomized map from one user's data to their report. The bound to use is that of Feldman, McMillan & Talwar (2021). Its idea is that each other user is, with probability $e^{-\varepsilon_0}$, a **clone** whose report looks exactly like one of your two possible reports.

That reduction gives a small pair of count distributions, whose exact $\delta(\varepsilon)$ is the **numeric clone bound**. It holds for every ε_0-DP randomizer, even one chosen adaptively, as when each user randomizes their own gradient in shuffled stochastic gradient descent (SGD). The same paper also gives a closed form, which needs enough clones to hide among,

$$\varepsilon_0 \le \ln \frac{n}{16\ln(2/\delta)}. \tag{35.3}$$

When that holds, the shuffled output is (ε, δ)-DP with

$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right). \tag{35.4}$$

For large n its leading term gives the shape to remember, because the central ε falls like $\sqrt{\ln(1/\delta)/n}$ with a constant that grows like $e^{\varepsilon_0/2}$. Where the condition fails, though, the closed form says nothing.

Table 35.1.: Central ε at $\delta = 10^{-6}$ after shuffling n reports that are each ε_0-DP. "n/a" marks where the condition of Equation 35.4 fails.

n	ε_0	Exact, randomized response	Clones, numeric	Clones, closed form
1,000	4	1.6910	3.9900	n/a
10,000	2	0.0870	0.1550	0.5009
100,000	2	0.0247	0.0450	0.1862
100,000	4	0.0847	0.1698	0.5346

Table 35.1 shows what generality costs. Where many clones are expected, the numeric clone bound is 1.44 to 2.00 times the exact value for randomized response. The closed form is 3.10 to 4.75 times the numeric bound where it applies. So use the closed form to reason about scaling, and compute the numeric bound to deploy.

35.4. Choosing ε_0 and deploying a shuffler

The practical question runs backwards, because you want the largest ε_0 that meets a target central guarantee, which means the least noise per report. So compute the bound for a few candidate values of ε_0 and keep the largest that meets the target. The choice of bound can decide feasibility, and a larger ε_0 buys accuracy. Take 100,000 users and a target of 0.2 at $\delta = 10^{-6}$. Among the values in Table 35.1, the closed form allows ε_0 up to 2, while the numeric clone bound allows 4.

Before you report a shuffled ε, check that the deployment meets the assumptions of the bound.

- **The order is uniformly random and hidden.** With a fixed or leaked order, or metadata such as arrival time or network address, the analyst knows which report is yours, and the guarantee falls back to ε_0.
- **The shuffler is trusted not to collude with the analyst.** If they collude, again only ε_0 remains.
- **The other users are honest.** Users who collude with the analyst reveal their own reports, so count only honest users in n.
- **The computation uses the number of reports actually shuffled.** The bound gets worse as n falls, so use the smallest batch you will release, not the number of users enrolled.
- **Each user sends one report per batch.** Otherwise the reports compose like any other releases.

35.5. Amplification by iteration

Suppose projected noisy SGD makes one pass over n records and releases only the final model. Then the noise of every later step protects a record used early (Feldman et al. 2018). The guarantee needs every step to be a **contraction**, a map that never pushes two models further apart. Write β for the smoothness of the loss, the most its gradient changes per unit of distance. A gradient step on a convex loss is then a contraction when the step size is at most $2/\beta$. A projection onto a convex set is one too. With Lipschitz losses as well, the record used at the t-th step gets one Gaussian step's Rényi differential privacy (RDP) curve. That curve is divided by the $n - t + 1$ noisy steps from t to the end.

So the guarantee depends on the position, and a report must state the worst one. For a pass over 10,000 records with noise multiplier 1, the last record gets $\varepsilon = 5.2985$ at $\delta = 10^{-5}$, converted as in RDP conversions in the full edition. That is a single Gaussian step, with no amplification at all, while the record in the middle gets 0.0707.

35.6. Hidden state: privacy that stops growing

Full-batch noisy gradient descent (GD) uses every record at every step, so composition charges in proportion to the number of steps. On a bounded convex set, though, the runs on D and D' can never drift further apart than the set's **diameter** d_K, the largest distance between two of its points. Altschuler & Talwar (2022) prove that the privacy loss of the last iterate then stops

growing. Write s for the per-step shift between the runs and σ for the per-step noise, both in parameter units. When d_K/s is an integer, the RDP of the last iterate stays below

$$\varepsilon(\alpha) \le \frac{2\alpha d_K s}{\sigma^2} \tag{35.5}$$

however long you train.

For example, on the unit interval with $s = 0.01$ and $\sigma = 0.1$, the bound stops at 4.0000 at order 2, while composition charges 20.0000 for 2000 steps. **Strong convexity** gives a plateau without a bounded set, because each step then shrinks the gap between the runs. The plateau arrives after about as many steps as the loss's condition number (Chourasia, Ye & Shokri 2021).

These results need two conditions, and deep learning breaks both. No mainstream library, Opacus or `dp_accounting` included, offers last-iterate accounting. The first is that **only the last iterate is released**. Checkpoints, logged models or picking the best checkpoint put you back to composition. The second is that every step is a contraction, which requires a convex loss. Annamalai (2024) constructs a non-convex loss for which the last iterate of differentially private SGD (DP-SGD) leaks as much as all iterates together. So never report a hidden-state ε for non-convex training, and account for it with the ordinary DP-SGD composition instead. Use it for convex models, such as private logistic regression, that release only their final weights.

35.7. Code

This snippet computes the exact central ε of shuffled binary randomized response from Equation 35.1, so you can use it to choose ε_0. With SciPy, `scipy.stats.binom.pmf` gives the binomial probabilities directly.

```
from math import exp, lgamma, log


def log_binom_pmf(k: int, n: int, p: float) -> float:
    return lgamma(n + 1) - lgamma(k + 1) - lgamma(n - k + 1) + k * log(p) + (n - k) * log(1


def shuffled_rr_pair(n: int, eps0: float) -> tuple[list[float], list[float]]:
    theta = exp(eps0) / (1 + exp(eps0))
    b = [exp(log_binom_pmf(y, n - 1, 1 - theta)) for y in range(n)] + [0.0]
    P = [theta * b[y] + (1 - theta) * (b[y - 1] if y else 0.0) for y in range(n + 1)]
    Q = [(1 - theta) * b[y] + theta * (b[y - 1] if y else 0.0) for y in range(n + 1)]
    return P, Q


def hockey_stick(P: list[float], Q: list[float], eps: float) -> float:
    return sum(max(p - exp(eps) * q, 0.0) for p, q in zip(P, Q))


def shuffled_rr_epsilon(n: int, eps0: float, delta: float) -> float:
```

```
    P, Q = shuffled_rr_pair(n, eps0)
    lo, hi = 0.0, eps0  # shuffling never does worse than the local eps0
    for _ in range(50):
        mid = (lo + hi) / 2
        worst = max(hockey_stick(P, Q, mid), hockey_stick(Q, P, mid))
        lo, hi = (mid, hi) if worst > delta else (lo, mid)
    return hi


print(f"{shuffled_rr_epsilon(10_000, 2.0, 1e-6):.4f}")  # 0.0870
print(f"{shuffled_rr_epsilon(100_000, 4.0, 1e-6):.4f}")  # 0.0847
print(f"{shuffled_rr_epsilon(1_000, 4.0, 1e-6):.4f}")  # 1.6910
```

35.8. Cheat sheet

Table 35.2.: Which accountant for which kind of amplification. All use replace-one neighbours.

Situation	Accountant	Conditions and pitfalls
Binary randomized response behind a shuffler	Exact count pair, Equation 35.1	The normal approximation fails with few 1-reports
Any ε_0-DP randomizer behind a shuffler	Numeric clone bound	1.44 to 2.36 times the exact value for randomized response
One-pass noisy SGD, last iterate	$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2(n-t+1)}$	Convex, smooth, Lipschitz; report the last record's value
Full-batch GD on a bounded convex set	Plateau $\frac{2\alpha d_K s}{\sigma^2}$	Only the last iterate released
Non-convex training, or checkpoints released	Ordinary composition	No hidden-state amplification

Rules of thumb

- Choose ε_0 with the numeric bound at the smallest batch size you will release.
- Don't deploy with the closed form or the normal approximation when users are few or ε_0 is large, because both fail there.
- A leaked order, a colluding shuffler or identifying metadata turns the guarantee back into ε_0.
- Never claim hidden-state privacy for deep learning, and for convex models release only the final weights.

35.9. Further reading

- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021): the clone bound and how to compute it. The full edition's script, shuffling_iteration.py, computes it too.
- Annamalai, "It's Our Loss: No Privacy Amplification for Hidden State DP-SGD With Non-Convex Loss" (AISec 2024): why hidden-state results fail for deep learning.

Derivations. Every result here is derived step by step in the full edition's chapter.

36. Adaptive composition: filters, odometers and individual accounting

36.1. Why this matters

Every composition theorem so far assumes that each step's privacy parameter, and the number of steps, are fixed before the run (Advanced and optimal composition). Real training breaks this whenever you stop on the validation loss or lower the noise as training stalls. Stochastic gradient descent (SGD) with differential privacy (DP), called DP-SGD, adds a third case, since a record with a small gradient costs less than the worst case. Plugging the values that occurred into a fixed-parameter theorem gives a report that can be wrong several times as often as it claims.

Fixed-step DP-SGD with a schedule chosen in advance needs none of this. Otherwise, which tool you need depends on what you fixed in advance. If only the stopping time adapts within a planned maximum, report ε for the maximum. If you fixed a budget and want to spend it adaptively, a **privacy filter** decides before each step whether it still fits. If you fixed nothing, a **privacy odometer** reports what you have spent so far, at a small extra price.

36.2. Why plugging in the realized parameters fails

Advanced composition bounds the total privacy loss of a run (Numerical accounting in the full edition), except with a failure probability δ, when the number of steps is fixed in advance. Applying it **ex post**, at a stopping time chosen from the outputs, is a different claim that Rogers et al. (2016) showed to be invalid.

A simple rule breaks it. Run randomized response with $\varepsilon = 0.1$ on one person's bit, step after step, so that the total loss is a random walk. The plug-in bound is correct at every fixed step, but an analyst may stop the first time the walk crosses it and report the bound at that step. At $\delta = 0.01$, the full edition finds this report wrong with probability 0.0318 within 10,000 steps, over three times its claim. Given enough steps, the walk crosses the bound with probability 1.

Two kinds of report escape the problem. For pure DP, basic composition, the plain sum of the per-step parameters, bounds the loss of every output, so it is valid however you stop. And a filter escapes, because it fixes its budget in advance and makes no running claim that a stopping rule could catch out.

36.3. Early stopping within a planned maximum

The commonest adaptive run needs no new theory. You fix the noise, the sampling rate and a maximum number of steps before training, and you stop early when the validation loss stops improving. **Then report the ε of the planned maximum, not of the steps you ran.** The run that stopped early releases a prefix of the full planned run. Anything computed from that prefix, including the decision to stop, is post-processing of the full run, so the full run's guarantee covers it.

For example, plan 100 full-batch Gaussian steps whose noise multiplier is 10. That run is $(5.299, 10^{-5})$-DP, so if you stop at step 60, you still report 5.299.

The rule has three conditions. First, the stopping decision must look only at released outputs, or at data you account for separately, because a decision that reads the private data leaks through the stopping time. Second, the maximum, the noise and the sampling rate must really be fixed. If you might extend training past the plan, you need a filter or an odometer. Third, report with the same accountant and δ you calibrated with. Choosing hyperparameters from several runs is a separate cost, which Hyperparameter tuning accounts for.

Library accountants are not odometers. In Opacus, `PrivacyEngine.make_private_with_epsilon` calibrates the noise for a target ε and δ over a planned number of `epochs`. Its `get_epsilon(delta)`, like the one in Google's `dp_accounting`, composes the steps taken so far and converts them once. That is valid only at a time fixed in advance, so call it at the planned end, or report the target you calibrated for.

36.4. The Rényi filter

When the noise itself changes during training, use a Rényi filter. Rényi differential privacy (RDP) measures each step by a curve over orders α (Rényi DP in the full edition). Fix one order and an RDP budget B before the run. Each step then has a **spend**, its RDP at that order, which may depend on earlier outputs but must be known before the step runs, so it is called **predictable**. Write $\varepsilon_t(\alpha)$ for the spend of the t-th step. The filter runs that step only if the spends so far, its own included, fit in the budget,

$$\sum_{j \le t} \varepsilon_j(\alpha) \le B. \tag{36.1}$$

Then the stopped run is (α, B)-RDP, as if the spends had been fixed in advance (Feldman and Zrnic 2021).

Each condition matters, because a spend that depended on its own step's output could be small exactly where the output reveals the most. The budget must also hold on every path, not on average. And the order must be fixed before the run, although you may track several and stop when any one runs out.

To aim at a DP target, write ε_g for the global privacy level you want. Then set the budget from the conversion of RDP conversions in the full edition, run backwards,

$$B = \varepsilon_g - \frac{\ln(1/\delta)}{\alpha - 1}. \tag{36.2}$$

For example, a target of $(5.299, 10^{-5})$ at order 5.8 gives $B = 2.900$. A full-batch Gaussian step spends $\alpha/(2\sigma_t^2)$, where σ_t is its noise multiplier, so with σ_t at 10 the budget lasts 100 steps. Now suppose training plateaus at step 60 and you halve the noise. Each later step then costs four times as much, so the filter allows 10 more steps. Either way the run is $(5.8, 2.900)$-RDP, whatever rule decided the switch, provided that rule looks only at released outputs.

36.5. Filters for pure differential privacy

Interactive query systems give each query a pure-DP parameter ε_t, chosen from earlier answers. The **basic filter** of Rogers et al. (2016) continues while they sum to at most the global ε_g. It is exact, but its budget grows like the number of steps.

For square-root growth, use the **Whitehouse filter** of Whitehouse et al. (2023). It continues while

$$\sqrt{2\ln(1/\delta)\sum_t \varepsilon_t^2} + \tfrac{1}{2}\sum_t \varepsilon_t^2 \le \varepsilon_g, \tag{36.3}$$

summing over the steps so far and the one about to run, and its stopped run is (ε_g, δ)-DP. It reads like advanced composition because it is the Rényi filter applied to zero-concentrated differential privacy (zCDP) (Concentrated DP in the full edition), so adaptivity costs essentially nothing.

For example, a budget of 1 at $\delta = 10^{-6}$ lets the Whitehouse filter run 349 steps of 0.01. That is slightly more than advanced composition allows with the number of steps fixed in advance. So use the Whitehouse form, checked before each step with that step's parameter included.

For an analytics service, use a library that enforces the budget for you. OpenDP's `make_privacy_filter` wraps an odometer built with `make_fully_adaptive_composition` and rejects any query that would push the total past its bound. Tumult Analytics tracks a `Session`'s remaining budget, such as a `RhoZCDPBudget`, and exposes it as `Session.remaining_privacy_budget`.

36.6. Odometers

Open-ended analyses and training to a target accuracy have no budget to stop at. They need a running answer to "how much have I spent?" that holds at every step at once. The **Ville odometer** gives one. Fix a grid Λ of orders before the run, with $|\Lambda|$ orders in it, and write $S_t(\alpha)$ for the RDP spend so far at order α. With probability at least $1 - \delta$, simultaneously for every step t, the loss so far is at most

$$U_t = \min_{\alpha \in \Lambda} \Big[S_t(\alpha) + \frac{\ln(|\Lambda|/\delta)}{\alpha - 1} \Big]. \tag{36.4}$$

This is the usual RDP conversion at the best order of the grid, with δ replaced by $\delta/|\Lambda|$, and that replacement is the whole price. So keep the grid small.

Over the part's grid of orders, the price for full-batch Gaussian steps is 15 to 19%. For example, the 100-step run above gets 6.253 instead of 5.299. Lécuyer (2021) builds a slightly looser odometer from doubling Rényi filters.

An odometer is a tail bound, not a DP guarantee. To turn it into an (ε_g, δ)-DP guarantee, stop by a rule that keeps the running bound at most ε_g. If you have a budget anyway, a Rényi filter at one order fixed in advance is cheaper, because it needs no union over orders.

36.7. Individual privacy accounting in DP-SGD

A DP-SGD step clips each record's gradient to a norm of at most the clipping norm C, and adds Gaussian noise with standard deviation σC. The usual analysis charges every record as if its clipped gradient always had norm C, but a record that the model fits moves the noisy sum less. Write $\bar{g}_{i,t}$ for the clipped gradient of record i at step t. With full-batch steps, that record's spend is

$$\varepsilon_{i,t}(\alpha) = \frac{\alpha \|\bar{g}_{i,t}\|^2}{2\sigma^2 C^2} \le \frac{\alpha}{2\sigma^2}. \tag{36.5}$$

This spend is predictable, because the gradient depends only on the record and the current model. So the **individual filter** of Feldman and Zrnic (2021) runs one Rényi filter per record and drops a record once its own spend would pass B. Every record then has the guarantee of the worst-case run, while records with small gradients keep training long after.

In the full edition's simulation with the order and budget above, the median typical record has an individual ε of 3.17 after 100 steps, against 5.299 for the worst case.

Read the per-record numbers with care. A per-record ε from the realized spend is an estimate, not a guarantee, because it is a fixed-time bound at a random quantity. It is also sensitive, since a small spend says that the model fits that record well, so show it only to the record's owner. There is a fairness cost too, because the records the model serves poorly are dropped first, and these often come from under-represented groups (Yu et al. 2023; Disparate impact). Under Poisson subsampling, it needs every record's gradient norm at every step, sampled or not (Koskela et al. 2023).

36.8. Code

A filter or an odometer is a sum of spends and a test. This snippet codes the Rényi filter for a noise schedule, and prints the numbers of the example above.

```
from math import log


def renyi_filter(sigmas: list[float], alpha: float, budget: float) -> int:
    """Steps of a full-batch Gaussian noise schedule that a Renyi filter lets through."""
    spent, steps = 0.0, 0
    for sigma in sigmas:
        cost = alpha / (2 * sigma**2)
        if spent + cost > budget + 1e-12:  # tolerance for floating-point sums
            break
        spent, steps = spent + cost, steps + 1
    return steps
```

```
alpha, budget = 5.8, 2.9
print(renyi_filter([10.0] * 1000, alpha, budget))  # 100
print(renyi_filter([10.0] * 60 + [5.0] * 1000, alpha, budget) - 60)  # 10
print(f"{budget + log(1 / 1e-5) / (alpha - 1):.3f}")  # 5.299
```

36.9. Cheat sheet

Table 36.1.: Which adaptive tool for which situation. Spends may depend on earlier outputs but must be known before their step runs.

Situation	Use	Report
Early stopping within a planned maximum	Nothing new	ε of the planned maximum
Noise chosen during training, budget fixed	Rényi filter at a fixed order	(α, B)-RDP, with $B = \varepsilon_g - \ln(1/\delta)/(\alpha - 1)$
Pure-DP queries chosen adaptively	Whitehouse filter, Equation 36.3	(ε_g, δ)-DP
No budget, stop whenever	Ville odometer, Equation 36.4	The running bound, valid at every step
Per-record spend in DP-SGD	Individual filter, Equation 36.5	The run's (α, B); per-record values as estimates

Rules of thumb

- If you early-stop within a planned maximum number of steps, report ε for the maximum.
- Never report a fixed-time bound at a time chosen from the outputs, and that includes a library's `get_epsilon` called when you decide to stop.
- An odometer must fix its grid of orders in advance. Picking the best order after the run without paying $\ln|\Lambda|$ is the plug-in mistake again.
- Compute per-record spends from the clipped gradient norms, keep them out of logs, and remember that the records dropped first are the ones the model already serves worst.

36.10. Further reading

- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023): the filter that matches advanced composition, and odometers for pure-DP steps.
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021): the Rényi filter and per-record accounting.

Derivations. Every result here is derived step by step in the full edition's chapter.

37. Choosing an accountant

37.1. Why this matters

You have trained a model with differential privacy (DP), using private stochastic gradient descent (SGD), called DP-SGD (Amplification by subsampling in the full edition). Each record joined each batch with probability q, the sampling rate, here 0.01, and the noise multiplier σ was 1. After 10,000 steps, a number this chapter writes k, you must state the ε at which the model is $(\varepsilon, 10^{-5})$-DP, in a paper or a model card.

A procedure that bounds or estimates the true ε is an **accountant**, and on this run the accountants disagree by a factor of over a thousand. That is because they make different kinds of statement. An **upper bound** comes with a proof, so the run really is as private as it reports, while an **approximation** has none and may land below the truth. A certified **lower bound** is no guarantee, but it exposes any reported ε that is too small. So the rule of this chapter is short. **Report the tightest upper bound whose assumptions hold for the code you ran, and report enough that someone else can recompute it.**

37.2. Every accountant on one run

Accountants differ in how much of each step's privacy they keep, filling in the rest with a worst case. From least to most, they are per-step composition and Rényi differential privacy (RDP) in the full edition, the central limit theorem (CLT) of Gaussian differential privacy (GDP) in the full edition, and the privacy loss distribution (PLD) in the full edition.

Table 37.1.: DP-SGD with $q = 0.01$ and $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps under each accountant. The PLD columns use the grid $h = 10^{-5}$.

k	Basic	Advanced	RDP, Mironov	RDP, improved	GDP CLT	PLD upper	PLD lower
Kind	Upper	Upper	Upper	Upper	Approxi-mation	Upper	Lower
1	0.20	1.13	1.3175	0.9555	0.0368	0.1995	0.1994
1,000	839.39	1258.64	2.5380	2.1014	1.6177	1.8332	1.8232
10,000	11296.93	24379.55	7.4287	6.7127	6.0071	6.2377	6.1377

The proofs put the RDP bounds in a fixed order. With the optimal conversion, the smallest valid value for the curve, at the end,

$$\varepsilon_{\text{RDP, Mironov}} \;\geq\; \varepsilon_{\text{RDP, improved}} \;\geq\; \varepsilon_{\text{RDP, optimal}} \;\geq\; \varepsilon_{\text{true}}. \tag{37.1}$$

Per-step composition, basic or advanced, is useless beyond a handful of steps, off by a factor of over a thousand after the full run. RDP is safe, and with the improved conversion it is 7.6% above the PLD at 10,000 steps, but far more on short runs. The CLT falls below the certified lower bound at every k, so a paper that quotes it, such as 1.62 after 1,000 steps, claims a guarantee the run does not have.

37.3. The inverse problem: noise for a target ε

Practitioners usually fix the budget, the steps and the batch size, and then ask how much noise they need. For an accountant A, write $\varepsilon_A(\sigma)$ for the value it reports at noise multiplier σ. The noise it requires for a target ε^* is then

$$\sigma_A(\varepsilon^*) = \min\{\sigma > 0 : \varepsilon_A(\sigma) \le \varepsilon^*\}. \tag{37.2}$$

More noise never raises a correct accountant's ε, so a bisection finds it. Return the end of the bracket where the target was checked, because that end is safe.

A loose accountant turns its slack directly into extra noise. For a target of 3 over 10,000 steps, the pessimistic PLD needs $\sigma = 1.584$, while RDP with the improved conversion needs 1.662. Across targets, the improved conversion costs up to 4.9% more noise than the PLD, and Mironov's conversion far more. The CLT asks for less noise than the PLD, which at a target of 8 is provably too little. So calibrate σ with the accountant you will report.

37.4. Where each accountant is tight

An accountant is tight only when its summary describes the mechanism exactly, so work through these questions in order.

1. **Does the code match the accountant's assumptions?** Check this first (Section 37.5), because a mismatch costs more than any choice of accountant.
2. **Are any parameters chosen during training?** If only the stopping time adapts within a planned maximum, account for the maximum. If the noise or budget adapts, use a Rényi filter (Adaptive composition in the full edition).
3. **Is it the Gaussian mechanism without subsampling?** Then the composed pair is exactly Gaussian, and GDP gives the exact ε. RDP with the improved conversion is 8.0% above it on a hundred releases at $\sigma = 10$, and zCDP gives the same as RDP here.
4. **Is it DP-SGD, the Poisson-subsampled Gaussian?** Then use the pessimistic PLD or a privacy loss random variable (PRV) accountant, with the optimistic value as the error bar. Avoid zero-concentrated differential privacy (zCDP), because its single slope cannot express amplification by subsampling.
5. **Are the steps pure DP, such as randomized response or the Laplace mechanism?** The PLD is exact when the loss takes few values. A hundred randomized-response steps at 0.1 give 4.3068, against 4.6154 from RDP.
6. **Do local reports pass through a shuffler, or is only the last model released?** See Amplification by shuffling and by iteration in the full edition.

37.5. Matching the accountant to the code

Every number in Table 37.1 assumes a model of the training run that the accountant cannot check against your code. Many papers reported the Poisson-sampling number while training with shuffled fixed-size batches, a statement about a different algorithm (Ponomareva et al. 2023). So check each item of Table 37.2 before you report.

Table 37.2.: A checklist for matching the accountant to the training code.

Item	The accountant assumes	Check in the code	If it does not match
Sampling scheme	Poisson sampling at rate q	Is each record included independently, so batch sizes vary?	Fixed-size random batches: account with $\sigma/2$. Shuffle and partition: report the unamplified guarantee
Neighbouring relation	Add or remove one record	Which relation does the claim state?	Replace-one costs more per step
Privacy unit	One example	Can one person contribute several examples?	Clip and sample per user, or pay for group privacy
Tuning and adaptivity	Settings fixed in advance	Tuning on private data, early stopping, noise schedules?	Account for the tuning runs; planned maximum, or a filter
Steps counted	Every step on private data	Warm-up runs, restarts, evaluation passes that release results?	Count them all
Noise scale	Noise standard deviation σC	Is σ measured against the clipping norm actually used?	Recompute σ from the noise added

The sampler is the most expensive item. Lebeda et al. (2024) show that such a run is covered by the Poisson accountant at noise multiplier $\sigma/2$. After the full run, the PLD upper bound rises from 6.24 to 43.42. That factor of 6.96 dwarfs any choice of accountant. Shuffle and partition is worse, because no amplification theorem covers it. What remains certain is one Gaussian step per epoch for zero-out neighbours, which gives $\varepsilon = 91.8173$ over a hundred epochs at $\sigma = 1$.

In a code review, a data loader with `shuffle=True` and a fixed `batch_size` is shuffle and partition, whatever `sample_rate` the accountant was given. Opacus samples by the Poisson scheme by default (`poisson_sampling=True`), so keep that default.

37.6. What to report

An ε without its assumptions cannot be checked or compared, so Ponomareva et al. (2023) and the US National Institute of Standards and Technology (NIST) ask for the items below. Choose the slack δ with care, because with n records, publishing one random record in full meets a δ of about $1/n$. So Ponomareva et al. ask for a δ much smaller than $1/n$, and NIST gives $1/n^2$ as a common recommendation.

Reading ε off the composed PLD at another δ is cheap, so report several values, not a single point. For the running example with 50,000 records, the pessimistic PLD gives 9.0502 at $1/n^2$, against 6.2377 at 10^{-5}. Round upper bounds up, never to the nearest value, so 9.0502 becomes 9.06. A filled-in statement for the running example reads as follows.

- **Setting.** DP-SGD in the central model. It covers every checkpoint and the final model.
- **Unit and neighbours.** One training example per person, with neighbours that add or remove one example.
- **Training.** Poisson sampling at rate 0.01 over 50,000 examples for 100 epochs, with clipping norm 1 and noise multiplier 1.
- **Accountant.** The pessimistic PLD on the grid 10^{-5}, in both directions, with the library and its version.
- **Guarantee.** The model is $(6.24, 10^{-5})$-DP and $(9.06, 4.0 \times 10^{-10})$-DP. The optimistic values, 6.1377 and 8.9378, bound the accountant's own error.
- **Not covered.** Say what the guarantee leaves out, such as hyperparameters tuned on a public proxy dataset, pretraining on public data, or preprocessing statistics computed from the private data.

37.7. Checking your accountant

An accountant is code, and code has bugs, so run checks whose answers you know. First, set $q = 1$, which makes each step a plain Gaussian mechanism. Ten steps at $\sigma = 1$ are then exactly $(17.8566, 10^{-5})$-DP, and every upper bound must lie above it, as RDP's 19.0536 does. Second, the PLD bracket on plain Gaussian steps must contain the exact value, which the calculator in Numerical accounting in the full edition gives. Third, the order of Equation 37.1 must hold, with no upper bound below the optimistic PLD. Fourth, ε must never decrease as the number of steps grows, and never increase as the noise grows.

Then compare two maintained libraries, and pin their versions in your report. The full edition's numbers agree with Opacus 1.6.0, `dp_accounting` 0.6.0 and `prv_accountant` 0.2.0. Disagreements usually have one of three causes. If two RDP accountants differ at each order by a fixed amount, 0.3612 at order 10, one of them uses Mironov's conversion. Fractional orders are a weak spot of `dp_accounting` 0.6.0, where it returns a loose bound, so compare at integer orders. And if two PLD or PRV brackets fail to overlap, one library has a bug.

37.8. Code

In Google's `dp_accounting`, you describe the run as an event and compose it into an accountant. Restricting RDP to integer orders avoids the fractional-order weakness.

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(
    sampling_probability=0.01, event=dp_accounting.GaussianDpEvent(noise_multiplier=1.0))
run = dp_accounting.SelfComposedDpEvent(event=step, count=10_000)

pld = dp_accounting.PLDAccountant(value_discretization_interval=1e-5)
pld.compose(run)
eps_pld = pld.get_epsilon(target_delta=1e-5)       # should lie in [6.1377, 6.2377]
rdp = dp_accounting.RdpAccountant(orders=list(range(2, 65)))   # same compose / get_epsilon
```

For the inverse problem, `dp_accounting.calibrate_dp_mechanism` searches for the noise multiplier that meets a target ε and δ. Opacus has `opacus.accountants.utils.get_noise_multiplier`, whose `accountant` argument defaults to `"rdp"`, so pass `"prv"` to match the default of `PrivacyEngine`. Opacus's `"gdp"` accountant uses the CLT, so never report it as a guarantee. For a cross-check, Microsoft's `prv_accountant` installs a command-line tool that prints a lower bound, an estimate and an upper bound.

```
compute-dp-epsilon --sampling-probability 0.01 --noise-multiplier 1.0 --delta 1e-5 --num-com
```

37.9. Cheat sheet

Table 37.3.: Which accountant for which mechanism.

Mechanism	Use	Avoid
Gaussian, no subsampling	GDP, exact	Per-step (ε, δ) composition
Poisson-subsampled Gaussian (DP-SGD)	Pessimistic PLD or PRV, with the lower bound as error bar	zCDP; the CLT as a guarantee
Pure-DP steps	PLD, exact for finite outputs, or RDP with the mechanism's own curve	Advanced composition
Fixed-size or shuffled batches	Poisson accountant at $\sigma/2$, or the unamplified bound	The Poisson number at σ

Rules of thumb

- Report the pessimistic PLD, with the optimistic value beside it, and calibrate σ with the same accountant you report.
- Use RDP for quick bounds, filters and many heterogeneous mechanisms.
- Never report a CLT number as a guarantee, and check any reported number against the PLD lower bound.
- Check the sampler first, because accounting fixed-size or shuffled batches as Poisson sampling is the most expensive error in practice.
- Never publish an ε without its accountant and its δ. Give the training parameters and the library version, and ε at several δ, including one well below $1/n$.

37.10. Further reading

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): matching the accounting to the pipeline, and what to report.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST SP 800-226, 2025): what an evaluator will ask of your claim.

Derivations. Every result here is derived step by step in the full edition's chapter.

Part V.

Differential privacy in machine learning

This part is for anyone who trains, ships or reviews a model under differential privacy (DP). It needs the Gaussian mechanism and composition from Part 3 and an accountant from Part 4. Audits also reuse the membership-inference attack of Part 2.

Every private training method has two halves. The privacy half is computed. The main algorithm is stochastic gradient descent (SGD) with clipped per-example gradients and Gaussian noise, called DP-SGD. Its accountant derives ε from the noise multiplier, the sampling rate and the number of steps alone. The utility half, accuracy, is measured, not proved. So each chapter asks two questions: is the ε correct, and is the accuracy at that ε good enough?

The chapters follow the decisions of a project, and their order matters. First choose the privacy unit, a record or a person, as User-level differential privacy in the full edition explains. Then fix the budget, a privacy level ε and a small slack δ, with Choosing ε in the full edition. Only then choose the method. That is usually DP-SGD in the full edition, or for a convex model one of the methods of Private empirical risk minimization in the full edition. Sometimes it is better to answer queries privately, as in Private prediction in the full edition, or to fine-tune a public model, as in Public data and private fine-tuning in the full edition. When batches cannot be sampled at random, follow the regularized leader (FTRL) with correlated noise takes over, in DP-FTRL and matrix factorization in the full edition.

Next, pay for the search over settings with Hyperparameter tuning and private selection in the full edition. Finally, check which groups lose accuracy with Disparate impact in the full edition, and test the model with planted records in Auditing private training in the full edition. The two commonest mistakes are picking the method before the unit and the budget, and treating the accountant's ε as the end of the work.

Adaptive data analysis in the full edition stands apart. It uses the same noise to keep a reused validation or test set honest, which protects conclusions rather than people.

The tools are Opacus for PyTorch, TensorFlow Privacy, and JAX Privacy, with Google's `dp_accounting` or Microsoft's `prv_accountant` for accounting. The running examples, a logistic regression and a small neural network, come with scripts that reproduce every number. By the end, you will be able to train a private model, defend its settings and its ε, and audit it before you ship it.

38. Private empirical risk minimization

38.1. Why this matters

Most of machine learning is **empirical risk minimization (ERM)**, which chooses the weights that minimize the average training loss, usually plus a penalty that keeps them short. The trained weights can give records away (Membership inference). Differential privacy (DP) bounds how much replacing one record can change what you release, through a privacy level ε and a small slack δ. So randomness has to enter somewhere between the data and the released weights.

There are three places to put it, and each gives a classic algorithm. Output perturbation trains as usual and adds noise to the final weights, while objective perturbation adds a random term to the training objective and minimizes it exactly. Gradient perturbation goes furthest back and adds noise to every gradient step. Each one hides with noise how far one record can move some quantity, its sensitivity. For a convex model with a ridge penalty, objective perturbation is usually the most accurate. For anything else only gradient perturbation works, and with clipping and random batches it becomes stochastic gradient descent (SGD) with noise, DP-SGD in the full edition.

38.2. The setting and the noise

A training set D holds a number n of records, each with d features and a label of plus or minus one. Write $\ell(w, z)$ for the loss of the weights w on a record z. With a ridge of strength λ, training minimizes the objective J,

$$J(w; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i) + \frac{\lambda}{2} \|w\|^2, \tag{38.1}$$

and its minimizer is written w^*. Two datasets are neighbours here when one **replaces** a record of the other. Every sensitivity is built from one more number L, a bound on the length of each record's loss gradient. For logistic regression with features of length at most 1, $L = 1$. A private release $\tilde{w}$ is judged by its **excess risk**, the amount by which its objective exceeds the smallest possible value.

The running example is logistic regression with ten features and a ridge of 0.01, at a target of $(1, 10^{-5})$. Every Gaussian method here calibrates its noise exactly (The Laplace and Gaussian mechanisms). For this target the noise must be 3.7306 times the sensitivity, and the classic formula asks for 30% more. The snippet in Section 38.10 computes the factor for any target.

For pure ε-DP, with no slack, the noise vector instead has a density that falls exponentially with its length, with a scale equal to the sensitivity divided by ε.

The guarantee holds only if your pipeline enforces the bounds it assumes. Clip or normalize every feature vector to length at most 1, with a bound fixed from public knowledge, because a norm computed from the private data leaks. Keep logistic labels at plus or minus one, and clip the labels of a linear regression to a public range, because L grows with them. Fix λ in advance, since choosing it by validation on private data is a search that must be paid for (Hyperparameter tuning in the full edition). Finally, the ridge must cover every weight, the intercept included. Scikit-learn's default solver leaves the intercept unpenalized, so drop it and append a constant feature instead, counted in the norm bound.

38.3. Output perturbation

Output perturbation treats training as a black box (Chaudhuri & Monteleoni 2008). When the loss is convex, the ridge makes the objective **strongly convex**, and that pins down how far one record can move the minimizer,

$$\|w^*(D) - w^*(D')\| \le \frac{2L}{\lambda n}. \tag{38.2}$$

Write $\mu(\varepsilon, \delta)$ for the largest safe ratio of sensitivity to noise, which is 0.2681 for the running example's target. The release adds to the minimizer a Gaussian vector Z, with a standard deviation σ on each coordinate,

$$\tilde{w} = w^*(D) + Z, \qquad Z \sim \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \frac{2L}{\lambda n\, \mu(\varepsilon, \delta)}. \tag{38.3}$$

With ten thousand records the sensitivity is 0.02, and the noise is 0.07461 per coordinate.

What goes wrong is the solver. The bound holds for the exact minimizer, and an optimizer that stops where the gradient has length g adds $2g/\lambda$ to the sensitivity. In the running example, a tolerance of 10^{-3} raises the sensitivity to 0.22, eleven times the exact value. So stop on a gradient length below L/n, checked at the end, never on a fixed step count.

38.4. Objective perturbation

Objective perturbation (Chaudhuri, Monteleoni & Sarwate 2011) adds a random linear term to the objective and minimizes the result exactly. All of the objective's curvature resists that term, so data that add curvature also reduce the damage.

It needs a loss of a particular form. The loss must depend on the weights only through the margin, the label times the inner product of the weights and the features. Its slope must be at most 1, its curvature between 0 and a known constant c, and the features of length at most 1. Logistic regression qualifies with $c = \frac{1}{4}$. A small share of the budget pays for the curvature, and the noise gets what is left,

$$\varepsilon' = \varepsilon - 2\ln\Big(1 + \frac{c}{n\lambda}\Big). \tag{38.4}$$

The noise b is the pure-DP noise with scale $2/\varepsilon'$, and the release is

$$\tilde{w} = \arg\min_{w} \Big[J(w; D) + \frac{1}{n}\langle b, w\rangle \Big]. \tag{38.5}$$

This is ε-DP. If ε' comes out zero or negative, raise the ridge until the curvature share takes at most half of the budget. With ten thousand records in the running example, the noise keeps 0.9950 of the budget.

When the data's curvature near the minimizer is about h in every direction, objective perturbation beats output perturbation by a factor of about $(h/\lambda)^2$. In the running example h is about three times the ridge, and the excess risk is ten times smaller.

The price is exact minimization, which no solver delivers, and floating-point rounding can itself leak (Discrete mechanisms). The loss must also be smooth, so a hinge loss must be smoothed first.

38.5. Gradient perturbation: noisy gradient descent

The third method makes each training step private and composes the steps. Its simplest form is full-batch gradient descent from zero weights, with a step size η and a fixed number of steps, written T. At every step t, fresh Gaussian noise, written ξ_t, is added to the gradient, so the update is

$$w_{t+1} = w_t - \eta(\nabla J(w_t; D) + \xi_t), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \frac{2L}{n} \cdot \frac{\sqrt{T}}{\mu(\varepsilon, \delta)}. \tag{38.6}$$

Releasing every iterate is (ε, δ)-DP for any loss whose per-record gradients have length at most L, convex or not. The noise grows like $\sqrt{T}$ because Gaussian steps compose exactly, by the rules of Gaussian differential privacy.

So **the step count is a privacy parameter**. Too few steps leave optimization error, and too many drown the solution in noise. For a strongly convex objective you can choose T from public quantities only, such as the largest curvature the logistic loss allows, and with ten thousand records this rule picks 218 steps. Choosing T by its validation score instead is a computation on the data that must be paid for.

38.6. Noisy gradient descent without a ridge

Many convex models have no useful ridge. Then keep the weights in a ball whose radius R is fixed in advance from public knowledge. Shrink them back onto it after each noisy step, and release the average of the iterates. The noise is the same as in Equation 38.6, and the excess risk falls like $RL\sqrt{d}/(n\varepsilon)$, which is optimal up to logarithms. The cost is computation, since ten thousand records already need 179,629 full passes, so practice runs the same idea on random minibatches.

38.7. How the three compare

For strongly convex objectives, the best excess risk that any (ε, δ)-DP algorithm can guarantee is of order

$$\frac{L^2\, d}{\lambda\, n^2 \varepsilon^2}, \tag{38.7}$$

up to logarithms. So a model with more parameters loses more accuracy at the same privacy level. And n and ε appear only as their product, so halving ε costs the same accuracy as halving the data. On the running example, objective perturbation has the lowest excess risk at every dataset size.

Pure-DP noise also beats Gaussian noise there, but only because the dimension is small, since its mean squared length grows like d^2 rather than d, so prefer Gaussian noise for large models.

How much data do you need before the noise stops hurting? Predictions are more forgiving than excess risk. With a thousand records, objective perturbation lowers held-out accuracy from 0.747 to 0.736, and output perturbation lowers it to 0.681. With ten thousand records, all four releases come within 0.002 of non-private accuracy, so a low-dimensional convex model can be made private almost for free. As a rule, privacy adds little to the sampling error once the number of records is well above $d \ln(1/\delta)/\varepsilon^2$ (Bassily et al. 2019).

38.8. When the model is not convex

A neural network's loss is not convex, and that breaks output and objective perturbation, which both need a unique minimizer that moves little. Gradient perturbation keeps its guarantee, because its proof used only the gradient bound. But a network's gradients have no bound, so you **clip** each record's gradient to a length C (Beyond global sensitivity), which biases what is optimized. Clipping, noise, random minibatches and an accountant together make DP-SGD in the full edition.

38.9. Libraries

For a linear model you rarely need to write any of this. IBM's diffprivlib offers `LogisticRegression(epsilon=..., data_norm=...)`, modelled on scikit-learn's, which implements objective perturbation. Set `data_norm`, the largest feature-vector length, yourself and clip to it, because if you leave it unset diffprivlib computes it from the data and warns that this leaks. Its `LinearRegression` takes `epsilon` with `bounds_X` and `bounds_y`, the public ranges of the features and labels. Both stop an iterative solver at a tolerance, so the exact-minimizer condition holds only approximately.

Output perturbation needs only an ordinary solver and a noise draw. Scikit-learn's `LogisticRegression` minimizes C times the summed loss plus half the squared weights, so $C = 1/(\lambda n)$ gives the objective of this chapter.

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(C=1 / (lam * n), fit_intercept=False, tol=tol, max_iter=max_iter)
clf.fit(X, y)                                    # rows of X with length <= 1
sigma = 2 * L / (lam * n * mu)                   # mu from the snippet below
w = clf.coef_.ravel() + np.random.default_rng().normal(0, sigma, X.shape[1])
```

Choose `tol` well below L/n and check the final gradient length, since the default tolerance voids the sensitivity bound. For a model trained by gradient steps, use Opacus as in DP-SGD in the

full edition. Its accountant assumes add/remove neighbours rather than replacement, so state the relation next to each ε.

38.10. Code

The snippet finds the largest safe Gaussian ratio by bisection and sizes the noise of output perturbation for the running example.

```
from math import erfc, exp, sqrt


def phi(x: float) -> float:
    """Standard normal cumulative distribution function."""
    return 0.5 * erfc(-x / sqrt(2))


def gaussian_mu(eps: float, delta: float) -> float:
    """Largest sensitivity-to-noise ratio mu for which the Gaussian mechanism is (eps, delta
    lo, hi = 1e-6, 50.0
    for _ in range(200):
        mid = (lo + hi) / 2
        d = phi(mid / 2 - eps / mid) - exp(eps) * phi(-mid / 2 - eps / mid)
        lo, hi = (lo, mid) if d > delta else (mid, hi)
    return lo


mu = gaussian_mu(1.0, 1e-5)
print(f"{mu:.4f} {1 / mu:.4f}")  # 0.2681 3.7306
print(f"{2 / (0.01 * 10_000) / mu:.5f}")  # 0.07461
```

38.11. Cheat sheet

Table 38.1.: Which private ERM method to use, with $\mu = \mu(\varepsilon, \delta)$ the largest safe Gaussian ratio, 0.2681 at $(1, 10^{-5})$.

Method	Noise	Needs	Use when
Output perturbation	$\sigma = 2L/(\lambda n \mu)$ on w^*	Convex loss, ridge, near-exact minimizer	Training must stay a black box
Objective perturbation	Scale $2/\varepsilon'$, $\varepsilon' = \varepsilon - 2\ln(1 + c/(n\lambda))$	Margin loss with bounded slope and curvature, $\|x\| \le 1$, exact minimizer	Convex linear model; the data add curvature

Method	Noise	Needs	Use when
Noisy gradient descent	$\sigma = (2L/n)\sqrt{T}/\mu$	Bounded or clipped gradients; T chosen privately	Any model; the basis of DP-SGD

Rules of thumb

- Stop exact-minimizer methods on a gradient tolerance well below L/n, never on a step count.
- The number of steps of noisy gradient descent is part of the privacy budget, and so is a λ tuned on private data.
- Set feature and label bounds from public knowledge, and include the intercept in the ridge or drop it.

38.12. Further reading

- Chaudhuri, Monteleoni & Sarwate, "Differentially Private Empirical Risk Minimization" (JMLR 2011): output and objective perturbation.
- Iyengar et al., "Towards Practical Differentially Private Convex Optimization" (IEEE S&P 2019): objective perturbation that tolerates inexact solvers, and a benchmark of the three methods.

Derivations. Every result here is derived step by step in the full edition's chapter.

39. DP-SGD

39.1. Why this matters

Nearly every deep-learning system that satisfies differential privacy (DP) runs one algorithm. It is stochastic gradient descent (SGD) with each example's gradient clipped to a fixed length and Gaussian noise added to every step, called DP-SGD (Abadi et al. 2016). Clipping bounds one record's influence for any model (Private empirical risk minimization in the full edition). An **accountant** then adds up the cost of every step into one privacy level ε with a small slack δ.

The privacy analysis never looks inside the model, so the settings split in two. The noise, the sampling rate and the number of steps decide ε, while the clipping norm and the step size are free to be tuned for accuracy. Most failures come from code that silently breaks the proof, so a checklist matters as much as the settings.

39.2. The algorithm and its guarantee

A training set holds n records, and the gradient of one record's loss is its **per-example gradient**, written g_i for record i. DP-SGD repeats five operations at every step.

1. Sample a batch. Each record enters independently with probability q, the **sampling rate**, which is called Poisson sampling.
2. Compute the per-example gradient of each record in the batch.
3. Clip each gradient longer than the **clipping norm** C to length C.
4. Add Gaussian noise with standard deviation σC to each coordinate of the sum, where the **noise multiplier** σ measures the noise in units of C.
5. Divide by the expected batch size, the fixed number $B = qn$, and step against the result, scaled by the step size η.

Write w_t for the weights at step t and S_t for its batch. Then the update, with fresh Gaussian noise ξ_t, is

$$w_{t+1} = w_t - \frac{\eta}{B}\Big(\sum_{i \in S_t} g_i \min\Big(1, \frac{C}{\|g_i\|}\Big) + \xi_t \Big), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I). \tag{39.1}$$

Releasing every intermediate set of weights is then (ε, δ)-DP for **add/remove neighbours**, datasets that differ because one has a record the other lacks. The Rényi differential privacy (RDP) accountant computes ε from the sampling rate, the noise multiplier and the number of steps alone (Amplification by subsampling).

This part's running example takes 10,000 steps with $q = 0.01$ and $\sigma = 1$, which costs $\varepsilon = 6.71$ at $\delta = 10^{-5}$. To pick σ for a target, fix the rate and the number of steps and let the accountant solve for it. With the same settings, $\varepsilon = 1$ needs a noise multiplier of 4.13.

39.3. The implementation checklist

The guarantee rests on four conditions, each a property of your code that a reviewer can check. Code that breaks one still trains and reports as before.

First, one record must move the clipped sum by at most C. Batch normalization breaks this, because its statistics let one record change every other record's gradient, so private models use group or layer normalization instead. The clipping norm must also be fixed in advance, since a C set from the current batch makes the noise depend on the data.

Second, the accountant must assume the sampler the code actually runs (Amplification by subsampling). Many data loaders shuffle once per epoch and cut the data into fixed batches. A record is then never absent from an epoch, and the true ε can be far above the Poisson number. Fixed-size batches drawn afresh at every step are covered under add/remove if you give a Poisson accountant the batch's fraction of the data as its rate and half your noise multiplier.

Third, divide the noisy sum by the public number B, never by the actual batch size, because the actual size changes when the target record is added.

Fourth, the accountant must count every step that touched the data, including those of runs discarded during tuning in the full edition.

Beyond the proof, state the neighbouring relation next to every ε, since under replacement one record can move the sum by $2C$. Preprocessing that looks at the data, such as normalizing by the mean, must itself be private. And a person with many records needs user-level privacy in the full edition.

39.4. Clipping and the step size

The clipping norm leaves privacy alone but controls both of its costs. A small C clips more gradients, so records with large gradients count for less, and they are often rare examples (Disparate impact in the full edition). A large C instead means more noise, since the noise is σC.

When every gradient is clipped, only the product ηC of the step size and the clipping norm matters. So the practical recipe is to clip hard and tune one number. On the running example, holding ηC at 0.25 keeps test accuracy at 0.9599 or above for every C up to 0.3. With the step size held fixed instead, a clipping norm of 10 drops accuracy to 0.8347.

What goes wrong is that gradient norms drift. The median per-example gradient norm of the running example falls from 3.592 to 0.006 in a thousand steps, so a C that clips everything early clips only a minority later. Adaptive clipping, which tracks a quantile of the norms privately, follows the drift for a small share of the budget, and Opacus offers it as `clipping="adaptive"`.

39.5. Batch size at a fixed budget

Suppose the privacy budget and the number of epochs, passes over the data, are fixed. Should you take many small steps or a few large ones? For a small sampling rate and a noise multiplier well above 1, the noise multiplier must grow like $\sqrt{q}$. Since q is B/n, the noise on the averaged gradient falls with the batch size,

$$\sigma \propto \sqrt{\frac{B}{n}}, \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{Bn}}. \tag{39.2}$$

Small batches are even worse than this law says, because they need a noise multiplier near or below 1, where each step costs more than the law assumes. For example, take a budget of $\varepsilon = 4$ over 100 epochs. Going from a batch of 10 to one of 1,000 then cuts the noise per averaged gradient by a factor of 18.6, where the law predicts 10.

Fewer steps are the price, and a larger step size pays it. At scale, De et al. (2022) and Anil et al. (2022) report that very large batches are a main ingredient of accurate models. So choose the largest batch that your hardware and your epoch count allow, and scale the step size up with it.

39.6. Making it fast

Computed one record at a time, DP-SGD can run up to two orders of magnitude slower than ordinary training. Of the three fixes, two are free. Vectorized per-example gradients, through `torch.func.vmap` with `grad`, JAX's `vmap` or Opacus's backward hooks, compute exactly the same clipped sum in parallel, so they change only the running time. **Ghost clipping** saves memory as well. A dense layer's per-example gradient is the outer product of the layer's input and the error flowing back into it. Its length is therefore the product of their lengths, so the norms come without forming the gradients.

Microbatching is the costly fix. It averages fixed groups of records and clips the averages, but one record can then flip its group's clipped average, so the sensitivity doubles to $2C$. Write m for the number of records per group. The noise on each coordinate of the averaged update then becomes

$$2m \cdot \frac{\sigma C}{B} \tag{39.3}$$

at the same ε. On the running example, groups of 4 cut test accuracy from 0.9589 to 0.9336, so use it only as a last resort, and check that your library charges for it.

39.7. What privacy costs in accuracy

The running example is a network with one hidden layer of 64 tanh units, trained on 10,000 synthetic records with C and η both 0.5. Table 39.1 trains it at four budgets.

Table 39.1.: Test accuracy of the running example under DP-SGD at four budgets, with $\delta = 10^{-5}$.

	Noise multiplier	Test accuracy (mean $\pm$ standard deviation over 3 runs)
$\varepsilon = 0.5$	7.72	0.8209 ± 0.0180
$\varepsilon = 1$	4.13	0.9175 ± 0.0161
$\varepsilon = 4$	1.36	0.9548 ± 0.0019
$\varepsilon = 8$	0.917	0.9604 ± 0.0019
Clipped, no noise	0	0.9649 ± 0.0009

Clipping alone costs nothing here, so the whole loss comes from the noise, and it grows steeply as ε shrinks. Going from $\varepsilon = 8$ to 4 costs half a point, while halving it from 1 to 0.5 costs almost 10.

Model size matters because the noise grows with the number of trained weights while the clipped signal does not. Write p for the number of trained weights. The noise vector on the averaged update is then about

$$\frac{\sigma C \sqrt{p}}{B} \tag{39.4}$$

long, which in the running example is about ten times the length of the clipped signal. So the best private model is often smaller than the best non-private one, unless public pretraining in the full edition does most of the work.

39.8. The Opacus workflow

To make a PyTorch training loop private, Opacus wraps an ordinary model, optimizer and data loader. First, `ModuleValidator.fix(model)` replaces batch normalization with group normalization. A `PrivacyEngine` then returns private versions of all three. Its `make_private_with_epsilon` picks the noise multiplier for a target ε, δ and number of epochs, while `make_private` takes `noise_multiplier` directly. Both take `max_grad_norm`, the clipping norm, and by default replace the loader with one that draws Poisson batches (`poisson_sampling=True`). The engine's default accountant is the privacy loss random variable (PRV) accountant, `"prv"`, and `accountant="rdp"` selects the Rényi one.

```
from opacus import PrivacyEngine
from opacus.validators import ModuleValidator

model = ModuleValidator.fix(model)                # BatchNorm -> GroupNorm
optimizer = torch.optim.SGD(model.parameters(), lr=lr)
engine = PrivacyEngine()                          # accountant="prv" by default
model, optimizer, loader = engine.make_private_with_epsilon(
    module=model, optimizer=optimizer, data_loader=loader,
    target_epsilon=4.0, target_delta=1e-5, epochs=epochs, max_grad_norm=C)
for epoch in range(epochs):
    for x, y in loader:                           # Poisson batches
```

```
        optimizer.zero_grad()
        criterion(model(x), y).backward()        # per-example gradients
        optimizer.step()                         # clip, add noise, step
print(engine.get_epsilon(delta=1e-5))
```

To size the noise before training, `opacus.accountants.utils.get_noise_multiplier(target_epsilon=..., target_delta=..., sample_rate=..., epochs=...)` solves for σ. When a large batch does not fit in memory, `BatchMemoryManager` from `opacus.utils.batch_memory_manager` splits it into smaller physical batches but still steps once per logical batch, so the accounting is unchanged. Passing `grad_sample_mode="ghost"` turns on ghost clipping, and `make_private` then also returns a wrapped loss function. In TensorFlow, TensorFlow Privacy's `DPKerasSGDOptimizer` takes `l2_norm_clip`, `noise_multiplier` and `num_microbatches`, and the last should equal the batch size.

Tune in a fixed order. First fix ε, a δ below $1/n$ and the number of epochs, since they are the contract. Then choose a large batch, thousands of examples if you can, and let the accountant set the noise multiplier. Finally, set C near or below the median per-example gradient norm, measured on public or proxy data, and search the step size, which now stands for ηC. Count that search as Hyperparameter tuning in the full edition explains.

When a private model is much worse than its non-private twin, the noise is too long for the signal. Change the batch size first, then train fewer weights, for instance by starting from a public model. If training diverges, lower ηC. And if ε looks too good, look for a shuffling loader, a microbatch setting or uncounted tuning runs.

39.9. Cheat sheet

Table 39.2.: DP-SGD settings and checks.

Setting or check	Rule
What sets ε	The sampling rate, the noise multiplier and the number of steps only
Sampler	Poisson; fixed-size batches use rate m/n and $\sigma/2$; never shuffle-and-partition
Model code	Group or layer normalization; divide by $B = qn$
Clipping norm	Fixed in advance, below the typical gradient norm; then tune ηC
Batch size	As large as you can, with the step size scaled up
Speed	Vectorize or ghost-clip; microbatches of m multiply the noise by $2m$
Model size	Noise length about $\sigma C\sqrt{p}/B$; train fewer weights

Setting or check	Rule
Report	ε, δ, the neighbouring relation, every run counted

Rules of thumb

- Read the data loader and the model before trusting an ε.
- Find the ε below which accuracy collapses on your task before you promise a budget.
- Report ε for every step of training, not for the last epoch, and at the δ you state.
- Choose C and the number of epochs on public data, or pay for the runs that chose them.

39.10. Further reading

- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016): DP-SGD and the moments accountant.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): what to clip, how to tune and what to report.

Derivations. Every result here is derived step by step in the full edition's chapter.

40. Hyperparameter tuning and private selection

40.1. Why this matters

Stochastic gradient descent (SGD) with clipped gradients and Gaussian noise satisfies differential privacy (DP) at the level its accountant reports for one run, and is called DP-SGD in the full edition. But nobody trains once. You try a grid of step sizes, score each model on a validation set and ship the best, and every run and every score touched private data. So the search leaks, and you must pay for it honestly.

The answer depends on one choice. With a fixed number of runs, releasing the best can cost as much as releasing all of them. For example, one run of this part's running example costs a privacy level ε of 6.71, but a grid of 10 runs costs 27.23. Now draw the *number* of runs at random, from a distribution fixed in advance. The same expected number of runs then costs between 10.08 and 13.67.

40.2. The procedure and the report

First, spend private runs only where you must, because the cheapest tuning is tuning you don't do on private data. Runs on public or proxy data cost nothing, and so do published DP-SGD settings for similar tasks. Fix a large batch, the number of epochs and a small clipping norm in advance. Below the typical gradient norm only the clipping norm's product with the step size matters, so that product is often the only thing left to search (DP-SGD in the full edition). Use tuners such as Optuna or Ray Tune on private data only for random search, with the number of runs drawn first.

Second, fix the search before you touch the private data. Write down the candidate set, the distribution of the number of runs, the score and the validation set. Deciding to "try a few more", or restarting a search whose result disappoints, makes the search depend on the data, and no theorem covers that.

Third, make the score private, with a disjoint validation set and noise on each count of correct predictions (Section 40.6). Fourth, account for the whole search. For two or three candidates, charge for every run by composition (Section 40.4). For more, use random search with a random number of runs (Section 40.5). Count every run that touched the private data, including crashed runs, debugging runs and evaluations used for early stopping.

Log the candidate set, the drawn number of runs, and each run's hyperparameters, accounting settings and noisy score. Opacus's `get_epsilon` reports one run only, so compose the runs in an accountant yourself. If you trained 50 configurations at $\varepsilon = 2$ each, the honest answer is not 2 but the composition of all 50, which grows almost linearly at large ε (Table 40.1).

Finally, report two numbers. The ε of the released run says what the model alone would leak, while the ε of the whole procedure is what the release actually guarantees. For the Poisson search of Section 40.7, a complete report reads as follows.

> DP-SGD with Poisson sampling rate 0.01, clipping norm 0.5, noise multiplier 1.53 and 1,000 steps. One run satisfies $(0.98, 10^{-5})$-DP. The step size was selected by random search over $\{0.0625, 0.125, \dots, 8\}$, with a Poisson number of runs with mean 8. Each run was scored by its correct predictions on a disjoint validation set of 5,000 records, plus Gaussian noise of standard deviation 20. The whole procedure satisfies $(1.99, 10^{-5})$-DP by the Rényi bound of Papernot & Steinke (2022, Theorem 6), for add/remove neighbours at the record level.

An audit in the full edition tests only the released model, so it cannot catch a missing tuning charge.

40.3. The best of k runs can cost k runs

A common argument says that choosing among eight step sizes reveals at most three bits. It is wrong, because the release is a whole private run, picked because of how it came out. If each run is a randomized response that comes out "good" or "bad", the best of k runs is bad only if every run is bad. The evidence from that event multiplies across runs, so the best of k runs is exactly $k\varepsilon$-DP.

Tuning without privacy and then training once privately does not escape this. Even a single chosen regularization weight can reveal whether a few outliers were in the training set (membership inference).

40.4. Paying by composition

The safe answer is to charge for every run. The best run is computed from the runs' outputs alone, so it costs at most what releasing all of them costs (Properties of DP). Rényi differential privacy (RDP) describes each run by a curve $\varepsilon(\lambda)$ over orders λ (Rényi DP). The curves of k runs add, so the whole search has a curve ε_A of at most

$$\varepsilon_A(\lambda) \le k\,\varepsilon(\lambda). \tag{40.1}$$

For DP-SGD this is simple, because k runs of a number of steps T cost exactly as much as one run of kT steps.

In the running example, ten runs cost 4.06 times one run, because the conversion to ε with a small slack δ pays its fixed cost only once. To keep ten runs at the budget of one, each run needs a noise multiplier of 2.45, which costs accuracy.

40.5. A random number of runs

The way past the $k\varepsilon$ barrier is to draw the number of runs at random, independently of the data. Then "every run was bad" is mostly explained by the search stopping early. Liu & Talwar (2019) showed this first for **random stopping**, which stops after each run with a fixed probability and costs 3ε for ε-DP runs.

For DP-SGD, use the version of Papernot & Steinke (2022), which works with RDP curves. First fix the candidate set and a distribution for the number of runs K. It is either Poisson or truncated negative binomial, a family with a shape parameter η. Then draw K and run that many candidates drawn at random from the set. Release the best by a noisy score. The whole search then costs about one run, plus a term that grows only with the logarithm of the expected number of runs,

$$\varepsilon_A(\lambda) \le \varepsilon(\lambda) + \frac{\ln \mathbb{E}[K]}{\lambda - 1} + (\text{a term set by the shape of } K). \tag{40.2}$$

For pure ε-DP runs it tends to $(2+\eta)\varepsilon$, so the logarithmic distribution ($\eta = 0$) gives 2ε and the geometric ($\eta = 1$) gives 3ε. Google's `dp_accounting` computes it, by composing in an `RdpAccountant` a `RepeatAndSelectDpEvent(event, mean, shape)` that wraps one run's event. Its shape is infinite for Poisson, 1 for geometric and 0 for logarithmic.

```
import dp_accounting
run = dp_accounting.SelfComposedDpEvent(
    dp_accounting.PoissonSampledDpEvent(q, dp_accounting.GaussianDpEvent(sigma)), steps)
search = dp_accounting.dp_event.RepeatAndSelectDpEvent(run, mean=8, shape=float("inf"))
acc = dp_accounting.rdp.RdpAccountant()
acc.compose(search)
print(acc.get_epsilon(1e-5))
```

Table 40.1 evaluates the bounds on the running example.

Table 40.1.: The ε at $\delta = 10^{-5}$ of tuning with DP-SGD runs of the running example (one run: $\varepsilon = 6.71$), by the number of runs and how it is chosen.

Expected runs	10	100	1,000
Composition	27.23	145.17	1035.93
Poisson	13.67	67.77	302.57
Geometric ($\eta = 1$)	11.31	14.32	16.76
Logarithmic ($\eta = 0$)	10.08	11.95	13.46

A smaller η gives a cheaper bound but more often stops after a single run, and the Poisson bound suits runs that are already very private. A random number of runs may also miss the best candidate. On a grid of 8 with 8 runs on average, it never tries the best one with probability 0.4667.

The theorems have three conditions, and each is easy to break. First, each run must draw its candidate independently from a fixed distribution, which is random search. So Bayesian

optimization, which picks each candidate from earlier scores, is not covered and needs adaptive composition. Second, the number of runs must be drawn independently of the data. Third, the score that ranks the runs must be part of each run's private output.

40.6. Selecting by a noisy validation score

When the candidates are fixed, such as public pretrained checkpoints, only the choice among them uses private data. That is a pure **selection** problem, which Selection solved. Score each candidate by its number of correct predictions on a private validation set. One record changes such a count by at most 1, while an accuracy divides by a size that is itself private. Then the **exponential mechanism** at a budget ε picks a candidate whose expected shortfall from the best count is at most

$$\frac{2}{\varepsilon}(\ln k + 1) \tag{40.3}$$

correct predictions, among k candidates. On a validation set of m records, that is an accuracy gap of at most $2(\ln k + 1)/(\varepsilon m)$. For example, choosing among eight trained models on 5,000 validation records costs about a tenth of a point of accuracy at a budget of 0.1.

With disjoint training and validation sets, each record sits in one of them, so a search over trained candidates costs the larger of the training cost and the selection cost, not their sum.

The random-number bounds need a private score too. An exact validation accuracy leaves the validation records unprotected, so add Gaussian noise to each run's count of correct predictions. With noise of standard deviation 20, one noisy count costs 0.182. Eight of them stay below the training runs' cost at every order the conversion uses, so they add nothing to the total.

40.7. Tuning the running example

Table 40.2 tunes the running example's step size over eight values at a total budget of $\varepsilon = 2$. Each strategy picks the noise multiplier that keeps it within budget.

Table 40.2.: Tuning the step size of the running example at a total budget of $\varepsilon = 2$, $\delta = 10^{-5}$, with mean and standard deviation of the released model's test accuracy over repetitions of the whole procedure.

Strategy	σ	ε of one run	Total ε	Test accuracy
No tuning: step size 0.5 from DP-SGD in the full edition	1.03	1.97	1.97	0.9509 ± 0.0015
Grid of 8, single-run ε reported	1.03	1.97	5.60	0.9479 ± 0.0030

Strategy	σ	ε of one run	Total ε	Test accuracy
Tune on proxy data, then one run	1.03	1.97	1.97	0.9487 ± 0.0025
Grid of 8, composition	2.07	0.66	1.99	0.9430 ± 0.0022
Random search, Poisson, mean 8	1.53	0.98	1.99	0.9433 ± 0.0060
Random search, logarithmic, mean 8	1.36	1.17	2.00	0.9372 ± 0.0200

The second row is the common reporting bug. A team runs the grid at the noise of a single run and reports that run's 1.97, when the honest cost is 5.60. It then looks better than an honest search of the same grid.

The honest searches lose about a point against one untuned run. Poisson random search beats composition only slightly, because with one run per step size on average it often misses one, and the logarithmic distribution often stops after one run. Tuning on proxy data, a related task labelled by a different random network, is nearly free.

40.8. Cheat sheet

Table 40.3.: Which tuning strategy for which search.

Situation	Strategy	Cost
Good hyperparameters from public or proxy data	One private run	One run
Two or three candidates	Fixed grid, composition	k runs of T steps = one run of kT steps
Many candidates, small per-run budget	Random search, Poisson number of runs	One run plus a term in the log of the mean
Many candidates, larger per-run budget	Random search, logarithmic or $\eta = 0.5$	Near $(2 + \eta)$ runs for pure DP; may try few
Fixed candidates, private validation set	Exponential mechanism on correct-prediction counts	Gap at most $2(\ln k + 1)/\varepsilon$ predictions
Trained candidates, disjoint validation set	Noisy counts	The larger of training and selection costs

Rules of thumb

- Fix the search before touching the private data, and never "try a few more" or restart it.
- Report the whole search's ε, never the selected run's alone.
- Account for Bayesian optimization as adaptive composition, not as a fixed candidate set.
- Never select by exact accuracy on a private validation set.

40.9. Further reading

- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022): the Poisson and truncated negative binomial bounds for DP-SGD.
- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019): random stopping and the 3ε bound.

Derivations. Every result here is derived step by step in the full edition's chapter.

41. DP-FTRL and matrix factorization

41.1. Why this matters

Stochastic gradient descent (SGD) with differential privacy (DP), the DP-SGD in the full edition of this part, owes much of its accuracy to **Poisson sampling**. Many pipelines cannot honestly claim that sampling, because they cut fixed batches, read data in arrival order or take whichever phones check in. Using the Poisson accountant anyway can understate the privacy loss ε considerably (Amplification by subsampling).

Giving up amplification honestly is expensive. On this part's running example, five passes at $\varepsilon = 2$ on a fixed order need a noise multiplier of 4.458 instead of 0.928, and test accuracy falls from 0.9460 to 0.8941. This chapter wins that loss back by correlating the noise across steps. The first such method used the follow the regularized leader (FTRL) optimizer and is called DP-FTRL. Its modern form, the **matrix factorization mechanism**, closes nearly all of the gap.

41.2. The matrix mechanism

Plain gradient descent subtracts each step's noisy sum of clipped gradients, scaled by the step size η over the batch size B, from the starting weights w_0:

$$w_{t+1} = w_0 - \frac{\eta}{B}\hat{S}_t, \qquad \hat{S}_t = \hat{x}_1 + \cdots + \hat{x}_t, \tag{41.1}$$

where $\hat{x}_t$ is the noisy gradient sum of step t. A training run is therefore a release of **prefix sums**, and momentum or a learning-rate schedule only changes the fixed weights with which they are combined. Every model is computed from them, so it is post-processing and inherits their privacy (Post-processing, group privacy and composition).

Stack the true gradient sums as the rows of a matrix x. The prefix sums are then Ax, where A has ones on and below the diagonal. The **matrix mechanism** splits A into an encoder R and a decoder L with $A = LR$. It adds Gaussian noise z to the encoded gradients and decodes the result:

$$\hat{S} = L(Rx + z) = Ax + Lz. \tag{41.2}$$

Each entry of the noise z has standard deviation sC, where C is the clipping norm and the **noise scale** s plays the role of DP-SGD's noise multiplier.

Privacy depends only on the columns of the encoder. A record used at one step shifts Rx by that step's column of R times its clipped gradient. So the encoder's sensitivity, in units of C, is the length of its longest column r_j. The mechanism is then a Gaussian mechanism whose ratio of sensitivity to noise is

$$\mu = \frac{\operatorname{sens}(R)}{s}, \qquad \operatorname{sens}(R) = \max_j \|r_j\|. \tag{41.3}$$

To calibrate, look up the largest ratio that meets your target (ε, δ), written $\mu(\varepsilon, \delta)$, with the exact Gaussian calibration of f-DP and Gaussian DP. Then set $s = \mathrm{sens}(R)/\mu(\varepsilon, \delta)$. This chapter uses $\delta = 10^{-5}$ throughout, and at $\varepsilon = 2$ the ratio is 0.502.

Accuracy depends on the rows of the decoder. Write ℓ_t for the row that produces the t-th prefix sum. Each coordinate of its error has standard deviation $sC\|\ell_t\|$, which is C/μ times the **error per unit of privacy**

$$e_t = \mathrm{sens}(R)\,\|\ell_t\|. \tag{41.4}$$

Two conditions make all this valid during training. First, both factors must be lower-triangular, so that each step's release uses only gradients that already exist. Second, a record's slot in the data order must be fixed in advance, independently of the data. The sensitivity above also assumes neighbours that add or remove a record. If they replace a record instead, its gradient can change by $2C$, and every sensitivity in this chapter doubles.

In code, the mechanism is noisy SGD that adds the correlated noise $R^{-1}z$ to the gradient sums. You keep a buffer of past noise draws and never form a large matrix.

41.3. Choosing a factorization

For a single pass, each factorization trades the encoder's column length against the decoder's row length. The numbers below are for one epoch of $T = 100$ steps.

Independent noise, the identity encoder, is DP-SGD without amplification. Its prefix-sum noise is a random walk, with a final error per unit of privacy of 10.

The **binary tree**, the encoder of the original DP-FTRL (Kairouz et al. 2021), adds noise to the sum over every node of a binary tree over the steps. Its error grows like $\log T$, with a largest error of 6.93 over one epoch.

The **square-root factorization** uses the same lower-triangular matrix for both factors, a square root of A. You can compute it with `scipy.linalg.sqrtm` on the prefix-sum matrix, or from the closed-form coefficients in the snippet below, which are constant along each diagonal. Over one epoch its sensitivity is 1.591, and its final error is 2.53, a quarter of independent noise's.

Optimized factorizations exist in research code (Denisov et al. 2022), but they cost time cubic in T and gain little, so the free square root is the default.

41.4. Several passes: banding and the data order

Over k passes a record enters the encoder through k columns. When no two columns have a negative inner product, as for the identity, the tree and the square root, the worst case is a record that contributes the same full-length gradient each time. The sensitivity is the length of the sum of its columns, maximized over the participation patterns that the data order allows:

$$\mathrm{sens}(R) = \max_{\text{patterns}} \Big\| \sum_{i=1}^{k} r_{\tau_i} \Big\|, \tag{41.5}$$

where $\tau_1, \dots, \tau_k$ are the steps of one pattern.

Repeated participation costs correlated noise more, because the square root's columns for nearby steps overlap, so its sensitivity grows faster than $\sqrt{k}$. **Banding** fixes this by keeping only the first b diagonals of the square root. Suppose every two participations of a record are at least b steps apart, a condition called b**-min-separation**. Then the columns no longer overlap, and the sensitivity is at most $\sqrt{k}$ times the longest column, as for independent noise. Its decoder rows are longer, so banding pays only when participations repeat.

The running example reuses one public random permutation every epoch, so each record takes part exactly 100 steps apart, which is the band used below.

Table 41.1.: Sensitivity (in units of C) and error of the final prefix sum per unit of privacy, for E participations 100 steps apart, $T = 100E$.

Epochs E	Independent: sens / final e_T	Square root	Banded square root
1	1.000 / 10.00	1.591 / 2.53	1.591 / 2.53
10	3.162 / 100.00	9.154 / 16.54	5.031 / 15.69
20	4.472 / 200.00	17.191 / 32.09	7.115 / 29.85

At 20 epochs, the banded sensitivity is exactly $\sqrt{20}$ times its one-epoch value, while the full square root's is more than twice as large. So the data order is part of the mechanism, and a permutation kept for every epoch guarantees a separation of exactly one epoch. Reshuffling every epoch can put a record at the end of one epoch and the start of the next, one step apart, and that breaks the banded bound.

41.5. Which method when

Table 41.2 trains the network of DP-SGD in the full edition with each method at equal ε, with DP-SGD on Poisson sampling and the Rényi differential privacy accountant.

Table 41.2.: Test accuracy (mean of three runs) at equal ε, $\delta = 10^{-5}$.

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 1	0.9343	0.9066	0.9134	0.9361	0.9358
2, 10	0.9493	0.8641	0.9096	0.9398	0.9491
8, 1	0.9371	0.9395	0.9412	0.9461	0.9434
8, 10	0.9520	0.9410	0.9484	0.9530	0.9549

Three patterns guide the choice. Independent noise without amplification is a poor option, trailing DP-SGD by 8.5 points at $\varepsilon = 2$ and ten epochs, while the banded square root comes within 0.02 points. At large ε and few epochs, amplification gains little, so correlated noise can even beat DP-SGD. At small ε over many epochs, DP-SGD keeps its lead, and only banding keeps up.

You can choose before training anything, by computing each candidate's noise in the final prefix sum, sC times the length of the decoder's last row. On the running example, the method with the least of it was the most accurate, or nearly so, in every configuration.

Federated learning is the main use, because the server cannot sample devices at a known rate. If a device contributes at most once every b rounds, a banded mechanism with band at most b gives a provable guarantee, and Google trained its Gboard keyboard language models this way with DP-FTRL (Xu et al. 2023).

You rarely write the mechanism yourself. Google's JAX Privacy library implements matrix-factorization noise for JAX training, including banded factorizations. TensorFlow Federated provides tree-aggregation noise for federated training through `tff.aggregators.DifferentiallyPrivateFactory`. Opacus assumes Poisson sampling, so it is the wrong tool here. Whatever the library, check that its sensitivity uses the participation pattern and the neighbouring relation you actually have.

41.6. Code

To check a library's configuration, the snippet computes a banded square root's sensitivity for your separation and number of epochs.

```
from math import sqrt


def sqrt_coeffs(t: int) -> list[float]:
    r = [1.0]
    for k in range(1, t):
        r.append(r[-1] * (2 * k - 1) / (2 * k))
    return r


def toeplitz_sensitivity(c: list[float], t: int, sep: int, epochs: int) -> float:
    best = 0.0
    for offset in range(sep):
        steps = range(offset, t, sep)[:epochs]
        col = [sum(c[i - p] for p in steps if 0 <= i - p < len(c)) for i in range(t)]
        best = max(best, sqrt(sum(v * v for v in col)))
    return best


T = 100  # steps per epoch
band = sqrt_coeffs(T)  # the banded encoder keeps the first T coefficients
print(f"{toeplitz_sensitivity(band, T * 20, T, 20):.3f}")  # 7.115
print(f"{toeplitz_sensitivity(band, T * 10, T, 10):.3f}")  # 5.031
```

Divide the sensitivity by $\mu(\varepsilon, \delta)$ to get the noise scale.

41.7. Cheat sheet

Table 41.3.: Which mechanism for which pipeline.

Situation	Use	What to expect
You can implement Poisson sampling	DP-SGD with its accountant	Strongest at small ε and many epochs
Fixed order, one pass	Square-root factorization	Close to the optimized factorization, with nothing to optimize
Fixed order, several passes	Banded square root, band at most the separation	Sensitivity at most $\sqrt{k}$ times one column

Rules of thumb

- If you cannot implement Poisson sampling, do not account as if you did.
- Fix the data order before training, independently of the data, and keep the same permutation every epoch.
- Compare methods by the noise in the final prefix sum at equal ε and the same number of epochs.
- Never claim amplification on top of a matrix mechanism unless its analysis allows it, as the banded analysis of Choquette-Choo et al. (2023) does.
- Never reuse the single-participation sensitivity after reshuffling every epoch.

41.8. Further reading

- Denisov, McMahan, Rush, Smith & Thakurta, "Improved Differential Privacy for SGD via Optimal Private Linear Operators on Adaptive Streams" (NeurIPS 2022): the mechanism for training.
- Choquette-Choo, Ganesh, McKenna, McMahan, Rush, Thakurta & Xu, "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023): banding and min-separation.

Derivations. Every result here is derived step by step in the full edition's chapter.

42. Private prediction: PATE and in-context learning

42.1. Why this matters

Training a model with differential privacy (DP) by stochastic gradient descent (SGD), as DP-SGD in the full edition does, makes every later query free, but you must own the training loop. The model may be a large language model (LLM) that you can prompt but not train, or one that answers only a few dozen questions in its life. In those cases you can train ordinary models, keep them secret, and make each *answer* private instead.

The idea is "one record, one vote". You split the private records into k disjoint shards, train one model, a **teacher**, on each, and release a noisy winner of their vote. Because a record lives in one shard, it can change only one vote. This chapter says when such answers beat a private model, how private aggregation of teacher ensembles (PATE) turns them into a free-to-use model, and how the vote protects an LLM's prompt.

42.2. The noisy vote: GNMax

Assign records to shards by a fixed rule chosen before looking at the data, such as a hash of a record identifier, or of a user identifier if the privacy unit is a user. Adding or removing a record then changes one teacher, which can move one vote from one label to another. The list of vote counts, the **vote histogram**, therefore moves by at most $\sqrt{2}$ in Euclidean length. The fixed rule is a condition, because if you cut shards by position in a file, inserting one record shifts every later record and can change every teacher. Likewise, if one person's records land in several shards, that person moves several votes.

GNMax (Papernot et al. 2018) adds Gaussian noise to each count and releases only the label with the largest noisy count. Write σ for the noise's standard deviation. Rényi differential privacy (RDP) describes a mechanism by its privacy loss $\varepsilon(\alpha)$ at each order α (Rényi DP). In those terms each answer costs a straight line:

$$\varepsilon(\alpha) = \frac{\alpha}{\sigma^2}. \tag{42.1}$$

The lines of Q answers add up, even when each query is chosen after seeing the earlier answers. Now write $\rho(\varepsilon, \delta)$ for the largest slope whose line still converts to your target, a privacy loss ε with slack δ (From Rényi DP to (ε, δ)). The noise that Q answers need is then

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon, \delta)}}. \tag{42.2}$$

This chapter uses $\delta = 10^{-5}$ throughout. A budget of $\varepsilon = 1$ then allows a slope of 0.0306, and $\varepsilon = 4$ allows 0.3731.

Table 42.1.: Noise per count for Q answers at $(\varepsilon, 10^{-5})$, from the RDP accountant.

Budget	Answers Q	GNMax σ
$\varepsilon = 1$	1	5.72
	100	57.2
	1,000	181
	10,000	572
$\varepsilon = 4$	1	1.64
	1,000	51.8

Two facts follow from Equation 42.2. The noise grows like the square root of the number of answers, and it does not depend on the teachers at all. So a teacher can be any model, or even a human annotator who saw one shard. The original PATE used **LNMax**, with Laplace noise. It needs less noise for a single answer, but about twice as much from 100 answers on, so use GNMax for anything beyond one or two answers.

Publishing the noisy counts as well as the label releases a whole vector, which the analysis does not cover. And since a gap can never exceed the number of teachers, the noise for 10,000 answers at $\varepsilon = 1$ swamps any ensemble with fewer than a few thousand teachers.

42.3. How accurate a noisy vote is

On two labels, the noisy vote returns the plurality label unless the difference of the two noises exceeds the **gap** g between the two counts. That difference is Gaussian with standard deviation $\sqrt{2}\sigma$, so the plurality wins with probability

$$\Pr[\text{plurality}] = \Phi\Big(\frac{g}{\sqrt{2}\,\sigma}\Big), \tag{42.3}$$

where Φ is the standard normal distribution function. The noiseless plurality is a ceiling, because noise can only pull an answer toward a coin flip.

Take this part's running example from DP-SGD in the full edition, whose 10,000 private records have two labels, and split them among 100 teachers. With the noise for 1,000 answers at $\varepsilon = 4$, a lopsided vote of 98 to 2 returns the plurality with probability 0.9051. But a close vote of 52 to 48 does so with probability 0.5218, barely better than a coin.

More teachers mean smaller shards and weaker teachers, so the right number is found by measuring. On the running example, shrinking the shards to 20 records cuts a teacher's accuracy to 0.6459. Yet the noiseless plurality stays between 0.9336 and 0.9403, because weak teachers err nearly independently. The gaps therefore grow with the number of teachers, and 500 teachers did best. So add teachers as long as the median gap keeps growing, and measure the noiseless vote first, since no budget gets you above it.

42.4. Private answers or a private model

The same budget can train a private model, whose answers are free, or pay for answers one at a time. A private model's accuracy does not depend on the number of queries, while the vote's falls as answers accumulate. So private answers are at least as accurate up to a **break-even** number of queries $Q^*(\varepsilon)$, and the private model wins beyond it.

Table 42.2.: DP-SGD on the running example against GNMax answers: the largest number of queries for which the answers' expected accuracy is at least the private model's.

ε	DP-SGD accuracy	Break-even Q^*	Best number of teachers
0.5	0.8164	58	500
1	0.9128	18	500
2	0.9438	0	—
4	0.9573	0	—

At small budgets DP-SGD loses a lot of accuracy, so the vote wins a window of queries, but from $\varepsilon = 2$ on DP-SGD beats even the ensemble's ceiling. To decide for your task, estimate the private model's accuracy at your budget, and compute the vote's expected accuracy from Equation 42.3 and the teachers' gaps on held-out queries. Then compare them at your expected number of queries.

The table assumes you *can* run DP-SGD, which is false for LLMs behind an interface.

42.5. PATE: from private answers to a private model

PATE spends the budget once. It labels Q *public, unlabelled* inputs with GNMax answers and trains a new model, the **student**, on them. The student is released and the teachers never are. Its weights and predictions cost exactly what the Q answers cost.

The recipe has four conditions. You need public inputs from the same distribution as the private data, which is easy for images or text and impossible for most medical records. The pool must really be public, because nothing protects an input that is someone's private record. The number of labels needs tuning, since more labels mean more noise on each. And the teachers stay on a server you trust, because they are non-private models open to every attack of Part 2.

On the running example, PATE loses to DP-SGD at every budget measured. At $\varepsilon = 4$, the best student of 250 teachers reaches 0.8898, against 0.9573 for DP-SGD. The weak teachers often disagree, and PATE wins only where teachers agree on most inputs and the student also learns from unlabelled inputs. On handwritten digits, Papernot et al. (2018) report a student within one point of the non-private accuracy.

42.6. Data-dependent accounting and Confident-GNMax

The cost α/σ^2 is a worst case over all vote histograms. But when nearly all teachers agree, one record almost never changes the answer, and a **data-dependent analysis** (Papernot et al. 2018) charges such a query far less. On tasks where teachers are nearly unanimous, as on digits, ε falls by a large factor. On the running example the saving is only about 20%, because few queries have large gaps.

The trap is that a data-dependent ε is computed from the votes, so publishing it leaks information about the data. Papernot et al. release it with noise scaled to its smooth sensitivity (Beyond global sensitivity) and pay for that release. Until it is released privately, a data-dependent ε is a measurement, not a guarantee.

Two refinements exist. **Confident-GNMax** answers only the queries on which a noisy check finds the teachers in agreement, which gave a student of 0.8894 on the running example, a small gain over plain GNMax. **Stable aggregation** (Bassily, Thakkar & Thakurta 2018) releases the exact plurality of lopsided votes, but it is mainly a tool for proofs.

42.7. Private in-context learning

An LLM can learn a task from labelled examples placed in its prompt, called **exemplars**. If the exemplars are private, the answers leak them. Duan et al. (2023) show that GPT-3, prompted with one private example, favours that example's correct label enough for a membership inference attack.

The fix is private prediction with prompts as teachers (Duan et al. 2023; Wu et al. 2024). You split the private exemplars into disjoint groups by a fixed rule such as a hash, prompt the model once per group with the query, and release the GNMax label of the answers. So each answer costs α/σ^2 as before, whatever the LLM computes.

Wu et al. also draw a fresh Poisson sample of the exemplars for each query, at a sampling rate r, and split only the sample into groups. So each answer costs the subsampled Gaussian of Amplification by subsampling, whose noise multiplier here is $\sigma/\sqrt{2}$. If $\varepsilon_{r,s}(\alpha)$ is that curve at rate r and multiplier s, then Q answers cost

$$\varepsilon(\alpha) = Q\,\varepsilon_{r,\,\sigma/\sqrt{2}}(\alpha). \tag{42.4}$$

Within one query the groups must still be disjoint, or one record would move two votes. Subsampling cuts the noise sharply, from 181 to 7.39 in the full edition's simulation of 1,000 answers at $\varepsilon = 1$.

In-context learning is where private prediction is strongest, because prompted LLMs agree with each other far more than small models do. Wu et al. report 91.97% sentiment accuracy at $\varepsilon = 1$, about as good as one non-private prompt. **PromptPATE** (Duan et al.) goes one step further, as PATE does. It labels a few public inputs privately and uses them as the exemplars of one public prompt, which then answers any number of queries at no further cost. Check whether a reported ε for it is data-dependent before treating it as a guarantee.

42.8. Code

The vote itself is a few lines of NumPy: count each label, add Gaussian noise to the counts and take the largest. Account for the answers with a maintained library rather than by hand. In Google's `dp_accounting`, one GNMax answer is a `GaussianDpEvent` whose noise multiplier is $\sigma/\sqrt{2}$. Compose Q of them with `SelfComposedDpEvent`, or wrap each in a `PoissonSampledDpEvent` for subsampled in-context learning. Then read ε off an `RdpAccountant` or a `PLDAccountant`. The full edition's script private_prediction.py reproduces this chapter's numbers.

42.9. Cheat sheet

Table 42.3.: Which private-prediction tool for which situation; GNMax noise is $\sigma = \sqrt{Q/\rho(\varepsilon, \delta)}$.

Situation	Use	Watch out for
Few queries, small budget, or a model you cannot train privately	GNMax over teachers on hashed shards	Release the label only; noise grows like $\sqrt{Q}$
Many queries, public unlabelled inputs available	PATE student	Teachers stay secret; tune the number of labels
Private exemplars in a prompt	Vote over disjoint groups, or PromptPATE	Never let one prompt with private exemplars answer users
Many queries, DP-SGD possible, $\varepsilon \geq 2$	A private model	On the running example the vote never wins there

Rules of thumb

- Count your queries before choosing between private answers and a private model.
- Measure the noiseless vote first, because it caps every private answer.
- Add teachers as long as the vote gaps grow.
- Assign shards and exemplar groups by a hash, never by position.
- Report the data-independent ε, or pay for a private release of the data-dependent one.
- Fix the number of answers Q in advance and stop answering when it is reached, since an open-ended query stream has no guarantee.

42.10. Further reading

- Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson, "Scalable Private Learning with PATE" (ICLR 2018): GNMax, the data-dependent analysis and Confident-GNMax.
- Duan, Dziedzic, Papernot & Boenisch, "Flocks of Stochastic Parrots" (NeurIPS 2023), and Wu, Panda, Wang & Mittal, "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024): attacks on prompts, PromptPATE and subsampled voting.

Derivations. Every result here is derived step by step in the full edition's chapter.

43. Public data and private fine-tuning

43.1. Why this matters

Training from random weights with the differential privacy (DP) version of stochastic gradient descent (SGD), DP-SGD in the full edition, loses a great deal of accuracy at a privacy loss ε of 1 and below. The most accurate private models are built differently. They start from a model **pretrained** without privacy on a large public dataset, and only **fine-tune** it privately on the sensitive data. Such models are far more accurate than those trained privately from scratch, as Tramèr & Boneh (2021) and later work report.

Fine-tuning only a small part of the model, such as the last layer or a low-rank update to each matrix, often helps further. That is because DP-SGD adds noise to every weight it trains, while the useful signal is bounded by clipping and usually lies in a few directions. This chapter gives the order in which to try the options and what public data does *not* protect.

43.2. How many weights to train

The privacy argument of DP-SGD needs only that one record changes the clipped gradient sum by at most the clipping norm C, whichever weights are trained. So the choice of trainable weights leaves ε unchanged and changes only how much noise lands on the model. Public data are free for a related reason, because anything computed from public data alone, such as the starting weights, is a fixed input as far as the private data are concerned. The condition is that nothing in it was chosen by looking at the private data. That includes normalization statistics and tokenizers, which are fine when fitted on public data and a leak when fitted on private data.

The noise grows with the number k of trainable weights. Write σ for the noise multiplier and B for the expected batch size. Then the noise on the averaged gradient has length about

$$\text{noise length} \approx \frac{\sigma C \sqrt{k}}{B}. \tag{43.1}$$

The signal, the average clipped gradient, is at most C long whatever k is, and it can even grow when fewer weights are trained, because shorter gradients are clipped less.

This part's running example is a small network with 3,329 weights, pretrained on a related public task and then fine-tuned on 10,000 private records. It trains for 10,000 steps with a clipping norm of 0.5 and a batch size of 100, at a slack δ of 10^{-5}. At $\varepsilon = 1$ the noise multiplier is 4.13, and the noise length is 1.191 for all weights against 0.166 for the 65 weights of the last layer.

Measured against the signal, the noise is 92.0 times larger for all weights but only 6.7 times for the last layer. So the signal needs about 8,455 steps to show through the noise for all weights, and only 44 for the last layer.

Counting weights overstates the cost, though. Noise hurts only in directions in which the loss is curved enough to notice it within the run (Li, Liu et al. 2022). Write h_j for the curvature of the loss in its j-th principal direction. With step size η, the number of directions the noise has time to fill is the **effective dimension**

$$d_{\text{eff}} = \sum_{j=1}^{k} \Big(1 - (1 - \eta h_j)^{2T}\Big), \tag{43.2}$$

and the noise raises the loss in proportion to it. On the running example, full fine-tuning has an effective dimension of 451 out of 3,329 weights. So its noise is 51 times larger in squared length than the last layer's, but it costs only about 7.5 times more. A larger step size or a longer run fills more flat directions with noise, which is why private fine-tuning wants smaller step sizes than training from scratch.

Freezing weights has a cost too, a **bias**, because no amount of private data can move the model along a frozen direction that the task needs.

43.3. Linear probing first

The simplest option freezes everything but the last layer and trains a linear classifier on the frozen features, which is **linear probing**. Tramèr & Boneh (2021) showed that it beats end-to-end private training on standard image benchmarks at moderate budgets. It turns private deep learning back into the convex problem of Private empirical risk minimization in the full edition, so it is the safe first baseline.

Its accuracy is capped by the features. The error of a private probe is the privacy cost of convex empirical risk minimization (ERM) in k dimensions plus the **feature gap**. That gap is how far the best linear model on the features falls short of the best model of any kind, and it does not shrink with more records or a larger ε. The best possible probe reaches 0.9536, against 0.9606 for a network trained non-privately from scratch. The private probe stays within 0.006 of its ceiling at every budget, but it can never pass it. So measure the probe's non-private ceiling first, and if it already meets your target, stop there. To train the probe privately, feed the frozen features to `diffprivlib.models.LogisticRegression`, or run DP-SGD on the one layer.

43.4. Low-rank adapters and full fine-tuning

When the features are not quite right, the probe stops at its ceiling, and you need to change the earlier layers. **Low-rank adaptation** (LoRA, Hu et al. 2022) freezes each pretrained matrix W_0 and learns a correction $UV^\top$ whose rank, the number r of columns of the two factors, is small. For a matrix with d_{in} inputs and d_{out} outputs, that is

$$k = r(d_{\text{in}} + d_{\text{out}}) \tag{43.3}$$

trainable weights instead of $d_{\text{in}} d_{\text{out}}$.

Training starts with the factor U at zero, so the model begins exactly at the pretrained one, and with the factor V random. That random draw must be independent of the private data. The

adapter's per-example gradients are outer products, so the ghost clipping of DP-SGD in the full edition applies and no per-example matrix is ever formed. Yu et al. (2022) showed that private fine-tuning of language models with LoRA matches or beats private full fine-tuning, at a fraction of the cost.

Full fine-tuning is not ruled out. Li, Tramèr et al. (2022) matched or beat the adapters by fine-tuning all the weights of language models, with large batches, a step size tuned for private training, and a fine-tuning objective that resembles the pretraining one. The evidence on accuracy goes both ways, but the adapter is cheaper to store, compute and clip.

In PyTorch, the usual stack is Hugging Face's `peft` library for the adapter and Opacus for DP-SGD. Build a `LoraConfig(r=..., target_modules=...)` and wrap the base model with `get_peft_model(model, config)`, so that only the adapter's weights require gradients. Then pass the model, its optimizer and data loader to the `make_private` method of Opacus's `PrivacyEngine`, which computes per-sample gradients only for the trainable parameters. For large language models, the `private-transformers` package of Li et al. and Microsoft's `dp-transformers` wrap this setup, with ghost clipping to save memory.

A third option projects each private gradient onto a subspace estimated from public gradients and adds noise only there (Zhou, Wu & Banerjee 2021). It keeps the same ε and trades noise for the bias of the signal it projects away.

43.5. Which method at which budget

On the running example, DP-SGD fine-tunes the last layer, a rank-4 adapter on the first layer plus the last layer, or all weights, against a baseline trained from scratch. Each method's step size was chosen at $\varepsilon = 1$ on half of the private test set, and accuracy is reported on the other half.

Table 43.1.: Test accuracy (mean of two runs), $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights k	3,329	3,329	65	525
Step size	0.25	0.1	0.3	0.1
$\varepsilon = 0.5$	0.8589	0.9395	0.9478	0.9464
$\varepsilon = 1$	0.9246	0.9599	0.9497	0.9616
$\varepsilon = 2$	0.9518	0.9663	0.9519	0.9653
$\varepsilon = 8$	0.9611	0.9677	0.9514	0.9655

On this task, pretraining is worth about a factor of eight in ε, since the adapter at $\varepsilon = 1$ matches training from scratch at $\varepsilon = 8$.

Which weights to train depends on the budget. At $\varepsilon = 0.5$ the last layer is best, because its noise is 7.2 times shorter than full fine-tuning's and its ceiling is not yet binding. From $\varepsilon = 1$ on, it is stuck near that ceiling, while the adapter and full fine-tuning keep improving. The adapter is never more than 0.0022 below the best method at any budget, so it is the robust middle choice.

The step size matters as much as the method. A step size three times too large costs full fine-tuning 1.3 points of accuracy. Their best step size is 2.5 times smaller than the one for training from scratch. The adapter's rank, by contrast, is cheap to get roughly right, since ranks from 1 to 16 differ by less than the run-to-run spread.

43.6. What public data does not buy

The guarantee is exactly this. For every fixed public dataset, the map from the private dataset to the fine-tuned model is (ε, δ)-DP. With respect to the public dataset, there is no guarantee of any kind. Non-private pretraining can memorize individual records (Memorization and training-data extraction), and the fine-tuned model inherits whatever the pretrained weights encode (Tramèr, Kamath & Carlini 2024).

This matters most for a person whose record is in both datasets, because only the private copy is protected, and a membership attack against the pretrained model (Membership inference) has no bound at all. Web-scale "public" datasets contain posts, profiles and leaked documents, so a private set of medical notes or emails may well overlap with them.

Before you train and when you report, go through this checklist.

1. Deduplicate the private set against the public data. That protects copies, not people, because exact matching misses paraphrases and records about the same person.
2. Check that the public data resemble the private task, because benchmarks with a "private" set close to curated public data overstate what you will get on genuinely sensitive data.
3. Choose the public model, the step size, the adapter rank and the architecture on public data, or pay for every private run. Keeping the best of three privately fine-tuned models spends the budget of all three runs, unless you use a private selection procedure (Hyperparameter tuning in the full edition).
4. Say what the public data are, how they were obtained and whether overlap was checked, and state that the guarantee covers the private data given the pretrained model.

43.7. Code

Before choosing what to fine-tune, compare the candidates' noise lengths at the noise multiplier your accountant gives.

```
from math import sqrt


def noise_length(sigma: float, clip: float, batch: float, trainable: int) -> float:
    return sigma * clip * sqrt(trainable) / batch


def lora_weights(rank: int, d_in: int, d_out: int) -> int:
    return rank * (d_in + d_out)
```

```
adapter = lora_weights(4, 51, 64) + 65  # first-layer adapter plus the 65-weight last layer
print(adapter)  # 525
print(f"{noise_length(4.13, 0.5, 100, 3329):.3f}")  # 1.191
print(f"{noise_length(4.13, 0.5, 100, adapter):.3f}")  # 0.473
print(f"{noise_length(4.13, 0.5, 100, 65):.3f}")  # 0.166
```

43.8. Cheat sheet

Table 43.2.: Which weights to fine-tune privately. The noise length is $\sigma C\sqrt{k}/B$ for k trainable weights, whatever they are.

Situation	Train	Why
Very small budget, or a probe already meets the target	Last layer (linear probe)	Fewest weights, convex, known ceiling
Moderate budget, features not quite right	Low-rank adapter plus last layer	Never far from the best; cheap to clip and store
Larger budget, large batches, tuned step size	All weights	Pays for the effective dimension, not every weight

Rules of thumb

- Start from a model pretrained on public data that resembles the private task.
- Measure the linear probe's ceiling before anything else.
- Use large batches and a step size smaller than for training from scratch, and tune them on public data.
- Never call the pretrained model private, since nothing protects its training data.
- Expect little from a public model far from the task.

43.9. Further reading

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021): linear probing on good features.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022), and Yu et al., "Differentially Private Fine-tuning of Language Models" (ICLR 2022): full and parameter-efficient private fine-tuning, and the hyperparameters that make each work.

Derivations. Every result here is derived step by step in the full edition's chapter.

44. User-level differential privacy

44.1. Why this matters

Every differential privacy (DP) guarantee in this part so far protects one record. On the running example, stochastic gradient descent (SGD) with clipping and noise, the DP-SGD in the full edition of this part, reaches a privacy loss ε of 6.71 per record. But people do not contribute one record each. A messaging user contributes hundreds of messages and a patient dozens of visits, and together a person's records can reveal far more than any one of them.

The fix rests on one idea: **the neighbouring relation is the privacy unit**. Whatever differs between two neighbouring datasets is what the guarantee protects. So to protect people, make neighbours differ in all of one person's records, bound how far one person can move the computation, and account with that bound. This chapter shows what a record-level ε means for a person, and three ways to train with a guarantee for the person instead. To make it concrete, the running example's 10,000 records are spread very unevenly over 1,000 users, so that the largest holds 348.

44.2. The privacy unit

Take a mechanism M, and two datasets D and D' that differ by adding or removing all the records of one user. The mechanism is **user-level** (ε, δ)-DP if, for every such pair and every set S of outputs,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{44.1}$$

Example-level DP is the same inequality for datasets that differ in one record. User-level DP is therefore not a new notion, and every property and accountant of Part 4 holds for it unchanged. Only the sensitivity changes. The right unit, whether a record, a user, a household or a device, answers the question "whose participation do I want to hide?" Report it with every ε (Choosing an accountant).

The textbook answer for a person with k records is **group privacy** (Post-processing, group privacy and composition). An example-level (ε, δ)-DP mechanism is user-level $(k\varepsilon, \delta_k)$-DP for users with at most k records, where the person's slack is

$$\delta_k = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{44.2}$$

Ten records protected at $\varepsilon = 1$ each give their owner $\varepsilon = 10$, and the slack is worse, because it multiplies the per-record δ by up to $e^{(k-1)\varepsilon}$. On the running example, even with the per-record δ tuned so that the user's slack is 10^{-5}, a user with two records gets $\varepsilon = 18.61$, and a user with ten gets no guarantee at all.

44.3. What an example-level model gives a user

Group privacy treats DP-SGD as a black box, but you know exactly how a user's records enter each step. Each step includes every record independently with the sampling rate q. For a user with k records, what matters is the chance that some record of theirs is in the batch. While kq is small, **a user with k records looks like one record sampled at rate kq**. So you can screen a user's exposure by rerunning the record-level accountant at the rate kq. The user's ε then grows roughly linearly in their number of records. An exact accountant for a user with k records also exists (Charles et al. 2024), and it is a little worse than the screen.

Table 44.1.: The user-level ε at $\delta = 10^{-5}$ of the running example's example-level DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps) for a user with k records. "None" means no $(\varepsilon, 10^{-5})$ guarantee.

Records of the user k	1	2	5	10	50
Exact user accountant	6.72	15.54	52.36	146.20	2161.43
Group privacy, Equation 44.2	6.71	18.61	118.12	none	none
One record at rate kq	6.71	15.32	50.11	134.50	1673.08

At ten records there is no protection to speak of. Still, the direct accountant beats group privacy for every user with more than one record, so never use group privacy for DP-SGD.

The rate-kq screen is within 8% of the exact accountant up to ten records. It is slightly optimistic, so report the exact value, and do not discount it as pessimistic, because a person's similar records have partly aligned gradients.

44.4. User-level DP-SGD

The better route makes the user the unit that DP-SGD samples, clips and accounts for (McMahan, Ramage, Talwar & Zhang 2018). Starting from weights chosen without looking at the data, each step does five things.

1. **Sample users.** Put each user in the batch independently with probability q, now the user sampling rate.
2. **Compute one vector per user.** From each sampled user's records alone, compute their average gradient, or the change in weights after a few local SGD steps.
3. **Clip per user.** Rescale that vector to length at most the clipping norm C.
4. **Add noise.** Sum the clipped vectors and add Gaussian noise with standard deviation σC per coordinate, where σ is the noise multiplier.
5. **Step.** Divide by the expected number of sampled users and take a gradient step.

Because one user moves the sum by at most C, however many records they have, the guarantee is user-level DP with exactly the accountant of example-level DP-SGD. A common variant clips each record instead and keeps at most k records per user. It has the guarantee of DP-SGD with noise multiplier σ/k, so it needs k times more noise.

Changing the unit has two consequences. First, **you pay for the number of units**. On the running example, with 1,000 steps and an expected batch of 100 units, sampling records needs

$\sigma = 0.778$ for $\varepsilon = 4$. Sampling users must use a rate ten times higher, and then it needs $\sigma = 3.78$. Second, **records inside a user are free**. They can be averaged, used for several local steps or subsampled without changing ε. More records make each user's vector a better estimate, so the signal improves while the noise stays fixed (Levy et al. 2021).

The data plumbing comes first. Carry a user identifier through the pipeline, and shard and sample by it. Per-example libraries such as Opacus clip each row of a batch, so with them you write a data loader that yields users and compute each sampled user's clipped vector yourself. Federated frameworks are built around this pattern, for example TensorFlow Federated's `tff.learning.dp_aggregator`, with a device as the user. Whatever the implementation, three things break the proof:

1. A user's vector depends on other users' data, for example through normalization by population statistics.
2. One person is spread over several accounts, so the unit is the account and the person is again a group.
3. Users are iterated in a fixed shuffled order instead of Poisson-sampled, which is a different mechanism (Matrix factorization in the full edition).

44.5. Contribution bounding

If your pipeline is built around records, you can keep it and still protect users. **Contribution bounding** keeps at most k records of each user, the cap, and runs example-level DP-SGD on what remains. The result is user-level DP with the ε that the user accountant gives for k records, and users with fewer records are covered too. The condition is that each user's kept records are chosen without looking at other users' data, for example at random. Otherwise adding one user could change which records others keep. Analytics libraries apply the same bound to queries, for example PipelineDP's `max_partitions_contributed` and `max_contributions_per_partition` parameters, and Tumult Analytics' `MaxRowsPerID` truncation.

Choosing the cap trades noise against bias (Amin et al. 2019). A larger cap keeps more data but needs more noise, while a smaller cap discards records and reweights the data toward light users. On the running example at a user-level $\varepsilon = 4$, raising the cap from 1 to 50 multiplies the noise by 5.8. Accuracy then falls from 0.8917 to 0.6401. The best cap depends on the data, so take it from public knowledge of the population or choose it privately (Hyperparameter tuning in the full edition). Trying caps on test accuracy spends privacy that the accountant does not see.

The two techniques also combine. You can cap each user's records only to bound the compute, and still clip one vector per user. The cap then costs data but no noise.

44.6. Heterogeneous users

User-level training changes the objective, not only the noise. Example-level training weights each user by their number of records, while user-level training with averaged vectors weights users equally. For a keyboard model, weighting people equally keeps a few very active users from defining everyone's suggestions. For a model meant to fit the overall distribution of records, it

under-weights heavy users, and a per-user cap discards their extra records outright (Disparate impact in the full edition).

If heavy users should count for more, sum a user's gradients instead of averaging them before clipping. Their influence then rises with their records until the sum reaches C. Both choices stay inside the per-user computation, so ε does not change.

44.7. Which strategy

Table 44.2 compares three routes on the running example, each with 1,000 steps and an expected batch of 100 units. Strategy (a) runs example-level DP-SGD on all records and accounts for the heaviest user.

Table 44.2.: Test accuracy and noise multiplier at the same user-level budget, $\delta = 10^{-5}$, one run each. The non-private model reaches 0.9612.

User-level ε	(a) All records, $k = 348$	(b) User-level DP-SGD	(c) At most 2 records per user
1	0.5318 ($\sigma = 449$)	0.8065 ($\sigma = 12.9$)	0.7084 ($\sigma = 14.4$)
2	0.5582 ($\sigma = 237$)	0.9028 ($\sigma = 6.89$)	0.8374 ($\sigma = 7.62$)
4	0.5713 ($\sigma = 128$)	0.9347 ($\sigma = 3.78$)	0.8810 ($\sigma = 4.18$)
8	0.5478 ($\sigma = 70.3$)	0.9442 ($\sigma = 2.18$)	0.9333 ($\sigma = 2.4$)

Keeping the record as the unit fails. Strategy (a) stays near chance at every budget, because the heaviest user dictates the noise for everyone. User-level DP-SGD is the best at every budget, and at $\varepsilon = 4$ it is 2.7 points below the non-private model. Its cost is compute, since each step processes all the records of about 100 users. The cap is the fallback when the pipeline must stay record-based. It trails by 5 to 10 points at the smaller budgets and comes close at $\varepsilon = 8$.

Single runs vary by about a point, so read the large gaps, not the small ones. Published comparisons on language models mostly agree. Chua et al. (2024) find that user-level DP-SGD generally beats contribution bounding for fine-tuning, while Charles et al. (2024) find that sampling examples sometimes wins.

44.8. Code

The rate-kq screen needs only a record-level accountant. In Google's `dp_accounting`, it is one subsampled Gaussian step composed over the run:

```
import dp_accounting

step = dp_accounting.PoissonSampledDpEvent(
    sampling_probability=k * q,
    event=dp_accounting.GaussianDpEvent(noise_multiplier=sigma),
)
```

```
accountant = dp_accounting.rdp.RdpAccountant()
accountant.compose(dp_accounting.SelfComposedDpEvent(step, steps))
print(accountant.get_epsilon(target_delta=1e-5))
```

With $k = 1$ and q the user sampling rate, the same code accounts for user-level DP-SGD. The script user_level.py builds the user population, implements the exact user accountant and trains the three strategies.

44.9. Cheat sheet

Table 44.3.: Which route to a user-level guarantee.

Situation	Use	What you get
Example-level model already trained	User accountant, or the rate-kq screen	User's ε grows roughly linearly in k, then faster
You can group records by user	User-level DP-SGD	Records' guarantee, with users as units; pays for the number of users
Pipeline must stay record-based	Contribution bounding with a public cap	User accountant for k records; loses data
Heavy users should count more	Sum before clipping instead of averaging	Same ε, different objective

Rules of thumb

- Name the privacy unit with every ε.
- Screen a user's exposure under an example-level model by rerunning the accountant at rate kq.
- Don't use group privacy for DP-SGD.
- Prefer sampling and clipping users, and fall back on a public cap.

44.10. Further reading

- Charles, Ganesh, McKenna, McMahan, Mitchell, Pillutla & Rush, "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024): the tight user accountant, and sampling examples against sampling users.
- Chua, Ghazi, Huang, Kamath, Kumar, Liu, Manurangsi, Sinha & Zhang, "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024): contribution bounding against user-level DP-SGD.

Derivations. Every result here is derived step by step in the full edition's chapter.

45. Disparate impact

45.1. Why this matters

Training the part's running example with differential privacy (DP) at a privacy budget ε of 4 costs less than one point of test accuracy (private gradient descent in the full edition). The private model reaches 0.9548, against 0.9610 without privacy. That number is an average over the whole test set, so it says nothing about who loses the point. Bagdasaryan, Poursaeed & Shmatikov (2019) studied networks trained with the private form of stochastic gradient descent (SGD), called DP-SGD. They found that the lost accuracy falls mostly on the classes and subgroups that are under-represented in the training data, and called this the **disparate impact** of DP on model accuracy.

So a model that is "less than a point worse" on average can be ten points worse for one group of people. Those groups are often the ones a privacy guarantee is meant to protect, such as small populations, people with rare conditions and records that are unusual in other ways.

45.2. Evaluating a private model per group

Before you look at any results, decide which groups matter, and give each group its own test records. Then train a non-private baseline with the same data and recipe. A gap between groups may already exist without privacy, so the quantity that DP is responsible for compares each group with itself.

Write acc_g for a model's accuracy on the test records of group g. The **privacy cost** π_g is the accuracy the group loses when the same training is made private,

$$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]. \tag{45.1}$$

The average runs over training runs, because private training is random. DP has disparate impact when different groups have different privacy costs, and the difference between two groups' costs is the **privacy cost gap**.

Every accuracy needs an interval. Each test record is either classified correctly or not, so a group's accuracy is a binomial proportion, and the exact Clopper–Pearson interval covers it (Hypothesis testing). In SciPy, `scipy.stats.binomtest(correct, m).proportion_ci(method="exact")` returns it, and in statsmodels `proportion_confint(correct, m, alpha=0.05, method="beta")` does. The gap between two groups needs its own interval. Write $\hat{p}_g$ for a group's measured accuracy and m_g for its number of test records. For a first group a and a second group b, measured on disjoint test records with large counts, an approximate 95% interval for the gap is

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}. \tag{45.2}$$

These intervals are wider than most people expect. Suppose a 2,000-record test split holds a 5% minority, so only 100 of its records come from the minority. On one model trained at $\varepsilon = 4$, 88 of them were correct, which gives an interval 13.6 points wide. Against the majority's 1,812 out of 1,900, the measured gap of 0.074 has the interval $[0.010, 0.138]$ by Equation 45.2. So the gap is real, but it could be anywhere from one point to fourteen. The width falls only like one over the square root of the number of records. So a 5-point gap shows only with a few hundred test records from the group, and a 1-point gap only with thousands.

A larger test set does nothing about the second source of variation, which is training itself. The next run gives another model, and a small group's accuracy varies far more between runs. In the running example at $\varepsilon = 4$, the minority's run-to-run standard deviation is almost six times the majority's. So a comparison of two recipes on one run each can easily reverse for the small group, and you should train three or more times.

`fairlearn.metrics.MetricFrame`, given a `sensitive_features` column, computes any metric per group in its `by_group` table, without intervals. The evaluation can also leak. Per-group accuracies on private test data are statistics of that data, so publishing them needs its own privacy analysis (Choosing ε in the full edition).

45.3. What to expect

To show the effect, the full edition gives the running example a minority. Of its 10,000 training records, 500 come from a different region of input space. Their inputs are longer, and their labels follow a rule that the majority's records do not teach. Without privacy the model learns both groups well, with a gap of only 1.2 points, so any large gap after private training is caused by privacy (Table 45.1).

Table 45.1.: Test accuracy of each group (mean ± standard deviation over 3 runs) of the running example with a 5% minority, trained with SGD, with clipping alone, and with DP-SGD at three budgets ($\delta = 10^{-5}$, noise multiplier σ). The privacy cost is Equation 45.1.

Training	Majority accuracy	Minority accuracy	Privacy cost: majority	minority
Non-private SGD	0.9529 ± 0.0018	0.9411 ± 0.0052	—	—
Clipping only, $\sigma = 0$	0.9581 ± 0.0015	0.9007 ± 0.0048	-0.0051	0.0404
$\varepsilon = 8$, $\sigma = 0.917$	0.9550 ± 0.0006	0.8553 ± 0.0110	-0.0020	0.0858
$\varepsilon = 4$, $\sigma = 1.36$	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0059	0.0934
$\varepsilon = 1$, $\sigma = 4.13$	0.8822 ± 0.0032	0.6133 ± 0.0173	0.0707	0.3278

The minority pays almost all of the cost. At $\varepsilon = 4$ the majority loses 0.59 points and the minority 9.34, which is 15.7 times as much. Even at $\varepsilon = 8$, where the majority loses nothing, the minority loses 8.58 points. An overall accuracy hides this, because a test set with 95% majority records reports a loss of about one point at $\varepsilon = 4$.

The cost depends steeply on the group's share of the data. At $\varepsilon = 4$, a minority with 1% of the training records loses 34.88 points and ends at chance, while one with 20% loses only 2.80.

45.4. Why small groups pay: clipping and noise

Both causes, clipping and noise, are blind to groups. The clipping row of Table 45.1 adds no noise, so it gives no privacy at all. Yet the minority already loses 4.04 points there while the majority gains 0.51, so clipping alone is disparate.

Clipping caps every record's pull on the update at the clipping norm C, so it cuts back hardest the groups whose gradients are long. Gradients are long where the model fits a group badly, as for rare patterns, and where the group's inputs are large, as for outlying records. A small group that has not been learned yet has both, so training gives it less weight than its share of the data.

The noise is the same for every group, while a group's signal grows with its size. So a small group's signal is the weakest against the noise, and its privacy cost grows roughly like one over its **share** of the training records. The two causes also compound, since a group whose gradients clipping halves pays like a group half its size.

What you need from this is a diagnosis. Log the median per-example gradient norm and the fraction of records clipped, per group, during training. In Opacus, each parameter's `p.grad_sample` holds the per-example gradients after the backward pass, so their norms can be grouped by a label you keep beside the batch.

45.5. Mitigations and what they cost

Every fix moves the cost around rather than removing it, as Table 45.2 shows.

Table 45.2.: Mitigations for disparate impact, and when to use them.

Fix	What happens	Use it?
Upweight the group's loss	Clipping scales the longer gradients back to C, so clipped records gain nothing	No
Smaller clipping norm	More records clipped, so the group's effective share falls further	No
Larger clipping norm, with the step size times C fixed	Less clipping bias, but the noise σC grows relative to short gradients	Rarely; on the running example neither direction helped
Clipping norm set from the largest observed gradient	Depends on the data, so it breaks the privacy proof	Only if chosen privately and charged

Fix	What happens	Use it?
Oversample the group	Its records' ε is set by their higher sampling rate, so σ must rise	Only with that accounting
Global scaling with an adaptive bound (Esipova et al. 2023)	The same as a larger clipping norm with a smaller step; keeps the update's direction	Worth a try
Group-aware clipping or fairness constraints (research)	Equalizes the groups' clipping or loss	Only with lawful group labels, spending budget on group statistics
More data from the group, a public pretrained model, or a larger ε	Raises the group's signal or lowers the noise	The fixes that work

Oversampling raises a group's pull but costs that group's privacy, because a record's guarantee depends on its own chance of being sampled (Amplification by subsampling). The rule must be fixed in advance, with the accountant at the highest rate. Sampling the 5% minority four times as often, at $\varepsilon = 4$ for its records, raises the noise multiplier from 1.36 to 4.71. Without the higher noise, the minority would silently get $\varepsilon = 20.96$, the weakest privacy for the group already worst served. With it, the gap narrows from 9.93 points to 3.99, but by **levelling down**. Both groups lose accuracy, the majority 8.01 points.

Some of the cost cannot be avoided by any algorithm, because learning a small group's own rule is private estimation on its few records (Lower bounds). So the reliable fixes are more data from the group, or a starting point that already serves it, such as a public pretrained model (Public data and private fine-tuning in the full edition).

45.6. Code

This function computes the interval for the accuracy gap, Equation 45.2, and prints the one quoted above.

```
from math import sqrt


def gap_interval(pa: float, ma: int, pb: float, mb: int) -> tuple[float, float]:
    """Approximate 95% interval for the accuracy gap pa - pb, measured on disjoint test sets
    se = sqrt(pa * (1 - pa) / ma + pb * (1 - pb) / mb)
    return pa - pb - 1.96 * se, pa - pb + 1.96 * se


print("%.3f %.3f" % gap_interval(0.954, 1_900, 0.880, 100))  # 0.010 0.138
```

The full edition's script trains the running example with a minority and computes every number in this chapter:

```
uv run python code/part05-dp-ml/disparate_impact.py
```

45.7. Cheat sheet

Table 45.3.: The decisions of this chapter.

Question	Answer	Conditions
Does DP hurt this group?	Its privacy cost, Equation 45.1, against the non-private baseline	Same data and recipe, averaged over runs
Is the gap real?	Clopper–Pearson per group, Equation 45.2 for the gap	Disjoint test records, large counts
How many test records?	A few hundred per group for a 5-point gap, thousands for 1 point	Width falls like $1/\sqrt{m_g}$
How big will the cost be?	Roughly one over the group's share, more with long gradients	Grows as ε falls
Which fix?	More data or a public model, then global scaling	See Table 45.2

Rules of thumb

- Report accuracy and privacy cost per group, with intervals and the spread over runs, next to the non-private model's.
- Never report overall accuracy alone, and never report the gap alone, because levelling down closes it too.
- Never compare two recipes on one seed each, because a small group's accuracy varies most between runs.
- Oversample a group only with the accountant at its higher sampling rate.
- Treat published per-group metrics on private test data as a release, and remember that the group labels themselves are sensitive.

45.8. Further reading

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019): the observation.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023): clipping's direction error, and global scaling with an adaptive bound.

Derivations. Every result here is derived step by step in the full edition's chapter.

46. Choosing ε

46.1. Why this matters

Every model trained with differential privacy (DP) ships with a privacy budget ε. The chapter on private stochastic gradient descent (SGD), DP-SGD in the full edition, measured what each budget costs in accuracy. That is half of the choice, and the other half is what each budget means for the people in the data. Without it, ε gets chosen by habit ("everyone uses 8"), and the choice does not survive a reviewer who asks what the number promises.

The answer reads privacy as a test. An attacker who wants to know whether a record was used for training runs a membership test, and every test has two error rates. The budget ε, together with its small slack δ, caps one rate in terms of the other. But the training algorithm has its own curve of the best rates an attacker can reach, which is usually much lower. Choosing ε well means computing that curve, reading it where your threat lives, and writing down what it says.

46.2. Justifying a choice

A defensible choice fixes the claim before the number, in six steps.

1. **Unit and neighbours.** Say whether the privacy unit is a record, a user or a device, and whether neighbours add or remove a unit or replace it. Say over what period the budget is spent, counting every run and search that touched the data (Hyperparameter tuning in the full edition).
2. **Threat.** Say whether the concern is membership, reconstruction or attribute inference, what the attacker knows beforehand, and what it sees, because this decides where you read the curve.
3. **Whole guarantee.** Compute the trade-off curve with a tight accountant (Section 46.3), choose δ small (Section 46.5), and read the curve at the threat's operating points.
4. **Evidence.** The curve is a theorem about an algorithm, not about your code. So run an empirical audit whose lower bound is consistent with the claim (Auditing private training in the full edition).
5. **Utility.** Plot accuracy against ε next to the risk, and pick the smallest budget past the bend where accuracy stops rising steeply.
6. **Comparison.** Only now compare with deployments (Section 46.6) and with the tiers of Ponomareva et al. (2023) in the rules of thumb.

On the running example, measure the risk in step 5 as the largest true positive rate (TPR) of a membership test at a false positive rate (FPR) of 1%. The first two doublings of the budget, from 1 to 4, raise test accuracy by 2.66 and then 1.07 points, while that TPR stays below 0.0724.

But the next doubling gains only 0.56 points, and it nearly triples the TPR, to 0.2092. So past $\varepsilon = 4$, each point of accuracy is bought with a much larger worst-case risk.

A team that chooses the $\varepsilon = 4$ run can then write the statement in Table 46.1, a template for any model card.

Table 46.1.: A privacy statement for the running example's $\varepsilon = 4$ run, following the six steps.

Item	Statement
Training	DP-SGD with Poisson sampling rate 0.01, noise multiplier 1.36, clipping norm 0.5 and 10,000 steps, on 10,000 records, one record per person
Unit and period	One record under add/remove neighbours; this one run, and no other run touched the training data
Assumptions	Poisson sampling and per-example clipping as the accountant assumes, checked by an audit
Guarantee	The Rényi differential privacy (RDP) accountant gives $(3.993, 10^{-5})$-DP and $(5.209, 10^{-8})$-DP; a privacy loss distribution (PLD) accountant gives $\varepsilon \leq 3.729$ at $\delta = 10^{-5}$
Membership	Against an attacker who knows every other record and sees every iterate, any membership test has TPR at most 0.01312 at FPR 10^{-3} and at most 0.0724 at FPR 10^{-2}
Utility	Test accuracy 0.9548, against 0.9610 without privacy

46.3. The pair and the whole curve

A membership test says "member" or "non-member" about one target record. Its TPR is the chance that it says "member" when the target was in the training set. Its FPR is the chance that it says so when the target was not. Every test against an (ε, δ)-DP mechanism satisfies

$$\mathrm{TPR} \leq \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}. \tag{46.1}$$

Attacks are judged at a low FPR, because an attack that rarely flags a non-member identifies people with confidence (Membership inference). Yet at $\varepsilon = 8$ the pair caps the TPR at 0.9997, which constrains almost nothing.

A real training run satisfies DP for a whole range of pairs, with one $\delta(\varepsilon)$ for each ε. The best TPR an attacker can reach at a given FPR is the lowest of all their lines, which is called the **trade-off curve**:

$$\mathrm{TPR}^*(\mathrm{FPR}) = \min_{\varepsilon}\left(e^{\varepsilon}\,\mathrm{FPR} + \delta(\varepsilon)\right). \tag{46.2}$$

For DP-SGD, a PLD accountant computes this curve, and its two roundings bracket the truth (Numerical accounting). Table 46.2 compares the answers for the running example, sampled at rate 0.01 for 10,000 steps.

Table 46.2.: The largest TPR of a membership test against the running example's DP-SGD at $\delta = 10^{-5}$, from the pair alone and exactly from the PLD bracket.

ε	Noise multiplier	Pair, FPR 10^{-3}	Exact, FPR 10^{-3}	Exact, FPR 10^{-2}
4	1.36	0.05461	[0.01215, 0.01312]	[0.06739, 0.0724]
8	0.917	0.9997	[0.05555, 0.0589]	[0.1987, 0.2092]
16	0.695	1	[0.2714, 0.2808]	[0.5447, 0.5588]

At an FPR of 10^{-3}, the curve of the $\varepsilon = 8$ run allows a TPR of at most 0.0589, far below the pair's cap. So a budget of 8 means little as a bare pair, but something real for DP-SGD once you report its curve.

Use the PLD bracket to certify, while an RDP accountant also gives a valid but looser curve. The Gaussian differential privacy (GDP) approximation in the snippet below is a quick check, not a bound, since at $\varepsilon = 16$ it reports 0.3234, above the exact value. With Google's `dp_accounting`, build the run's PLD and read the curve through Equation 46.2, passing a grid of ε to `get_delta_for_epsilon`:

```
from dp_accounting.pld import privacy_loss_distribution as pld
import numpy as np

run = pld.from_gaussian_mechanism(1.36, sampling_prob=0.01).self_compose(10_000)
eps = np.linspace(0.0, 20.0, 2001)
tpr = np.min(np.exp(eps) * 1e-3 + run.get_delta_for_epsilon(eps))  # TPR at FPR 1e-3
```

For ε alone, `dp_accounting.pld.PLDAccountant` and Opacus's default `accountant="prv"` give the same kind of tight answer (Numerical accounting). The curve describes a strong attacker who sees every intermediate model, and a typical record attacked through the final model is much safer.

The same curve bounds reconstruction (Reconstruction attacks). Suppose an attacker knows the target is one of a number m of equally likely records. It then succeeds with probability at most the curve read at an FPR of $1/m$, so at most 0.0589 for a thousand candidates at $\varepsilon = 8$.

46.4. What an attacker can believe

Most people ask how sure someone can become that they were in the data. Write p for the attacker's **prior**, its probability of "member" before it sees the model. If no output has a privacy loss above ε, its belief afterwards is at most

$$\Pr[\text{member} \mid \text{output}] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{46.3}$$

So DP bounds the change of belief, not the belief itself. At $\varepsilon = 4$, an attacker who starts at 1% can end at 35.55%. That is the worst output, though, and in DP-SGD the realized loss is usually far below ε.

Base rates matter as much as the loss. An attacker who acts on a flag is right with probability

$$\Pr[\text{member} \mid \text{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}}. \tag{46.4}$$

Suppose an attacker with a 1% prior flags at an FPR of 10^{-2}, against the $\varepsilon = 8$ run. Its flags are then right with probability at most 0.1744, so five out of six are false. A statement about beliefs must therefore name the attacker, because an insider with a prior of 0.5 starts much closer to certainty.

46.5. How small δ must be

A deliberately bad mechanism shows what δ permits. It publishes each of n records in full, independently with probability δ, and it is $(0, \delta)$-DP. Yet it publishes at least one record with probability

$$1 - (1-\delta)^n \approx 1 - e^{-n\delta}. \tag{46.5}$$

For the running example's 10,000 records at $\delta = 1/n$, that is 0.6321. So $n\delta$ must be much smaller than 1. Here n counts privacy units, which means people when one person has many records (User-level differential privacy in the full edition). NIST SP 800-226 reports the common recommendation that δ be at most $1/n^2$, and calls values above 10^{-5} suspicious.

For DP-SGD a smaller δ is cheap, because its privacy loss has a light tail. Moving the $\varepsilon = 4$ run to $\delta = 1/n^2$ raises its budget only to 5.209.

46.6. Comparing with deployed values

"Industry uses ε between 1 and 10" hides what every ε needs, its **privacy unit** and the **period** over which it is spent. The Census Bureau's 2020 redistricting data used zero-concentrated differential privacy (zCDP), whose guarantee is a single number ρ. It spent $\rho = 2.56$ on persons, reported as $\varepsilon = 17.14$ at $\delta = 10^{-10}$, with one person's record replaced as the unit. Google's Gboard language models, trained privately in federated learning, had device-level guarantees for a whole training run, from about 1 to 14 at the same δ. Apple's local DP is stated per donation, with QuickType at $\varepsilon = 8$ and two donations a day, so up to 16 a day.

So convert both budgets to the same unit, neighbouring relation and δ before comparing. A zCDP parameter ρ converts to a pair by

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{46.6}$$

An add/remove guarantee (ε, δ) implies only the replace-one guarantee $(2\varepsilon, (1 + e^{\varepsilon})\delta)$. A budget per day adds up over time, to 365 a year at $\varepsilon = 1$ per day.

So the running example's $\varepsilon = 8$ run is not "less than half the Census budget", because at $\delta = 10^{-10}$ it is at 11.627, and a replacement can double that.

46.7. Code

This snippet turns a claimed pair into the TPR it allows, and approximates DP-SGD's curve with GDP.

```
from math import exp, sqrt
from statistics import NormalDist


def pair_tpr_cap(eps: float, delta: float, fpr: float) -> float:
    """The largest TPR any test can reach at a given FPR, from the pair (eps, delta) alone."
    return min(1.0, exp(eps) * fpr + delta, 1 - exp(-eps) * (1 - delta - fpr))


def gdp_tpr(q: float, steps: int, sigma: float, fpr: float) -> float:
    """GDP estimate of DP-SGD's TPR at an FPR (sampling rate q, noise multiplier sigma); not
    mu = q * sqrt(steps * (exp(1 / sigma**2) - 1))
    return NormalDist().cdf(NormalDist().inv_cdf(fpr) + mu)


print(f"{pair_tpr_cap(4, 1e-5, 1e-3):.5f}")  # 0.05461
print(f"{gdp_tpr(0.01, 10_000, 1.36, 1e-3):.5f}")  # 0.01244
```

The full edition's script computes the exact curves and every other number in this chapter, in two to three minutes:

```
uv run python code/part05-dp-ml/choosing_epsilon.py
```

46.8. Cheat sheet

Table 46.3.: Which tool answers which question when choosing ε.

Question	Tool	Notes
What does DP-SGD allow?	The trade-off curve, Equation 46.2	Certify with the PLD, check with GDP
How sure can an attacker become?	Equation 46.3 and Equation 46.4	Name the prior; base rates dominate
Can a record be reconstructed?	The curve at FPR $1/m$	State the prior, "one of m"
How small a δ?	$\delta \leq 1/n^2$	n counts privacy units
Comparing with a deployment?	Equation 46.6; replace-one doubles ε	Same unit and δ first

Rules of thumb

- Report the curve, not just the pair, as the TPR at FPR 10^{-3} and 10^{-2}.
- A budget of at most 1 is strong for any mechanism. One up to 10 needs DP-SGD's curve as justification, and one above 10 needs the curve and an audit.
- Default to $\delta \leq 1/n^2$, since it costs DP-SGD little, and never use $\delta \geq 1/n$.
- Never read the line $e^{\varepsilon}\,\mathrm{FPR} + \delta$ as DP-SGD's actual risk, because its curve lies far below it.
- Never quote ε without its unit, δ and period, and never compare a daily budget with a lifetime one.

46.9. Further reading

- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, 2025): privacy units, δ, and the hazards of large budgets.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023): tiers and reporting.

Derivations. Every result here is derived step by step in the full edition's chapter.

47. Auditing private training

47.1. Why this matters

The running example trains a network with differential privacy (DP), using the private form of stochastic gradient descent (SGD) called DP-SGD in the full edition. Its noise multiplier σ, the noise's standard deviation in units of the clipping norm, is 1. The Rényi differential privacy (RDP) accountant turns its configuration into a privacy level ε of 6.71 at a slack δ of 10^{-5}. That number is a proof about an algorithm, while what ships is code, and a bug in the noise or the sampling barely changes the accuracy.

An **audit** checks the code by attacking it. It plants records called **canaries**, trains with and without them, and counts how often a membership test guesses right. DP caps how often any test can be right, so a test that is right too often proves that the code is less private than claimed. The reverse never holds, because a weak attack also fails against a leaky model.

47.2. Which audit for which question

Use the cheapest audit that answers your question (Table 47.1). The per-step audit also works as a regression test (Section 47.9).

Table 47.1.: Which audit answers which question. "Here" means the running example, whose claim is 6.71.

Question	Audit	Cost	Blind to
Does the code clip and add the noise it claims?	Per-step audit with a Dirac canary in every step (Section 47.3)	Two training runs	Sampling and accounting bugs
Is the claimed ε right, accounting included?	End-to-end white-box audit with the likelihood-ratio statistic (Section 47.6)	Thousands of runs per hypothesis	Bugs smaller than its shortfall, about 3 units here
A cheap end-to-end check of a slow model	One-run audit with many white-box canaries (Section 47.7)	One run	Most bugs, since its best bound here was 2.53

Question	Audit	Cost	Blind to
What can a realistic attacker learn from the released model?	Black-box one-run audit with mislabeled canaries (Section 47.8)	One run, plus controls	The mechanism's worst case

47.3. Known bugs, and which audit catches them

Read the code for each bug below, since an audit catches only some of them.

- **Variance passed where a standard deviation is expected.** With the running example's clipping norm of 0.5, this halves the noise. A per-step audit catches it.
- **Noise scaled for the mean but added to the sum,** which makes it too small by the batch size. A per-step audit catches it at once.
- **Clipping applied after averaging, or noise from a weak seed** with only 100 possible values. Nasr et al. (2023) found both in real code with per-step audits.
- **The accountant told the wrong numbers,** such as 20,000 records instead of 10,000 because augmented copies were counted. The noise is right, so only an end-to-end audit can catch it, and only if the error is large.
- **Batches shuffled but accounted as Poisson sampling,** which can make the true ε several times the reported one (Amplification by subsampling). No cheap audit exercises the sampling, so read that code yourself.

The per-step audit puts a Dirac canary (Section 47.5) into every step of one run and leaves it out of another. Each step's noisy value on the canary's coordinate, in units of the clipping norm, is one trial, and for correct code it follows

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1, \sigma^2) \text{ with it.} \tag{47.1}$$

The audit bounds the separation μ of these two distributions from below with Equation 47.5, and correct code has $\mu = 1/\sigma$. On the running example, two runs of 10,000 steps bound μ by 0.886 for the correct code, against the claimed 1, while the variance bug gives 1.745 and is refuted.

Audits catch errors only in the unsafe direction, so code that adds too much noise passes every audit.

47.4. From an attack to a lower bound

A **trial** is one training run with or without the canary, and the attack says "in" when a score, such as minus the canary's loss, is above a threshold. Its **true positive rate (TPR)** is the chance that it says "in" when the canary was trained on. Its **false positive rate (FPR)** is the chance that it says "in" when the canary was left out. DP requires, for every test,

$$\mathrm{TPR} \le e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \le e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{47.2}$$

The rates are measured from n trials with the canary and n without, so the bound must use their pessimistic ends. Write $\bar{\alpha}$ for the one-sided Clopper–Pearson upper bound on the FPR (Hypothesis testing). Write $\underline{\tau}$ for the lower bound on the TPR, both at level $\beta/2$, where β is the chance that the audit is wrong. For a count of `k` events in `n` trials, SciPy gives the upper bound as `scipy.stats.beta.ppf(1 - beta / 2, k + 1, n - k)`, and statsmodels' `proportion_confint(..., method="beta")` gives the same interval. With probability at least $1 - \beta$, the code is then not (ε, δ)-DP for any level below

$$\varepsilon_{\text{lo}} = \max\Big\{ \ln \frac{\underline{\tau} - \delta}{\bar{\alpha}},\ \ln \frac{1 - \bar{\alpha} - \delta}{1 - \underline{\tau}} \Big\}. \tag{47.3}$$

The bound needs independent trials and a threshold chosen without looking at the counted trials. So choose the threshold on half of the trials, and count on the other half.

Suppose 1,000 runs with the canary give 500 true positives, and 1,000 without give one false positive. The point estimate is 6.21, but confidence is expensive, and the certified bound is only 4.43. Even a perfect attack hits a ceiling. With n trials per hypothesis, it can certify at most about

$$\varepsilon_{\max}(n) \approx \ln \frac{n}{\ln(2/\beta)}. \tag{47.4}$$

So each extra unit of ε costs a factor of e in trials. At 95% confidence, 1,000 trials per hypothesis certify at most 5.60, and 10,000 at most 7.90. Certifying the running example's 6.71 takes at least 3,038, so no feasible end-to-end audit can confirm that a large claim is tight.

47.5. Canaries and threat models

An **input-space canary** is a record designed to stand out, such as a mislabeled example. A **gradient-space canary** has a gradient that the auditor writes directly. The strongest is the **Dirac canary** of Nasr et al. (2023). Its gradient has length exactly the clipping norm, all on one coordinate that no other record's gradient touches.

The threat model says what the attacker sees. In the **black-box** model it sees only the final model. In the **white-box** model it sees every intermediate model, and so it can recover each step's noisy gradient sum. Against a Dirac canary seen white-box, the most powerful test thresholds a **likelihood-ratio statistic** built from those sums, which the full edition derives. A simpler score that adds up the canary coordinate's total change is weaker, lowering the most the audit can certify on the running example from 6.01 to 4.35.

47.6. Tight audits of the mechanism

An end-to-end white-box audit with Dirac canaries faces the accountant's own worst case, but it still falls far short of the claim, because at small δ the level is decided at a tiny FPR. For the running example that rate is 8.1×10^{-8}, far below the $3/n$ that n trials can measure. Nasr et al. (2023) avoid the tail by assuming a Gaussian trade-off curve. Write Φ for the standard normal distribution function. A single test then bounds the separation μ from below,

$$\mu \geq \Phi^{-1}(\text{TPR}) - \Phi^{-1}(\text{FPR}), \tag{47.5}$$

read at a moderate FPR, where trials are cheap. Its conversion to ε holds only if the curve really is Gaussian.

On the running example, the rigorous bound Equation 47.3 reaches 3.63 with 50,000 trials per hypothesis, about half the claim. The Gaussian-shape estimate reaches 5.33.

47.7. One-run auditing

Steinke, Nasr & Jagielski (2023) get many trials from one run. Choose a number m of canaries, include each one by its own independent fair coin, train once and score every canary. For a list of r guesses, guess "in" for the $r/2$ highest scores and "out" for the $r/2$ lowest. Under ε-DP, the number W of correct guesses is no more likely to reach any count v than a binomial count is:

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}. \tag{47.6}$$

The certified bound is the largest level whose tail at the observed count is below β.

The coins must be independent and fair, and trying several list lengths needs a Bonferroni correction. Under (ε, δ)-DP, the paper adds a term of at most $2m\delta$.

On the running example, one run with 1,000 white-box Dirac canaries certifies 2.22, against 2.84 from 10,000 runs per hypothesis.

47.8. Black-box audit of the trained network

To see what the released network gives away, plant **mislabeled canaries** and attack the final model with a loss score or the likelihood-ratio attack of Membership inference. Run a non-private control next to it.

The non-private model memorizes its canaries and certifies 2.96, but the DP-SGD model certifies only 0.13. A model with clipping but no noise, which has no finite ε at all, certifies only 0.24. So a black-box bound of 0.5 against a claimed 8 does not show that the model is safe, or that its real ε is 0.5. It measures this attacker only.

47.9. Auditing in continuous integration

The per-step audit is cheap enough to run as a unit test on every commit. Feed the code's own clip-and-noise step a Dirac canary's gradient, written by hand. In Opacus, for example, write it into a parameter's `p.grad_sample` before the `DPOptimizer` runs `clip_and_accumulate` and `add_noise`, and read the noisy gradient afterwards. Test the code path that ships, not a copy of it. Run the steps with the canary and without, record each step's noisy value on the canary's coordinate, and bound μ with Equation 47.5. The test fails when the bound exceeds $1/\sigma$.

Fix the seeds so that a failure can be reproduced, and use a β below 5%, or correct code fails for one seed in twenty.

47.10. Code

This snippet computes the certified bound of Equation 47.3 from counts, and prints the number quoted in the text.

```
from collections.abc import Callable
from math import exp, lgamma, log


def binom_cdf(k: int, n: int, p: float) -> float:
    """Pr[Bin(n, p) <= k], summed in logs so that large n is safe."""
    log_terms = (lgamma(n + 1) - lgamma(i + 1) - lgamma(n - i + 1) + i * log(p) + (n - i) *
    return min(1.0, sum(exp(t) for t in log_terms))


def bisect(f: Callable[[float], float], target: float, lo: float, hi: float) -> float:
    """The x in [lo, hi] with f(x) = target, for f decreasing in x."""
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if f(mid) > target else (lo, mid)
    return lo


def eps_lower_bound(fp: int, tp: int, n: int, delta: float, beta: float = 0.05) -> float:
    """The eps certified with confidence 1 - beta from fp false and tp true positives in n t
    fpr_hi = bisect(lambda p: binom_cdf(fp, n, p), beta / 2, 1e-12, 1 - 1e-12)
    tpr_lo = 1 - bisect(lambda p: binom_cdf(n - tp, n, p), beta / 2, 1e-12, 1 - 1e-12)
    return max(log((tpr_lo - delta) / fpr_hi), log((1 - fpr_hi - delta) / (1 - tpr_lo)))


print(f"{eps_lower_bound(1, 500, 1_000, 1e-5):.2f}")  # 4.43
```

The full edition's script runs every audit:

```
uv run python code/part05-dp-ml/auditing.py
```

47.11. Cheat sheet

Table 47.2.: The results of this chapter.

Result	Formula	Conditions
Certified bound	Equation 47.3, Clopper–Pearson at $\beta/2$	Independent trials; threshold chosen on other trials

Result	Formula	Conditions
Ceiling	Equation 47.4, 5.60 at 1,000 trials	A perfect attack
One run	Equation 47.6, plus at most $2m\delta$	Canaries included by independent fair coins

Rules of thumb

- An audit can refute a claim, never prove one, so report its threat model with its number.
- Budget trials before you start, at least $e^{\varepsilon}\ln(2/\beta)$ per hypothesis, and never call an audit tight with too few.
- Choose thresholds and list lengths on separate data, or pay with a Bonferroni correction.
- Read the sampling and accounting code yourself, because the cheap audits cannot see them.
- Never read a small black-box bound as the model's real ε.

47.12. Further reading

- Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis, "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023): Dirac canaries, per-step audits and real bugs.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023): the one-run audit.

Derivations. Every result here is derived step by step in the full edition's chapter.

48. Adaptive data analysis

48.1. Why this matters

Machine learning reuses its evaluation data all the time. For example, you tune hyperparameters on a validation set, look at the scores and try new settings, and a competition shows a leaderboard score after every submission. Each time, the next question depends on the answers to the earlier ones, which makes the analysis **adaptive**. The practical question is how many such questions one holdout set can answer before its answers stop telling you anything about new data.

The answer depends on how the questions are answered. Exact scores on a holdout set break quickly. Giving each question its own fresh slice of data keeps the answers valid, but it needs data in proportion to the number of questions. Differential privacy (DP) does better. Answers that reveal little about any single record cannot be used to overfit the records behind them. So with noisy answers, the data you need grows only like the square root of the number of questions.

48.2. What to do in practice

What decides the cost is the number of adaptive **rounds**, not the number of queries. A round is a batch of questions fixed before any of its answers is seen. Table 48.1 gives the method for each situation.

Table 48.1.: How to reuse a holdout set, by situation.

Situation	What to do	Cost in records
Questions fixed before any answer is seen	Answer them exactly on one holdout set	Grows like $\ln k$ in the number k of questions
A final evaluation	Keep a test set and use it once	One round
Few adaptive rounds, plenty of data	Give each round its own slice of data	Grows like the number of rounds
Many adaptive rounds, scarce data	Answer through Gaussian noise (Section 48.6)	Grows like $\sqrt{k}$
A training set to explore and a holdout set to confirm	Thresholdout (Section 48.7)	Pays only when the two disagree
Tuning that is already private	Nothing more to do, since privacy protects the winner's score	Free
A public leaderboard	Release improvements, not scores, as in the Ladder	Resists the boosting attack

Choosing the best of k models trained without the validation set is one round. With a number n of validation records, the winner's score then overstates its true score by at most $\sqrt{\ln(2k)/(2n)}$ on average. Your search may already be private in the validation set, at a privacy level ε with slack δ (Hyperparameter tuning and private selection in the full edition). Then the winner's score overstates its true score by at most $e^{\varepsilon} - 1 + \delta$ on average (Section 48.5). On a leaderboard, Blum & Hardt's **Ladder** reports a new score only when a submission beats the best so far by more than a fixed step.

The classical fix inside one dataset is nested cross-validation, which keeps the folds that score the search away from the folds that choose it. Whatever you use, report how many evaluations a validation number has survived. Practice is often milder than the worst case. The risk is real when the records are few, the adaptive rounds are many, and someone has an incentive to climb a leaderboard.

48.3. Reusing a holdout set overfits

A **statistical query** asks a question about each record whose answer is a number between 0 and 1, such as "does this classifier get this record right?". Its empirical value is its average over the n records you hold, and its population value is its average over new data. If all k queries are fixed before the data are drawn, then on average the largest error among their exact answers is at most

$$\sqrt{\frac{\ln(2k)}{2n}}. \tag{48.1}$$

The number of queries hardly matters, because it enters only through a logarithm.

That guarantee fails as soon as a query depends on earlier answers. Blum & Hardt's **boosting attack** shows it on a leaderboard whose holdout labels are fair coin flips, so that nothing can beat 50% on new data. The attacker submits k random labellings, reads their exact scores, and submits the majority vote of those that scored above 50%. With 1,000 submissions on 1,000 holdout records, the final submission reports 0.7182. The overfitting is close to

$$\frac{1}{\pi}\sqrt{\frac{k}{2n}}, \tag{48.2}$$

so it depends only on the ratio of submissions to holdout records.

What goes wrong in practice is rarely a deliberate attack. A long hyperparameter search, run with scikit-learn's `GridSearchCV` on one validation split, does the same thing, because the winner is the setting that happened to fit that split's noise best. So its validation score overstates its test score.

48.4. Sample splitting and its cost

The simplest fix never lets a record answer a question it helped to choose. Split the sample into k equal parts and answer each query on its own part. Then, however the queries were chosen, the largest error is on average at most

$$\sqrt{\frac{k\ln(2k)}{2n}}. \tag{48.3}$$

The data you need now grows linearly with k. With queries in r rounds, one part per round is enough, and the k in front of the logarithm becomes r.

Take a validation set of 10,000 records and a sequential search over 1,000 configurations, each chosen from the earlier scores. Splitting restores validity but leaves the bound at 0.6165, useless for scores between 0 and 1. Bringing it down to 0.05 takes 1,520,181 records, against 1,521 for fixed queries.

48.5. Differential privacy implies generalization

Both results below use (ε, δ)-DP for samples that differ by *replacing* one record, which doubles an add-or-remove ε.

The **transfer lemma** covers a single query chosen by a private algorithm. If the algorithm is (ε, δ)-DP, the chosen query's empirical and population values differ on average by at most

$$e^{\varepsilon} - 1 + \delta. \tag{48.4}$$

The **transfer theorem** covers a whole interaction. Suppose a mechanism answers k adaptive queries, the whole exchange is (ε, δ)-DP, and its answers are within α of the empirical values on average. Then its largest error against the population is on average at most

$$\alpha + e^{\varepsilon} - 1 + \delta. \tag{48.5}$$

The bound holds whatever the analyst does with the answers. For small ε, privacy at level ε costs about ε in accuracy, so answering to within 0.05 needs ε around 0.05 or less. Bassily et al. (2016) proved versions of the theorem that hold with high probability rather than on average.

This DP protects the validity of the answers, not the people in the holdout set, so a holdout set of personal data needs its own privacy accounting.

48.6. Answering with Gaussian noise

The simplest private mechanism adds Gaussian noise with a standard deviation σ to each empirical value. Composing the k answers gives a privacy level ε (Concentrated differential privacy), and with this level the transfer theorem bounds the expected largest error by

$$\sigma\sqrt{2\ln(2k)} + e^{\varepsilon} - 1 + \delta. \tag{48.6}$$

The first term is the noise, and the rest is the transfer term. More noise shrinks the transfer term but grows the noise term, so there is a best σ. At that balance the records you need grow like $\sqrt{k}$, because the privacy loss of k Gaussian answers grows like $\sqrt{k}$.

To use it, choose the σ and δ that minimize the bound, as the snippet below does. Table 48.2 compares the three approaches.

Table 48.2.: Records needed to bring each bound on the expected largest error down to 0.05. The fixed-query column is valid only for queries chosen in advance.

Queries k	Fixed in advance	Sample splitting	Gaussian noise
1	139	139	7,300
1,000	1,521	1,520,181	764,337
1,000,000	2,902	2,901,731,548	33,393,779

The constants are large, so even a single query needs 7,300 records with noise, because the transfer term must itself be below 0.05. So splitting stays cheaper up to a few hundred adaptive queries, and only beyond that does the square root win.

In the other direction, when each record has many features, no efficient mechanism can answer much more than about n^2 adaptive queries (Hardt & Ullman 2014, assuming one-way functions exist).

48.7. The reusable holdout: Thresholdout

Thresholdout (Dwork et al. 2015) answers from the training set while they agree and pays privacy only when they do not. It takes a threshold T, a noise scale σ and a budget of B detections. Below the noisy threshold it returns the training value at no cost, and above it, it counts an **overfitting detection** and returns a noisy holdout value. This is the sparse vector technique of Selection, and no mainstream library ships it, but it takes a few lines of NumPy:

```
rng = np.random.default_rng()
noisy_t = T + rng.laplace(scale=2 * sigma)
def answer(train_value, holdout_value):
    global noisy_t, detections
    if abs(holdout_value - train_value) + rng.laplace(scale=4 * sigma) <= noisy_t:
        return train_value
    detections += 1                      # stop answering once detections reaches B
    noisy_t = T + rng.laplace(scale=2 * sigma)
    return holdout_value + rng.laplace(scale=sigma)
```

Write Δ for the most one holdout record can move a query. A run that stops at its B-th detection is DP in the holdout records at

$$\varepsilon = \frac{2B\Delta}{\sigma}, \tag{48.7}$$

however many queries were answered below the threshold. The threshold sets the resolution of every answer, and when the budget is spent the holdout set must stop answering.

In the full edition's experiment, an analyst screens noise features on a training set and confirms them on a holdout set. A holdout set answered exactly then reports 0.6842 for a classifier whose true accuracy is 0.5, while Thresholdout reports 0.5217. But its formal guarantee is weak, because the run triggered 325.8 detections on average, about $\varepsilon = 29.1$.

48.8. Code

This snippet computes the three bounds for 1,000 queries on 10,000 records, and checks one entry of Table 48.2.

```
from math import expm1, log, sqrt


def fixed_queries(n: int, k: int) -> float:
    """Expected largest error of k queries chosen before the data were drawn."""
    return sqrt(log(2 * k) / (2 * n))


def splitting(n: int, k: int) -> float:
    """Expected largest error when each adaptive query gets its own n/k records."""
    return sqrt(k * log(2 * k) / (2 * n))


def gaussian(n: int, k: int, sigma: float, delta: float) -> float:
    """Transfer-theorem bound for k adaptive queries answered with Gaussian noise sigma."""
    rho = k / (2 * n**2 * sigma**2)
    eps = rho + 2 * sqrt(rho * log(1 / delta))
    return sigma * sqrt(2 * log(2 * k)) + expm1(min(eps, 50.0)) + delta  # cap avoids overfl


def best_gaussian(n: int, k: int) -> float:
    """The smallest Gaussian bound over a grid of noise levels and deltas."""
    grid = [10 ** (e / 100) for e in range(-600, 1)]
    return min(gaussian(n, k, s, d) for s in grid for d in grid)


print(f"{fixed_queries(10_000, 1000):.4f}")  # 0.0195
print(f"{splitting(10_000, 1000):.4f}")  # 0.6165
print(f"{best_gaussian(10_000, 1000):.4f}")  # 0.4095
print(f"{best_gaussian(764_337, 1000):.4f}")  # 0.0500
```

The full edition's script runs the attacks and the reusable-holdout experiment:

```
uv run python code/part05-dp-ml/adaptive_data_analysis.py
```

48.9. Cheat sheet

Table 48.3.: The results of this chapter.

Method	Expected largest error	Conditions
Fixed queries	Equation 48.1	Queries chosen before the data were drawn
Sample splitting	Equation 48.3, with k the number of rounds	Each round on its own records
Transfer theorem	$\alpha + e^{\varepsilon} - 1 + \delta$	Whole interaction DP under replacement
Gaussian noise	Equation 48.6, records growing like $\sqrt{k}$	Noise σ on every answer
Thresholdout	Privacy $\varepsilon = 2B\Delta/\sigma$ for B detections	Answers below the threshold are free

Rules of thumb

- Count adaptive rounds, not queries.
- Keep a test set that is used once, at the end, and never for model selection.
- Privacy at level ε costs about ε in accuracy, so validity needs a small ε.
- Below a few hundred adaptive queries, splitting is cheaper than noise.
- A large holdout set does not make many rounds of tuning safe, since adaptive error grows with the number of rounds.
- On a leaderboard, release improvements, never exact scores.

48.10. Further reading

- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis" (*Science* 2015): Thresholdout and the experiment this chapter scales down.
- Blum & Hardt, "The Ladder: A Reliable Leaderboard for Machine Learning Competitions" (ICML 2015): the boosting attack and a leaderboard that resists it.

Derivations. Every result here is derived step by step in the full edition's chapter.

A. Glossary

Acronyms used in the book and the pages that use them.

AEAD Authenticated encryption with associated data. Used in Cryptography basics.
API Application programming interface. Used in Threat models and privacy games.
AUC Area under the curve. Used in Hypothesis testing, Membership inference.
BSC Binary symmetric channel. Used in Information theory.
CDF Cumulative distribution function. Used in The Laplace and Gaussian mechanisms.
CDP Concentrated differential privacy. Used in Concentrated differential privacy.
CLT Central limit theorem. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant.
CPA Chosen-plaintext attack. Used in Cryptography basics.
DLG Deep Leakage from Gradients. Used in Gradient inversion.
DNA Deoxyribonucleic acid. Used in Lower bounds: what privacy costs.
DP Differential privacy. Used in Property inference, Gradient inversion, Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Query release and private multiplicative weights, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Private mean estimation, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training, Adaptive data analysis.
DPI Data processing inequality. Used in Divergences, Information theory.
EMD Earth mover's distance. Used in Syntactic anonymity and how it breaks.
ERM Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Private empirical risk minimization, Public data and private fine-tuning.
FFT Fast Fourier transform. Used in Numerical accounting with privacy loss distributions.
FL Federated learning. Used in Gradient inversion.
FNR False negative rate. Used in Hypothesis testing.
FPR False positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Property inference, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.
FTRL Follow the regularized leader. Used in Differential privacy in machine learning, DP-FTRL and matrix factorization.

GCM Galois/Counter Mode. Used in Cryptography basics.
GD Gradient descent. Used in Optimization, Amplification by shuffling and by iteration.
GDP Gaussian differential privacy. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Choosing an accountant, Choosing ε.
GDPR General Data Protection Regulation. Used in Linkage and re-identification.
GPU Graphics processing unit. Used in Membership inference.
HIPAA Health Insurance Portability and Accountability Act. Used in Syntactic anonymity and how it breaks.
HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.
HMAC Hash-based message authentication code. Used in Cryptography basics, Linkage and re-identification.
KL Kullback–Leibler. Used in Divergences, Hypothesis testing, Information theory.
LLM Large language model. Used in Private prediction: PATE and in-context learning.
LR Likelihood ratio. Used in Hypothesis testing.
LWE Learning with errors. Used in Cryptography basics.
MAP Maximum a posteriori. Used in Information theory, Attribute inference and model inversion.
ML Machine learning. Used in Membership inference, Choosing an accountant, DP-SGD, Choosing ε.
MWEM Multiplicative weights exponential mechanism. Used in Query release and private multiplicative weights.
NN Nearest neighbour. Used in Algorithmic stability and generalization.
PATE Private aggregation of teacher ensembles. Used in f-DP and Gaussian differential privacy, Private prediction: PATE and in-context learning.
PII Personally identifiable information. Used in Memorization and training-data extraction.
PIN Personal identification number. Used in Information theory.
PLD Privacy loss distribution. Used in From Rényi DP to (ε, δ) and to tests, Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε.
PMF Probability mass function. Used in Discrete and floating-point-safe mechanisms.
PRF Pseudorandom function. Used in Cryptography basics.
PRG Pseudorandom generator. Used in Cryptography basics.
PRV Privacy loss random variable. Used in Numerical accounting with privacy loss distributions, Choosing an accountant, DP-SGD.
RDP Rényi differential privacy. Used in Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, DP-SGD, Hyperparameter tuning and private selection, Private prediction: PATE and in-context learning, Choosing ε, Auditing private training.
RMSE Root-mean-square error. Used in The Laplace and Gaussian mechanisms, Beyond global sensitivity, Private mean estimation.
ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and privacy games, Membership inference.
SGD Stochastic gradient descent. Used in Optimization, Algorithmic stability and generalization, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting,

Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training.

SQL Structured Query Language. Used in Reconstruction attacks.

TPR True positive rate. Used in Hypothesis testing, Threat models and privacy games, Membership inference, Property inference, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.

TV Total variation. Used in Divergences, Hypothesis testing, Cryptography basics, Property inference, Syntactic anonymity and how it breaks.

XOR Exclusive or. Used in Cryptography basics.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification, Syntactic anonymity and how it breaks.

B. Index

AboveThreshold, 124, 129
adaptive composition, 162, 208, 210, 234
advanced composition, 119, 123, 162, 166, 188, 208, 213
amplification by shuffling, 162, 203, 210
amplification by subsampling, 162, 173, 191, 210, 267
analytic Gaussian mechanism, 115
approximate differential privacy, 153–155, 160
approximate zCDP, 175
attribute inference, 51, 66, 260
auxiliary knowledge, 52

basic composition, 111, 112, 119, 123, 188
binomial distribution, 9, 26, 234
black-box attack, 52, 71, 140, 267, 271

canary, 56, 61, 74, 76, 266, 267, 271
central limit theorem, 185, 194, 197, 209, 213, 214
Chernoff bound, 9, 14, 15
clipping, 3, 15, 37, 124, 140, 159, 160, 211, 212, 223, 227, 230, 251, 254, 256–259, 261, 267
Clopper–Pearson interval, 14, 15, 26, 54, 259, 270
composition, 92, 100, 106, 111, 112, 116, 119, 123, 137, 162, 166–168, 173, 185, 188, 191, 197, 202, 208, 210, 213, 216, 231, 233, 234
composition attack, 92

deduplication, 76
Deep Leakage from Gradients, 81
differential privacy, 1, 51, 61, 65, 76, 80, 85, 90, 96, 100, 106, 112, 118, 123, 129, 130, 135, 140, 145, 150, 155, 160, 162, 168, 173, 179, 185, 197, 208, 210, 214, 216, 222, 228, 234, 239, 249, 254, 259, 261, 265, 271
discrete Gaussian distribution, 143–145, 150, 179
discrete Laplace distribution, 143, 145
dominating pair, 197
DP-SGD, 162, 168, 173, 179, 185, 188, 190, 191, 194, 203, 208–210, 212, 213, 216, 222, 226–228, 230–232, 234, 237, 239, 242, 244, 251, 253, 254, 256, 261, 262, 264, 265

eccentricity, 86
empirical risk minimization, 37, 43, 216, 221, 222
entropy, 30–32, 74, 81, 90, 92
exponential mechanism, 124, 128, 132, 175, 233

f-DP, 162, 185
fingerprinting lemma, 155
frequency oracle, 150
fully adaptive composition, 208
Fundamental Law of Information Recovery, 51

Gaussian differential privacy, 162, 181, 184, 185, 194, 209, 210, 213, 264
Gaussian mechanism, 100, 115, 118, 176, 184, 210, 216
generalization gap, 61
global sensitivity, 100, 143
gradient inversion, 51, 80
group privacy, 100, 111, 112, 116, 168, 211, 251, 254

hidden state, 202, 203
homomorphic encryption, 45
honest-but-curious, 52

imputation baseline, 66, 90
individual privacy accounting, 208

k-anonymity, 90, 92
Kullback–Leibler divergence, 21

l-diversity, 90, 92
label-only attack, 52, 61
Laplace distribution, 3
Laplace mechanism, 118, 210
likelihood ratio attack, 59, 61
Lipschitz, 33, 34, 37, 43, 202
local differential privacy, 150
local sensitivity, 140
loss attack, 60, 61

McDiarmid's inequality, 14
membership inference, 51, 56, 61, 216
meta-classifier, 69, 71
min-entropy, 30–32
model inversion, 66
moments accountant, 168, 228
Mondrian, 88
mutual information, 31, 32

Narayanan–Shmatikov attack, 86
neighbouring datasets, 100, 106, 111, 114, 116, 129, 168, 187, 191, 211, 228
normal distribution, 56

objective perturbation, 221, 222
onion effect, 61
optimal composition, 100, 119
output perturbation, 221

parallel composition, 111, 112
permutation invariance, 71
Pinsker's inequality, 21
Poisson distribution, 9, 175, 187, 190, 191, 196, 197, 210–214, 223, 227, 230, 231, 233, 234, 237, 239, 252, 261, 267
Poisson sampling, 9, 190, 196, 197, 211, 212, 214, 223, 230, 239, 261, 267
post-processing, 31, 100, 110–112, 116
privacy filter, 119, 123
privacy game, 51, 56
privacy loss, 162, 173, 178, 191, 210, 261
privacy loss distribution, 162, 173, 178, 191, 194, 196, 197, 209, 210, 212–214, 261, 262, 264
privacy loss random variable, 210
privacy profile, 118, 123
property inference, 51, 71
propose-test-release, 140
PRV accountant, 196, 197
pseudorandom function, 44
pure differential privacy, 118, 130, 140, 145, 153–155, 167, 179, 187, 208, 210, 213, 233

quantitative information flow, 32
quasi-identifier, 85, 90

randomized response, 150, 164, 199, 202, 210
RDP curve, 164, 173, 191
re-identification, 51, 90, 106
reconstruction attack, 51, 52, 98
report-noisy-max, 128, 154
ROC curve, 26, 54, 56
Rényi differential privacy, 162, 164, 166–168, 173, 178, 179, 185, 188, 190, 191, 194, 197, 208–210, 213, 214, 241, 261
Rényi divergence, 21
Rényi filter, 208, 210

sampling without replacement, 190, 191, 227
secure aggregation, 81, 145
sensitivity, 100, 116, 118, 132, 137, 140, 143–145, 164, 167, 168, 175, 179, 237, 239
shadow model, 59, 61, 68, 69
shuffle model, 150, 200, 202, 210
simulation paradigm, 49
smooth sensitivity, 140
snapping mechanism, 145
sparse vector technique, 116, 127, 129
stochastic gradient descent, 37, 43, 162, 168, 173, 179, 185, 188, 190, 191, 194, 202, 203, 208–210, 212, 213, 216, 222, 226–228, 230–232, 234, 237, 239, 242, 244, 251, 253, 254, 256, 261, 262, 264, 265
strong convexity, 33, 34, 37, 38
subsample-and-aggregate, 140
subsampled Gaussian mechanism, 175, 196, 210, 213

t-closeness, 90, 92
threat model, 1, 51, 54, 56, 271
trade-off function, 3, 26, 185, 260, 264
truncated concentrated differential privacy, 175

unary encoding, 150
uniform stability, 43
unit of privacy, 100, 106, 116, 168, 237, 251, 253, 254

variance, 3, 6, 9, 14, 37, 56, 59, 61, 159, 160, 267
vulnerability, 31, 32

white-box attack, 52, 266

zero-concentrated differential privacy, 175, 176, 178, 179, 210, 213, 264